

Inside RavenDB 8.0

Oren Eini

October 2025

Table of Contents

Dedication.....	1
I: Introduction.....	3
1. Welcome to RavenDB	5
1.1. What is RavenDB?	5
1.2. Fascinating information about a boring database.....	6
1.3. About this book.....	6
1.4. Who is this for?.....	7
1.5. Who am I?	7
1.6. In this book.....	8
1.7. Summary	12
2. Zero to RavenDB	13
2.1. Setting RavenDB on your machine	13
2.2. Your first database.....	16
2.3. Working with the RavenDB Studio	20
2.4. Your first RavenDB program.....	23
2.5. The client API	30
2.6. Document identifiers in RavenDB	45
2.7. Document metadata	53
2.8. Distributed compare-exchange operations with RavenDB	54
2.9. Document extensions: beyond JSON documents	56
2.10. Cluster-wide transactions	57
2.11. Testing with RavenDB.....	57
2.12. Summary	60
3. Beyond mere documents: Attachments, Counters, Times Series and Revisions	63
3.1. Attachments	63
3.2. Counters.....	68
3.3. Time Series	72
3.4. Time Series rollup & retention policies	81
3.5. Incremental Time Series	83
3.6. Revisions	86
3.7. Summary	95
4. Document Modeling	97
4.1. Beyond relational data modeling.....	97
4.2. Using RavenDB as a key/value store	99
4.3. Modeling considerations in RavenDB.....	100
4.4. Common modeling scenarios	103
4.5. Advanced modeling scenarios	109
4.6. Additional considerations for document modeling.....	113

4.7. ACID vs. BASE	120
4.8. Summary	122
5. Deep Dive Into the RavenDB Client API	125
5.1. Writing documents	126
5.2. Working with attachments	126
5.3. Working with the document metadata.....	128
5.4. Cross cutting concerns with session events.....	129
5.5. Change tracking and SaveChanges.....	131
5.6. Optimistic & pessimistic concurrency	132
5.7. Patching documents and concurrent modifications.....	138
5.8. Deferring commands	141
5.9. Bulk inserting documents to RavenDB.....	142
5.10. Reading documents.....	146
5.11. Aggressive caching	155
5.12. Changes API.....	156
5.13. Projecting data in queries	157
5.14. Cross-cutting concerns on the client.....	159
5.15. How RavenDB uses JSON.....	160
5.16. Summary	161
6. Advanced Documents Capabilities	163
6.1. Document Expiration	163
6.2. Refreshing Documents	165
6.3. Archiving Documents	167
6.4. Documents Compression	169
6.5. Schema validation	175
6.6. Remote attachments	175
7. Batch Processing With Subscriptions	177
7.1. Processing a batch of documents in a subscription.....	177
7.2. The first real subscription	184
7.3. Optimizing subscription processing	188
7.4. Error handling and recovery with subscriptions	196
7.5. Subscription deployment patterns	200
7.6. Using subscription for queuing tasks	202
7.7. The Outbox design pattern	203
7.8. Parallel work inside a subscription batch	204
7.9. Concurrent subscription processing	206
7.10. Versioned subscriptions.....	208
7.11. Subscriptions, background jobs and the role of the admin.....	211
7.12. Subscription and the RavenDB Cluster.....	213
7.13. Subscriptions behind the scenes	216
7.14. Subscribing to documents in the future.....	217

7.15. Summary	219
8. Large Language Models & AI Integration	221
8.1. Vector search.	222
8.2. Connecting RavenDB to AI Models	228
8.3. Embedding Generation.	233
8.4. Generative AI inside your database	248
8.5. Generative AI & Subscriptions	249
9. RavenDB Clusters	251
10. RavenDB on Cloud and the Edge	253
11. Sharding in RavenDB	255
12. Querying in RavenDB	257
13. Indexing in RavenDB	259
14. MapReduce and Aggregations in RavenDB	261
15. Sharing data and making friends with ETL	263
16. Production Deployments	265
17. Securing your Ravens.	267
18. Inside the Voron storage engine	269
19. Inside the Corax indexing engine	271
20. Operational Recipes	273

Dedication

To Rachel, with my love.

To Tamar, who think such topics are "soooo boring", I hope you'll change your mind at some point.

To Shahar, who would always fall asleep when I talked about the book, hopefully you'll like it better when you grow up.

Part I: Introduction

Chapter 1. Welcome to RavenDB

I used to be *the* database guy, going from client to client and getting their systems to work faster and better. I had the chance to see dozens of teams working with a wide variety of databases. The common thread across all of them was that databases are finicky beasts that require constant care and attention.

RavenDB was created specifically to address that issue. I wanted to have a database that would be *boring*, that would just hold my data and spit it back up again as needed. Something that I can setup, forget about and have it Just Work on its own. I spent the better part of two decades (since 2008) making this vision into reality.

Today, RavenDB is a multi-model, high-performance, non-relational, distributed database for online transactions processing, suitable for running anywhere from isolated edge deployments to geo-distributed cloud clusters. And in almost all scenarios, it is can be run in a setup-and-forget mode, where the database will take care of its own management.

Our goal is to make RavenDB a very boring database, one that allows you to focus on building your application, and not deal with the hassle of data management. That design decision had a major impact on how RavenDB was built and how it operates.

1.1. What is RavenDB?

RavenDB is a **multi-model** database. It primarily deals with documents, but it also supports binary data (BLOB), counters and time-series data. When talking about "documents" in the context of RavenDB, we're talking about structured information in the form of self-contained data (as opposed to Word or Excel documents). A document is usually in JSON or XML format. RavenDB is, essentially, a database for storing and working with JSON data. You can also store binary data and a few other types, but you'll primarily use RavenDB for JSON documents.

RavenDB is **high-performance**. It is designed to be fast, and it is. One of the driving forces behind RavenDB's design was to ensure that we are able to match or exceed your performance requirements. That shows in the overall design (which we'll discuss in detail later in the book), the actual database implementation and in the API and interaction model that RavenDB uses.

RavenDB is **non-relational**. Instead of being based on tables and rows, RavenDB is based on documents. This means that you don't have to deal with the classical Impedance Mismatch problem that you have with relational databases. You can store your data in a way that make sense from a business perspective. Complex data structures can be stored in a single document, and you can query and update them with ease.

RavenDB is **distributed**. You can run it on a single node, (suitable for development or running on the edge) or on a cluster of nodes to gain high availability, load balancing and geo distribution. A single cluster can host multiple databases, each of which can span some or all of the nodes in the cluster. You can run RavenDB on your own machines or used the hosted RavenDB service, which is a fully managed RavenDB in the cloud.

There are certain things that a database should provide; transactions are one (which RavenDB

obviously have), but you also need manageability and a certain degree of ease of use. A common thread we'll explore in this book is how RavenDB was designed to reduce the load on the operations team by dynamically adjusting to your workload and operational environment on the fly. Working with a database should be easy and obvious, and to a large extent, RavenDB facilitates that.

1.2. Fascinating information about a boring database

RavenDB is *meant* to be boring, it is an explicit design goal for us. This may sound strange, but the idea is that if RavenDB is boring, it is doing its job, and you can do yours. That said, the idea is that RavenDB is going to be boring to *use*. I find it fascinating, and I hope that after reading this book, you'll understand the lengths we went to in order to make RavenDB boring to use, reliable to deploy and a joy to work with.

This book isn't meant to provide documentation; RavenDB has quite a bit of documentation that tells you exactly how to use it. While informative, documentation tends to be dry and it hardly makes for good reading (or for an interesting time writing it). A blog post, on the other hand, tells a story—even if most of mine are technical stories. I like writing them, and I've been pleasantly surprised that a large number of people also seem to like reading them.

I'm a developer at heart. That means that one of my favorite activities is writing code. Next in the list is writing about writing code. Writing documentation, on the other hand, is so far down the list of my favorite activities that one could say it isn't even on the list. I do like writing blog posts, and I've been maintaining an active blog for close to fifteen years.^[1]

1.3. About this book

What you're reading now is effectively a book-length blog post. The main idea here is that I want to give you a way to *grok* RavenDB.^[2] This means not only gaining knowledge of what RavenDB does, but also all the reasoning behind the bytes. In effect, I want you to understand all the *whys* of RavenDB.

Although blog posts and books have very different structures, audiences and purposes, I'll still aim to give you the same easy-reading feeling of your favorite blogger. If I've accomplished what I've set out to do, this will be neither dry documentation nor a reference book. If you need either, you can read the online RavenDB documentation.^[3] This is not to say that the book is purely my musings; the content is also born from the training course we've been teaching for the past 15 years and the formalization of our internal onboarding training.

By the end of this book, you're going to have a far better understanding of how and why RavenDB is put together the way it is. More importantly, you'll have the knowledge and skills needed to make efficient use of RavenDB in your systems.

1.3.1. Acknowledgments

This book has been written and re-written several times. In fact, looking at the commit logs, I started writing it in July of 2014. Writing something of this scope, and at the same time pushing the product itself forward is not easy and would not have been possible without the help of many people.

On the technical side, I want to thank Adi Avivi, Anders Wåglund, Arkadiusz Paliński, Arkadiusz Palinski, Aviv Rahmany, Bartosz Piekarski, Dan Bishop, Damian Olszewski, Danielle Greenberg, Djordje Djukic, Egor Shamanaev, Elemar Junior, Efrat Shenhar, Federico Lois, Fitzchak Yitzchaki, Golan Nahum, Gracjan Sadowicz, Grisha Kotler, Grzegorz Lachowski, Idan Ben Shalom, Iftah Ben Zaken, Igal Merhavia, Jay Lokhande, Jonathan Matheus, Judah Himango, Karmel, Karmel Indych, Lev Skuditsky, Liel Nagar, Lukas Bickel, Maciej Aszyk, Maksym Smoliński, Marcin Lewandowski, Mateusz Bartosik, Maxim Buryak, Michael Yarichuk, Michał Leśniak, Paweł Pekról, Rafał Kwiatkowski, Shahar Hikri, Shiran Shalom, Stav Lagziel, Szymon Kulec, Tal Weiss and Tomasz Opalach.

The editors, who had the harsh task of turning raw text into a legible book. Erik Dietrich, Laura Lattimer, Katherine Mechling, Amanda Muledy and above all Danielle Greenberg. All the errors you find were inserted by myself after the last round of edits, I assure you.

The early readers of this book, who had gone above merely giving feedback and actively contributed to making this better. Andrej Krivulcik, Jason Ghent, Bobby Johnson, Sean Killeen, Gabriel Schmitt Kohlrausch, Cathal McHale, Daniel Palme, Alessandro Riolo, Clinton Sheppard, Jan Ove Skogheim, Daniel Wonisch and Stephen Zeng.

Thanks you all, it would have been much harder, and likely not possible, without you.

1.4. Who is this for?

I've tried to make this book useful for a broad category of users. Developers will come away with an understanding of how to best use RavenDB features to create awesome applications. Architects will gain the knowledge required to design and guide large-scale systems with RavenDB clusters at their cores. Operations teams will learn how to monitor, support and nurture RavenDB in production.

Regardless of who you are, you'll come away from this book with a greater understanding of all the moving pieces and how you can mold RavenDB to meet your needs.

This book mostly uses C# as the language for code samples, though RavenDB can be used with .NET, Java, Go, Node.js, Python, Ruby and PHP, among others. Most things discussed in the book are applicable even if you aren't writing code in C#.

All RavenDB official clients follow the same model adjusted to match different platform expectations and API standards. So regardless of the platform you use to connect to RavenDB, this book should still be useful.

1.5. Who am I?

My name is Oren Eini and over 18 years ago, I got frustrated working with relational databases. At the time, I was working mostly as a consultant for companies looking to improve the performance of data-driven applications. I kept seeing customer after customer really struggle with this, not because they were unqualified for the task, but because they kept using the wrong tools for the job.

At some point I got so frustrated that I sat down and wrote a blog post about what I thought an ideal datastore for OLTP applications (online transaction processing applications, a fancy way to say business applications) should look like. That blog post turned into a series of blog posts and then

into some weekend hacking. Fast-forward a few months, and I had the skeleton of what would eventually become RavenDB and a burning desire to make it a reality.

At the time, building RavenDB felt not like I was creating something from scratch, but like I was merely letting out something that was already fully formed. As I mentioned, that was over a while ago, and RavenDB has been running production systems ever since 2010.

In that time, we have learned a lot about what it takes to really make a database that *just works* and doesn't force you to jump through so many hoops. In particular, I came from a Microsoft-centric world, and that world had a big impact on the design of RavenDB. Most NoSQL solutions (especially at the time I started writing RavenDB) had a very different mental model for how they should operate. They put a lot of attention on speed, scale out or esoteric data models, often at the severe expense of ease of use, operational simplicity and what I consider to be fundamental features such as transactions.

I wanted to have a database that would *make sense* for building web applications and business systems; you know, the bread and butter of our industry. I wanted a database that would be ACID, because a database without transactions just didn't make sense to me. I wanted to get rid of the limitations of the rigid schema of relational databases but keep working on domain-driven systems. I wanted something that is fast but at the same time could just be thrown on a production server and would work without having to pay for an on-call babysitter.

A lot of the design of RavenDB was heavily influenced by the Release It!^[4] book, which I *highly* recommend. We tried to get a lot of things right from the get go, and with close to two decades in production to look back at, I think we did a good job there.

1.6. In this book...

One of the major challenges in writing this book was figuring out how to structure it. There are so many concepts that interweave with another and trying to understand them in isolation can be difficult. For example, we can't talk about modeling documents before we understand the kind of features that are available for us to work with. Considering this, I'm going to introduce concepts in stages.

1.6.1. Part I — The basics of RavenDB

Focus: Developers

This is the part you will want new hires to read before starting to work with RavenDB, as it contains a practical discussion on how to build an application using RavenDB. We'll skip over theory, concepts and background information in favor of getting things done; the more theoretical concepts will be discussed later in the book.

We'll cover setting up RavenDB on your machine, opening up the RavenDB Studio in the browser and connecting to the database from your code. After we get beyond the "hello world" stage, we'll introduce some of the basic concepts that you need to know in order to work with RavenDB: building a simple CRUD application, learning how to perform basic queries and in general working with the client API.

After covering the basics, we'll move into modeling documents in RavenDB; how to build your application so that it meshes well with document-based modeling; what sort of features you need to be aware of when designing the domain model and how to deal with common modeling scenarios; concurrency control and dealing with data that doesn't always match the document model (time series or binary data, for example).

Following on this high level discussion, we'll dive into the client API and explore the advanced options RavenDB offers: from lazy requests to reduce network traffic, to the optimal way to read and write a lot of data very quickly, perform partial document updates and see how caching is an integral part of the client API. We'll conclude the first part of the book with an overview of batch processing in RavenDB and how you can use highly available, reliable subscriptions to manage all sorts of background tasks in your application in a quite elegant fashion.

1.6.2. Part II — Distributed RavenDB

Focus: Architects

This part focuses on the theory of building robust and high performance systems using RavenDB. We'll go directly to working with a cluster of RavenDB nodes on commodity hardware, discuss data and work distribution across the cluster and learn how to best structure systems to take advantage of what RavenDB brings to the table.

We'll begin by dissecting RavenDB's dual nature of distributed RavenDB. RavenDB is using both a consensus protocol and a gossip protocol to build two layers of communication between the various nodes in the cluster. We'll learn why we use this dual-mode and how it adds tremendously to RavenDB's robustness in the presence of failures.

After going over the theory, we'll get practical: setting up RavenDB clusters, explore different topologies and study how clients interact with a cluster of RavenDB nodes. We'll cover distributed work, load balancing and ensuring high availability and zero downtime for your applications.

One key reason you'll want to use a distributed system is to handle bigger load. We'll cover how you can grow your cluster and even run RavenDB in a geo-distributed deployment with nodes all around the world. RavenDB clusters aren't just collections of machines. They are self-managing entities, sharing load and distributing tasks among the various nodes. We'll talk about how clusters are self-monitoring and self-healing and how RavenDB takes active steps to ensure the safety of your data at all times.

Sharding is a way to split enormous datasets across multiple nodes. We'll cover how RavenDB handles sharding and what kind of impact going to a sharded database has on your system design and architecture.

Modern systems are rarely composed of a stand-alone application. So to finish up this section, we'll explore how RavenDB integrates with other systems and databases. RavenDB was explicitly designed to make such integration easier. We'll go over how to create data flow that automatically synchronizes data to different destinations, be they RavenDB instances or even relational databases.

1.6.3. Part III — Querying and indexing

Focus: Developers and architects

This part discusses RavenDB indexes data to allow for quick retrieval of information, whether a single document or aggregated data spanning years. We'll cover all the different indexing methods in RavenDB and how each of them can be used to implement the features you want in your systems.

RavenDB has very rich querying and indexing support. We'll start by exploring the RavenDB Query Language (RQL) and the kind of queries that you can perform. We'll look at how RavenDB processes and optimizes your queries to answer them as fast as possible.

Then we'll get to the really fun stuff. RavenDB's queries can answer a lot more than just where `Status = 'Active'`. We'll look at full text queries, querying multiple collections at once and faceted search. We'll look at how RavenDB can find similar documents and suggest to the user different queries to try as the user tries to find a particular nugget of information.

Spatial queries (searching based on geographical data) will be covered in depth. We'll also cover how you can find documents not based on their own data, but on related documents' data. Similar to, but simpler and faster than JOIN from relational databases, the ability to efficiently find documents using related documents can greatly simplify and speed up your queries. One of the strengths of RavenDB is that it is schema-less by nature, and that doesn't stop at data storage. RavenDB also has very powerful capabilities for querying over dynamic data and user-generated content.

MapReduce in RavenDB is a very important feature. It allows RavenDB to perform lightning-fast aggregation queries over practically any dataset, regardless of size. We'll explore exactly how this feature works, the kind of behaviors it enables and what you can do with what are effectively free aggregation queries.

Finally, we'll go over the care and feeding of indexes in RavenDB: how you can create, deploy, monitor and manage them yourself. We'll talk about how the RavenDB query optimizer interacts with your indexes and how to move them between environments.

1.6.4. Part VI - AI & Large Language Models Integration

Focus: Developers and architects

This part discusses RavenDB's integration with AI and Large Language Models, starting with how vector search allows for advanced semantic queries. We'll explore embedding generation, which creates vectors from your data, enabling this type of search. Then we'll cover how you can apply Generative AI directly from within the database to enhance, summarize, and make decisions on your data.

You'll also learn how to build AI agents that leverage RavenDB's capabilities to become intelligent, production-ready systems with a very short development time. We'll dive into features like conversation memory and summarization, and see how you can expose RavenDB data and queries directly to the model. You'll also learn how to let the model securely invoke actions within your code.

The AI integration in RavenDB is designed to let you use the advanced features of AI models while protecting you from the risks of giving a potentially untrustworthy model full control. We'll discuss RavenDB's security approach, which allows you to manage the AI models and their access to your systems safely.

1.6.5. Part V — Security

Focus: Operations and architects

RavenDB is used to store business-critical data such as medical information and financial transactions. In this part, we'll go over all the steps that have been taken to ensure that your data is safe, the communication channels are secure and only authorized users are able to access your database.

We'll cover how to set up RavenDB securely. RavenDB's security model is binary in nature. Either you run RavenDB in an unsecured mode (only useful for development) or you run it in a secured mode. There are no half measures or multiple steps to take.

Setting up RavenDB securely is easy — although making it easy was certainly not easy — and once set up, RavenDB takes care of all aspects of securing your data. Data in transit is encrypted, and clients and servers mutually authenticate themselves. We'll discuss how RavenDB handles authentication and authorization, as well as how you can control who gets to the database and what they can access.

We'll also cover securing your data at rest. RavenDB supports full database encryption, ensuring that not even a single byte of your data is ever saved to disk in plain text. Instead, RavenDB will encrypt all data and indexes using 256-bit encryption. Your data will be decrypted on the fly as needed and only kept in memory for the duration of an active transaction.

We'll also cover other aspects of running an encrypted database: how you should manage encryption keys and how to back up and restore encrypted databases.

1.6.6. Part VI — Running in production

Focus: Operations

This part deals with running and supporting a RavenDB cluster or clusters in production, from spinning a new cluster, to decommissioning a downed node, to tracking down performance problems. We'll learn all you need (plus a bit more) in order to understand how RavenDB works and how to customize its behavior to fit your own environment.

We'll also explore using RavenDB Cloud, which manages all those details for you, so you won't need to deal with them.

We'll go over the planning ahead for the resources you'll need to handle expected load (and how to handle *unexpected* load) and the kind of deployment topologies you can choose from and their implications. We'll also go over the network, firewall and operating system configurations that can make or break a production environment.

Just getting to production isn't good enough; we are also going to cover how you can *stay* in

production and stay there healthily. We'll discuss monitoring and troubleshooting, what sort of details you need to keep an eye on and how RavenDB surfaces potential issues early on. We'll discover how you can dig into RavenDB and see exactly what is going on inside the engine. We'll go over the kinds of self-optimizations RavenDB routinely applies and how you can take advantage of them.

We'll cover common issues and how to troubleshoot them, diagnosing and resolving problems while keeping the cluster up and functioning. We'll learn how to plan for disaster recovery and actually apply the plan if (and when) disaster strikes.

We'll spend a whole chapter discussing backups and restores. RavenDB supports several options for backing up your data: offsite hot spares, full binary snapshots of your data and highly compressed backups meant for long term storage. We'll discuss backup strategies and options, including backing up directly to the cloud. More importantly, we'll cover how you can define and execute a restore strategy, a critical — though often (sadly) overlooked — part of your overall backup strategy.

Finally, we are going to close this book with a set of operational recipes. These are ready-made answers to specific scenarios that you might run into in production. These are meant to serve both as a series of steps for you to follow if you run into a particular scenario and as a way to give you better insight into the process of working with RavenDB in production.

1.7. Summary

So there's a lot going on in this book, and I hope you'll find it both interesting and instructive. But remember, the one thing it *isn't* meant to do is replace the documentation. The purpose of this book is to give you a full background on and greater understanding of how RavenDB works. I'm not covering the nitty-gritty details of every API call and what parameters should be passed to it.

In many cases, I have elected to discuss a feature, give one or two examples of its use and where it's best utilized and leave the reader with the task of reading up on the full details in the documentation.

This book is meant to be more than API listing. It is meant to tell a story, the story of how you can make the best use of RavenDB in your applications and environment. So, without further ado, turn the page and let's get started.

[1] You can find it at <http://ayende.com/blog>

[2] Grok means "to understand so thoroughly that the observer becomes a part of the observed — to merge, blend, intermarry, lose identity in group experience." From Robert A. Heinlein, *Stranger in a Strange Land*.

[3] Available at <https://ravendb.net/docs>

[4] Michael T. Nygard, *Release It! Design and Deploy Production-Ready Software*, Pragmatic Bookshelf, 2007, <https://pragprog.com/titles/mnee2/release-it-second-edition>

Chapter 2. Zero to RavenDB

The very first step we need to take in our journey to understand RavenDB is to get it running on our machine so we can actually get things done. I'm deferring discussion on what RavenDB is and how it works to a later part of this book because I think having a live version that you can play with will make it much easier to understand.

2.1. Setting RavenDB on your machine

For this section, I'm assuming that you're a developer trying to get a RavenDB instance so you can explore it. I'm going to ignore everything related to actual production deployments in favor of getting you set up in as few steps as possible. A full discussion on how to deploy, secure and run RavenDB in production is available in Chapter 16.



You can run RavenDB in the cloud as well, visit <https://cloud.ravendb.net> to get started. For now, we're going to focus on getting RavenDB running on your local machine and everything that entails. I'll cover cloud deployments in Chapter 10.

I'm going to go over a few quick install scenarios, and you can select whichever one makes the most sense for your setup. After that, you can skip to the next section, where we'll actually start using RavenDB.

2.1.1. Running on Docker

The easiest way to get RavenDB is probably via Docker. If you already have Docker installed, all you need to do is run the command in Listing 2.1.

Listing 2.1. Running RavenDB in Docker

```
docker run \
  -e RAVEN_Setup_Mode=None \ ①
  -e RAVEN_License_Eula_Accepted=true \ ②
  -p 8080:8080 \ ③
  ravendb/ravendb ④
```

- ① Disable setup wizard, run with default values
- ② Accept the EULA for RavenDB
- ③ Forward port 8080 from the container to port 8080 in the host machine
- ④ The docker image to run: ravendb/ravendb

Docker will now get the latest RavenDB version and spin up a new container to host it. Note that we are setting RavenDB up to skip the setup process, this is fine for now, since we want to get started as soon as possible. We'll go over RavenDB setup options in Chapter 16.

The output of the command in Listing 2.1 is shown in Listing 2.2.

Listing 2.2. RavenDB Server Output (Docker)

```

|-----|-----|
|  _  \   |  _  \   |  _  \   |
| ( _ ) |  _  _   |  _  _   |
|  _  // _ ' \ \ / / _ ' \   |
| | \ \ ( _ | \ V / _ / | | |  _ |  _ |
| _ | \ \ _ _ | \ / \ _ _ | | | _ _ _ / | _ _ _ /

```

You can now access your RavenDB instance using `http://localhost:8080`. If something is already holding port 8080 on your machine, you can map it to a different one using the `-p 8081:8080` option.

2.1.2. Running on Windows

To set up RavenDB on Windows, you'll need to go to <https://ravendb.net/download>, select the appropriate platform (Windows x64, in this case) and download the zip file containing the binaries.

Extract the file to a directory and then run the `run.ps1` script. This will run RavenDB in interactive mode, inside a console window, and will walk you through the process of setting up RavenDB. We can skip that process and run RavenDB directly, similar to how we run it using Docker, using the command in Listing 2.3.

Listing 2.3. Running RavenDB in Windows

```
.\Server\Raven.Server.exe
--Setup.Mode=None
--License.Eula.Accepted=true
--ServerUrl=http://127.0.0.1:8080
--browser
```

- ① Disable setup wizard, run with default values
- ② Accept the EULA for RavenDB
- ③ Bind to loopback (127.0.0.1) on port 8080

④ Open the browser to the RavenDB Studio on startup.

If something is already using port 8080, RavenDB will fail to start and give you an "address in use" error (more specifically, EADDRINUSE). You can customize the port and host from the command line by modifying the `ServerUrl` to: `--ServerUrl=http://localhost:8081`

That will run RavenDB on port 8081, avoiding the conflicting port issue. You might need to try another port as well if port 8081 is also taken on your machine, of course.

2.1.3. Running on Linux

To set up RavenDB on Linux, you'll need to go to <https://ravendb.net/download>, select the appropriate platform (Linux x64, most likely) and download the `tar.bz2` file containing the binaries. You can use `tar -xf RavenDB-8.0-linux-x64.tar.bz2` to decompress the archive.

Go to the RavenDB folder and then run `run.sh` script. This will run RavenDB in interactive mode, walking you through the setup wizard.

We can skip that process and run RavenDB directly, similar to how we run it using Docker, using the command in Listing 2.4.

Listing 2.4. Running RavenDB in Linux

```
./Server/Raven.Server.exe      \  
  --Setup.Mode=None           \ ①  
  --License.Eula.Accepted=true \ ②  
  --ServerUrl=http://127.0.0.1:8080 ③
```

① Disable setup wizard, run with default values

② Accept the EULA for RavenDB

③ Bind to loopback (127.0.0.1) on port 8080

If something is already using port 8080, RavenDB will fail to start and give you an "address in use" error (more specifically, EADDRINUSE). You can customize the port and host from the command line by modifying the `ServerUrl` to: `--ServerUrl=http://localhost:8081`

That will run RavenDB on port 8081, avoiding the conflicting port issue. You might need to try another port as well if port 8081 is also taken on your machine, of course.

You can now point your browser to `http://127.0.0.1:8080` to visit the RavenDB Studio.

2.1.4. Using the live demo instance

Without installing anything, you can point your browser to `http://live-test.ravendb.net` and access the public demo instance that we have available. This is useful for quick checks and verifications, but it isn't meant for anything more serious than that.

Obviously, all data in the live instance is public, and there are no guarantees about availability. We use this instance to try out the latest versions, so you should take that into consideration. In short, if you need to verify something small, go ahead and hit that instance. Otherwise, you'll need your

own version.

2.1.5. Using RavenDB Cloud

You can also run RavenDB in the cloud, using <https://cloud.ravendb.net/>. You can register a free account using: <https://cloud.ravendb.net/user/register> and then start a free private instance on the cloud. I'm not covering this yet since we have a full chapter to discuss that in Chapter 10. It's worth mentioning here since some people prefer to run their databases fully on the cloud, and RavenDB has great support for it.

For the rest of this chapter, we'll assume that you are running on a local instance or the public live-test one. If you are using a cloud instance, be sure to note the differences (running with secured setup, mostly). You can skip ahead to Chapter 10 to learn how to setup a cloud instance properly and then continue forward from here.

2.2. Your first database

At this point, you've already set up an instance of RavenDB to work with, and you've loaded the RavenDB Studio in your browser. For simplicity's sake, I'm going to assume from now on that you're running RavenDB on the local machine on port 8080. Point your browser to <http://127.0.0.1:8080>, and you should be greeted with an empty RavenDB instance. You can see how it looks in Figure 2.1.

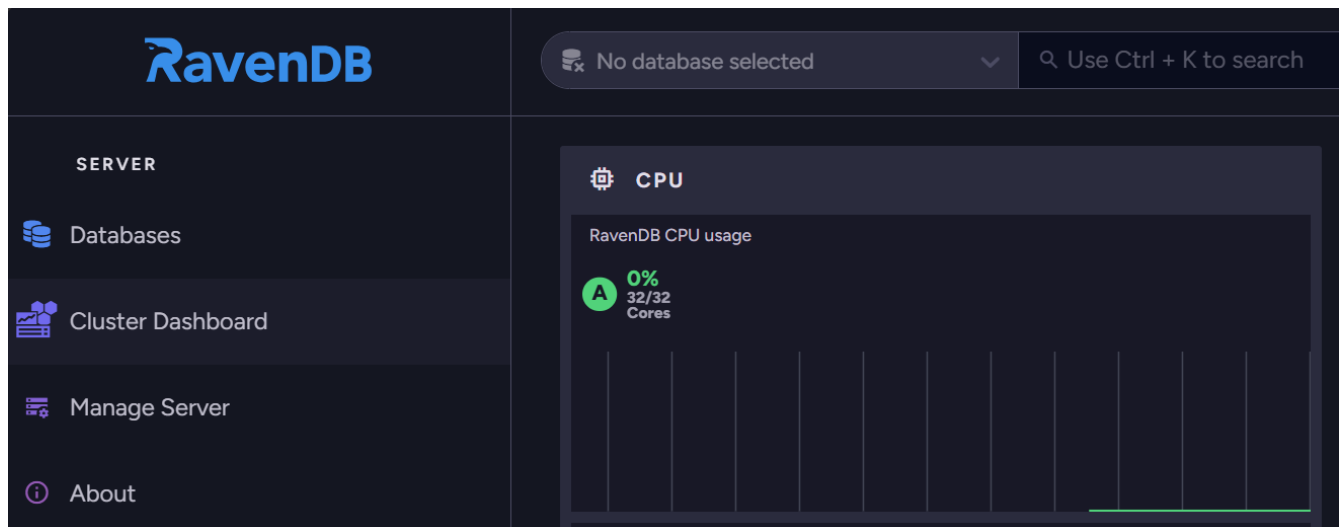


Figure 2.1. An empty RavenDB node

What we have right now is a RavenDB node that is a self-contained cluster.^[1] Now that we have a running node, the next step is to create a new database on this node.

You can do that by clicking Databases button on the left and then New Database button, naming the new database Northwind and selecting Quick Create to accept all the defaults. We'll discuss what all of those mean later in this book. You can see the create database dialog in Figure 2.2.

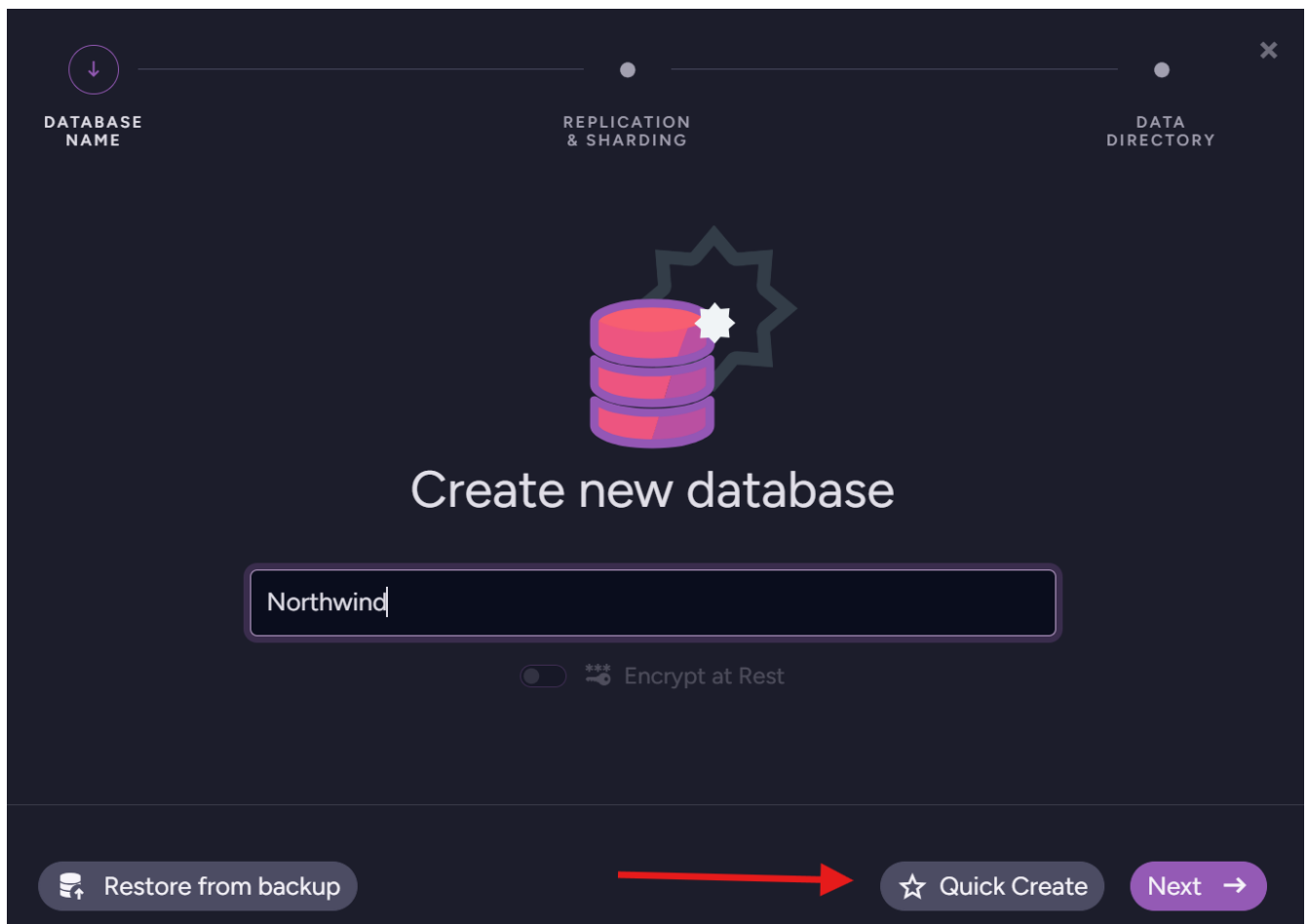


Figure 2.2. Create database dialog

Click the Quick Create button, and that's pretty much it. Your new database is ready. Click on the Databases button on the left to see what this looks, as shown in Figure 2.3.

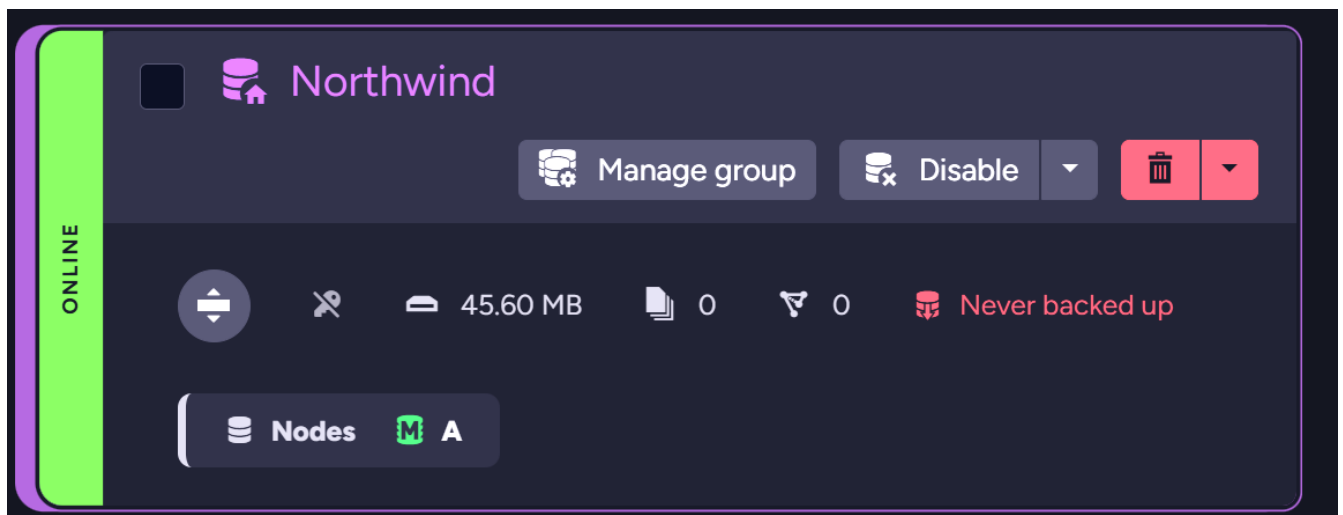


Figure 2.3. An empty new database

2.2.1. Creating sample data

Of course, this new database contains no data, which makes it pretty hard to work with. We'll use the sample data feature in RavenDB to have some documents to experiment with.

Click on the database name, then select the Tasks in the left menu and then Create Sample Data. The

page that this opens has a Create sample data button, click on the button.

This action will populate the database with the sample Northwind dataset. For those not familiar with Northwind, it's a sample dataset of an online store, and it includes common concepts such as orders, customers and products. Let's explore this dataset inside of RavenDB.

On the left menu, select Documents, and you'll see a view similar to what's pictured in Figure 2.4, showing the recently created documents and collections.

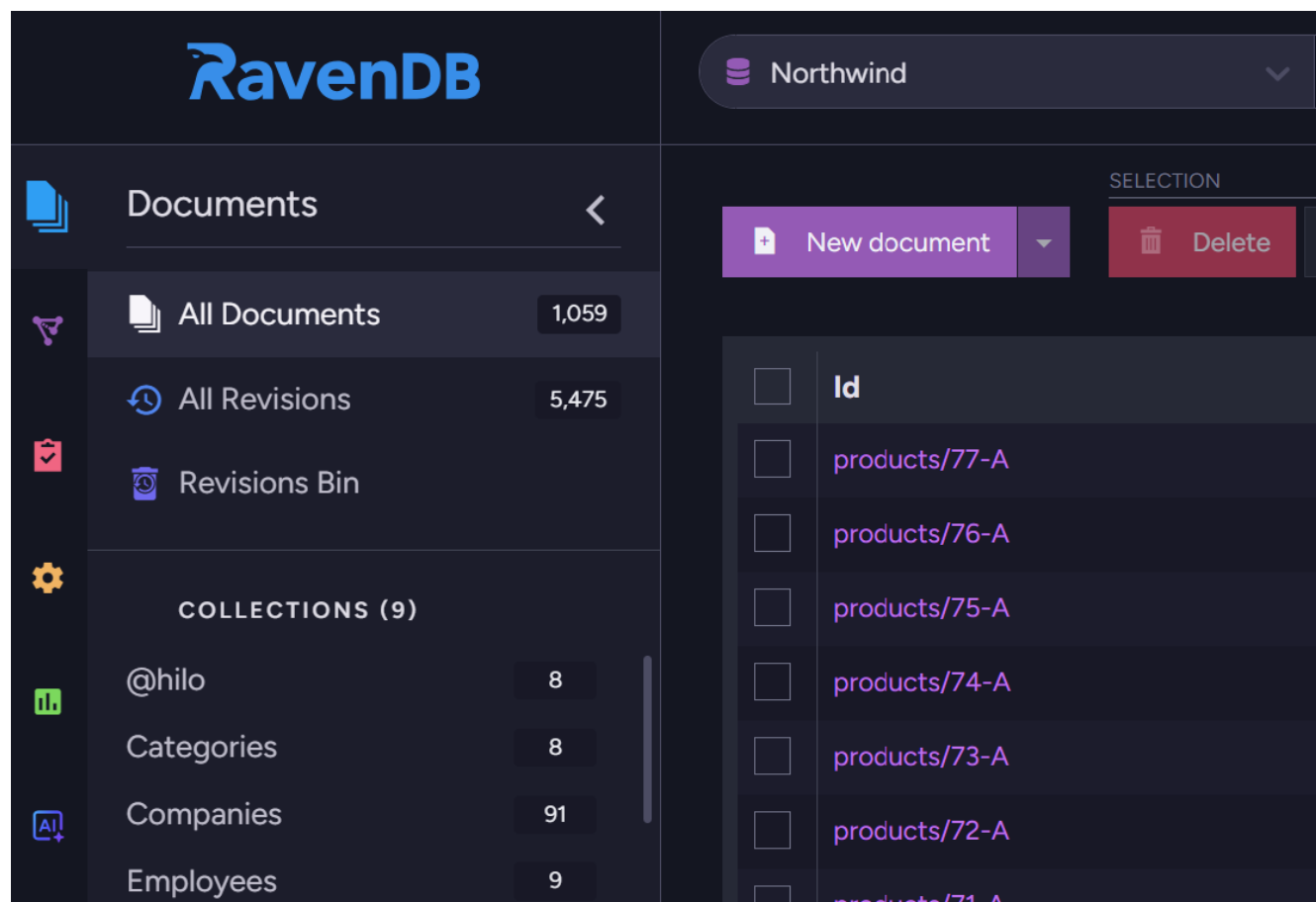


Figure 2.4. Northwind documents view

Collections are the basic building blocks inside RavenDB. Every document belongs to exactly one collection, and the collection typically holds similar documents (though it doesn't have to). These documents are most often based on the entity type of the document in your code. It's very similar to tables in a relational database, but unlike tables, there's no requirement that documents within the same collection will share the same structure or have any sort of schema.

Collections are very important to the way data is organized and optimized internally within RavenDB. We'll frequently use collections to group similar documents together and apply an operation to them (subscribing to changes, indexing, querying, ETL, applying AI operations, etc.).

2.2.2. Our first real document

Click on the Orders collection and then on the first document in the listing, which should be orders/830-A. The result is shown in Figure 2.5. For the first time, we're looking at a real JSON document inside of RavenDB.

orders/830-A



Save



Clone



Delete



Compare

```
1 {
2   "Company": "companies/65-A",
3   "Employee": "employees/1-A",
4   "Freight": 8.53,
5   "ShipTo": {
6     "City": "Albuquerque",
7     "Country": "USA",
8     "Line1": "2817 Milton Dr.",
9     "Region": "NM"
10  },
11  "Lines": [
12    {
13      "Discount": 0.2,
14      "PricePerUnit": 19,
15      "Product": "products/2-A",
16      "ProductName": "Chang",
17      "Quantity": 24
18    },
```

Figure 2.5. An Orders document

If you're used to working with non-relational databases, this is pretty obvious and not too exciting. But if you're mostly used to relational databases, there are several things to note here.

In RavenDB, we're able to store arbitrarily complex data as a single unit. If you look closely at

Figure 2.5, you'll see that instead of just storing a few columns, we can store rich information and work with nested objects (the `ShipTo` property) or arrays of complex types (the `Lines` property).

This means that we don't have to split our data to satisfy the physical constraints of our storage. A whole object graph can be stored in a single document. Modeling will be further discussed in Chapter 4, but for now I'll just mention that the basic modeling method in RavenDB is based around root aggregates which you may be familiar with from Domain Driven Design.^[2]

In the meantime, you can explore the different collections and the sample data in the Studio. We spent a lot of time and effort on the RavenDB Studio. Though it's pretty, I'll be the first to admit that looking at a syntax highlighted text editor isn't really *that* impressive. So let's see what kind of things we can do with the data as a database.

2.3. Working with the RavenDB Studio

This section will cover the basics of working with data within RavenDB Studio. If you're a developer, you're probably anxious to start seeing code. We'll get into that in the next section — no worries.

2.3.1. Creating and editing documents

When you look at a particular document, you can edit the JSON and click Save, and the document will be saved. There isn't really much to it, to be honest. Creating new documents is a bit more interesting. Let's create a new Category document.

Go to Documents in the left menu, click Categories under Collections and select New document in current collection, as shown in Figure 2.6.

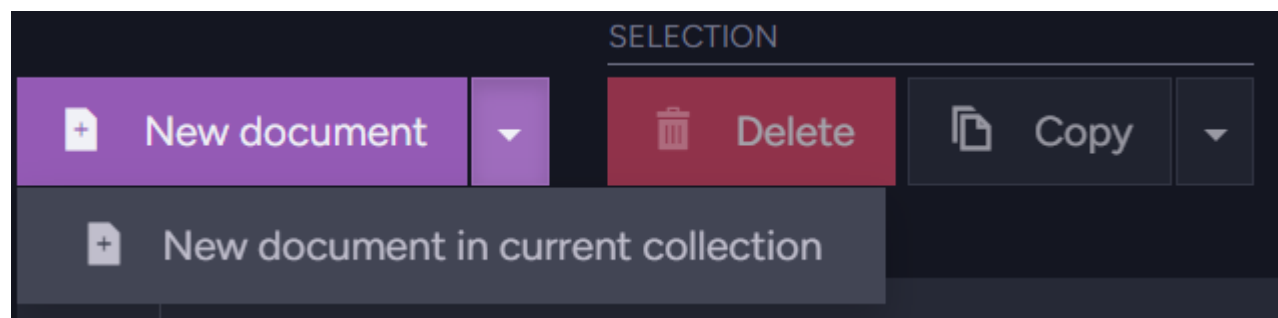


Figure 2.6. New document in current collection

This will open the editor with an empty, new document that's based on one of the existing categories. Note that the document ID is set to `Categories/`. Fill in some values for the properties in the new document and save it. RavenDB will assign the document ID automatically for you.

One thing that may not be obvious is that while the Studio generates an empty document based on existing ones, you're free to add or remove properties and values and modify the document's structure however you like. RavenDB is typically run in schema-less mode, although it offers schema validation for enforcing data consistency. This flexibility in document structure makes it much easier to evolve your data model and handle more complex data.

2.3.2. Patching documents

The first thing we'll learn is how to do bulk operations inside the Studio. Go to Documents on the left menu and click the Patch menu item. You'll be presented with the screen shown in Figure 2.7.



Figure 2.7. Adding field 'Patched' to all documents in the Categories Collection

Patching allows you to write a query that executes a JavaScript transformation that can modify the matching documents. To try this out, let's run a non-trivial transformation on the Categories documents. Using a patch script, we'll add localization support — the ability to store the category name and description in multiple languages.

Start by adding the code in Listing 2.5 to the query text.

Listing 2.5. Patching the Categories collection to add localization support

```
from Categories
update {
  this.Name = [
    { "Lang": "en-us", "Text": this.Name }
  ];
  this.Description = [
    { "Lang": "en-us", "Text": this.Description }
  ];
}
```

Click the Test button, and you can see the results of running this operation: a category document. You can also select which specific document this test will be tested on. The before-and-after results of running this script on categories/4-A are shown in Figure 2.8.

After

```
{
  "Name": [
    {
      "Lang": "en -us",
      "Text": "Dairy Products"
    }
  ],
  "Description": [
    {
      "Lang": "en -us",
      "Text": "Cheeses"
    }
  ],
  "@metadata": {
    "@collection": "Categories",
```

Before

```
{
  "Name": "Dairy Products",
  "Description": "Cheeses",
  "@metadata": {
    "@collection": "Categories",
```

Figure 2.8. Categories localization with JavaScript patching

Patch scripts allow us to modify our documents en masse, and they are very useful when you need to reshape existing data. They can be applied on a specific document, a whole collection or all documents matching a specific query.

It's important to mention that for performance reasons, such bulk operations can be composed of multiple, independent and concurrent transactions instead of spanning a single large transaction. Each such independent transaction processes a set of documents with full ACID properties (while the patch operation as a whole does not).

2.3.3. Deleting documents

If you want to delete a particular document in the Studio, you can simply go to the document and hit the Delete button. You can delete a whole collection by going to the collection page (in the left menu, choose Documents and then select the relevant collection in the Collections menu), selecting all the documents in the header row and clicking Delete.

2.3.4. Querying documents

The previous sections talked about how to create, update and delete documents. But for full CRUD support, we still need to read documents. So far, we've looked at documents whose IDs were already known to us, and we've looked at entire collections. In this section, we'll focus on querying documents based on their data.

In the left menu, go to Indexes and then to Query. This is the main screen for querying documents in the RavenDB Studio. Enter the following query and then click the query button: `from Companies where Address.Country = 'UK'`

You can see the results of this query in Figure 2.9. We'll cover querying in much more detail in Chapter 12.

Results 

1

from Companies

2

where Address.Country = 'UK'

 Index 'Auto/Companies/ByAddress.Country' ▼

was used to get the results in 0 ms

Previe...	Id	Name	ExternalId
	companies/4-A	Around the Horn	AROUT
	companies/11-A	B's Beverages	BSBEV
	companies/16-A	Consolidated Holdings	CONSH

Figure 2.9. All companies in the UK query results

The overview in this section was not meant to be a thorough walk-through of all options in RavenDB Studio, but only show you some basic usage so that you can get familiar with the Studio and be able to see the results of the coding done in the next section within the Studio.

2.4. Your first RavenDB program

We're finally at the good parts, where we can start slinging code around. For simplicity's sake, I'm going to use a simple console application to explore the RavenDB API. Typically, RavenDB is used in web/backend applications, so we'll also explore some of the common patterns of organizing your RavenDB usage in your application later in this chapter.

Most of the code samples in this book use C#, but the documentation can guide you on how to achieve the same results with any supported client.

Create a new console application with RavenDB, as shown Listing 2.6.

Listing 2.6. Installing RavenDB Client NuGet package

```
dotnet new console --name Rvn.Ch02
dotnet add .\Rvn.Ch02\ package RavenDB.Client
```

This will setup the latest client version for RavenDB on the project. The next step is to add a namespace reference by adding `using Raven.Client.Documents;` to the top of the `Program.cs` file.

And now we're ready to start working with the client API. The first thing we need to do is to set up access to the RavenDB cluster that we're talking to. This is done by creating an instance of `DocumentStore` and configuring it as shown in Listing 2.7.

Listing 2.7. Creating a document store pointed to a local instance

```
using var store = new DocumentStore
{
    Urls = ["http://127.0.0.1:8080"],
    Database = "Tasks"
}.Initialize();
```

This code sets up a new `DocumentStore` instance and lets it know about a single node — the one running on the local machine — and that we are going to be using the `Tasks` database. The document store is the starting location for all communication with the RavenDB cluster. It holds the configuration, topology, cache and any customizations that you might have applied.

Typically, you'll have a single instance of a document store per application (singleton pattern) and use that same instance for the lifetime of the application. However, before we can continue, we need to go ahead and create the `Tasks` database in the Studio so we'll have a real database to work with. You can create a new database using the same process we went through for the `Northwind` database earlier in this chapter.

The document store is the starting location for all RavenDB work, but the real workhorse is the session. The session is what will hold our entities, talk with the server and, in general, act as the front man to the RavenDB cluster.

2.4.1. Defining entities and basic CRUD

Before we can actually start using the session, we need *something* to actually store. It's possible to work with completely dynamic data in RavenDB, but that's a specific scenario covered in the documentation. Most of the time, you're working with your entities. For the purpose of this chapter, we'll use the notion of tasks to build a simple list of things to do.

Listing 2.8 shows what a class that will be saved as a RavenDB document looks like.

Listing 2.8. Entity class representing a *ToDo* task

```
public class ToDoTask
{
    public string Id { get; set; }
    public string Task { get; set; }
    public bool Completed { get; set; }
    public DateTime DueDate { get; set; }
}
```

This is about as simple as you can get, but we're only starting, so that's good. Let's create a new task inside RavenDB, reminding us that we need to pick up a bottle of milk from the store tomorrow. The code to perform this task (pun intended) is shown in Listing 2.9.

Listing 2.9. Saving a new task to RavenDB

```
using (var session = store.OpenSession())
```

```

{
    var task = new ToDoTask
    {
        DueDate = DateTime.Today.AddDays(1),
        Task = "Buy milk"
    };
    session.Store(task);
    session.SaveChanges();
}

```

We opened a new session and created a new `ToDoTask`. We then stored the task in the session and called `SaveChanges` to save all the changes in the session to the server. You can see the results of this in Figure 2.10.

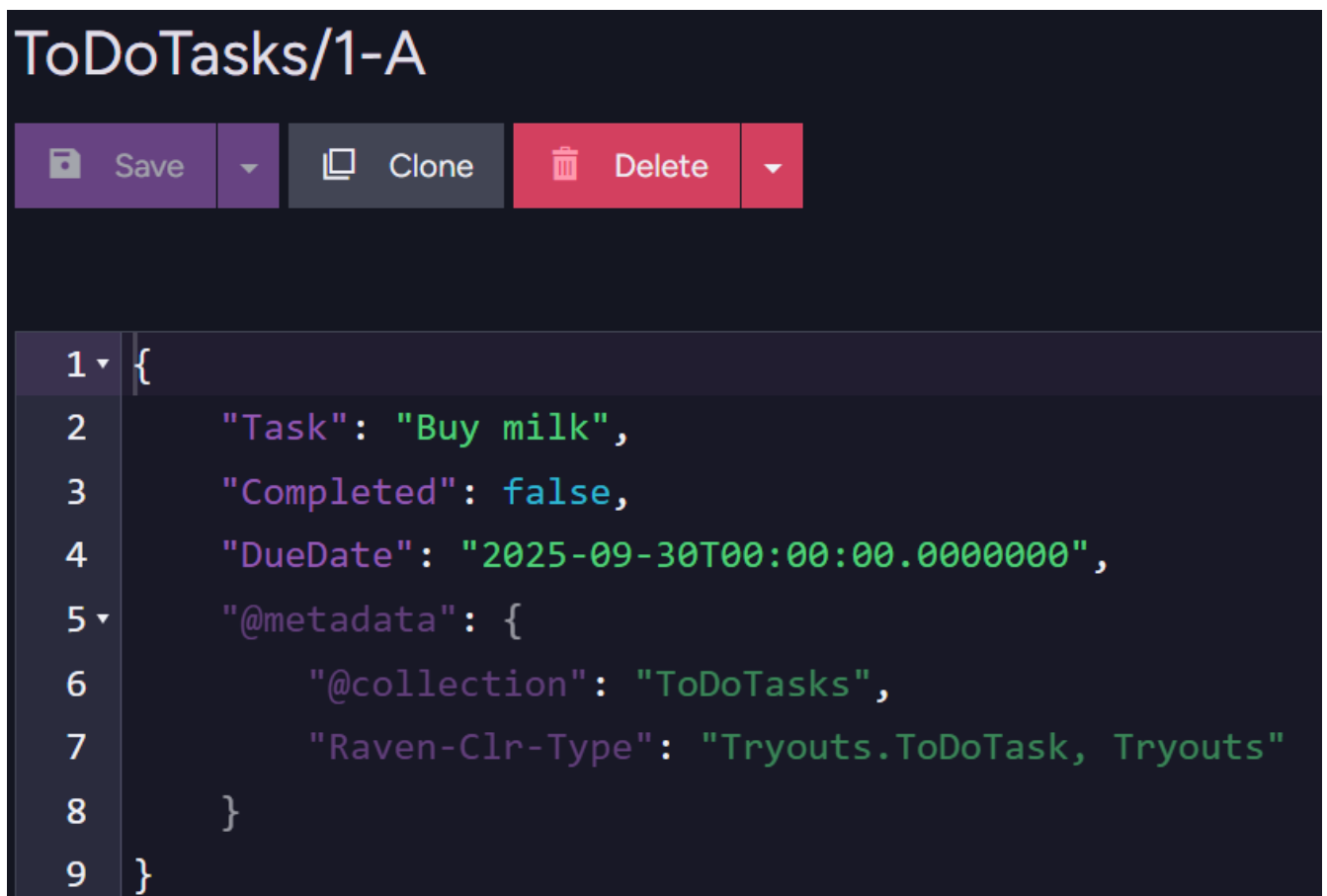


Figure 2.10. The newly created task document in the Studio

As it so happened, I was able to go to the store today and get some milk, so I need to mark this task as completed. Listing 2.10 shows the code required to handle updates in RavenDB.

Listing 2.10. Loading, modifying and saving a document

```

using (var session = store.OpenSession())
{
    ToDoTask task = session.Load<ToDoTask>("ToDoTasks/1-A");
    task.Completed = true;
    session.SaveChanges();
}

```

Several interesting things can be noticed even in this very small sample. We loaded the document and modified it, and then we called `SaveChanges`. We didn't need to call `Store` again. Because the task instance was loaded via the session, it was also tracked by the session, and any changes made to it would be sent back to the server when `SaveChanges` was called. Conversely, if the `Completed` property was already set to `true`, the RavenDB client would detect that and do nothing since the state of the server and the client match.



Asynchronous vs synchronous APIs

The RavenDB Client is natively asynchronous (it is using `async` to implement the synchronous API). In C#, you can choose which API to use by calling `OpenSession()` or `OpenAsyncSession()`. The `async` session has methods such as `LoadAsync`, `StoreAsync`, `SaveChangesAsync`, etc. We'll cover the `async` API later in this chapter.

The document session implements the Unit of Work ^[3] and Identity Map ^[4] design patterns. This makes it much easier to work with complex behaviors since you don't need to manually track changes to your objects and decide what needs to be saved and what doesn't. It also means that the only time the RavenDB client will send updates to the server is when you call `SaveChanges`. That, in turn, means you'll experience a reduced number of network calls. All of the changes will be sent as a single batch to the server. And because RavenDB is transactional, all those changes will happen as a single transaction, either completing fully or not at all.

Let's expand on that and create a few more tasks. You can see how this works in Listing 2.11.

Listing 2.11. Creating multiple documents in a single transaction

```
using (var session = store.OpenSession())
{
    for (int i = 0; i < 5; i++)
    {
        session.Store(new ToDoTask
        {
            DueDate = DateTime.Today.AddDays(i),
            Task = "Take the dog for a walk"
        });
    }

    session.SaveChanges();
}
```

Figure 2.11 shows the end result of all this playing around we've done. We're creating five new tasks and saving them in the same `SaveChanges` call, so they will be saved as a single transactional unit.

☐	@id	Task	Completed	DueDate
☐	ToDoTasks/6-A	Take the dog for a walk	false	2025-10-03T00:00:00.0000000
☐	ToDoTasks/5-A	Take the dog for a walk	false	2025-10-02T00:00:00.0000000
☐	ToDoTasks/4-A	Take the dog for a walk	false	2025-10-01T00:00:00.0000000
☐	ToDoTasks/3-A	Take the dog for a walk	false	2025-09-30T00:00:00.0000000
☐	ToDoTasks/2-A	Take the dog for a walk	false	2025-09-29T00:00:00.0000000
☐	ToDoTasks/1-A	Buy milk	true	2025-09-30T00:00:00.0000000

Figure 2.11. All the current task documents

2.4.2. Querying RavenDB

Now that we have all these tasks, we want to start querying the data. Before we get to querying these tasks from code, I want to show you how to query the data from the Studio. Go to Indexes and then Query in the Studio and you'll see the query page. Let us find all the tasks we still have to do, we can do that using the following query: `from ToDoTasks where Completed = false`. You can see the results of this in Figure 2.12.

Query

Syntax ?

```
1 from ToDoTasks where Completed = false
```

Results

 Index 'Auto/ToDoTasks/ByCompleted' was used to get the results in <1 ms 

Preview	@id	Task	Completed
	ToDoTasks/2-A	Take the dog for a walk	false
	ToDoTasks/3-A	Take the dog for a walk	false
	ToDoTasks/4-A	Take the dog for a walk	false
	ToDoTasks/5-A	Take the dog for a walk	false
	ToDoTasks/6-A	Take the dog for a walk	false

Figure 2.12. Querying for incomplete tasks in the Studio

We'll learn all about querying RavenDB in Chapter 12. For now, let's concentrate on getting results, which means looking at how we can query RavenDB from code. Let's say I want to know what kind of tasks I have for the next couple of days. In order to get that information, I can use the query in Listing 2.12. (Remember to add `using System.Linq;` to the top of the `Program.cs` file.)

Listing 2.12. Querying upcoming tasks using LINQ

```
using (var session = store.OpenSession())
{
    var tasksToDo =
        from t in session.Query<ToDoTask>()
        where t.DueDate >= DateTime.Today &&
            t.DueDate <= DateTime.Today.AddDays(2) &&
            t.Completed == false
        orderby t.DueDate
        select t;
    Console.WriteLine(tasksToDo.ToString());
}
```

```

foreach (var task in tasksToDo)
{
    Console.WriteLine($"{task.Id} - {task.Task} - {task.DueDate}");
}
}

```

Running the code in Listing 2.12 will result in the output shown in Listing 2.13.

Listing 2.13. Querying upcoming tasks output

```

from ToDoTasks where DueDate between $p0 and $p1
    and Completed = $p2 order by DueDate

ToDoTasks/2-A - Take the dog for a walk - 5/14/2017 12:00:00 AM
ToDoTasks/3-A - Take the dog for a walk - 5/15/2017 12:00:00 AM
ToDoTasks/4-A - Take the dog for a walk - 5/16/2017 12:00:00 AM

```

The query in Listing 2.12 shows us using LINQ to perform queries against RavenDB with very little hassle and no ceremony whatsoever. There is actually a *lot* going on behind the scenes, but we'll leave all of that to Chapter 12. You can also see that we can call `.ToString()` on the query to get the query text from the RavenDB client API.

Let's look at an aggregation query. The code in Listing 2.14 gives us the results of all the tasks per day.

Listing 2.14. Aggregation query on tasks

```

using (var session = store.OpenSession())
{
    var tasksPerDay =
        from t in session.Query<ToDoTask>()
        group t by t.DueDate into g
        select new
        {
            DueDate = g.Key,
            TasksPerDate = g.Count()
        };

    // from ToDoTasks
    // group by DueDate
    // select key() as DueDate, count() as TasksPerDate
    Console.WriteLine(tasksPerDay.ToString());

    foreach (var tpd in tasksPerDay)
    {
        Console.WriteLine($"{tpd.DueDate} - {tpd.TasksPerDate}");
    }
}

```

If you're familiar with LINQ, there isn't much to say about the code in Listing 2.14. It works, and it's obvious and easy to understand. If you *aren't* familiar with LINQ and working with the .NET platform, I strongly recommend learning it. From the consumer side, Linq is quite beautiful. Now, if you were to implement querying using LINQ, it's utterly atrocious — take it from someone who's done it a few times. But lucky for you, that isn't *your* problem. It's ours.

So far, we've explored the RavenDB API a bit, saved documents, edited a task and queried the tasks in various ways. This was intended to familiarize you with the API and how to work with RavenDB. The client API was designed to be very simple, focusing on the common CRUD scenarios. Deleting a document is as easy as calling `session.Delete`, and all the complex options that you would need are packed inside the `session.Advanced` property.

Now that you have a basic understanding of how to write a Hello World in RavenDB, we're ready to dig deeper and see the client API in all its glory.

2.5. The client API

We've already used a document store to talk with a RavenDB server. At the time, did you wonder what its purpose is? The document store is the main entry point for the whole client API. It holds the server URLs, for one. (So far we used only a single server but in many cases, our data can span across multiple nodes.) It also holds the default database we will want to operate on, as well as the X509 client certificate that will be used to authenticate ourselves to the server. Its importance goes beyond connection management, so let's take a closer look at it.

2.5.1. The document store

The document store holds all the client-side configuration, including serialization configuration, failover behavior, caching options and much more. In a typical application, you'll have a single document-store instance per application (singleton). Because of that, the document store is thread safe, with an initialization pattern that typically looks like the code in Listing 2.15.

Listing 2.15. Common pattern for initialization of the DocumentStore

```
public class DocumentStoreHolder
{
    private readonly static Lazy<IDocumentStore> _store =
        new Lazy<IDocumentStore>(CreateDocumentStore);

    private static IDocumentStore CreateDocumentStore()
    {
        var documentStore = new DocumentStore
        {
            Urls = // urls of the nodes in the RavenDB Cluster
            [
                "https://ravendb-01:8080",
                "https://ravendb-02:8080",
                "https://ravendb-03:8080",
            ],
            Certificate = new X509Certificate2("tasks.pfx"),
        };
    }
}
```



```

        Database = "Tasks",
    };

    documentStore.Initialize();
    return documentStore;
}

public static IDocumentStore Store
{
    get { return _store.Value; }
}
}

```

The use of lazy ensures that the document store is only created once, without you having to worry about double locking or explicit thread safety issues. And you can configure the document store as you see fit. The rest of the code can access the document store using `DocumentStoreHolder.Store`. That should be relatively rare since, apart from configuring the document store, the majority of the work is done using sessions.

Listing 2.15 shows how to configure multiple nodes, set up security and select the appropriate database. We'll learn about how to work with a RavenDB cluster in Chapter 9. We still have a lot to cover on the document store without getting to clusters, though.

Conventions

The client API, just like the rest of RavenDB, aims to *just work*. To that end, it's based on the notion of conventions: a series of policy decisions that have already been made for you. Those decisions range from which property holds the document ID to how the entity should be serialized to a document.

For the most part, we expect that you'll not have to touch the conventions. A lot of thought and effort has gone into ensuring you'll have little need to do that. But there's simply no way that we can foresee the future or anticipate every need. That's why most of the client API parts are customizable.

Customizations can be applied by changing various settings and behaviors via the `DocumentStore.Conventions` property. For example, by default, the client API will use a property named `Id` (case sensitive) to store the document ID. But there are users who want to use the entity name as part of the property name. So they'll have `OrderId` for orders, `ProductId` for products, etc.

[5]

The code in Listing 2.16 is how you can tell the client API to apply the `TypeName + Id` policy to find the ID property for your entities.

Listing 2.16. Customizing how the RavenDB client selects which property is used as the ID

```

documentStore.Conventions.FindIdentityProperty =
    prop => prop.Name == prop.DeclaringType.Name + "Id";

```

Don't worry. We won't go over all of the available options, since there are quite a few of them. Please refer to the online documentation to get the full list of available conventions and their effects. It might be worth your time to go over and quickly study them just to know what's available to you, even if they aren't something that you'll touch all that often (or ever).

Beside the conventions, there are certain settings available directly from the document store level that you should be aware of, like default request timeouts, caching configuration and event handlers. We'll cover all of those later on. But for now, let's focus on authentication.

Authentication

A database holds a lot of information. Usually, it's pretty important that you have control over who can access that information and what they can do with it. RavenDB fully supports this notion.

In development mode, you'll most commonly work in an unsecured mode, which implies that any connection will be automatically granted cluster administrator privileges. This reduces the number of things that you have to do upfront. But as easy as that is for development, for production, you'll want to run in a secure fashion. After doing so, all access to the server is restricted to authenticated users only.

Unsecured network-accessible databases are bad for you



By default, RavenDB will **refuse** to listen to anything but localhost in an unsecured mode. This is done for security reasons, to prevent admins from accidentally exposing RavenDB without authentication over the network. If you attempt to configure a non-localhost URL with authentication disabled, RavenDB will answer all requests with an error page explaining the situation and giving instructions on how to fix the issue.

You can let RavenDB know this is something you actually want, if you're running on a secure and isolated network. It requires an additional and explicit step to make sure this is your conscious choice and not an admin oversight.

RavenDB uses X509 client certificates for authentication. The good thing about certificates is that they're *not* users. They're not tied to a specific person or need to be managed as such. Instead, they represent specific access that was granted to the database for a particular reason. I find that this is a much more natural way to handle authentication, and typically X509 client certificates are granted on a per application / role basis.

A much deeper discussion of authentication, managing certificates and security in general can be found in the Chapter 17.

2.5.2. The document session

The session (also called "document session", but we usually shorten it to just "session") is the primary way your code interacts with RavenDB. If you're familiar with Hibernate (Java), Entity Framework (.NET) or Active Record (Ruby), you should feel right at home. The RavenDB session was explicitly modeled to make it easy to work with.



Defining "document" and "entity"

We tend to use the term "document" to refer both to the actual documents on the server and to manipulating them on the client side. It's common to say, "load that document and then..." But occasionally, we need to be more precise.

We make a distinction between a document and an entity (or aggregate root). A document is the server-side representation, while an entity is the client-side equivalent. An entity is the deserialized document that you work with in the client-side and save back to the database to become an updated server-side document.

We've already gone over the basics previously in this chapter, so you should be familiar with basic CRUD operations using the session. Let's look at the session with a bit more scrutiny. One of the main design forces behind RavenDB was the idea that it should *just work*. And the client API reflects that principle. If you look at the surface API for the session, here are the following high level options:

- `Load()` - Load a document or a set of documents into the session.
- `Include()` - Include related documents in the load/query results.
- `Delete()` - Register a deletion of a document from the session.
- `Query()` - Start a new query.
- `Store()` - Register a new document in the session to be saved to the server.
- `SaveChanges()` - Save all the changes in the session to the server.
- `Advanced` - Access to more advanced options.

Those are the most common operations that you'll run into on a day-to-day basis. And more options are available under the `Advanced` property.

Disposing the session



The .NET implementation of the client API holds resources that *must* be freed. Whenever you make use of the session, be sure to wrap the variable in a `using` statement or else do something to ensure proper disposal. Not doing so can force the RavenDB client to clean up using the finalizer thread, which can in turn increase the time it takes to release the acquired resources.

Load

As the name implies, this gives you the option of loading a document or a set of documents into the session. A document loaded into the session is managed by the session. Any changes made to the document would be persisted to the database when you call `SaveChanges`. A document can only be loaded once in a session. Let's look at the code in Listing 2.17.

Listing 2.17. Loading the same document twice

```
var t1 = session.Load<ToDoTask>("ToDoTasks/1-A");
var t2 = session.Load<ToDoTask>("ToDoTasks/1-A");

Assert.True(Object.ReferenceEquals(t1, t2));
```

Even though we called `Load<ToDoTask>("ToDoTasks/1-A")` twice, there's only a single remote call to the server and only a single instance of the `ToDoTask` class. Whenever you load a document, it's added to an internal dictionary that the session manages, and the session checks the dictionary to see if the document is already there. If so, it will return the existing instance immediately. This helps avoid aliasing issues and also generally helps performance.

For those of you who deal with patterns, the session implements the Unit of Work and Identity Map patterns. This is most obvious when talking about the Load operation, but it also applies to Query and Delete.

Load can also be used to read more than a single document at a time. For example, the code in Listing 2.18 loads three documents at once.

Listing 2.18. Loading multiple documents in a single call

```
Dictionary<string, ToDoTask> tasks = session.Load<ToDoTask>(
    "ToDoTasks/1-A",
    "ToDoTasks/2-A",
    "ToDoTasks/3-A"
);
```

This will result in a dictionary with all three documents in it, retrieved in a single remote call from the server. If a document we tried to load wasn't found on the server, the dictionary will contain null for that document ID.

Budgeting remote calls

Probably the easiest way to kill your application performance is to make a lot of remote calls. And a likely culprit is the database. It's common to see a web application making dozens of calls to the database to service a single request, usually for no good reason. In RavenDB, we've done several things to mitigate that problem. The most important among them is to allocate a budget for every session. Typically, a session would encompass a single operation in your system. The scope of the session is typically limited to handling a single request or a the processing of one message.

A session is limited by default to a maximum of 30 calls to the server. If you try to make more than 30 calls to the server, an exception is thrown. This serves as an early warning that your code is generating too much load on the system and as a circuit breaker.^[6]

You can increase the budget, of course, but just having that warning in place ensures that you'll think about the number of remote calls you're making.

The limited number of calls allowed per session also means that RavenDB has a lot of options to *reduce* the number of calls. When you call `SaveChanges()`, you don't need to make a separate call per changed entity; you can go to the database once. In the same manner, we also allow you to batch *read* calls. We'll discuss the Lazy feature in more depth in [ravnedb-client-api].

The client API is pretty smart about it. If you try to load a document that was already loaded

(directly or via Include), the session can serve it directly from the session cache. And if the document doesn't exist, the session will also remember that it couldn't load that document and will immediately return null rather than attempt to load the document again.

Working with multiple documents

We've seen how to work with a single document, and we even saved a batch of several documents into RavenDB in a single transaction. But we haven't actually worked with anything more complex than a `ToDoTask`. That's pretty limiting, in terms of the amount of complexity we can express. Listing 2.19 lets us add the notion of people who can be assigned tasks to the model.

Listing 2.19. People and Tasks model in RavenDB

```
public class Person
{
    public string Id { get; set; }
    public string Name { get; set; }
}

public class ToDoTask
{
    public string Id { get; set; }
    public string Task { get; set; }
    public bool Completed { get; set; }
    public DateTime DueDate { get; set; }

    public string AssignedTo { get; set; }
    public string CreatedBy { get; set; }
}
```

From looking at the model in Listing 2.19, we can learn a few interesting tidbits. First, we can see that each class stands on its own. We don't have a `Person` property on `ToDoTask` or a `Tasks` collection on `Person`. We'll learn about modeling more extensively in Chapter 4, but the gist of modeling in RavenDB is that each document is independent, isolated and coherent.

What does this mean? It means we should be able to take a single document and work with it successfully without having to look at or load additional documents. The easiest way to conceptualize this is to think about physical documents. With a physical document, I'm able to pick it up and read it, and it should make sense. References to other locations may be frequent, but there will usually be enough information in the document itself that I don't *have* to go and read those references.

In the case of the `ToDoTask`, I can look at my tasks, create new tasks or mark them as completed without having to look at the `Person` document. This is quite a shift from working with relational databases, where traversing between rows and tables is very common and frequently required.



Modeling data in the age of Large Languages Models

The reason we model documents to be independent, isolated, and coherent is that a single document holds all the information it needs to be understood. It turns out

that this approach is incredibly beneficial for AI models. Because the documents are self-contained, an AI can process them as they are, without needing to pull data from other sources to make sense of them. This simplifies the data for the model and makes it easier for the AI to work with.

Let's see how we can create a new task and assign it to a person. Listing 2.20 shows an interesting feature of RavenDB. Take a look and see if you can find the oddity.

Listing 2.20. Creating a new person document

```
using (var session = store.OpenSession())
{
    var person = new Person
    {
        Name = "Oscar Arava"
    };
    session.Store(person);           ①
    Console.WriteLine(person.Id);    ②
    session.SaveChanges();           ③
}
```

- ① Storing the Person entity in the session (and generating an ID for it)
- ② Printing the (newly generated) ID of the Person entity - *before* we save that to the database
- ③ Actually going to the database server and saving the new entity

RavenDB is transactional, and we only send the request to the server on `SaveChanges`. So how could we print the `person.Id` property before we called `SaveChanges`? Later in this chapter, we'll cover document identifiers and how they're generated, but the basic idea is that the moment we returned from `Store`, the RavenDB client ensured that we had a valid ID to use with this document. As you can see with Listing 2.21, this can be quite important when you're creating two documents at the same time, with references between them.

Listing 2.21. Creating a new person and assigning him a task at the same time

```
using (var session = store.OpenSession())
{
    var person = new Person
    {
        Name = "Oscar Arava"
    };
    session.Store(person);

    var task = new ToDoTask
    {
        DueDate = DateTime.Today.AddDays(1),
        Task = "Buy milk",
        AssignedTo = person.Id,
        CreatedBy = person.Id
    };
}
```

```
session.Store(task);
session.SaveChanges();
}
```

Now that we know how to write multiple documents and create associations between documents, let's see how we read them back. There's a catch, though. We want to do it efficiently.

Includes

RavenDB doesn't actually *have* references in the usual sense. There's no such thing as foreign keys, like you might be used to. A reference to another document is just a string property that happens to contain the ID of another document. What does this mean for working with the data? Let's say that we want to print the details of a particular task, including the name of the person assigned to it. Listing 2.22 shows the obvious way to do this.

Listing 2.22. Displaying the details of a task (and its assigned person)

```
using (var session = store.OpenSession())
{
    string taskId = Console.ReadLine();

    ToDoTask task = session.Load<ToDoTask>(taskId);
    Person assignedTo = session.Load<Person>(task.AssignedTo);

    Console.WriteLine(
        $"{task.Id} - {task.Task} by {assignedTo.Name}");

    // will print 2
    Console.WriteLine(session.Advanced.NumberOfRequests); ①
}
```

① The code prints how many requests we made to the server (one to load the task, and one to load the assigned person)

This code works, but it's inefficient. We're making *two* calls to the server here, one to fetch the task and another to fetch the assigned user. This is part of the budgeting and awareness program RavenDB has, aimed at reducing the number of remote calls and speeding up your applications.

Error handling

Listing 2.22 really bugged me when I wrote it, mostly because there's a lot of error handling that isn't being done: the task ID being empty, the task document not existing, the task not being assigned to anyone...you get the drift. I just wanted to mention that most code samples in this book will contain as little error handling as possible so as not to distract from the code that actually *does* things.

Having to go to the database twice is a pity because the server already knows the value of the `AssignedTo` property, and it could send the document that matches the value of that property at the

same time it's sending us the task. RavenDB's Includes functionality, which handles this in one step, is a favorite feature of mine because I still remember how excited I was when we finally figured out how to do this in a clean fashion. Look at Listing 2.23 to see how it works, and compare it to Listing 2.22.

Listing 2.23. Task and assigned person - single roundtrip

```
using (var session = store.OpenSession())
{
    string taskId = Console.ReadLine();

    ToDoTask task = session
        .Include<ToDoTask>(x => x.AssignedTo)           ①
        .Load(taskId);

    Person assignedTo = session.Load<Person>(task.AssignedTo); ②

    Console.WriteLine(
        $"{task.Id} - {task.Task} by {assignedTo.Name}");

    // will print 1
    Console.WriteLine(session.Advanced.NumberOfRequests);    ③
}
```

① Including the document referred to by the AssignedTo property

② Loading the related document (no remote call required)

③ Printing the number of remote calls (only one, since we included the referenced value)

The only difference between the two code listings is that in Listing 2.23 we're calling to Include before the Load. The Include method gives instructions to RavenDB: when it loads the document, it should look at the AssignedTo property. If there's a document with the document ID that's stored in the AssignedTo property, it should send it to the client immediately.

However, we didn't change the type of the task variable. It remains a ToDoTask. So what exactly did this Include method do here? What happened is that the session got a reply from the server, saw that there are included documents, and put them in its Identity Map. When we request the Person instance that was assigned to this task, we already have that information in the session and can avoid going back to the server to fetch the same document we already have.

The API is almost the same — and except for that call, everything else remains the same — but we managed to significantly cut the number of remote calls we make. You can Include multiple properties to load several referenced documents (or even a collection of them) efficiently. This is similar to a JOIN in a relational database, but it's much more efficient since you don't have to deal with Cartesian products and it doesn't modify the shape of the results.

Includes aren't joins

It's tempting to think about includes in RavenDB as similar to a join in a relational database. And there are similarities, but there are also fundamental differences. A join will modify the

shape of the output. It combines each matching row from one side with each matching row on the other, sometimes creating Cartesian products that can cause panic attacks for your DBAs.

And the more complex your model, the more joins you'll have, the wider your result sets become and the slower your application will become. In RavenDB, there's very little cost to adding includes. That's because they operate on a different channel than the results of the operation and don't change the shape of the returned data.

Includes are also important in queries. There, they operate *after* paging has applied, instead of before, like joins.

The end result is that includes don't modify the shape of the output, don't have a high cost when you use more than one of them and don't suffer from problems like Cartesian products.

Include cannot, however, be used to include documents that are referenced by included documents. In other words, Include is not recursive. This is quite intentional because allowing includes on included documents will lead to complex requests, both for the user to write and understand and for the server to execute. You *can* actually do recursive includes in RavenDB, but that feature is exposed differently (via the `declare` function mode, which we'll cover in Chapter 9).

Using multiple ``Include``s on the same operation, however, is just fine. Let's load a task, and with it we'll include both the assigned to person and the one who created the task. This can be done using the following snippet:

```
ToDoTask task = session.Include<ToDoTask>(x => x.AssignedTo)
                        .Include(x => x.CreatedBy)
                        .Load(taskId);
```

Now I can load both the `AssignedTo` person and the `CreatedBy` one, and there's still only a single round trip to the server. What about when both of them are pointing at the same document? RavenDB will return just a single copy of the document, even if it was included multiple times. On the session side of things, you'll get the same instance of the entity when you load it multiple times.

Beware of relational modeling inside of RavenDB

As powerful as the `Include` feature is, one of *the* most common issues we run into with RavenDB is people using it with a relational mindset — trying to use RavenDB as if it was a relational database and modeling their entities accordingly. `Include` can help push you that way because it lets you get associated documents easily.

We'll talk about modeling in a lot more depth in the next chapter, when you've learned enough about the kind of environment that RavenDB offers to make sense of the choices we'll make.

Delete

Deleting a document is done through the appropriately named `Delete` method. This method can

accept an entity instance or a document ID. The following are various ways to delete a document:

```
var task = session.Load<ToDoTask>("ToDoTasks/1-A");  
session.Delete(task); // delete by instance
```

And:

```
session.Delete("ToDoTasks/1-A"); // delete by ID
```

It's important to note that calling `Delete` doesn't actually delete the document. It merely marks that document as deleted in the session. It's only when `SaveChanges` is called that the document will be deleted.

Query

Querying is a large part of what RavenDB does. Not surprisingly, queries strongly relate to indexes, and we'll talk about those extensively in Part III. You've already seen some basic queries in this chapter, so you know how we can query to find documents that match a particular predicate, using LINQ.

Like documents loaded via the `Load` call, documents that were loaded via a `Query` are managed by the session. Modifying them and calling `SaveChanges` will result in their update on the server. A document that was returned via a query and was loaded into the session explicitly via `Load` will still have only a single instance in the session and will retain all the changes that were made to it.^[7]

Queries in RavenDB don't behave like queries in a relational database. RavenDB doesn't allow computation during queries, and it doesn't have problems with table scans or slow queries. We'll touch on exactly why and cover details about indexing in Part III, but for now you can see that most queries will *just work* for you.

Store

The `Store` command is how you associate an entity with the session. Usually, this is done because you want to create a new document. We've already seen this method used several times in this chapter, but here's the relevant part:

```
var person = new Person  
{  
    Name = "Oscar Arava"  
};  
session.Store(person);
```

Like the `Delete` command, `Store` will only save the document to the database when `SaveChanges` is called. However, it will give the new entity an ID immediately, so you can refer to it in other documents that you'll save in the same batch.

Beyond saving a new entity, `Store` is also used to associate entities of existing documents with the

session. This is common in web applications. You have one endpoint that sends the entity to the user, who modifies that entity and then sends it back to your web application. You have a live entity instance, but it's not loaded by a session or tracked by it.

At that point, you can call `Store` on that entity, and because it doesn't have a null document ID, it will be treated as an existing document and overwrite the previous version on the database side. This is instead of having to load the database version, update it and then save it back.

`Store` can also be used in optimistic concurrency scenarios, but we'll talk about this in more detail in Chapter 4.

SaveChanges

The `SaveChanges` call will check the session state for all deletions and changes. It will then send all of those to the server as a single remote call that will complete transactionally. In other words, either all the changes are saved as a single unit or none of them are.

Remember that the session has an internal map of all loaded entities. When you call `SaveChanges`, those loaded entities are checked against the entity as it was when it was loaded from the database. If there are any changes, that entity will be saved to the database.

It's important to understand that any change would force the entire entity to be saved. We don't attempt to make partial document updates in `SaveChanges`. An entity is always saved to a document as a single full change.

The typical way one would work with the session is:

```
using (var session = documentStore.OpenSession())
{
    // do some work with the session
    session.SaveChanges();
}
```

So `SaveChanges` is usually only called once per session, although there's nothing wrong with calling it multiple times. If the session detects that there have been no changes to the entities, it will skip calling the server entirely.

With this, we conclude the public surface area of the session. Those methods allow us to do about 90% of everything you could wish for with RavenDB. For the other 10%, we need to look at the `Advanced` property.

Advanced

The surface area of the session was carefully designed so that the common operations were just a method call away from the session, and that there would be few of them. But while this covers many of the most common scenarios, it isn't enough to cover them all.

All of the extra options are hiding inside the `Advanced` property. You can use them to configure the behavior of optimistic concurrency on a per-session basis using:

```
session.Advanced.UseOptimisticConcurrency = true;
```

Or you can define it once globally by modifying the conventions:

```
documentStore.Conventions.UseOptimisticConcurrency = true;
```

You can force a reload of an entity from the database to get the changes made since the entity was last loaded:

```
session.Advanced.Refresh(product);
```

And you can make the session forget about an entity completely (it won't track it, apply changes, etc.):

```
session.Advanced.Evict(product);
```

I'm not going to go over the Advanced options here. There are quite a few, and they're covered in the documentation quite nicely. It's worth taking the time to read about, even if you'll rarely need the extra options.

Hiding the session: avoid the IRepository mess

A common problem we see with people using the client API is that they frequently start by defining their own data access layer, usually named `IRepository` or something similar.

This is generally a bad idea. We've only started to scratch the surface of the client API, and you can already see there are plenty of valuable features (Includes, optimistic concurrency, change tracking). Hiding behind a generic interface typically results in one of two situations:

- Because a generic interface doesn't expose the relevant (and useful) features of RavenDB, you're stuck with using the lowest common denominator. That means you give up a lot of power and flexibility, and in 99% of cases, the interface won't allow you to switch between data store implementations.^[8]
- The second situation is that, because of issues mentioned in the previous point, you expose the RavenDB features behind the `IRepository`. In this case, you're already tied to the RavenDB client, but you added another layer that doesn't do much but increase code complexity. This can make it hard to understand what's actually going on.

The client API is meant to be easy to use and high level enough that you'll not need to wrap it for convenience's sake. In all likelihood, if you do wrap it, you'll just wind up forwarding calls back and forth.

One thing that's absolutely wrong to do, however, is to have methods like `IRepository.Get<T>(string id)` that will create and dispose of a session within the scope of the

Get method call. That cancels out a *lot* of optimizations, behaviors and functionality,^[9] and it would be a real shame for you to lose these features of RavenDB.

2.5.3. The Async Session

So far, we've shown only synchronous work with the client API. But async support is crucial for high performance applications. That's why RavenDB has full support for it. In fact, that's the recommended mode, and the synchronous version is actually built on top of the async version. The async API is exposed via the async session. In all respects, it's identical to the sync version.

Listing 2.24. Working with the async session

```
using (var session = documentStore.OpenAsyncSession())
{
    var person = new Person
    {
        Name = "Oscar Arava"
    };
    await session.StoreAsync(person);
    await session.SaveChangesAsync();
}

using (var session = documentStore.OpenAsyncSession())
{
    var tasksPerDayQuery =
        from t in session.Query<ToDoTask>()
        group t by t.DueDate into g
        select new
        {
            DueDate = g.Key,
            TasksPerDate = g.Count()
        };
    List<ToDoTask> tasksToDo = await tasksPerDayQuery.ToListAsync();

    foreach (var task in tasksToDo)
    {
        Console.WriteLine($"{task.Id} - {task.Task} - {task.DueDate}");
    }
}
```

Listing 2.24 shows a few examples of working with the async session. For the rest of the book, we'll use both the async and synchronous sessions to showcase features and behavior of RavenDB.

RavenDB splits the sync and async API because their use cases are quite different, and having separate APIs prevents you from doing some operations synchronously and some operations asynchronously. Because of that, you can't mix and use the synchronous session with async calls or vice versa. You can use either mode in your application, depending on the environment you're using. Aside from the minor required API changes, they're completely identical.

The async support is deep — all the way to the I/O issued to the server. In fact, as I mentioned earlier, the synchronous API is built on top of the async API and async I/O.

We covered the basics of working with the client API in this section, but that was mostly mechanics. We'll dive deeper into using RavenDB in the next chapter, where we'll also learn how it's all put together.

2.5.4. Going below the session

"Ogres are like onions," said Shrek. In a way, so is the client API. At the top, and what you'll usually interact with, are the document store and the document session. They, in turn, are built on top of the notion of Operations and Commands. An Operation is a high level concept, such as loading a document from the server.

Deep dive note

I'm going to take a small detour to explain how the client API is structured internally. This shouldn't have an impact on how you're *using* the client API, but it might help you better understand how the client is put together. Feel free to skip this section for now and come back to it at a later date.

The LoadOperation is the canonical example of this. A session Load or LoadAsync will translate into a call to the LoadOperation, which will run all the associated logic (Identity Map, Include tracking, etc.) up to the point where it will make a call to the server. That portion is handled by the GetDocumentCommand, which knows how to ask the server for a document (or a set of documents) and how to parse the server reply.

The same GetDocumentCommand is also used by the session.Advanced.Refresh method to get an updated version of the document from the server. You won't typically be using any of that directly, going instead through the session. Occasions to use an Operation directly usually arise when you're writing some sort of management code, such as Listing 2.25, which creates a new database on the cluster.

Listing 2.25. Creating a database named 'Orders' using Operation

```
var dbRecord = new DatabaseRecord("Orders");
var createDbOp = new CreateDatabaseOperation(dbRecord);
documentStore.Admin.Server.Send(createDbOp);
```

A lot of the management functionality (creating and deleting databases, assigning permissions, changing configuration, etc.) is available as operations that can be invoked in such a manner.

In other cases, you can use an Operation to run something that doesn't make sense in the context of a session. For example, let's say I wanted to delete all of the tasks in the database. I could do it with the following code:

```
store.Operations.Send(new DeleteByQueryOperation(
```

```
new IndexQuery { Query = "from ToDoTasks" }
));
```

The reason that the tasks are exposed to the user is that the RavenDB API, at all levels, is built with the notion of layers. The expectation is that you'll usually work with the highest layer: the session API. But since we can't predict all things, we also provide access to the lower level API, on top of which the session API is built, so you can use it if you need to.

2.6. Document identifiers in RavenDB

The document ID is a unique string that globally identifies a document inside a RavenDB database. A document ID can be any UTF8 string up to 2025 bytes, although getting to those sizes is extremely rare. You've already seen document IDs used in this chapter — `people/1-A`, `ToDoTasks/4-A` and the like. Using a Guid like `92260D13-A032-4BCC-9D18-10749898AE1C` is possible but not recommended because it's opaque and hard to read/work with.

By convention, we typically use the collection name as the prefix, a slash and then the actual unique portion of the key. But you can also call your document `hello/world` or `what-a-wonderful-world`. For the adventurous, Unicode is also a valid option. The character `U+1F426` is a valid document ID, and trying to use it in RavenDB is possible, as you can see in Figure 2.13. Amusingly enough, trying to include a raw emoji character broke the build for this book.

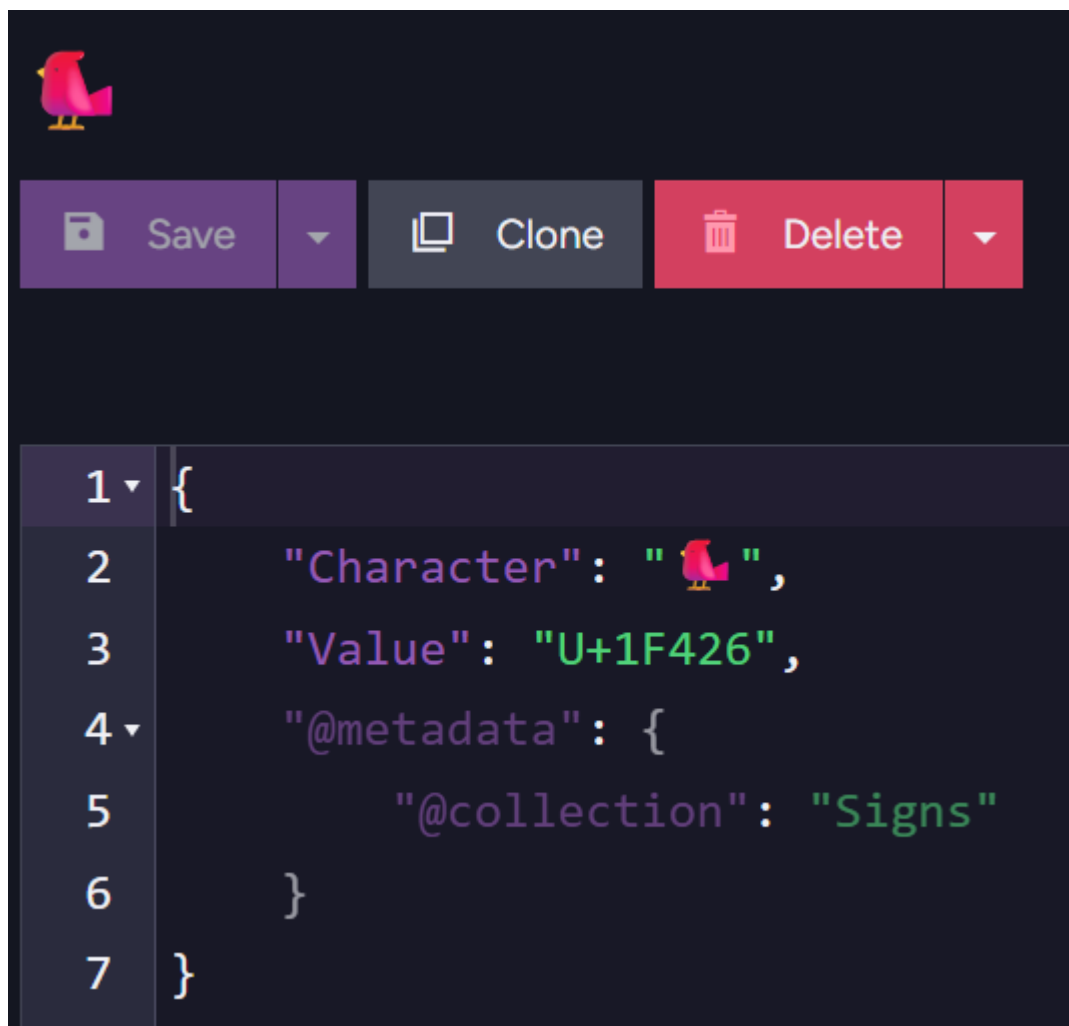


Figure 2.13. Unicode gave us Emojis, and the birdie document

While going full-on emoji for document identifiers might be going too far ^[10], using Unicode for document IDs means that you don't have to worry if you need to insert a Unicode character (such as someone's name).

RavenDB and Unicode

I hope it goes without saying that RavenDB has full support for Unicode. Storing and retrieving documents, querying on Unicode data and pretty much any related actions are supported. I haven't talked about it so far because it seems like an obvious requirement, but I think it's better to state this support explicitly.

So RavenDB document IDs are Unicode strings up to 2025 bytes in length, which must be globally unique in the scope of the database. This is unlike a relational database, in which a primary key must only be unique in the scope of its table. This has never been a problem because we typically use the collection name as the prefix to the document key. Usually, but not always, there's no requirement that a document in a specific collection will use the collection name prefix as the document key. There are a few interesting scenarios that open up because of this feature, discussed later in this section.

Human-readable document IDs

Usually, we strongly recommend to have document IDs that are human-readable (ToDoTasks/123-A, people/oscar@arava.example). We often use identifiers for many purposes. Debugging and troubleshooting are not the least of those.

A simple way to generate IDs is to just generate a new Guid, such as 92260D13-A032-4BBC-9D18-10749898AE1C. But if you've ever had to read a Guid over the phone, keep track of multiple Guid values in a log file or just didn't realize that the Guid in this paragraph and the one at the start of this section aren't, in fact, the same Guid...

If you're anything like me, you went ahead and compared the two Guid values to see if they actually didn't match. Given how hard finding the difference is, I believe the point is made. Guid values are not friendly, and we want to avoid having to deal with them on an ongoing basis if we can avoid it.

So pretty much the only thing we require is some way to generate a unique ID as the document ID. Let's see the strategies that RavenDB uses to allow that.

2.6.1. Semantic (external) document identifiers

The most obvious way to get an identifier is to ask the user to generate it. This is typically done when you want an identifier that's of some meaningful value. For example, people/oscar@arava.example or accounts/591-192 are two document IDs that the developer can choose. Listing 2.26 shows how you can provide an external identifier when creating documents.


```
using (var session = store.OpenSession())
{
    var person = new Person
    {
        Name = "Oscar Arava"
    };
    session.Store(person, "people/oscar@arava.example");
    session.SaveChanges();
}
```

The `people/oscar@arava.example` example, which uses an email address in the document identifier, is a common technique to generate a human-readable document identifier that makes it easy to locate a document based on a user provided value (the email). While the `accounts/591-192` example uses a unique key that's defined in another system. This is common if you're integrating with existing systems or have an external feed of data into your database.

2.6.2. Nested document identifiers

A special case of external document naming is when we want to handle nested documents. Let's consider a financial system that needs to track accounts and transactions on those accounts. We have our account document `accounts/591-192`, but we also have all the financial transactions concerning this account that we need to track.

We'll discuss this exact scenario in the next chapter, where we'll talk about modeling, but for now I'll just say that it isn't practical to hold all the transactions directly inside the account document. So we need to put the transactions in separate documents. We *could* identify those documents using `transactions/1234-A`, `transactions/1235-A`, etc. It would work, but there are better ways.

We're going to store the transaction information on a per-day basis, using identifiers that embed both the owner account and the time of the transactions: `accounts/591-192/txs/2017-05-27`. This document holds all the transactions for the 591-192 account for May 27th, 2017.

RavenDB doesn't care about your document IDs

RavenDB treats the document IDs as opaque values and doesn't attach any meaning to a document whose key is the prefix of other documents. In other words, as far as RavenDB is concerned, the only thing that `accounts/591-192` and `accounts/591-192/txs/2017-05-17` have in common is that they're both documents.

In practice, the document IDs are stored in a sorted fashion inside RavenDB, and it allows for efficient scanning of all documents with a particular prefix quite cheaply. But this is a secondary concern. What we're really trying to achieve here is to make sure our document IDs are very clear about their contents.

You might recall that I mentioned that RavenDB doesn't require documents within a given collection to have an ID with the collection prefix. This is one of the major reasons why —

because it allows you to nest document IDs to get yourself a clearer model of your documents.

2.6.3. Client-side identifier generation (hilo)

External identifiers and nesting document IDs are nice, but they tend to be the exception rather than the rule. For the most part, when we create documents, we don't want to have to think about what IDs we should be giving them. We want RavenDB to just handle that for us.

RavenDB is a distributed database

A minor wrinkle in generating identifiers with RavenDB is that the database is distributed and capable of handling writes on any of the nodes without requiring coordination between them. On the plus side, it means that in the presence of failures we stay up and are able to process requests and writes. On the other hand, it can create non-trivial complexities.

If two clients try to create a new document on two nodes in parallel, we need to ensure that they will not accidentally create documents with the same ID.^[11]

It's important to note, even at this early date, that such conflicts are part of life in any distributed database, and RavenDB contains several ways to handle them (this is discussed in Chapter 9 in more detail).

Another wrinkle that we need to consider is that we really want to be able to generate document IDs on the client, since that allows us to write code that creates a new document and uses its ID immediately, in the same transaction. Otherwise, we'll need to call to the server to get the ID, then make use of this ID in a separate transaction.

RavenDB handles this by using an algorithm called *hilo*. The concept is pretty simple. The first time you need to generate an identifier, you reserve a *range* of identifiers from the server. The server is responsible for ensuring it will only provide that range to a single client. Multiple clients can ask for ranges at the same time, and they will receive different ranges. Each client can then safely generate identifiers within the range it was given, without requiring any further coordination between client and server.

This is extremely efficient, and it scales nicely. RavenDB uses a dynamic range allocation scheme, in which the ranges provided to the client can expand if the client is very busy and generates a lot of identifiers very quickly (thus consuming the entire range quickly).

This is the default approach in RavenDB and the one we've used so far in this book. There's still another wrinkle to deal with, though. What happens if two clients request ID ranges from two different nodes at the same time? At this point, each node is operating independently (indeed, a network failure might mean that we aren't *able* to talk to other nodes). In order to handle this scenario properly, each range is also stamped with the ID of the node that assigned that range. This way, even if those two clients have managed to get the same range from each node, the generated IDs will be unique.

Let's assume the first client got the range 128 - 256 from node A and the second client got the same range from node B. The *hilo* method on the first client will generate document IDs like `people/128-A`,

people/129-A, and on the second client, it will generate people/128-B, people/129-B, etc. These are different documents. Using shorthand to refer to documents using just the numeric portion of the ID is common, but pay attention to the full ID as well.

It's important to note that this scenario rarely occurs. Typically, the nodes can talk to one another and share information about the provided ID ranges. Even if they can't, all clients will typically try to use the same server for getting the ranges, so you need multiple concurrent failures to cause this. If it *does* happen, RavenDB will handle it smoothly, and the only impact is that you'll have a few documents with similar IDs.

Dispose the document store to return unused range back to the system

When you take a range out using the hilo algorithm, it is marked as yours exclusively. If you aren't using the entire range, it will be lost when another client is using it. That can be annoying if you care about gaps in the numbers.

When you dispose the document store, RavenDB will attempt to return any unused range back to the system. Note that this is done simply for cosmetic reasons, to avoid the simplest scenario of running a process to save a single document and consuming the entire range. With hilo, there is no actual attempt to *ensure* that there would only be sequential values.

2.6.4. Server-side identifier generation

Hilo is quite nice, as it generates human-readable and predictable identifiers. However, it requires both client and server to cooperate to get to the end result. This is not an issue if you're using any of the client APIs, but if you're writing documents directly (using the RavenDB Studio, for example) or don't care to assign the IDs yourself, there are additional options.

You can ask RavenDB to assign a document ID to a new document when it is saved. You do that by providing a document ID that ends with the slash (/). Go into the RavenDB Studio and create a new document. Enter in the ID the value `tryouts/` and then click on the Save button. The generated document ID should look something like Figure 2.14.

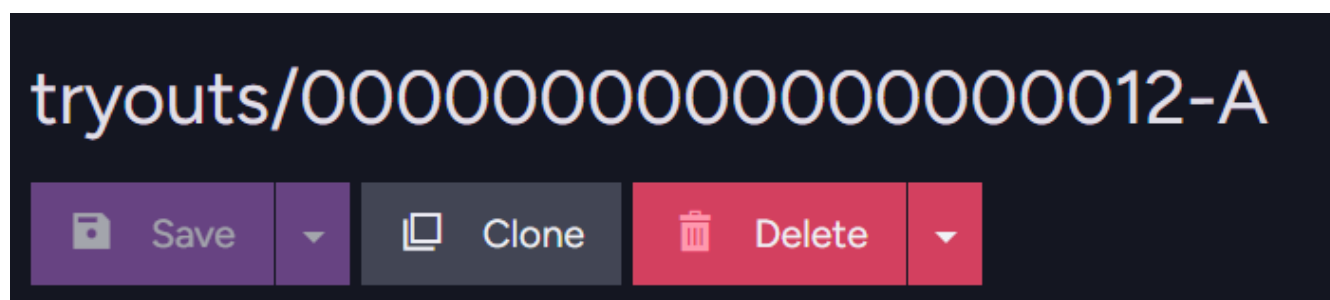


Figure 2.14. Server side generated document ID (tryouts/00000000000000000000000012-A)

When you save a document whose ID ends with a slash, RavenDB will generate the ID for you by appending a numeric value (the only guarantee you have about this value is that it's always increasing) and the node ID.

Don't generate similar IDs manually

Due to the way we implement server-side identifier generation, we can be sure that RavenDB will never generate an ID that was previously generated. That allows us to skip some checks in the save process (avoid a B+Tree lookup). Since server-side generation is typically used for large batch jobs, this can have a significant impact on performance.

If you manually created a document with an ID like `tryouts/000000000000000015-A` and RavenDB is about to create such an ID, RavenDB will detect the problem and move to the next number. However, given that you may also have documents with IDs like `tryouts/000000000000000016-A` we don't want to burn cycles trying to find a free number. So RavenDB will handle the problem by appending a GUID to the end of the ID.

The end result will look like: `tryouts/000000000000000016-E7GEsXQbsUuDxjSxW5+l/g`. In other words, everything will *work*, but it is likely to be quite surprising to see such an ID in your system. It's easier all around to just avoid creating documents with IDs that match the pattern of server-side generated IDs.

This kind of ID plays quite nicely with how RavenDB actually stores the information on disk, which is convenient. We'll give this topic a bit more time further down in the chapter. This is the recommended method if you just need to generate a large number of documents, such as in bulk insert scenarios, since it will generate the least amount of work for RavenDB.

The id generated by the `/` prefix contains the node id as well, since it is possible for two nodes to generate the same id on two nodes independently. For higher consistency, we need to turn to cluster-wide approaches, see the next section.

2.6.5. Identity generation strategy

All the ID generation strategies we've outlined so far have one problem: they don't give you any promises with regards to the end result. What they do give you is an ID you can be sure will be unique, but that's all. In the vast majority of cases, this is all you need. But sometimes you need a bit more.

If you really need to have consecutive IDs, you can use the identity option. Identity, just like in a relational database (also called sequence), is a simple always-incrementing value. Unlike the hilo option, you always have to go to the server to generate such a value.

Generating identities is very similar to generating server-side IDs. But instead of using the slash (`/`) at the end of the document, you use a pipe symbol (`|`). In the Studio, try to save a document with the document ID `tryouts|`. The pipe character will be replaced by a slash (`/`) and a document with the ID `tryouts/1` will be created. Doing so again will generate `tryouts/2`, and so on.

Invoices and other tax annoyances

For the most part, unless you're using semantic IDs (covered earlier in this chapter), you shouldn't care what your document ID is. The one case you care about is when you have an

outside requirement to generate absolute consecutive IDs. One such common case is when you need to generate invoices.

Most tax authorities have rules about not missing invoice numbers, to make it just a tad easier to audit your system. But an invoice document's *identifier* and the invoice *number* are two very different things.

It's entirely possible to have the document ID of invoices/843-C for invoice number 523. And using an identity doesn't protect you from skipping values because documents have been deleted or a failed transaction consumed the identity and now there's a hole in the sequence.

For people coming from a relational database background, the identity option usually seems to be the best one, since it's what they're most familiar with. But updating an identity happens in a *separate transaction* from the current one. In other words, if we try to save a document with the ID invoices| and the transaction fails, the identity value is still incremented. So even though identity generated consecutive numbers, it might still skip identifiers if a transaction has been rolled back.

Except for very specific requirements, such as a legal obligation to generate consecutive numbers, I would strongly recommend not using identity. Note my wording here. A legal obligation doesn't arise because someone wants consecutive IDs since they are easier to grasp. Identity has a real cost associated with it.

The biggest problem with identities is that generating them in a distributed database requires us to do a lot more work than one might think. In order to prevent races, such as two clients generating the same identity on two different servers, part of the process of generating a new identity requires the nodes to coordinate with one another.^[12]

That means we need to go over the network and talk to the other members in the cluster to guarantee we have the next value of the identity. That can increase the cost of saving a new document with identity. What's worse is that, under failure scenarios, we might not be able to communicate with a sufficient number of nodes in our cluster. This means we'll also be unable to generate the requested identity.

Don't generate similar IDs manually - part II

Just like with server-side ID generation, you need to be careful not to create documents that match the pattern of identity generated IDs. If you create a document with the ID tryouts/15 and the next identity value is 15, RavenDB will detect the conflict and *abort the transaction* with a concurrency error. Unlike server-side ID generation, we can't just move to the next number, since that would break the promise of consecutive IDs, so we have to fail the entire transaction.

Because we *guarantee* that identities are always consecutive across the cluster, if there's a failure scenario that prevents us from talking to a majority of the nodes, we'll not be able to generate the identity at all, and we'll fail to save the new document. All the other ID generation methods can work without issue when we're disconnected from the cluster, so unless you truly need consecutive IDs, use one of the other options.

RavenDB also has the notion of **cluster-wide transactions**, which we'll cover in the Chapter 9, ensuring that you can both generate identifiers *and* persist documents in an atomic fashion.

2.6.6. Performance implications of document identifiers

We've gone over a lot of options for generating document identifiers, and each of them have their own behaviors and costs. There are also performance differences among the various methods that I want to talk about.

Premature optimization warning

This section is included because it's important at scale, but for most users, there's no need to consider it at all. RavenDB is going to accept whatever document IDs you throw at it, and it's going to be *very* fast when doing so. My strong recommendation is that you use whatever document ID generation that best matches *your* needs, and only consider the performance impact if you notice an observable difference — or have crossed the hundreds of millions of documents per database mark.

RavenDB keeps track of the document IDs by storing them inside a B+Tree. If the document IDs are very big, it will mean that RavenDB can pack less of them in a given space.^[13]

The hilo algorithm generates document IDs that are lexically sortable, up to a degree (people/2-A is sorted after people/100-A). But with the exception of when we add a digit to the number^[14], values are nicely sorted. This means that for the most part we get nice trees and very efficient searches. It also generates the most human-readable values.

The server-side method using the slash (/) generates the best values in terms of suitability for storage. They're a bit bigger than the comparable hilo values, but they make up for it by being always lexically sorted and predictable as far as the underlying storage is concerned. This method is well suited for large batch jobs and contains a number of additional optimizations in its codepath. (We can be sure this is a new value, so we can skip a B+Tree lookup, which matters if you are doing that a *lot*.)

Semantic IDs (people/oscar@arava.example or accounts/591-192/txs/2017-05-17) tend to be unsorted, and sometimes that can cause people to want to avoid them. But this is rarely a good reason to do so. RavenDB can easily handle a large number of documents with semantic identifiers without any issue.

Running the numbers

If you're familiar with database terminology, then you're familiar with terms like B+Tree and page splits. In the case of RavenDB, we're storing document IDs separately from the actual document data, and we're making sure to coalesce the pages holding the document keys so we have a good locality of reference.

Even with a database that holds a hundred million documents, the whole of the document ID data is likely to be memory resident, which makes the cost of finding a particular document

extremely cheap.

The one option you need to be cautious of is the identity generation method. Be careful not to use it without careful consideration and analysis. Identity requires network round trips to generate the next value, and it will become unavailable if the node cannot communicate with a majority of the nodes the cluster.

2.7. Document metadata

Document data is composed of whatever it is that you're storing in the document. For the order document, that would be the shipping details, the order lines, who the customer is, the order priority, etc. You also need a place to store additional information that's unrelated to the document itself but is rather *about* the document. This is where metadata comes into play.

The metadata is also in JSON format, just like the document data itself. RavenDB reserves for its own use metadata property names that start with @ , but you're free to use anything else. By convention, users' custom metadata properties use Pascal-Case capitalization. In other words, we separate words with a dash, and the first letter of each word is capitalized while everything else is in lower case. RavenDB's internal metadata properties use the @ prefix, all lower cased, with words separated by a dash (e.g., @last-modified).

RavenDB uses the metadata to store several pieces of information about the document that it keeps track of:

- The collection name — stored in the @collection metadata property and determines where RavenDB will store the document. If the collection isn't set, the document will be placed in the @empty collection. The client API will automatically assign an entity to a collection based on its type. (You can control exactly how using the conventions.)
- The last modified date — stored in the @last-modified metadata property in UTC format.
- The client-side type — This is a client-side metadata property. So for .NET, it will be named Raven-Clr-Type; for a Java client, it will be Raven-Java-Class; for Python, Raven-Python-Type and... you get the point. This is used solely by the clients to deserialize the entity into the right client-side type.

You can use the metadata to store your own values. For example, Last-Modified-By is a common metadata property that's added when you want to track who changed a document. From the client side, you can access the document metadata using the code in Listing 2.27.

Listing 2.27. Modifying the metadata of a document

```
using (var session = store.OpenSession())
{
    var task = session.Load<ToDoTask>("ToDoTasks/1-A");
    var metadata = session.Advanced.GetMetadataFor(task);
    metadata["Last-Modified-By"] = person.Name;
    session.SaveChanges();
}
```

Note that there will be no extra call to the database to fetch the metadata. Whenever you load the document, the metadata is fetched as well. That metadata is embedded inside the document and is an integral part of it.

Changing a document collection

RavenDB does *not* support changing collections, and trying to do so will raise an error. You *can* delete a document and then create a new document with the same ID in a different collection, but that tends to be confusing, so it's best to be avoided if you can.

Once you have the metadata, you can modify it as you wish, as seen in Listing 2.27. The session tracks changes to both the document and its metadata, and changes to either one of those will cause the document to be updated on the server once `SaveChanges` has been called.

Modifying the metadata in this fashion is possible, but it's pretty rare to do so explicitly in your code. Instead, you'll usually use event handlers (covered in Chapter 4) to do this sort of work.

2.8. Distributed compare-exchange operations with RavenDB

RavenDB is meant to be run in a cluster. You can run it in single-node mode, but the most common (and recommended) deployment option is with a cluster. You already saw some of the impact this has had on the design of RavenDB. Auto-generated document IDs contain the node ID that generated them to avoid conflicts between concurrent work on different nodes in the cluster.

One of the challenges of any distributed system is how to handle coordination across all the nodes in the cluster. RavenDB uses several strategies for this, discussed in Part II of this book. At this point, I want to introduce one of the tools RavenDB provides specifically in order to allow users to manage the distributed state correctly.

If you've worked with multi-threaded applications, you are familiar with many of the same challenges. Different threads can be doing different things at the same time. They may be acting on stale information or modifying the shared state. Typically, such systems use locks to coordinate the work between threads. That leads to a whole separate issue of lock contention, deadlock prevention, etc. With distributed systems, you have all the usual problems of multiple threads with the added complication that you may be operating in a partial failure state. Some of the nodes may not be able to talk to other nodes (but can still talk to *some*).

RavenDB offers a simple primitive to handle such a scenario: the compare-exchange feature. A very common primitive with multi-thread solutions is the atomic compare-and-swap operation. From code, this will be `Interlocked.CompareExchange` when using C#. Because this operation is so useful, it's supported at the hardware level with the `CMPXCHG` assembly instruction. In a similar way, RavenDB offers a distributed compare-exchange feature.

Let's take a look at Listing 2.28, for a small sample of what this looks like in code.

Listing 2.28. Using compare exchange to validate unique username in a distributed system

```
var cmd = new PutCompareExchangeValueOperation<string>(
    key: "names/john",
    value: "users/1-A",
    index: 0);
var result = await store.Operations.SendAsync(cmd);
if (result.Successful)
{
    // users/1-A now owns the username 'john'
}
```

The code in Listing 2.28 uses `PutCompareExchangeValueOperation` to submit a compare-exchange operation to the cluster at large. This operation compares the existing index for `names/john` with the expected index (in this case, `0`, meaning we want to create a new value).

If successful, the cluster will store the value `users/1-A` for the key `names/john`. However, if there is already a value for the key and the index does not match, the operation will fail. You'll get the existing index and the current value and can decide how to handle things from that point (show an error to the user, try writing again with the new index, etc.).

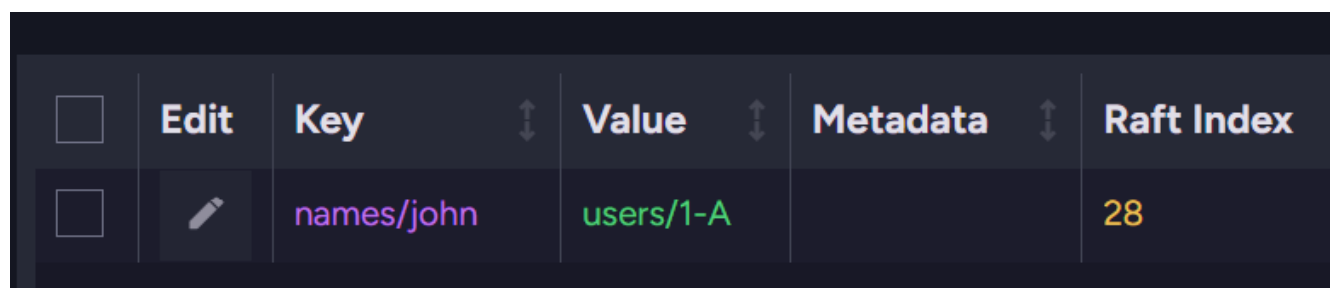


Cluster-wide operations & regular transactions

In Listing 2.28, we create a `names/john` compare exchange value that is tied to `users/1-A`. What happens if we fail immediately after the creation of the compare exchange value, but before we saved the `users/1-A`? We may have inconsistent state. RavenDB offers cluster-wide transactions for this scenario, see Listing 2.31 for more details.

The most important aspect of this feature is the fact that this is a cluster-wide, distributed operation. It is guaranteed to behave properly even if you have concurrent requests going to separate nodes. This feature is a low-level one; it is meant to be built upon by the user to provide more sophisticated features. For example, in Listing 2.28, we ensure a unique username for each user using a method that is resilient to failures, network partitions, etc.

You can see how this is exposed in the Studio in Figure 2.15.



☐	Edit	Key	Value	Metadata	Raft Index
☐		names/john	users/1-A		28

Figure 2.15. Viewing the compare exchange values in the Studio

We'll talk more about compare-exchange values in Chapter 9. For now, it's good to remember that they're there and can help you make distributed decisions in a reliable manner. A compare-exchange value isn't limited to just a string. You can also use a complex object, a counter, etc. However, remember that these are *not* documents. You can read the current value of compare-

exchange value using the code in Listing 2.29. Aside from checking the current value of the key, you get the current index, which you can then use in the next call to `PutCompareExchangeValueOperation`.

Listing 2.29. Reading an existing compare exchange value by name

```
var cmd = new GetCompareExchangeValueOperation<string>("names/john");
var result = await store.Operations.SendAsync(cmd);
```

Aside from getting the value by key, there is no other way to query for the compare-exchange values. Usually you already know what the compare-exchange key will be (as in the case of creating a new username and checking the name isn't already taken). Alternatively, you can store the compare-exchange key in a document that you'll query and then use the key from the document to make the compare-exchange operation.

If you know the name of the compare-exchange value, you can use it directly in your queries, as shown in Listing 2.30.

Listing 2.30. Querying for documents using `cmpxchg()` values

```
from Users
where id() == cmpxchg('names/john')
```

The query in Listing 2.30 will find a document whose ID is located in the `names/john` compare-exchange value. We'll discuss this feature again in Chapter 9. This feature relies on some of the low-level details of RavenDB distributed flow, and it will make more sense once we have gone over that.

2.9. Document extensions: beyond JSON documents

RavenDB is primarily a document database, storing JSON documents and letting you query and operate on them. That isn't the sum total of its capabilities. In this section, we'll briefly touch on those additional capabilities. As this section name suggest, RavenDB groups those additional options under the document's extension moniker. Let's explore what options RavenDB offers beyond JSON documents:

- **Attachments** - raw binary data that is being stored by RavenDB as-is, also known as BLOB (binary large objects). The easiest way to think about those is as attachments to email. RavenDB stores the attachments and allow you to download them afterward and index their contents. Examples: profile picture on a user profile, invoice PDF on an order form, etc.
- **Counters** - an 64 bits integer that counts (up or down). The reason this exists is that the counter allows concurrent and distributed operations to seamlessly merge without conflicts. Examples: visits counter on a page, current occupancy in a building, etc.
- **Time Series** - allows to track a value (or set of values) that changes over time. For example, tracking your heartrate throughout the day, a car's GPS location, etc. You not only track the current value, but also all the historical ones and are able to efficeintly execute aggregation operations on the values (what is my average heartrate on an hourly basis for the past week).

We'll cover all of those in depth in Chapter 3.

2.10. Cluster-wide transactions

RavenDB is always transactional. But there are two types of transactions that you can use, single-node transactions and cluster-wide transactions. We'll learn more about them in Chapter 9, but Listing 2.31 is a simple example of using cluster-wide transactions to do the same work as we did in Listing 2.28.

Listing 2.31. Using cluster-wide transaction to ensure a unique username in a distributed environment

```
using (var session = store.OpenSession(new SessionOptions {  
    TransactionMode = TransactionMode.ClusterWide    ①  
}))  
{  
    var user = new User("john", "john@example.com");  
    session.Store(user);                               ②  
    session.Advanced.ClusterTransaction  
        .CreateCompareExchangeValue("names/john", user.Id); ③  
    session.SaveChanges();                               ④  
}
```

- ① Open the session with cluster-wide transaction mode.
- ② Store an entity as we normally would (to go through the cluster).
- ③ Create a new compare exchange value (and ensure it is new) in the same transaction as storing the user.
- ④ Save both the compare exchange and the document store to the database in a cluster-wide transaction.

The main difference we have here is the use of `ClusterWide` as the `TransactionMode` when we open the session. What this code does is store a new `User` document in the session (and thus getting its ID). We then create a new compare exchange value off the session's cluster transaction and then we call to `SaveChanges`.

RavenDB ensures that the creation of the `names/john` compare exchange value and the saving of the new `User` document will happen in an atomic fashion. Two concurrent operations trying to also create a user named `john` is *guaranteed* to have one of them fail with a `ConcurrencyException` (and reverting all operations sent in the `SaveChanges` call) and one will succeed. We'll learn more about cluster-wide and how it works in Chapter 9.

2.11. Testing with RavenDB

This chapter is quite long, but I can't complete the basics without discussing testing. When you build an application, it's important to be able to verify that your code works. That has become an accepted reality, and an application using RavenDB is no exception.

In order to aid in testing, RavenDB provides the `Raven.TestDriver` NuGet package. The same capability exists for other platforms as well, check the online documentation for the full details. Using the test driver, you can get an instance of an `IDocumentStore` that talks to an in-memory database. Your tests will be very fast, they won't require you to do complex state setup before you

start and they will be isolated from one another.

Listing 2.32 shows the code for a simple test that saves and loads data from RavenDB.

Listing 2.32. Basic CRUD test using RavenDB Test Driver

```
public class BasicCrud : RavenTestDriver           ①
{
    public class Play
    {
        public string Id { get; set; }
        public string Name { get; set; }
        public string Author { get; set; }
    }

    [Fact]
    public void CanSaveAndLoad()
    {
        using (var store = GetDocumentStore())      ②
        {
            string id;
            using (var session = store.OpenSession())
            {
                var play = new Play
                {
                    Author = "Shakespeare",
                    Name = "As You Like It"
                };
                session.Store(play);
                id = play.Id;
                session.SaveChanges();
            }

            using (var session = store.OpenSession())
            {
                var play = session.Load<Play>(id);
                Assert.Equal("Shakespeare", play.Author);
                Assert.Equal("As You Like It", play.Name);
            }
        }
    }
}
```

① Inherits from the `RavenTestDriver` to enable using RavenDB directly in your tests.

② Use `GetDocumentStore` to get a live document store instance managed by the testing infrastructure.

There are two interesting things happening in the code in Listing 2.32. The code inherits from the `RavenTestDriver` class, and it uses the `GetDocumentStore` method to get an instance of the document store. Let's break apart what's going on.

The `RavenTestDriver` class is the base test driver, which is responsible for setting up and tearing down databases. All your RavenDB tests will use this class as a base class.^[15] Most importantly, from your point of view, is that the `RavenTestDriver` class provides the `GetDocumentStore` method, which generates a new in-memory database and is responsible for tearing it down at the end of the test. Each call to the `GetDocumentStore` method will generate a *new* database. It will run purely in memory, but other than that, it's fully functional and behaves in the same manner as a typical RavenDB server.

The code in Listing 2.32 is using `xunit` for testing, but there's no dependency on the testing framework from `Raven.TestDriver`. You can use whatever testing framework you prefer.

How does `Raven.TestDriver` work?

In order to provide fast tests and reduce environment noise, the test driver runs a single instance of the RavenDB server using an in-memory-only node binding to `localhost` and a dynamic port. Each call to the `GetDocumentStore` method will then create a new database on that single-server instance.

When the test is closed, we'll delete the database, and when the test suite is over, the server instance will be closed. This provides you with a test setup that's both very fast and that runs the exact same code as you will run in production.

2.11.1. Debugging tests

Sometimes a test fails and you need to figure out what happened. This is easy if you're dealing with in-memory state only, but it can be harder if your state resides elsewhere. The RavenDB test driver provides the `WaitForUserToContinueTheTest` method to make that scenario easier. Calling this method will pause the current test and open the RavenDB Studio, allowing you to inspect, validate and modify the content of the in-memory database (while the test is *still running*). After you've looked at the database state, you can resume the test and continue execution.

This makes it much easier to figure out what's going on because you can just *look*. Let's test this out. Add the following line between the two sessions in the code in Listing 2.32 and then run the test:

```
WaitForUserToContinueTheTest(store);
```

When the test reaches this line, a magical thing will happen, as shown in Figure 2.16. The Studio will open, and you'll be able to see and interact with everything that's going on inside RavenDB. One nice feature I like for complex cases is the ability to just export the entire database to a file, which lets me import it into another system later on for further analysis.

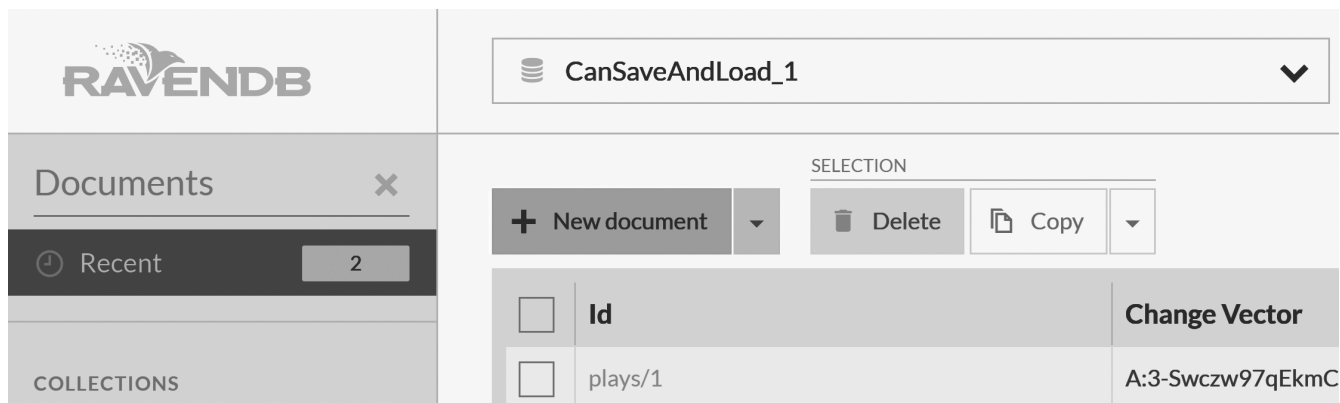
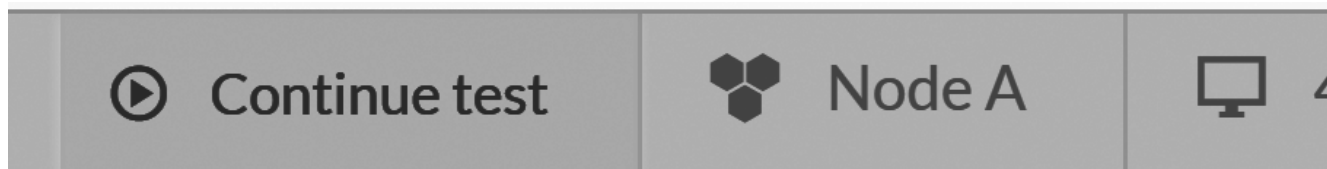


Figure 2.16. Peeking into running test instance mid-test

At this time, the rest of the test is suspended, waiting for you to confirm you’re done peeking inside. You can do that by clicking the button shown in Press this to continue the test, located at the bottom of the screen, after which your test will resume normally and (hopefully) turn green.

Press this to continue the test



The test driver can do quite a bit more (configure the database to your specifications, create relevant indexes, load initial data, etc.). You can read all about its features in the online documentation.

2.12. Summary

At this point in the book, we’ve accomplished quite a lot. We started by setting up a development instance of RavenDB on your machine.^[16] And we learned how to set up a new database and played a bit with the provided sample database.

We then moved to the most common tasks you’ll do with RavenDB:

- Creating/editing/deleting documents via the Studio.
- Querying for documents in the Studio.

The idea was to get you familiar with the basics of working with the Studio so you can see the results of your actions and learn to navigate the Studio well enough that it’s useful. We’ll talk more about working with the Studio throughout the rest of the book, but remember that the details are covered extensively in the online documentation and are unlikely to need additional verbiage.

Things got more interesting when we started working with the RavenDB API and wrote our first document via code. We looked at the very basics of defining entities to work with RavenDB (the next chapter will cover this exact topic in depth). We learned about creating and querying documents and were able to remind ourselves to buy some milk using RavenDB.

We dove deeper and discussed the architecture of the RavenDB client, as well as the use of Document Store and Document Session to access the cluster and a specific database, respectively. As a

reminder, the document store is the single access point to a particular RavenDB cluster, and it allows you to globally configure a wide range of behaviors by changing the default conventions.

The session is a single Unit of Work that represents a single business transaction against a particular database and is the most commonly used API to talk to RavenDB. It was designed explicitly to make it easy to handle 90% of pure CRUD scenarios, and more complex scenarios are possible by accessing the session. Advanced functionality.

From the client API, we moved to discussing how RavenDB deals with the crucial task of properly generating document identifiers. We looked at a few of RavenDB's identifier generation strategies and how they work in a distributed cluster:

- The hilo algorithm: generates the identifier on the client by collaborating with the server to reserve identifier ranges that can be exclusively generated by a particular client.
- Server-side: generates the identifier on the server side, optimized for very large tasks. It allows each server to generate human-readable, unique identifiers independently of each other.
- Identity: generates a consecutive numeric value using a consensus of the entire cluster. Typically the slowest method to use and only useful if you *really* need to generate consecutive IDs for some reason.

You can also generate the document ID yourself, which we typically call a semantic ID. Semantic IDs are identifiers that have meaning: maybe it's an external ID brought over from another system, or maybe it's a naming convention that implies the content of the document.

We briefly discussed document metadata and how it allows you to store out-of-band information about the document (auditing details, workflow steps, etc.) without impacting the document's structure. You can modify such metadata seamlessly on the client side (and access it on the server). RavenDB makes use of metadata to hold critical information such as the document collection, when it was last modified, etc.

Last but certainly not least, we discussed testing your applications and how the RavenDB test driver allows you to easily set up in-memory instances that will let you run your code against them. The test driver even allows you to stop a test midway through and inspect the running RavenDB instance using the Studio.

In this chapter, we started building the foundation of your RavenDB knowledge. In the next one, we'll build even further on that foundation. We'll discuss modeling data and documents inside RavenDB and how to best structure your system to take advantage of what RavenDB has to offer.

[1] Actually, that's not exactly the case, but the details on the state of a newly minted node are a bit complex and covered in more detail in Chapter 9.

[2] Eric Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Addison-Wesley, 2003.

[3] Unit of Work pattern: Maintains a list of objects affected by a business transaction and coordinates the writing out of changes and the resolution of concurrency problems. See <https://martinfowler.com/eaCatalog/unitOfWork.html>

[4] Identity Map pattern: Ensures that each object gets loaded only once by keeping every loaded object in a map. Looks up objects using the map when referring to them. See <https://martinfowler.com/eaCatalog/identityMap.html>

[5] I'll leave aside Id vs. ID, since it's handled in the same manner.

[6] A pattern taken from a wonderful book that heavily influenced the RavenDB design it *Release It! Design and Deploy Production-Ready Software*, Michael T. Nygard, Pragmatic Bookshelf, 2007.

[7] You can call `session.Advanced.Refresh` if you want to force the session to update the state of the document from the server.

- [8] The 1% case where it will help is the realm of demo apps with little to no functionality.
- [9] Unit of Work won't work. Neither will change tracking, optimistic concurrency you'll have to deal with manually, etc.
- [10] Although, when you think about it, there's a huge untapped market of teenage developers...
- [11] If the user explicitly specified the document ID, there's nothing that RavenDB *can* do here. But for IDs that are being generated by RavenDB (client or server), we can do better than just hope that we'll have no collisions.
- [12] This is done using the Raft consensus protocol, which covered in Chapter 9.
- [13] RavenDB is using B+Tree for on disk storage, and uses pages of 8KB in size. Bigger document IDs means that we can fit less entries in each page, and need to traverse down the tree, requiring us to do a bit more work to find the right document. The same is true for saving unsorted document IDs, which can cause page splits and increase the depth of the tree. In nearly all cases, that doesn't really matter.
- [14] Rolling from 99 to 100 or from 999 to 1000.
- [15] Not strictly necessary, but this is the easiest way to build tests.
- [16] The steps outlined in this chapter are meant to be quick and hassle-free, rather than an examination of proper production deployments. Check Chapter 15 for details on those.

Chapter 3. Beyond mere documents:

Attachments, Counters, Times Series and Revisions

RavenDB is all about documents, everything in RavenDB revolves around them. And documents in RavenDB are JSON objects that can be as complex and large as you want them to be.^[1] JSON is awesome, but it isn't always the best format to store your data. Luckily, RavenDB comes with several additional options to handle most common scenarios.

3.1. Attachments

Binary data is not well suited for JSON. You *can* use BASE64 to encode the value (at 25% overhead in storage), but that is rarely a good idea unless they are very small. Many applications need to handle large binary values, a profile picture or an uploaded PDFs are two such common examples.

RavenDB offers the attachments feature to handle this scenario. Conceptually, you can think about attachments in RavenDB in the same manner you do attachments in email. Let's see how this works first, and then discuss what is actually going on behind the scenes. In Listing 3.1 you can see how we create a new Invoice document and add a PDF file to it.

Listing 3.1. Storing an attachment in RavenDB

```
using (var session = store.OpenSession())
{
    Invoice invoice = CreateNewInvoice(out string pdfInvoice); ①
    session.Store(invoice);                                     ②
    using var pdfStream = File.OpenRead(pdfInvoice);           ③
    session.Advanced.Attachments
        .Store(invoice, "invoice.pdf", pdfStream, "application/pdf"); ④
    session.SaveChanges();                                     ⑤
}
```

- ① Here we create the invoice instance and get the pdfInvoice filename. This piece is not related to RavenDB.
- ② Storing the new invoice in the session (same as we did in the previous chapter).
- ③ Open a Stream from PDF file, to pass to the Attachments.Store() method.
- ④ Store the attachment stream (with a name and content type) into the session. Does *not* call to the server.
- ⑤ Atomically stores both the new invoice and its attachments into the database.

In Listing 3.1 you can see the basics of storing a new attachment in RavenDB. As the name suggests, we *attach* the stream to an existing document. Attachments do not exist on their own and are always associated with a document. As a result, deleting the document will also delete all its attachments. You cannot have a free-standing attachment.^[2]

In the Studio, you can see the attachments for a document on the right hand side, as you can see in Figure 3.1.

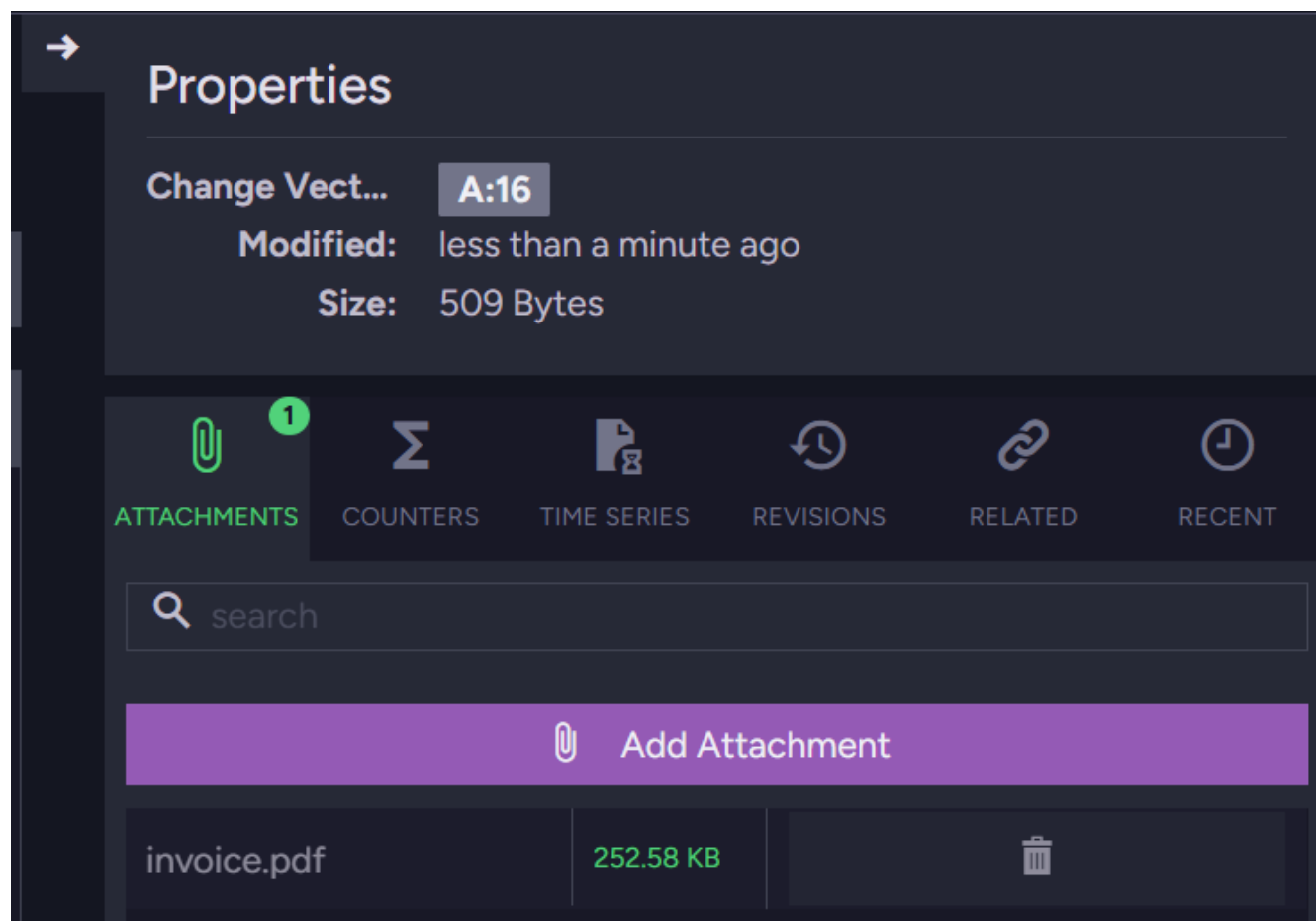


Figure 3.1. The invoice.pdf attachment on the Invoice document in the Studio

As you can see, in the RavenDB Studio we have a dedicated pane for attachments. You can click on the invoice.pdf in Figure 3.1 to download the file. We talked about document metadata in the previous chapter, and here is a good chance to see how RavenDB makes use of it in Listing 3.2.

Listing 3.2. The metadata element in the Invoice document after we attached a file to it.

```
{
  "@metadata": {
    "@attachments": [
      {
        "Name": "invoice.pdf",
        "Hash": "SszdDSioWjvo07yg0IGz52srIoicf4slUMy/js2WQKc=",
        "ContentType": "application/pdf",
        "Size": 258642
      }
    ],
    "@collection": "Invoices",
    "Raven-Clr-Type": "Tryouts.Invoice, Tryouts",
    "@change-vector": "A:16-woIsp5F3M0m00sjsTKlJ0g",
    "@flags": "HasAttachments",
    "@id": "invoices/1-A",
    "@last-modified": "2025-09-30T06:18:26.2639106Z"
  }
}
```

```
}  
}
```

- ① The `@attachments` array holds all the details about the attachment on this document.
- ② The name of the attachment, must be unique *within* this document. In other words, two documents can each have `invoice.pdf` attachment with no issues.
- ③ Cryptographic hash of the attachment - used for de-duplication of data, discussed more later.
- ④ The content-type of the attached file - this is not used by RavenDB directly, except to set the Content-Type header when downloading the file.
- ⑤ The size of the attachment in bytes.
- ⑥ RavenDB internal flags - indicating that this document has an attachment.

Listing 3.2 shows a *lot* of information about attachments. At first glance, this is just the metadata about the file we attached. On closer inspection, there is a wealth of information here that shows what happens behind the scenes. A document has an array for `@attachments`, and naturally you can have multiple attachments for the same document. Attachments names are scoped to the document that you attach them to. So you can have two Invoice documents each with their own `invoice.pdf` file. In the same document, however, attachment names are unique.

Attachments in the sample data

In the previous chapter, we created the Northwind database and seeded that with the sample data. The sample dataset contains attachments in the Employees and Categories collections. Figure 3.2 shows the attachment section for the `employees/1-A` document.

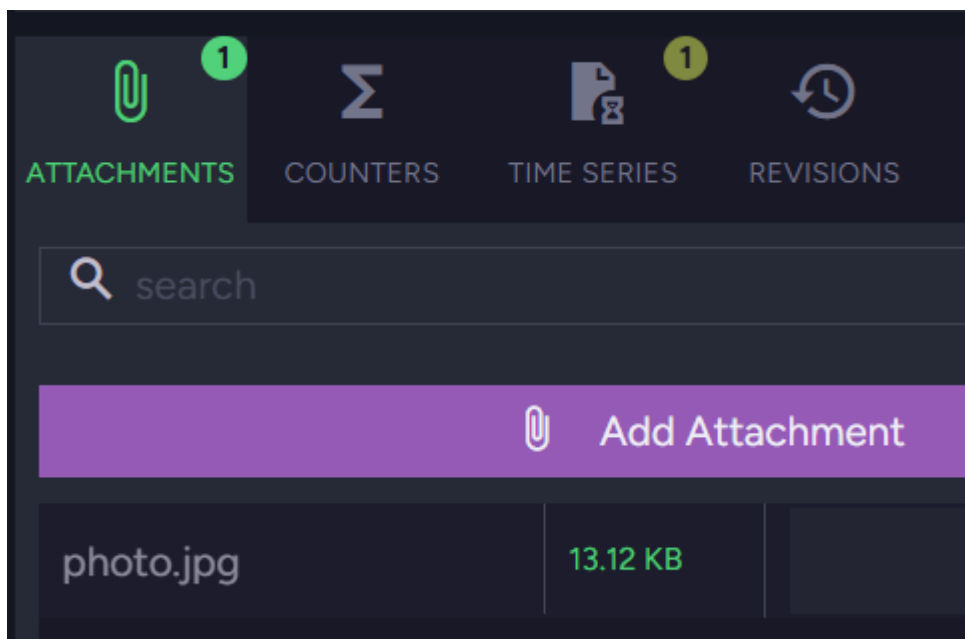


Figure 3.2. The employee's image in the sample dataset.

The attachment metadata also includes all the information you need *about* the attachment, except its content. You can access the `ContentType` or the `Size` of the attachment directly from the document metadata, without needing to make a separate remote call to get those details. There is also the `Hash`

element, which holds the cryptographic hash of the attachment's contents.



Attachments de-duplication

RavenDB automatically handles de-duplication for attachments when you add them to the database. This is done by computing a cryptographic hash of the attachment's content and seeing if we already have that stored. If the value is already stored, we'll just reference the stored data, rather than needing to hold duplicate copies. This is handled completely transparently to you and requires no attention. You *can* make interesting use cases using this information, however.

3.1.1. Working with attachments

Beyond just storing attachments, we can also do various (server-side) operations on them, such as:

- Rename - change the name of the attachment to a new one (no need to re-upload the file) on the same document.
- Move - move the attachment from the source document to *another* document (potentially renaming along the way).
- Copy - associate already existing attachment on a document with another document (no data is duplicated).
- Delete - delete the attachment from the document (if there are no other documents referencing the attachment, it will be remove).

A good example of how you make use of attachments is shown in Listing 3.3. Like all such calls in the session, calling Copy, Move or Rename does not actually perform the work but register it in the session to be executed as a single transaction when `SaveChanges` is called.

Listing 3.3. Copying the default profile image when we create a new user

```
using (var session = store.OpenSession())
{
    User person = new User("Oscar", "Arava");
    session.Store(person);
    session.Advanced.Attachments.Copy(           ①
        "users/default", "default-image.png",   ②
        person.Id, "profile-image.png");        ③
    session.SaveChanges();                       ④
}
```

- ① Copy the attachment one document to another (without going through the client).
- ② The source document and attachment name.
- ③ The destination document and attachment name.
- ④ Actually sending the Copy command to the database for execution.

Listing 3.3 shows creation of a new user and attaching the default profile image to it. Note that we don't need to download & upload the file to create it. Furthermore, we take advantage of RavenDB's attachment de-duplication to have a single copy of all the default profile images, across however

many numbers of users we have.

Attachments vs. BLOB

If you have worked with binary data with other databases, you are likely familiar with the concept of BLOB (binary large objects) columns. RavenDB's attachments are similar in concept, but quite different when you look in depth. The actual data for the attachment isn't stored alongside the document's data. In fact, given that a single copy of the attachment can be associated with many documents, it *can't* be stored inline.

To read the content of the attachment, you *must* explicitly load it. There is no option for doing something like `SELECT * ...` and accidentally reading very large values.

3.1.2. Reading attachments

The simplest way to read an attachment is shown in Listing 3.4. We get back an `AttachmentResult` which holds the attachment's `Details` and you can read the actual contents from the `Stream`.

Listing 3.4. Reading the attachment concept and writing to the output

```
using AttachmentResult result = session.Advanced.Attachments
    .Get("people/1-A", "profile-image.png");
result.Stream.CopyTo(outputStream);
```

Note that you should dispose the `AttachmentResult` you get from the operation when you are done with it. You can also use the `Get()` overload that accepts a number of `AttachmentRequest` values, to download multiple documents in a single remote call. You can also read a particular range of data from the attachment using the `GetRange` method. This allows advanced scenarios on your end if you are working with large attachments.

RavenDB's attachments has no size limit

Attachments in RavenDB have no size limits. You can store 10GB of video file inside of RavenDB (and stream it out using the `GetRange` method) without any issues.



Documents are limited in size in RavenDB (to about 2GB or so), and the attachment names and metadata are stored in the document itself. You should avoid having a "dumping ground" with a single document holds a *lot* (tens of thousands) of attachments. It is better to have a separate document per attachment if this is the case.

3.1.3. Attachments storage

RavenDB doesn't do *anything* with the attachment data. It simply holds them for you. This is in sharp contrast to storing JSON data, where RavenDB expose a lot of operations on documents and store them internally in an optimized format for its needs. Attachments are just stored. And this has important implications for your usage.

Consider storing a large `data.txt` file as an attachment. RavenDB will simply store that. If you were to compress using GZip or Zstd you'll likely get a far smaller file, which you can then upload as an attachment (`data.txt.gz`, for example). Usually storage space on database servers is *expensive* and you should attempt to reduce the amount you use needlessly.

RavenDB also supports the notion of retiring attachments (sometimes called remote attachments). That allows you to set a specific period of time before RavenDB will move the attachment's content to object storage (S3 or any compatible API backend) to store the attachment's contents. This is done in a manner that is completely transparent to your application.

After retiring an attachment to S3, you can still call to `Get()`, `GetRange()`, etc as you previously did. RavenDB will take care of proxying the request to the appropriate location. A major benefit of this is that you can have a transactional operation on both documents and attachments, then RavenDB will push the data to S3 (and transparently proxy as needed for reads) behind the scenes. See the online documentation for the full details on how to configure and run this option.



Indexing attachments contents

We'll cover indexing in RavenDB in Chapter 13, but it is important to note that you *can* index attachments' contents. However, if you'll retire attachments to object storage, you'll *not* be able to index the remote attachments. In practice, you'll usually only index small attachments and thus aren't usually candidates to retire to object storage.

3.2. Counters

Counters are a very simple concept, it is just a named 64 bits signed integer that can be attached to a document. In the sample data (which you can find in the Northwind database we created in the previous chapter) you can look at the `Products` collection to see usages of counters. You can see that in Figure 3.3.

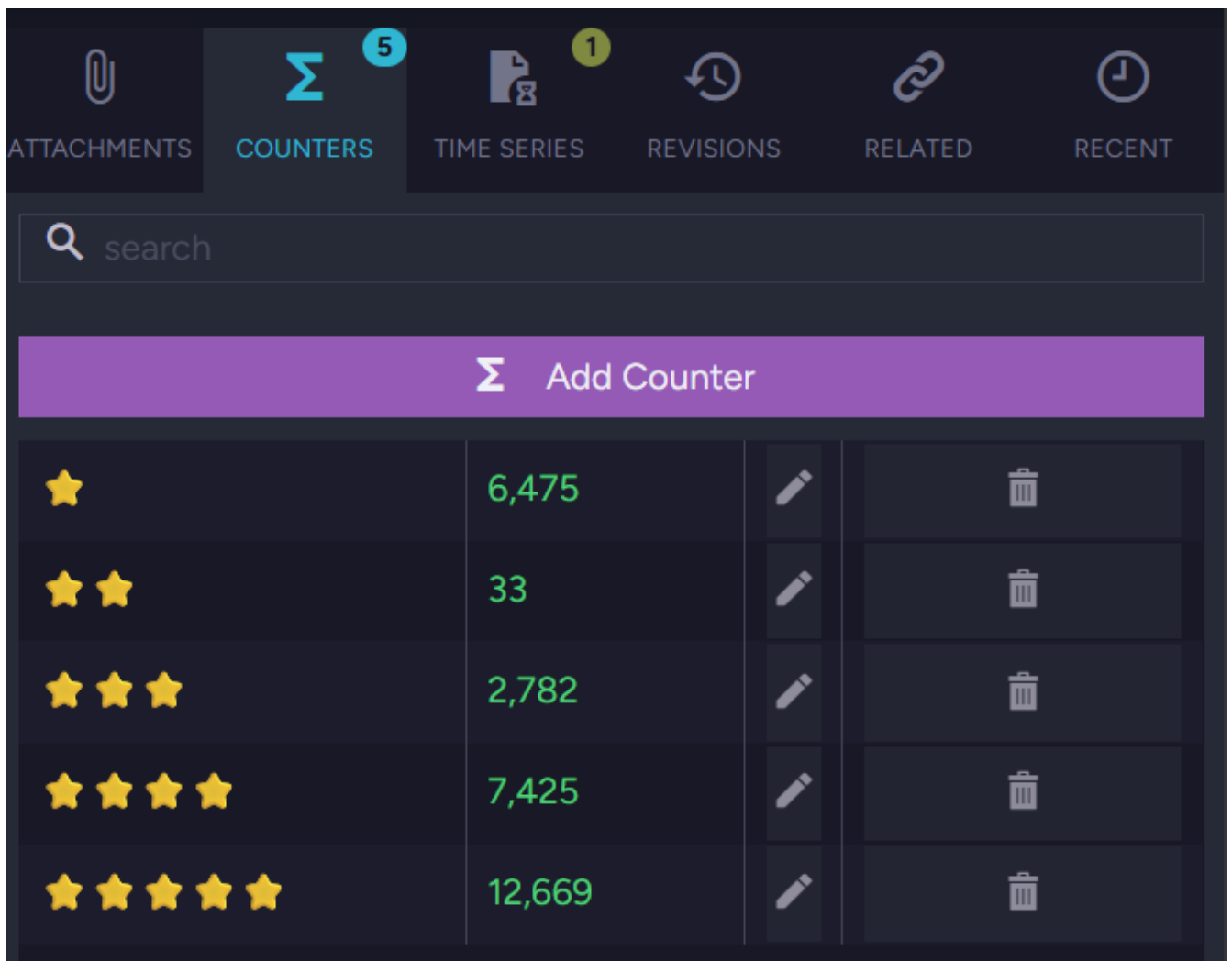


Figure 3.3. Counters usage in the Products collection for ranking products

In Figure 3.3 we were having fun, using emojis as the counter names. The idea is that we count how many times a product was rated one to five stars, with a dedicated counter to each rating.

Why would you want to use a Counter compared to a simple numeric property on the document? Let's consider the scenario of starring a product. You can use counters for this, as in Figure 3.3, or a property on the document. The key difference between the two is that counters allow you to increment or decrement a counter in a concurrent and distributed fashion safely.

Listing 3.5 shows how we can use the RavenDB client API to increment a counter on a document.

Listing 3.5. Creating a document and incrementing a counter vs. incrementing a property vs. patching the document

```
using (var session = store.OpenSession())
{
    var page = new Page("/index.html");           ①
    session.Store(page);
    session.SaveChanges();
}

using (var session = store.OpenSession())
{
```

```

    session.CountersFor("pages/1-A")
        .Increment("Visits");           ②
    session.SaveChanges();
}

using (var session = store.OpenSession())
{
    var page = session.Load<Page>("pages/1-A");
    page.Visits++;                       ③
    session.SaveChanges();
}

using (var session = store.OpenSession())
{
    session.Advanced.Defer(new PatchCommandData("pages/1-A",
        null, new PatchRequest           ④
        {
            Script = "this.Visits++;";
        }));
    session.SaveChanges();
}

```

- ① Create a Page document.
- ② Increment the Visits counter (creating it along the way) on the pages/1-A document.
- ③ Loading the page, incrementing the Visits property and saving the changes back to the database.
- ④ Using patching to increment the Visits property on the server side.

Listing 3.5 shows all the ways that we can record a visit to a page. The first is using a counter and the second is using a property on the document. What is the difference between the two? To increment a counter, we don't need to load the document, and RavenDB will take care of merging concurrent increments to the same counter (even if they happen on different nodes).

Counters are created on the fly

A counter is created the first time you call `Increment` on it, there is no need to create a counter before you can use it. This make it very useful to use dyanmic names for the counter to record information (for example: `visits/os:macosx`, `visits/browser:edge`). If you go that route, remember that the list of counter names is stored in the document metadata and that you don't want that to get out of hand. Dozens to low hundreds counters on a document is okay, thousands or more probably require a rethink on how you model the data.

On the other hand, to increment the `Visits` property on the document, you must load the document first, increment the property then save the *entire* document back to the database. There is also the option of using *patching*, the last example in Listing 3.5, we'll discuss patching documents in more dpeth in [ravnedb-client-api]. For now, just know that this allows you to do document modifications completely on the server side.

Consider the scenario of two concurrent visits to the same page. How would that work? If we use the option of loading the document and modifying the property, we've problem of losing one of those updates. If we use the patch approach, if we talk to multiple nodes at the same time, we run the risk of a conflict between them.

Decrementing a counter

RavenDB doesn't offer a Decrement operation on counters. You can pass a negative value to Increment in order to decrement the value.

Counters solves this problem completely, you don't need to read the previous value, and RavenDB will merge concurrent writes both in a single node and distributed across the cluster.

Following RavenDB's design philosophy, you can include a counter when you load a document, as you can see in Listing 3.6. Meaning you need a single remote call to fetch both pieces of information.

Listing 3.6. Reading a document & its counter in a single roundtrip to the database

```
var page = session.Load<Page>("pages/1-A", includes =>
    includes.IncludeCounter("Visits"));
long visits = session.CountersFor(page).Get("Visits") ?? 0;
```

Similar to attachments, we store information *about* the counters associated with a document in the document's metadata. The actual counters' data is kept separate from the document itself.

Listing 3.7. The metadata RavenDB keeps about counters in the document's metadata

```
{
  "@metadata": {
    "@collection": "Pages",
    "@counters": [
      "Visits"
    ],
    "Raven-Clr-Type": "Tryouts.Page, Tryouts",
    "@change-vector": "A:34-woIsp5F3M0m00sjsTKlJ0g",
    "@flags": "HasCounters",
    "@id": "pages/1-A",
    "@last-modified": "2025-09-30T18:39:19.8423899Z"
  }
}
```

① The counter names in this document

② RavenDB internal flags noting that this document has counters

Counters cannot overflow or underflow

Counters are 64 bits signed integers (long in C#/Java/C, etc). If an increment operation would cause the value to overflow or underflow, an exception will be raised. In practice, this is not something that you usually have to concern yourself with, but it is important to note that you are protected from this issue.

3.2.1. Deleting counters

RavenDB offers the Delete operation on counters, as you can see in Listing 3.8.

Listing 3.8. Deleting a named counter on a document

```
using (var session = store.OpenSession())
{
    session.CountersFor("pages/1-A")
        .Delete("Visits");
    session.SaveChanges();
}
```

Deleting counters is expected to be rare operation, and it has important interactions with concurrent Increment when working in a distributed environment. Consider the case of two concurrent requests to two separate nodes in the cluster. One to Delete the Visits counter on pages/1-A and the other to Increment the same counter. What will happen?

In this case, the Increment wins and the value will be retained. Some of the previous increments (before the Delete) that haven't been pushed to the cluster may be lost, however. In general, users are expected to avoid mixing Delete and Increment operations. That isn't a case of undefined behavior leading to nasal demons, mind. The behavior is well defined and tested, it is simply complex and involved a lot of distributed state and concurrency.

3.3. Time Series

Time Series is used when you care about a value and the way it changes over time. The classic example is your heartrate. My *current* heartrate, for example, is currently at 66 BPM. This doesn't tell you much about me or my health. A doctor looking at heartrate over a period of 24 hours, on the other hand, can probably tell a lot about the health of the patient. Another common example is currency exchange rate. If the USD/EUR is at 0.99 at the moment, that is interesting. But you can derive a lot of interesting information from looking at how this value changed over time.

RavenDB supports time series natively, and it stores them using an optimal format to reduce storage costs and improve queries efficiencies. In the sample dataset (check the Northwind database we created in the previous chapter), we have time series available on the Employees and Companies collections. You can see what this looks like for employees/1-A in Figure 3.4.

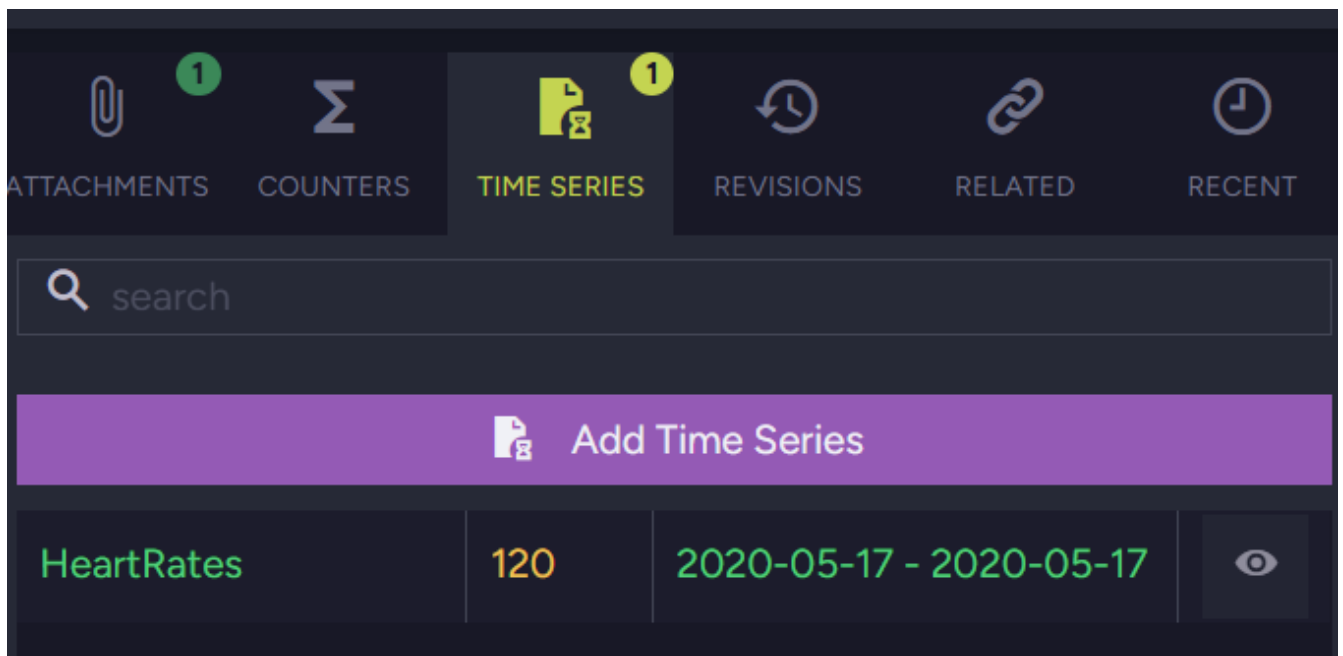


Figure 3.4. Time Series pane for the employees/1-A document

You can see the count of values (120) and the range of values for the HeartRates time series (in this case, a single day). If you'll click on the eye icon, you'll be able to see the actual values, showing in Figure 3.5.

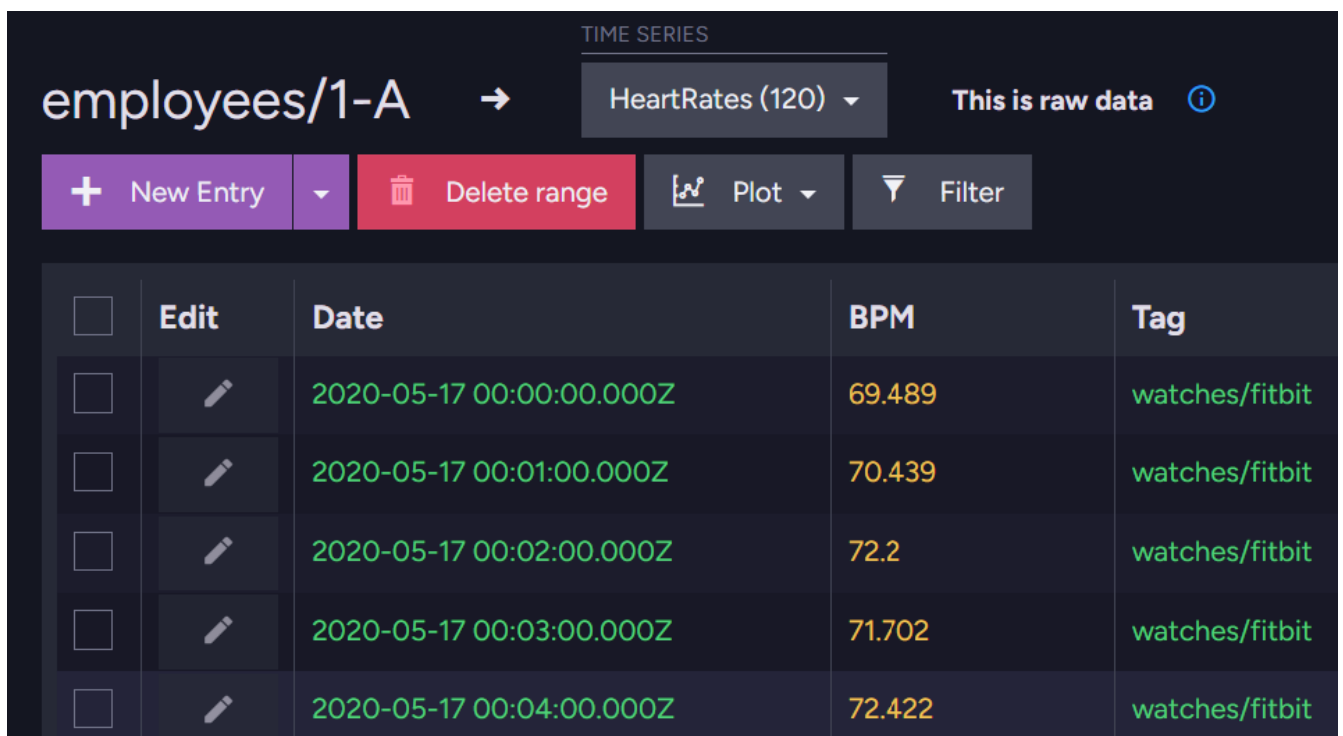


Figure 3.5. Detail view for the HeartRates time series for employees/1-A

Figure 3.5 shows a lot of interesting details. You can see that each value in a time series has a Date, a Value and a Tag. The Date is the instance in time when this value was recorded, the Value the BPM (beats per minute) in that instance and the Tag is a way to provide some additional information about that particular value.

Time Series tags

Each time series value has a tag associated with it. This tag is just free text that you can add. Its purpose is to record information about the value itself. In the case of Figure 3.5, you can see that the tag records that the data came from watches/fitbit. Another user may have that data provided through watches/apple-watch, etc.

It is recommended that the tag will be short and repetitive. A document ID that contains information about the source for that particular datum is best. In most cases, the data source doesn't change often, so you'll naturally get good results. The reason this matters relates to the way RavenDB actually stores this information, and is discussed in more depth later in this chapter.

You'll usually not care about individual values in a time series, you care a lot more about all the values together. Listing 3.9 shows how you can run a query on a time series and get aggregated information about it. Figure 3.6 shows the results of this query.

Listing 3.9. Getting average & max heart rate every 15 minutes from employees/1-A

```
from 'Employees'
where id() == 'employees/1-A'
select timeseries(from 'HeartRates' group by 15m select avg(), max())
```



Figure 3.6. Chart view for the query in Listing 3.9

Time Series are associated with a document, just like attachments and counters. You can see how we append a value to a time series on a document in Listing 3.10.

Listing 3.10. Appending a value to a time series

```
using (var session = store.OpenSession())
{
    session.TimeSeriesFor("pages/1-A", "DailyUsers")
        .Append(DateTime.Today, [2351], tag: "google/analytics");
}
```

```
    session.SaveChanges();  
}
```

In Listing 3.10 you can see that we append a value for today to the `DailyUsers` time series on `pages/1-A`. Like counters, there is no time series creation step. The time series for a document is created on the first append of a value. You can also see that we can append multiple values to a particular time in a time series. In Listing 3.10 we add a single value, but you can add up to 32 values to a time series.

The `Append` call accepts a `DateTime` as its first argument. Listing 3.10 pass `DateTime.Today` as the value, but it is important to understand that RavenDB time series operate *only* on UTC time. All values are converted to UTC when you append it. RavenDB offers the `offset` option when querying the time series, to allow you to match your time zone offset if needed.

The value of a time series - array of double values

We can append up to 32 values to a time series at a particular point in time. Those values are of type `double`, more precisely, double precision floating point IEEE 754 numbers. We recommend that you'll always pass the same number of values on each call, but it isn't *required*. If you pass 3 values in one call and 1 in the next call, it will just work for RavenDB. It is often a pitfall for clients that process the data and expect each entry in the time series to have the same number of values.

You cannot pass `NaN` as a value to the time series, RavenDB will reject such values.

What happens if we Append *another* value with an already existing date? Listing 3.11 explores RavenDB's behavior in this case.

Listing 3.11. Overwriting an existing value

```
using (var session = store.OpenSession())  
{  
    session.TimeSeriesFor("pages/1-A", "DailyUsers")  
        .Append(DateTime.Today, [4381], tag: "google/analytics");  
    session.SaveChanges();  
}  
  
using (var session = store.OpenSession())  
{  
    var series = session.TimeSeriesFor("pages/1-A", "DailyUsers")  
        .Get(DateTime.Today.AddDays(-1), DateTime.Today.AddDays(1));  
    foreach (var item in series)  
    {  
        // prints: 9/30/2025 9:00:00 PM - 4381  
        Console.WriteLine(item.Timestamp + " - " + item.Value);  
    }  
}
```

- ① Overwrite an existing value by sending new values for existing date in the time series
- ② Reading a range of values from the time series and printing it

In Listing 3.10 we overwrote the value from Listing 3.10 by sending new values for an existing time. Note that this time needs to be *exactly* the same time value (down to the millisecond). Listing 3.10 also shows how we can read a range of values from a time series and print it out.

The include feature of RavenDB also support time series, as you can see in Listing 3.12.

Listing 3.12. Loading a document and a time series range in a single remote call

```
var page = session.Load<Page>("pages/1-A", include =>
    include.IncludeTimeSeries("DailyUsers",
        DateTime.Today.AddDays(-1),
        DateTime.Today.AddDays(1)));
var series = session.TimeSeriesFor(page, "DailyUsers")
    .Get(DateTime.Today.AddDays(-1), DateTime.Today.AddDays(1));
```

When we read from a time series, we always need to pass a start & end markers. This is because time series in RavenDB are design to hold a *large* number of values. We have tested RavenDB with billions of entries in the time series, see more about the storage aspects of time series later in this chapter. You can usually not read the entire time series (nor does it make sense), so the ability to request all values within a range allow you to fetch a reasonable chunk for processing or viewing.

Date values in time series

The date value in time series is *always* in UTC and is using millisecond granularity. The DateTime type in C# can actually go to lower granularity than millisecond, but RavenDB will truncate all dates to millisecond granularity.

Like attachments and counters, you can see in Listing 3.13 how RavenDB records the existence of a time series in a document.

Listing 3.13. Metadata section of the pages/1-A document, with both counters and timeseries

```
{
  "@metadata": {
    "@collection": "Pages",
    "@counters": [
      "Visits"
    ],
    "@timeseries": [                                ①
      "DailyUsers"
    ],
    "Raven-Clr-Type": "Tryouts.Page, Tryouts",
    "@change-vector": "A:37-woIsp5F3M0m00sjsTKlJ0g",
    "@flags": "HasCounters, HasTimeSeries",          ②
    "@id": "pages/1-A",
  }
}
```

```
    "last-modified": "2025-09-30T22:26:42.7164247Z"
  }
}
```

- ① The names of the time series of this document
- ② RavenDB flags indicating that this document has both counters and time series

3.3.1. Querying time series

We saw in Listing 3.9 previously how we can query time series. We'll cover queries in far more depth in Chapter 12 but I want to cover some of the basics of what you can do with time series before we get any further.

All time series queries start by first finding the relevant document and then querying over the appropriate time series and range. Listing 3.14 shows how you can query a time series and get its raw values. You can copy the query in the RavenDB Studio Query page (in the Northwind database) to execute it and see the results.

Listing 3.14. Get raw time series results in a date range from HeartRates on employees/1-A.

```
$start = '2020-05-16'           ①
$end   = '2020-05-18'           ②

from 'Employees'                ③
where id() == 'employees/1-A'    ④
select timeseries(               ⑤
    from 'HeartRates'           ⑥
    between $start and $end      ⑦
    where Tag = 'watches/fitbit'
)
```

- ① The parameters for the query.
- ② Which collection we are targeting.
- ③ Filtering the documents to find.
- ④ The actual time series query expression to run for each document.
- ⑤ Which time series to select from.
- ⑥ Defining the range that we scan in the time series
- ⑦ Filtering the raw results from the time series in the selected range.

Listing 3.14 shows off quite a few features in a concise form, see the comments immediately following it. I'm not going to cover everything about time series queries. If you are interested in the full details, check the online documentation. Note that we are filtering the time series results by tag, so we'll only get entries that came from watches/fitbit. You can also filter on the raw values (using `where Values[0] > 70`).

Note that filtering on the time series results, you'll be *scanning* through the results. If you are going through a large range, that can take a while.

When querying on time series, Listing 3.14 shows querying by id. You can also use a query that would return multiple results, such as where `Address.city = 'London'`. In this case, you'll get a separate time series result for each of the match documents of the top query.

Usually when querying over a time series, you aren't interested in the raw values, but aggregated to a given time period. We saw that in Listing 3.14, where we grouped the results on 15 minutes boundary. Let's see another such query and discuss that in detail. In this case, we are querying over the `StockPrice` time series in the `Companies` collection.

Listing 3.15. Get aggregated stock price details for the first half of 2020, on a monthly basis, for companies/88-A.

```
$start = '2020-01-01'
$end = '2020-06-01'

from 'Companies'
where id() == 'companies/88-A'
select timeseries(
  from 'StockPrices' between $start and $end
  group by '1 month'
  select avg(), min(), max(),
         first(), last(), sum()
)
```

① Grouping the results by 1 month.

② Selecting which aggregation functions to perform on the results.

Listing 3.15 shows off a few interesting details. When you run this query in the studio, you'll see the results in Figure 3.7. You can see that we select multiple aggregation options for the data, and that the query grouped the data on a monthly basis.

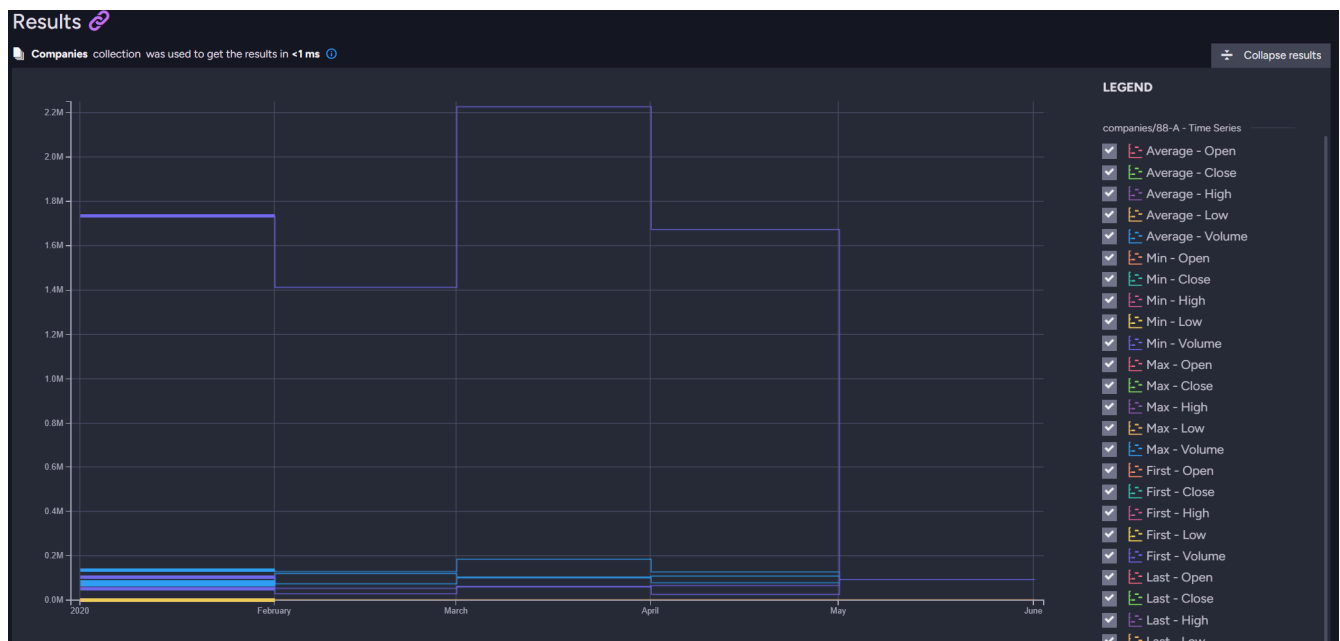


Figure 3.7. Chart view of the Listing 3.15 query.

You can group by milliseconds, seconds, minutes, hours, days, months, quarters and years. For

example, you can group by 6 hours or 30 seconds. The interesting thing about how RavenDB deals with aggregations and group by for time series is when you need to aggregate over large amounts of data with coarse granularity.

Consider a use case where we record heartrate on a per second basis, and needing to visualize that to the user as average, highest and lowest heart rate on an hourly basis for the past week. Let's compute the number of results we have to go through. A week has a total of 604,800 seconds, so that is the number of results we have. Because we output on an hourly basis, we have a total of 168 hours in a week. You can see the query in Listing 3.16.

Listing 3.16. Aggregate heart rates in hour granularity for the past week.

```
from 'Employees'
where id() == 'employees/1-A'
select timeseries(
    from 'HeartRates'
    between $weekAgo and $now
    group by 1h ①
    select avg(), min(), max()
)
```

① Grouping in 1 hour granularity.

How does RavenDB processes the query in Listing 3.16. It isn't going to run through the 604,800 entries we have. Internally, RavenDB stores the time series entries in segments, and it is able to condense the results of many entries into a single segment. The actual compression ratio depends on the exact data you have. In this case, I was able to fit 608,400 time series entries in 378 segments. With about 1,600 entries per segment. In terms of raw size, that is under 4MB in size.

RavenDB aggregations are able to take advantage of these low level storage details. Each segment already stores the aggregated values for each segment and the aggregation process can skip going through 604,800 entries and deal with aggregation on the 378 segments. In other words, you can do *fast* aggregations without paying the cost for them.

3.3.2. Naming time series values

In Figure 3.7 you can see the legend on the right side, with names such as Open, Close, etc. Those are the names of the values we record for the StockPrice time series on the Companies collection. RavenDB doesn't usually care for the names of the values, it is just an array of values (and you refer to them by position). For humans, it is much nicer to be able to name the value positions properly.

In the RavenDB Studio, you can go to Settings and then Time Series to define the naming of the values in the time series. You can see how that looks like in Figure 3.8.

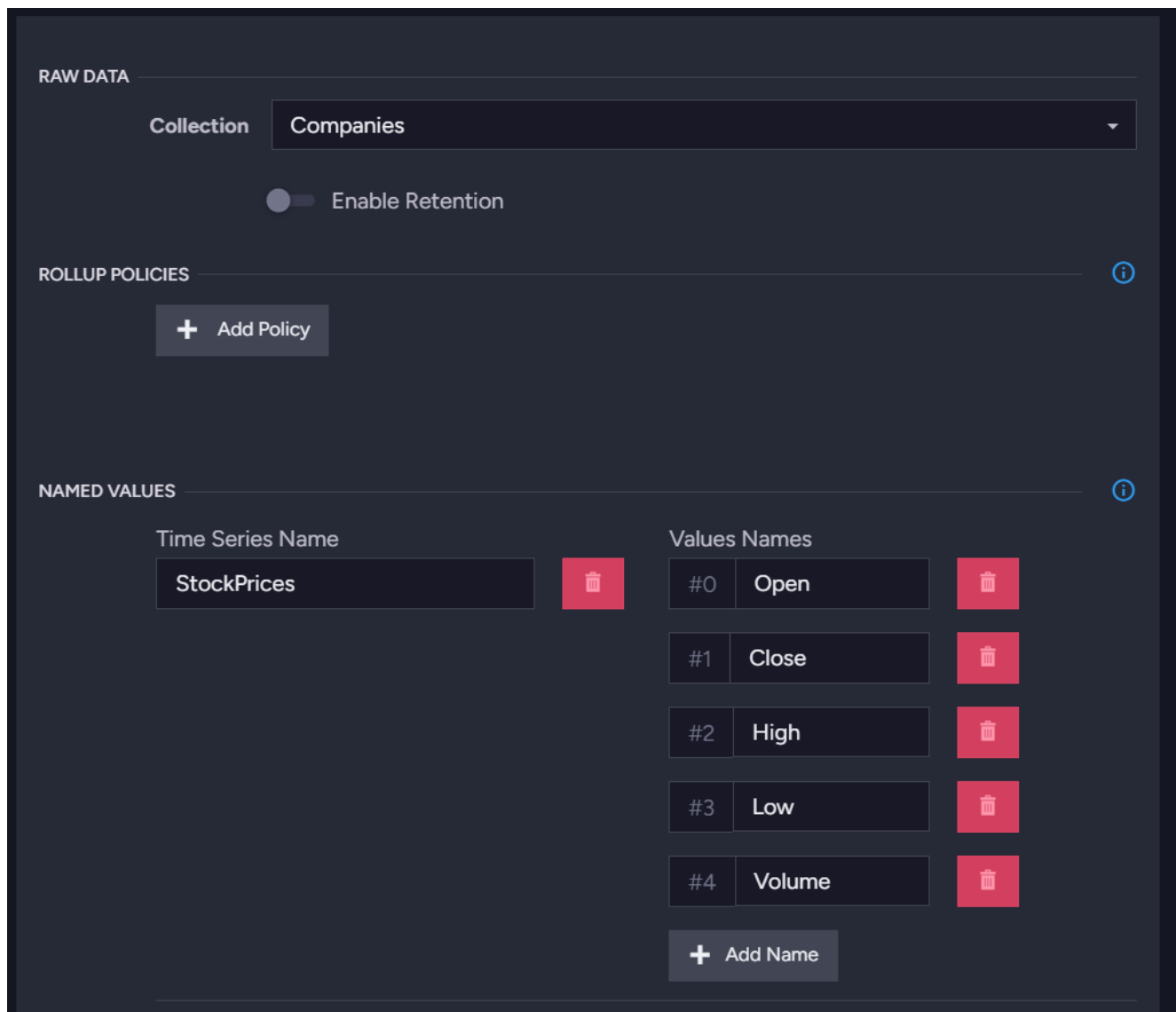


Figure 3.8. Configuring the names for the values of the StockPrice time series for Companies.

3.3.3. Deleting time series data

You can delete data from a time series by specifying a specific time series entry to remove by its date or by specifying a range and removing all the entries included in it. If you removed all the entries in the time series, RavenDB will delete the time series from the document. You can see how this looks like in Listing 3.17.

Listing 3.17. Deleting data from a time series

```
session.TimeSeriesFor("pages/1-B", "DailyUsers")
    .Delete(); ①

session.TimeSeriesFor("pages/1-B", "DailyUsers")
    .Delete(DateTime.Today); ②

session.TimeSeriesFor("pages/1-B", "DailyUsers")
    .Delete( ③
        DateTime.Today.AddDays(-7),
        DateTime.Today.AddDays(-1));
```

- ① Deleting the entire time series (by deleting all its values).
- ② Deleting a specific time series entry at the given date.
- ③ Deleting all the time series entries from a week ago to yesterday.

In a similar fashion to the counters, mixing Delete and Append is likely to produce unexpected results. If you Delete an entire time series (or even just a range) and you have concurrent Append to the time series (or the range being deleted), the Append will take precedence, but some of the values may be lost. It is the user responsibility to ensure that this is handled properly.

Optimizing time series storage requirements

All the entries inside a time series are compressed using Gorilla compression.^[3] This allows RavenDB to store large amounts of time series data in a very efficient manner.

The reason this matters enough to cover in this book is that the compression format benefits greatly from predictability. Let's assume you wish to record your heart rate every minute. In practice, one value will be recorded in 12:22:00.8027074 and the next will be in 12:23:01.3754696, etc. It is more efficient for RavenDB if you'll truncate those values to 12:22:00 and 12:23:00. That way, it can more easily predict the values and use less bits to store them.

In the same sense, if the values you are storing are mostly the same, we can compress them more efficiently. Consider what level of accuracy you need from your values. For example, for heart rate, you can truncate to a single decimal digit.

In general, if you need to store large amount of information in a time series, those considerations can save you a *lot* of space.

3.4. Time Series rollup & retention policies

RavenDB time series are meant to store *large* amount of data. We have tested the implementation with billions of data points in a single time series. It is easy to get to big sizes with a time series. Consider a simple scenario of keeping track of a user's heartrate on a per second basis. That means that you'll record 31,536,000 entries per year. Assuming a good compression ratio, that should take about 250MB or so.

250MB is a reasonable size to keep, but what happens if I have a million users, and each one of them has a time series like that? Now we are talking about 238 *terabytes* of data. And that is a horse of a different order of magnitude indeed. You can store data of this size in RavenDB, but you'll need to go with sharded approach. See Chapter 11 for more details on that approach.

Often enough, you don't need to keep the full data at the original granularity forever. In our heartrate example, we want to keep the data at a second granularity for two weeks, then we need the data at per-minute granularity for another 6 months, then we can drop to per-hour level. RavenDB supports this using rollups and retention policies, as you can see in Figure 3.9.

RAW DATA

Collection: Employees

☒ Enable Retention

Retention Time: KEEP DATA FOR: 14 days

ROLLUP POLICIES

Policy Name: Min ⓘ 🗑️

Aggregation Time: GROUP EVERY: 1 minutes

☒ Enable Retention

Retention Time: KEEP DATA FOR: 6 months

▼

Policy Name: Hour 🗑️

Aggregation Time: GROUP EVERY: 1 hours

☐ Enable Retention

Figure 3.9. Defining rollup & retention policies for time series on the Employees collection.

Figure 3.9 shows how we define the policies to do rollups (aggregate on a minute and hourly level) automatically as well as the retention policies (when RavenDB will delete the data). The space savings this entails are quite huge. Going from per second to per minute level, we reduce the cost by an order of magnitude. And going to hourly basis reduce us by another order of magnitude. Storing a full year of data for a million users will now cost us tens of gigabytes, and not hundreds of terabytes.

RavenDB allows you to define rollup and retention policy at the collection level only. All the time series on the collection share the same rollup and retention policy.

3.4.1. How time series rollups work?

When executing a rollup, RavenDB create *another* time series. In the case of the rollup policies we defined in Figure 3.9 and the HeartRates time series, we'll have: HeartRates@Min and HeartRates@Hour rollup time series. Those time series will store the aggregated results for each of the values in the the source time series.

In the case of the HeartRates time series, the HeartRates@Min rollup will contain the Count, Min, Max, First, Last, Sum values each each minute. In other words, the rollup time series will have 6 "columns" for each of the values in the source time series. The HeartRates@Hour rollup will aggregate

the values from the previous rollup, but here it will have the same 6 aggregated values per source value in the original time series.

Time series in RavenDB are limited to a maximum of 32 values per entry. Rollup time series also share this limitation, but because they hold 6 aggregated values for each value in the source time series, RavenDB is unable to do rollups if you have more than 5 values in the source time series.

Time series have a lot of depths to explore

Time series in RavenDB have a lot of capabilities. I touched most of the core concepts in this chapter, but there is more depth to explore. I would recommend looking into the online documentation to see what other options and capabilities you have available.

3.4.2. Working with multiple values in a time series

Up to this point, we look into time series that mostly worked with a single value. RavenDB time series can hold up to 32 values for a single point in time. Consider the case where we have a `Vehicle` document and we want to track its location over time. The code in Listing 3.18 shows how we can do that.

Listing 3.18. Recording the current location of a vehicle using a time series with two values (lat/lng)

```
void RecordCurrentLocation(string vehicleId, float lat, float lng)
{
    using var session = store.OpenSession();
    session.TimeSeriesFor(vehicleId, "Location")
        .Append(DateTime.UtcNow, [lat, lng]);
    session.SaveChanges();
}
```

Listing 3.18 shows the natural extension of what we have done so far, simply passing an array of values to the `Append` method. This will create a time series with the recorded locations of the car over time. You can then read the data and draw the vehicle location over time.

One important thing to note about the code in Listing 3.18 is a very subtle detail. Typically, GPS coordinate are provided to you as `float`. However, RavenDB time series values are always `double`. That *usually* doesn't cause a problem, but there are some cases where conversion from `float` to `double` result in a meaningful change in your domain. That is *very* unlikely, but worth calling out explicitly.

3.5. Incremental Time Series

We previously used a RavenDB Counter to track the total number of visits on a page. It would be nice if we could also show the visits on a time line. At first glance, it looks easy enough to do so using time series. Listing 3.19 shows one such attempt to use time series to track page visits over time.

Listing 3.19. Trying to track the number of page visits using time series - wrong approach

```
// Note: this is the _wrong_ way to do this, see the explanation below
session.TimeSeriesFor("pages/1-A", "Visits")
    .Append(DateTime.UtcNow, 1);
```

The code in Listing 3.19 will record a count of one on every visit to the page. We can then get the total visits per hour for the past week using the code in Listing 3.20.

Listing 3.20. Reporting the hourly page visits for the past week using time series.

```
from 'Pages'
where id() == 'pages/1-A'
select timeseries(
    from 'Visits' between $weekAgo and $now
    group by 1h
    select sum()
)
```

The output we get from Listing 3.20 is *exactly* what we want from this feature. It sums the number of visits from the individual entries we appended to the time series. It also plays very well with the rollups feature, allowing RavenDB to tally the visits to the right granularity that you want.

The problem with the code in Listing 3.19 is that it is **wrong**. It is wrong in a pretty subtle way, the problem is what happens when you have two visits on the *same* millisecond. In this case, calling Append on the same time will be treated as an updated value, rather than incrementing the already existing value. If you'll use the approach in Listing 3.19, you'll lose some visits when they happen on the same millisecond. That isn't what we want. An ideal option would be to have the same behavior we have in counters, where we can increment a value concurrently and safely, even when running in a distributed manner.

That is what *incremental time series* are for. An incremental time series is a normal time series, whose name starts with: "INC:". You cannot Append to an incremental time series, but must always use Increment. On the client API, you'll also use IncrementalTimeSeriesFor instead of TimeSeriesFor. [properly-tracking-page-visits-time-series] shows how this is done.

Listing 3.21. Track the number of page visits using time series - properly

```
session.IncrementalTimeSeriesFor("pages/1-A", "Visits")
    .Increment(DateTime.UtcNow, 1);
```

In [properly-tracking-page-visits-time-series], if this is called concurrently on the same exact time (and even if the requests are handled by different nodes), RavenDB will properly tally up all the values. In the sample dataset (in the Norhtwind database) we use an incremental time series to track the number of views in the Products collection using the INC:Views incremental time series. Listing 3.22 shows how you can get daily view counts for a product for two months. And Figure 3.10 shows the output of the query.

Listing 3.22. Reporting daily product views from the sample dataset.

```

$start = '2021-12-01'           ①
$end = '2022-01-31'

from 'Products'
where Category = 'categories/1-A' ②
select timeseries(
    from 'INC:Views'             ③
    between $start and $end      ④
    group by 1d                  ⑤
    select sum()                 ⑥
)

```

- ① The date range we'll query each of the time series on.
- ② Return results for all the products in categories/1-A (Beverages).
- ③ Selecting which time series we'll use - there is no difference here between incremental and standard time series
- ④ Specifying the date range for the time series that we'll operate on.
- ⑤ The grouping granularity (1 day, in this case).
- ⑥ The aggregation to perform. For incremental time series, it is almost always sum().

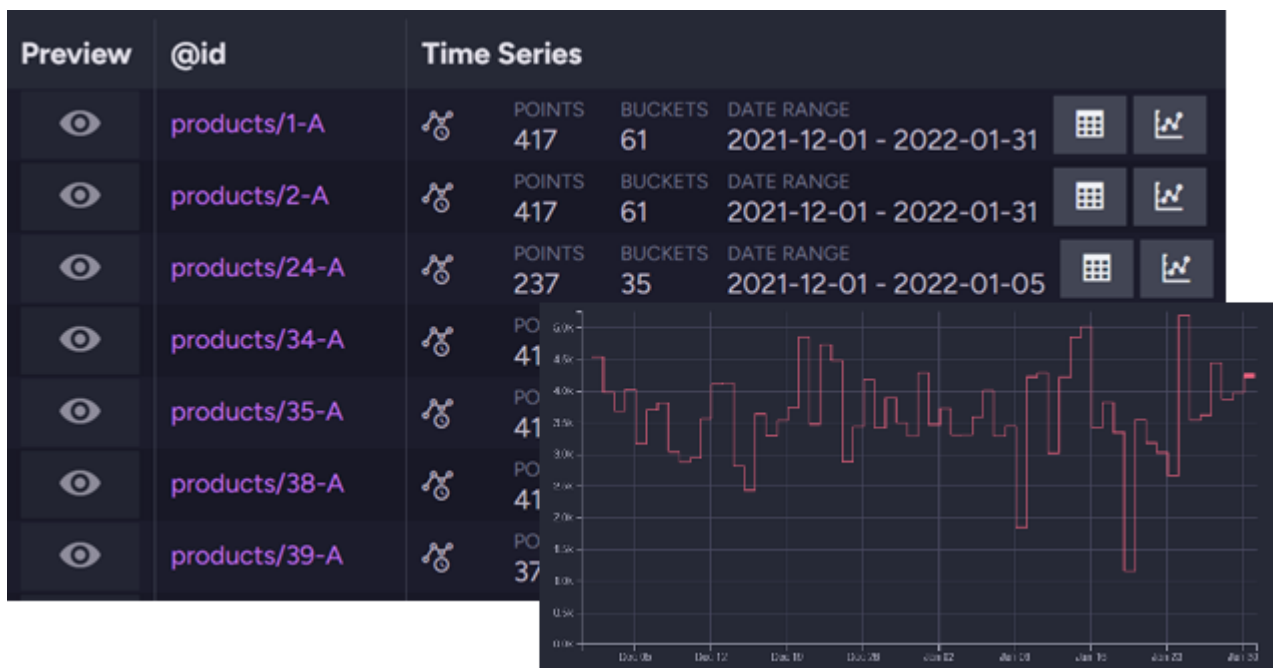


Figure 3.10. Query results for Listing 3.22, both the full results and the chart for a single result

In Figure 3.10 you can see the output of a time series query that returns multiple results. For each result in the top level query, RavenDB executes the time series query and returns the results back to the caller. In this case, the result of Listing 3.22 has 12 matching products, and each time series in it has 61 results.

Incremental Time Series at the right granularity

In Listing 3.21, we used `DateTime.UtcNow` to record a visit to the page. That gives us millisecond level precision on the number of visits to the page. It is almost *never* useful to record stuff like page visits with millisecond precision. We'll usually operate on those at maximum granularity of an hour, or maybe a minute.

That being the case, it makes sense to call `Increment` using the minimal granularity that you will ever work with the data. In the case of page visits, that is probably a minute. So we'll use the following approach instead:

```
var now = DateTime.UtcNow;
var nowTrimmed = new DateTime(now.Year, now.Month, now.Day,
                               now.Hour, now.Minute, 0);
session.IncrementalTimeSeriesFor("pages/1-A", "Visits")
    .Increment(nowTrimmed, 1);
```

This will record the values at the right granularity from the start, savings us more space and computation costs down the line. Remember that you can always aggregate at a higher value (per hour, for example, if you recorded at a minute granularity) at query time.

Reducing the total number of entries is highly recommended.

Incremental time series are meant to solve a very specific problem, how to count values over time. In the same sense, counters in RavenDB solves the same problem, for cases where you don't care about the time aspect. Like counters, you can also decrement values by using a negative number when you call `Increment`.

There is a a major difference between the incremental time series and counters that you should be aware of. Counters are using signed 64 bits integer number as their code data type, while incremental time series are built on top of regular time series. As such, their core data type is an array of double values. We haven't shown a code example of that, but you can absolutely send multiple values to incremental time series.

Incremental time series also differs from standard time series in that they do not have a tag that you can use. What would be the tag for the same entry if two calls passed two different values? Instead, RavenDB is using the tag internally to manage the concurrent handling of incremental time series.

3.6. Revisions

Certain classes of applications have very strict change control requirements for their data. For example, most medical, banking, payroll and insurance applications have strict "never delete since we need to be able to see all changes on the system" rules. One particular system I worked with had the requirement to keep all changes for a minimum of seven years.

With RavenDB, this kind of system is much easier. That's because RavenDB has built-in support for

handling revisions. Revisions are an immutable snapshot of a document, created either manually or using automatically by RavenDB. That allows us to go back in time and look at the previous states of a document.

We'll start looking at manual handling of revisions. In the RavenDB Studio, open the employees/1-A document, and then click on the Revisions tab on the right, shown in Figure 3.11.

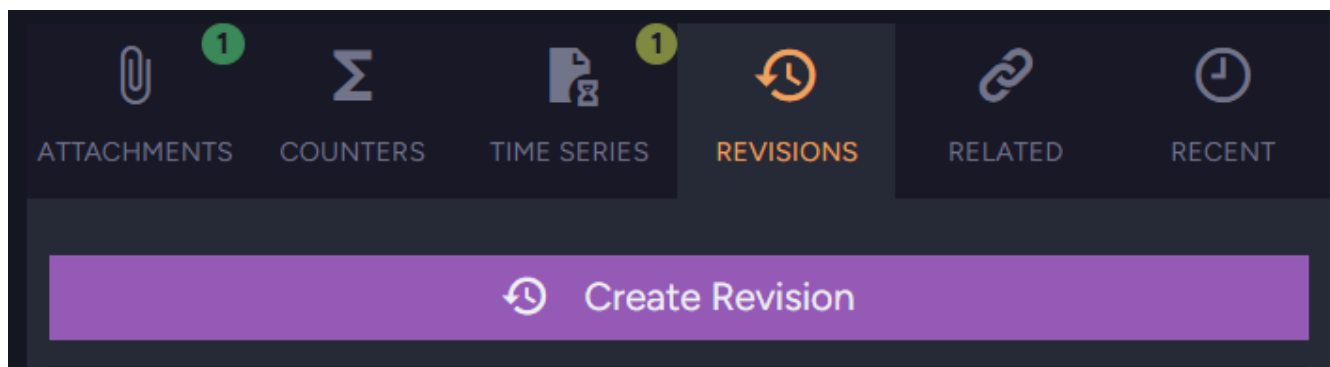


Figure 3.11. The revisions tab in the documents properties and the Create Revision button

Clicking on Create Revision will create a revision with the current time. You can click on the revision to see its state, which would be identical to the current document. Go back to employees/1-A, update the first name from "Nancy" to "Max" and save the document. Now click on Create Revision again. You now have two revisions for the employees/1-A document. You can click on either of them to see how the older revisions is named "Nancy" and the newer one is "Max". You can also click on the Compare button to see the actual difference between them, shown in Figure 3.12.

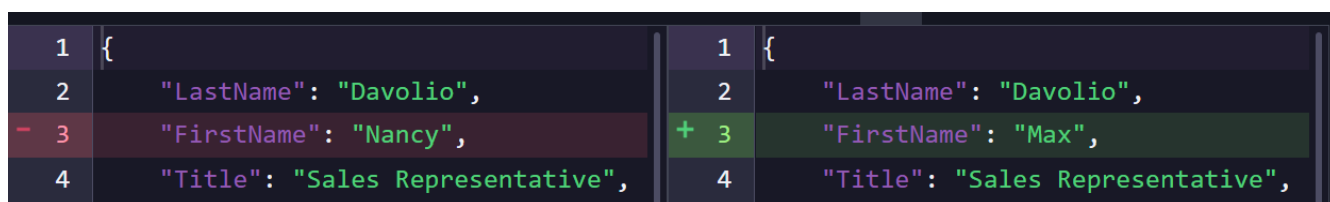


Figure 3.12. Showing the difference between two revisions in RavenDB Studio

RavenDB does not offer any option to modify revisions, they are intentionally immutable. You can create a revision yourself, using the Studio or the client API, as shown in Listing 3.23.

Listing 3.23. Creating a revision of a document using the client API

```
using (var session = store.OpenSession())
{
    session.Advanced.Revisions
        .ForceRevisionCreationFor("pages/1-A");
    session.SaveChanges();
}
```

In Listing 3.23 we are effectively doing the same as Create Revision. RavenDB will create an immutable snapshot of the document's current state (see more below on attachments, counters and time series interaction with revisions, though).

The client API also allows to do more interesting things, such as creating a revision and modifying the document as part of the same transaction, as you can see in Listing 3.24.

Listing 3.24. Modifying a document and creating a revision of its previous state in a single atomic transaction

```
using (var session = store.OpenSession())
{
    var page = session.Load<Page>("pages/1-A");
    page.Title = "Hello World";
    session.Advanced.Revisions
        .ForceRevisionCreationFor(page);
    session.SaveChanges();
}
```

In Listing 3.24, it doesn't matter whether we first update the document or called `ForceRevisionCreationFor`, in both cases, we'll first create the revision (capture the *before* state of the document) and then modify it.

Using `ForceRevisionCreationFor` to manually create revisions is viable for some use cases, where there are specific stages in which we want to create revisions. For example, you may have a page in draft mode and not care about tracking revisions in that stage. But once it moved to published state, you want to keep a record of all changes.

It is far more common to want RavenDB to handle that for you, and you can configure RavenDB to automatically manage revisions for you. In the Studio go into the Northwind database, and on the left side menu, click Settings and then Document Revisions. Figure 3.13 shows the revision configuration for the database.

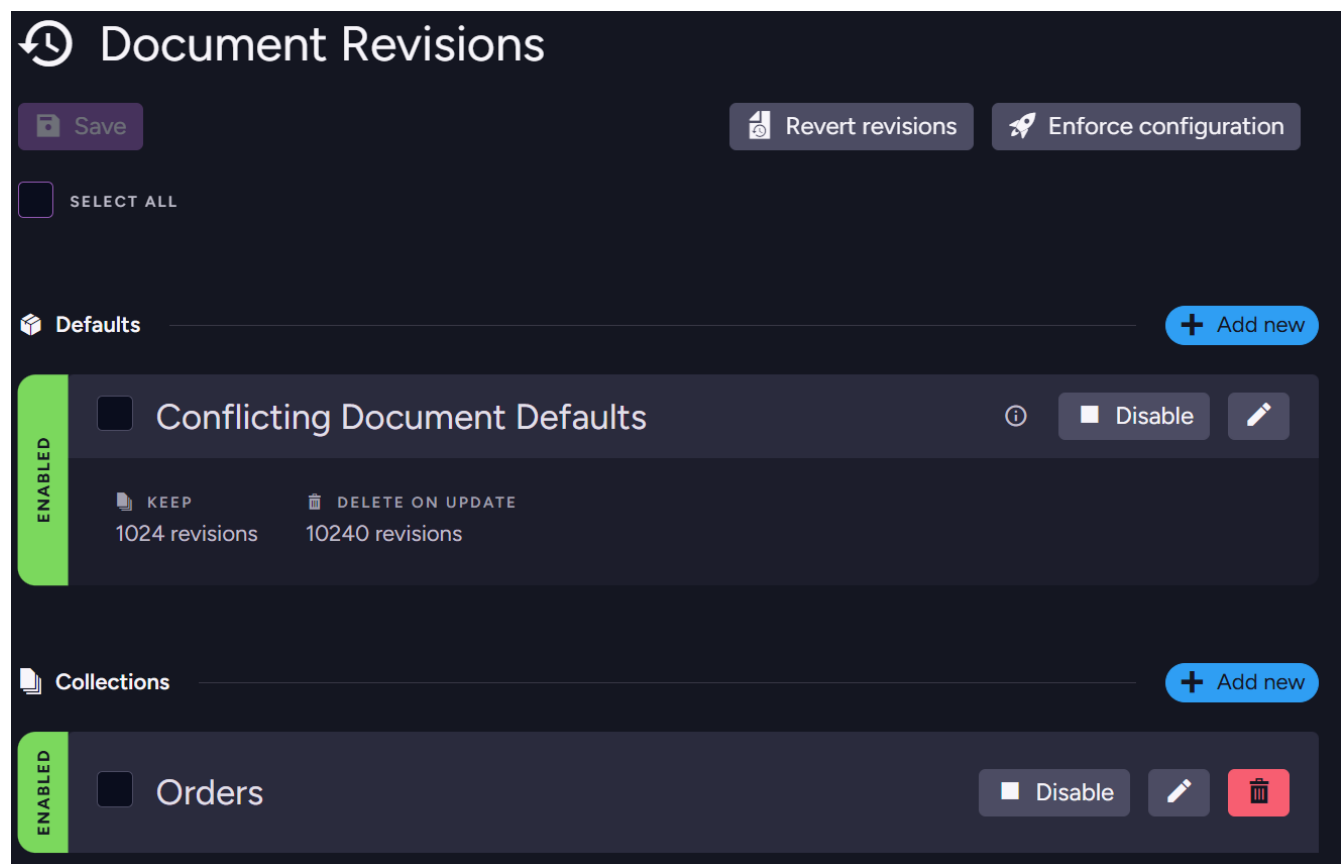


Figure 3.13. Revision configuration for the Northwind database

For now, we'll ignore the Conflicting Document Defaults configuration in Figure 3.13, we'll discuss that in Chapter 9. You can see that in the Northwind database, there is revision configuration for the Orders collection. You can click on the pencil icon to see more details on that. The revision configuration is mostly about when we can expire revisions (discussed later). The most important aspect is that if we have a revision configuration for a collection, RavenDB will create a revision for the document automatically whenever the document changes.

Revisions in a cluster

In a cluster, revisions are going to be replicated to all the nodes in the database. Conflicts cannot occur with revisions, since each revision has its own distinct signature. One of the most common usages of revisions is to store the entire document history when you have automatic conflict resolution. We'll cover this behavior in depth in Chapter 9.

In the case of the Northwind database, we have revisions configured for the Orders collection, and if you'll look at the document orders/830-A you can see all the revisions that were created for the document.

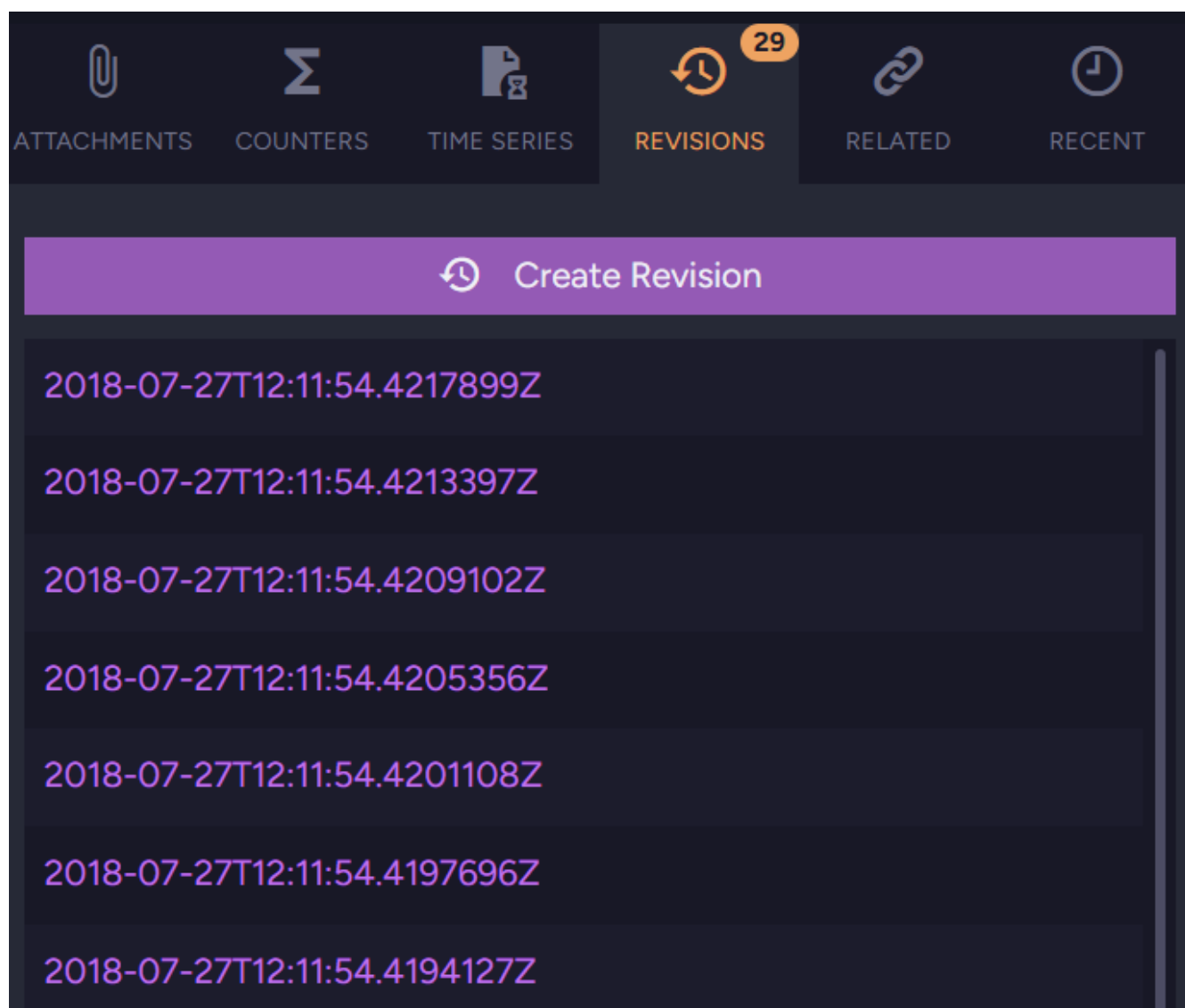


Figure 3.14. Revisions view for the orders/830-A

If you'll modify the orders/830-A document and save it, you'll see a new revision created automatically by RavenDB for you.

RavenDB's revisions are immutable record of changes to the document. RavenDB offers no way to modify or delete a revision. That is great from the perspective of creating an audit trail, but it isn't always desirable in terms of used disk space. RavenDB does offers the ability to expire revisions based on time or number of revisions. See Figure 3.15 for the details.

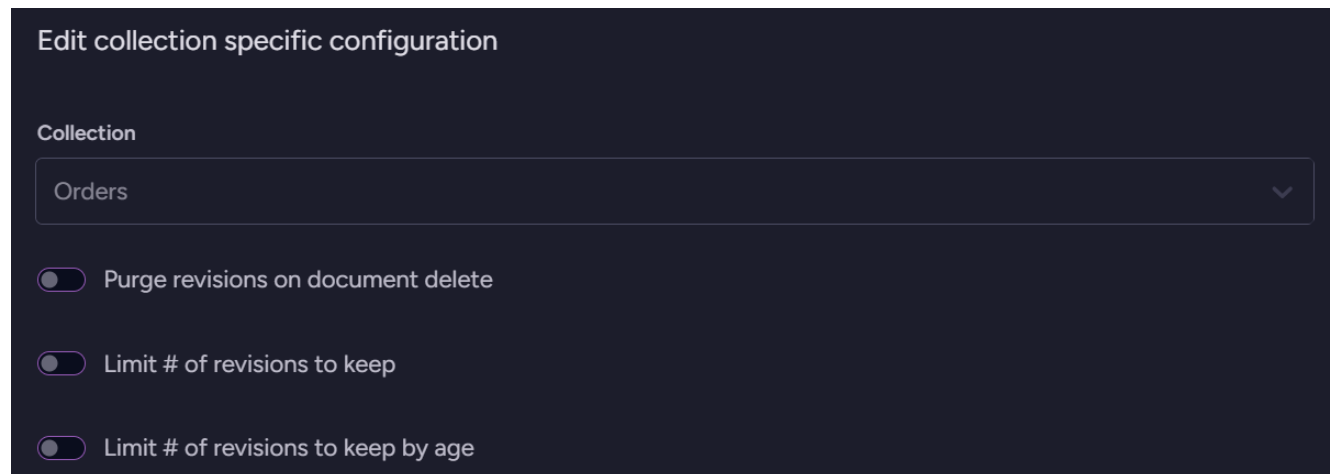


Figure 3.15. Configuration options for revisions on the Orders collection

Figure 3.15 shows how we can limit the number of revisons for a document based on age (delete revisions that are older than 60 days, for exmaple) or number of revisions (keep maximum of 100 revisions). If you define both, both conditions needs to match for the revisions to be removed.

Pay attention to the Purge revisions on document delete option. When configuring revisions, by default we'll *keep* the revisions even after the document itself has been deleted. You can access the revisions for deleted documnets in the Revision Bin. You can see that in Figure 3.16.

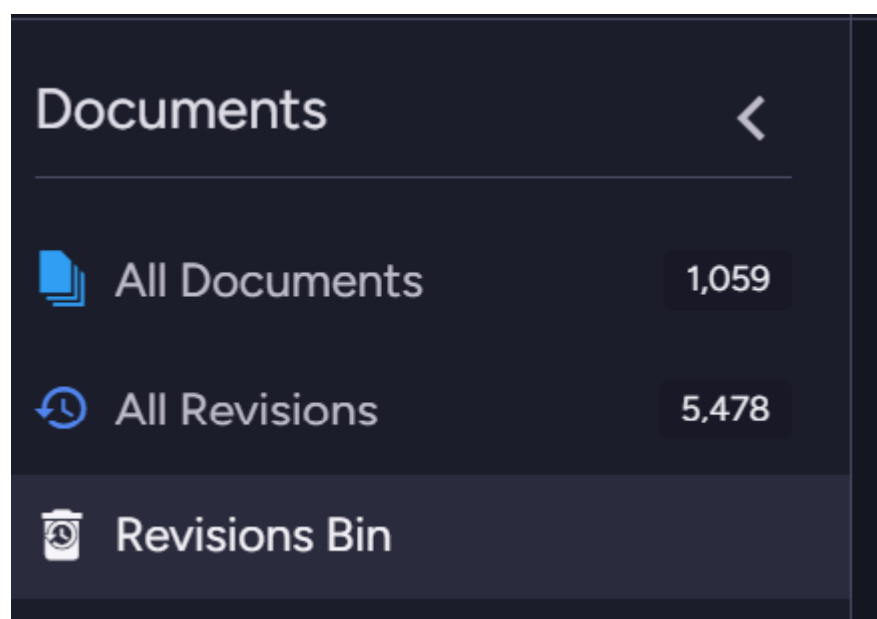


Figure 3.16. The revision bin allows you to access the revisions of deleted documents.

You can also work with revisions in a programatic manner, as you can see in the following snippet:

```
List<Page> revisions = session.Advanced.Revisions
    .GetFor<Page>("pages/1-A", 0, 10);
```

This snippet get the last 10 revisions on the pages/1-A document. You can also use this API to page through revisions and use `GetCountFor("pages/1-A")` to get the total count of revisions for a document. A common use case for this is to allow a user to browse through the past versions of a document in your system.

A really interesting option you have with revision is to fetch the first revision before a particular point in time, as you can see here:

```
var pageBefore2020 = session.Advanced.Revisions
    .Get<Page>("pages/1-A", new DateTime(2020,1,1));
```

This will get you the first revision of the page whose date is earlier than 2020-01-01.

Revisions are compressed

When using revisions, you are likely to end up with a *lot* of revisions. To reduce the storage cost of revisions, RavenDB compress all revisions automatically for you. That is an internal detail, not externally exposed, but important for operations to know when using revisions.

RavenDB optimize the compression format based on the actual documents been compressed, generating the appropriate compression dictionary to get good compression ratio across many documents.

I've found that some applications, without the regulations requiring versioning, make use of revisions just because they give the users an easy way to look at the changes on an entity over time. That was especially true in one example I can recall, where the user in question was the business expert in charge of the whole system. This feature was very valuable to her since it allowed her to see exactly what happened and who took the action. It was quite interesting to see how much value she got out of this feature.

In terms of cost, revisions obviously increase the amount of disk space required. But given today's disk sizes, that isn't usually a significant concern. Aside from disk space utilization, revisions don't actually have any impact on the system. Revisions are also quite important for ensuring that transaction boundaries are respected when replicating changes in the cluster (discussed in Chapter 9), handling historical subscriptions (discussed in Chapter 7) and ETL processes.

3.6.1. Revisions allow for time travel

A really nice capability that you get from having revisions is the ability to go back in time to a previous revision. Listing 3.25 shows how you can revert a page to its status in 2020-01-01.

Listing 3.25. Reverting a document to a past revision

```
using (var session = store.OpenSession())
{
    var pastPage = session.Advanced.Revisions
        .Get<Page>("page/1-A", new DateTime(2020,1,1)); ①
    session.Advanced.Evict(pastPage);                    ②
    session.Store(pastPage);                             ③
    session.SaveChanges();                               ④
}
```

- ① Fetch the page version as it was in 2020-01-01.
- ② Force the session to evict (forget about) the revision instance.
- ③ Store the pastPage as a *document*, so it will be saved back to the database.
- ④ Actually save the old version of the revision as the new value of the document.

You can also perform a mass revert of documents from the RavenDB Studio. Go to Settings and then to Document Revisions and check the Revert Revisions option as shown in Figure 3.17.

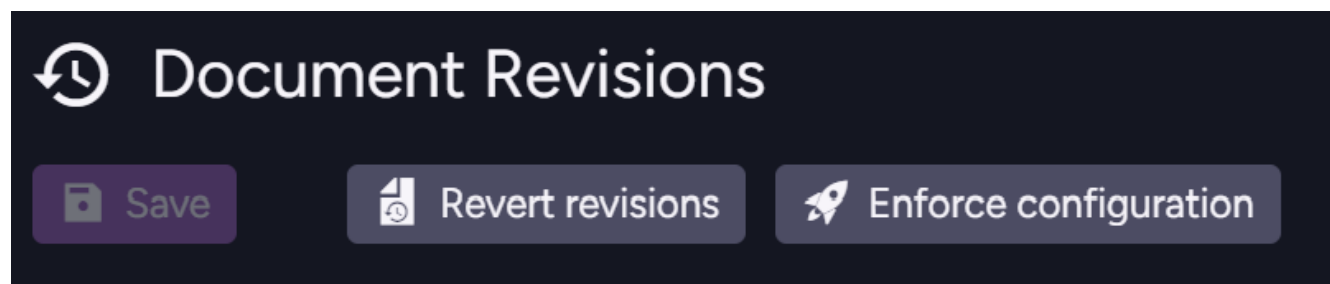


Figure 3.17. The Revert Revisions button allows to mass revert documents to a previous state

Clicking on the Revert Revisions button shown on Figure 3.17 will show the dialog in Figure 3.18.

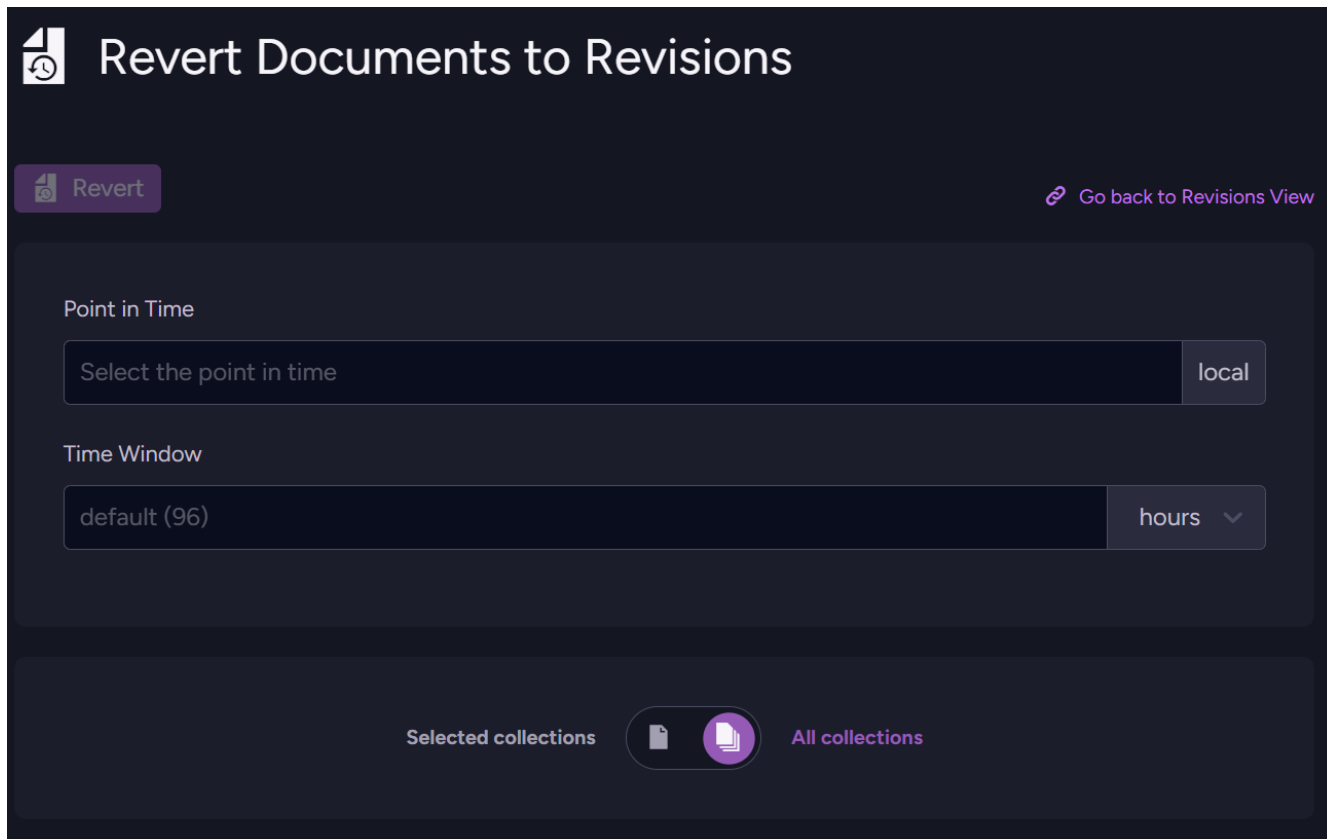


Figure 3.18. Revert Revisions allows you to select to what time to revert documents to, on all documents or for specific collections

Figure 3.18 shows the options to do mass revert of documents to a previous state, based on their revisions. You can specify the time to revert the documents to and whether it applies to all documents or just to specific collections.

The Time Window option in Figure 3.18 is interesting, from an implementation perspective. The use case for mass revert revision is that you'll want to go back to a previous time *recently*. The way Revert Revisions is implemented is that we scan through all the revisions in RavenDB and apply the first revision before the given date. The Time Window option determine how far back we should look for matching revisions. The longer the Time Window, the deeper RavenDB will scan through all the revisions in the database to find candidates to revert.

As you can imagine, if you specify a very large Time Window or Point in Time that is in the far past, RavenDB may have to scan through a *lot* of revisions.

Oppsies & point in time reverts

A key use case for the Revert Revisions is when you made an oppsie. You pushed a bad version to production and want to revert all changes made in the past day, or you run an update query on the wrong database, etc. Revert Revisions allows you to avoid a full restore-from-backup scenario and revert the database state back to its previous state.

It does require that you'll have revisions already configured, of course, but it can be a true saver when the bell tolls for you.

3.6.2. Interactions of revisions with attachments, counters and time series

Revisions create an immutable snapshot of a document. Attachments, counters and time series are document extensions that are stored *separately* from the document itself. That means that when you use those features in conjunction with revisions, there are a few things to keep in mind.

Revisions handle attachments seamlessly. Remember that attachments are stored internally in RavenDB based on their cryptographic hash, for de-duplication purposes. Revisions are able to take advantage of that and simply create another reference to the attachments' content in question. In other words, if you create a revision for a document with an attachment and then update the attachment on the document, the revision's attachment will keep the *same* attachment it had when it was created.

For counters and attachments, revisions handle things a bit differently. Listing 3.26 will make it easier to explain what exactly is going on.

Listing 3.26. Metadata for a revision on the products/75-A document that has both counters and timeseries

```
{
  "@metadata": {
    "@collection": "Products",
    "@counters-snapshot": {           ①
      "": 3832,
      "": 47,
      "": 2488,
      "": 34989,
      "": 48077
    },
    "@timeseries-snapshot": {        ②
      "INC:Views": {
        "Count": 536,
        "Start": "2021-11-30T22:00:00.0000000Z",
        "End": "2022-02-17T06:31:30.0000000Z"
      }
    },
    "@change-vector": "A:8896-BdphfhgaDUaCc4sSTDq2Qw",
    "@flags": "HasRevisions, Revision, HasCounters, HasTimeSeries, ForceCreated",
    "@id": "products/75-A",
    "@last-modified": "2025-10-02T09:22:59.3194788Z"
  }
}
```

① Snapshot of the counter *values* at the time of the revision creation.

② Snapshot of the timeseries *metadata* at the time of the revision creation.

Listing 3.26 shows two different behaviors for revisions with counters and time series. For counters, we record the *current values* of the counters at the time the revision was created. This is simple to do because we can capture the entire state of the counter in a single 64 bits number. We use the `@counters-snapshot` in the metadata to let you access the counters stored state explicitly.

For time series, we face a far different problem. RavenDB's time series are meant to be *large*. It is common to have hundred of thousands or millions of values in a single time series. If we were to duplicate all of that on each revision, we'll very quickly run out of disk space. Instead of trying to duplicate the time series, we record in the time series snapshot the range of values that existed in the time series at the time the revision was created.

Revisions from an older version of your software

One thing you should note when using the revisions feature is that over time, as your software evolves, you might see revisions from previous versions of your application. As such, the revision document might be missing properties or have properties that have been removed or had their type changed.

Because revisions are immutable, it isn't possible to run migration on them, and you need to take that into account. When working with revisions, you might want to consider working with the raw document, rather than turning it into an instance of an object in your model.

In most cases, time series are added to in a time based manner. So new values in the time series will be outside the range you have in the snapshot. In practice, it is fairly rare to want to deal with revisions on time series, and just being able to tell what the range at the revision creation time is sufficient. Having said that, it is important to understand that the time series data itself is *not* maintained in the revision, unlike counters and revisions.

3.7. Summary

In this chapter we went over some of the more interesting capabilities of RavenDB, beyond dealing with JSON documents. As much as I like JSON, it isn't always the best option to store your data, and RavenDB isn't limited just to dealing with documents.

The idea is that we can provide dedicated support for storing binary data, time series and distributed counters. Instead of you having to cobble together a solution, RavenDB has full fledged native support for them.

Attachments allows you to attach binary files to documents. RavenDB will store the data internally, managed de-duplication for the attachments so you'll only have a single copy of the data. You can also define retirement policy to move attachments from RavenDB out to object storage (S3 compatible) to reduce storage costs. Even after retiring the attachments to object storage, RavenDB will allow you to keep using the RavenDB API to access them. Transparently hiding the actual storage location of the attachment data and ensuring that you can have proper transactions composed of both files and documents.

Counters deal with the narrow task of counting values. They provide a cheap way to count values at high scale without worrying about concurrency or distributed state. Unlike document properties that require loading before updating, counters can be incremented directly through client API methods. This eliminates read-modify-write requirement and make it cheaper to work with counters.

Time series allow you to efficiently record changes to values over time. RavenDB can handle time

series with billions of entries and once stored, you can query, aggregate and filter the time series data easily. You can also define rollup and retentions policies to reduce the amount of data that you need to hold on to. Time series are a great fit for a number of domains, such as medical, finance and spatial.

Incremental time series combine the benefits of counters and time series, allowing you to concurrently increment values at specific points in time. RavenDB will then take care of dealing with both concurrency handling and dealing with distributed state so you'll end up with the right final tally without needing to concern yourselves with the details.

Finally, Revisions allow you to create immutable record of changes to your documents. This helps great when you *must* have a proper audit log of changes in your system, including proper verification that the data cannot be modified behind your back. Revisions are also useful to expose "looking back" features to your users, as well as implementing revert feature in your application.

In this chapter, we explored the more esoteric types of data that you can work with in RavenDB. The reasons we call RavenDB a multi model database, rather than just a document database. In the next chapter, we are going to move back to JSON documents and explore how best to model your data within RavenDB.

[1] Well, actually... documents in RavenDB are limited to about 2GB in size, and the largest record document that *I* personally saw in RavenDB was ~800MB. That is *not* recommended, see Chapter 4 for more details on that.

[2] You can have a document just to hold an attachment, of course

[3] Gorilla compression is a time series compression method from Facebook. See <https://www.vldb.org/pvldb/vol8/p1816-teller.pdf>(Gorilla: A Fast, Scalable, In-Memory Time Series Database paper) for more details.

Chapter 4. Document Modeling

Modeling your data is crucially important as different modeling decisions will have a different impact on your data and the amount of work that needs to be done by the database. It requires your understanding of how RavenDB, or any other database, is actually storing information and what features it has available for you to utilize.

If you get it wrong, you may be forcing the database to do an exponential amount of additional work. On the other hand, if you play to the database's strengths, you can get a robust and easy-to-build system. So, overall, no pressure whatsoever.

Relational-based modeling is so frequently encountered that, in most cases, you don't even see people talk about the relational part. They refer to relational data modeling as just "data modeling." But when you try to apply a relational modeling solution to a non-relational system, the end result is usually...suboptimal.

Documents aren't flat

Documents, unlike a row in a relational database, aren't flat. You aren't limited to just storing keys and values. Instead, you can store complex object graphs as a single document. That includes arrays, dictionaries and trees. Unlike a relational database, where a row can only contain simple values and more complex data structures need to be stored as relations, you don't need to work hard to map your data into a document database.

That gives you a major advantage. It simplifies a lot of common tasks, as you'll see in this chapter.

The problem is that this is extremely well-entrenched behavior, to the point where most people don't even realize they're making decisions based on what would work best for a relational database. So the first section of this chapter is going to deal with how to get away from the relational mindset, and the second part will focus on how to model data for RavenDB.

4.1. Beyond relational data modeling

You've likely used a relational database in the past. That means you've learned about normalization and how important it is. Words like "data integrity" are thrown around quite often. But the original purpose of normalization had everything to do with reducing duplication to the maximum extent.

A common example of normalization is addresses. Instead of storing a customer's address on every order that he has, we can simply store the address ID in the order, and we've saved ourselves the need to update the address in multiple locations. You can see a sample of such a schema in Figure 4.1.

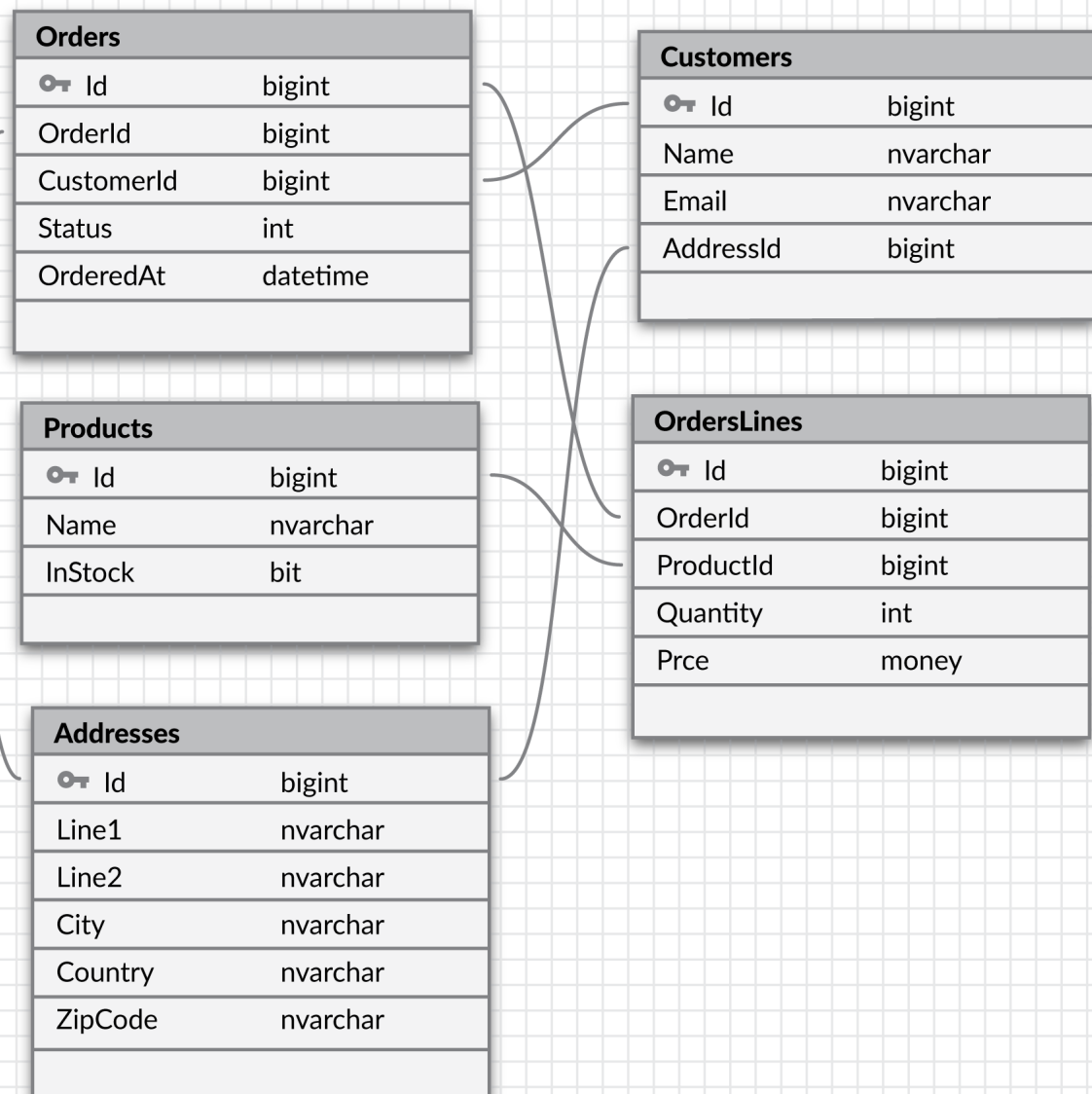


Figure 4.1. A simple relational schema for orders

You've seen (and probably written) such schemas before. And at a glance, you'll probably agree that this is a reasonable way to structure a database for order management. Now, let's explore what happens when the customer wishes to change his address. The way the database is set up, we can just update a single row in the addresses table. Great, we're done.

Well, it *would* be great...except we've just introduced a subtle but deadly data corruption to our database. If that customer had existing orders, both those orders and the customer information now point at the same address. Updating the address for the customer therefore will also update the address for *all of its orders*. When we look at one of those orders, we won't see the address it was shipped to but rather the *current* customer address.

When a data modeling error means calling the police

In the real world, I've seen such things happen with payroll systems and paystubs (or "payslips," across the pond). An employee had married and changed her bank account information to a new shared bank account with her husband. The couple also wanted to

purchase a home, so they applied for a mortgage. As part of that, they had to submit paystubs from the past several months. The employee asked her HR department to send over her most recent stubs. When the bank saw paystubs made to an account that didn't exist, they suspected fraud. The mortgage was denied and the police were called. An unpleasant situation all around.^[1]

It's common for this issue to be blamed on bad modeling decisions (and I agree). The problem is that a more appropriate model is complex to build and work with, expensive to query and hard to reason about in general.

A lot of the wisdom about data modeling is limited by only seeing the world through relational eyes. The relational model offers us tables, rows and columns to store our data, and it's up to us to hammer the data into the right shape so the relational database can accept it. Along the way, we have to deal with an impedance mismatch between how software (and our minds) model the data and how we're forced to store it to the relational database.

A document database like RavenDB doesn't solve this problem completely. It's entirely possible to construct models that would be a poor fit for the way RavenDB stores data. However, the way most business applications (and in general OLTP systems) think about their data is an excellent fit for RavenDB.

You can read more about this by looking at Domain Driven Design^[2] and, in particular, about the concept of an Aggregate Root.

What are aggregates?

One of the best definitions I've read is Martin Fowler [https://martinfowler.com/bliki/DDD_Aggregate.html]'s:

A DDD aggregate is a cluster of domain objects that can be treated as a single unit. An example may be an order and its line items. These will be separate objects, but it's useful to treat the order (together with its line items) as a single aggregate. An aggregate will have one of its component objects be the aggregate root. Any references from outside the aggregate should only go to the aggregate root. The root can thus ensure the integrity of the aggregate as a whole.

In the context of RavenDB, this is highly relevant since every RavenDB document is an aggregate and every aggregate is a document. Modeling techniques for aggregates work well for document-oriented design, and that gives you a great resource for modeling in the real world.

But before we can start running, we need to learn to walk. So let's start by learning how to use RavenDB with the most basic of modeling techniques: none.

4.2. Using RavenDB as a key/value store

RavenDB is a document database, but it's also quite good at being just a key/value store. That's mostly accidental, but as part of making sure we have a fast database, we also significantly reduced

the cost of storing and loading documents without any of the other benefits of RavenDB.

With the restriction that the data must be JSON, using RavenDB as a key/value store makes a lot of sense. It also makes sense to use RavenDB to cache information, to store the shopping cart during the purchase process or just to hold on to the user session data: all classic models for key/value stores.

RavenDB doesn't impose any additional costs on storing/loading documents by default, so you get to use a fast database with the simplest of all access models. Typically, the complexity in key/value modeling resides in generating the appropriate key and in what sort of operations you can do on it.

For the purpose of using RavenDB as a key/value store, it's likely the following features will be most relevant:

- You can generate the document identifier independently of the collection used.
- Loading saved document(s) can be done in a single call by providing the ID(s) to load.
- RavenDB provides automatic expiration for documents, which you can use with caching/session data.
- You can perform searches using document identifiers as a prefix (and even using glob-like searches).
- Includes can be used to fetch related data without having to make multiple remote calls.

The nice thing about using RavenDB as a key/value store for such information is that you aren't *limited* to just those key/value operations. If you're storing shopping cart data inside RavenDB, you can also start pulling data from the stored carts. You can see the most popular item being purchased or do projections on inventory or any of a whole host of things that will provide you with useful knowledge.

In a typical key/value store (Redis, for example), you'll have to manually track such things. However, RavenDB allows you to query and aggregate the data very easily.

But using RavenDB just for key/value data is somewhat of a waste, given what it's capable of doing with your data. So, without further ado, let's dive into document modeling in RavenDB.

4.3. Modeling considerations in RavenDB

The reason we cover modeling in this chapter is that I want you to get a feel for some of RavenDB's capabilities before we start discussing modeling techniques. The kind of features that RavenDB has to offer are directly relevant to the models you'll write.

RavenDB as the application database

A common bad practice is to integrate between different applications using a shared database. This has been looked down upon for quite a while. RavenDB makes no attempt to cater to those who use it as shared database. Instead, it shines in its role as an application database.

Typically, shared database solutions suffer because they attempt to force disparate applications to treat the data in the same manner. This is a pretty bad idea, since different applications tend to treat even very similar concepts in a variety of ways. The most common example is a billing application and a fulfillment application. Both have the concept of a customer, and both actually refer to the same thing when they talk about a customer. But the fulfillment application needs to keep track of very different data from the billing application. What will the fulfillment app do with the `CreditScore` field, and what can the billing app gain from looking at the `ShippingManifest` details?

Each application should model its data independently and evolve it as it needs to. Keeping everything in a shared database leads to required coordination between the various application teams. That results in increased complexity in evolving the data model as the various applications are being worked on.

However, even though the different applications are separate, they still need to share data between them. How do you do that with RavenDB? We'll talk about that a lot more in Chapter 15, but the answer is that each application has its own database, and we can set up information flow between the servers by using RavenDB's built-in ETL capabilities.

In this way, each application is independent and can evolve and grow on its own, with well-defined boundaries and integration points to its siblings.

We typically encourage you to use DDD techniques for modeling in RavenDB since they fit so well. A core design principle for modeling documents is that they should be independent, isolated and coherent, or more specifically,

- **Independent**, meaning a document should have its own separate existence from any other documents.
- **Isolated**, meaning a document can change independently from other documents.
- **Coherent**, meaning a document should be legible on its own, without referencing other documents.

These properties are easier to explain using negation. First, a document isn't independent if it can't stand up on its own. A good example of a dependent document is the `OrderLine`. An `OrderLine` doesn't really have a meaning outside the scope of an `Order`, so it should be a `Value Object` inside the `Order` document. An `Entity` is defined as something with a distinct and independent existence. And that's how you treat documents: as entities. If a document exists only as part of a larger whole, then the whole thing should be re-factored/re-modeled to a single document.

Second, a document isn't isolated if changing one document requires you to also update additional documents. For example, if updating the `CustomerBilling` document necessitates updating the `Customer` document, they aren't isolated. A document should be able to change independently from other documents. Otherwise, there's an unclear transaction boundary involved. With each document isolated, the transaction boundary (meaning what can change together) is much clearer. It's drawn at the document level.

Lastly, a document isn't coherent if it's not possible to make sense of it with just the information it contains. If you need to go and look up additional state or information from other documents, the

document isn't coherent. A good example is the `Order` document and the `ShippingMethod`. If, to ship an order, we need to go and look at the `Customer` document, then the `Order` document is not coherent.

4.3.1. Looking at physical documents

A great trick for document modeling is to consider the data as, well...documents. What I mean by that is *physical* documents: the kind you can actually hold in your hand and get a paper cut from. A lot of the same considerations that apply in the real world apply to document modeling.

If I hand you a document and tell you that you need to fill out *this* form and then go and update *that* form, you'll rightly consider the process a normal government behavior/bureaucratic/Kafkaesque.^[3] If I gave you a form and told you that, in order to understand it, you had to consult another document... you get my point.

When modeling, I find that it helps to picture the document in its printed form. If it makes sense as a printed page, it's probably valid in terms of document modeling.

4.3.2. Denormalization and other scary beasts

Anyone taking a relational modeling course knows that "store a fact only once" is a mantra that's just short of being sacred (or just plain sacred). The basic idea is that if you store a fact only a single time, you are preventing update anomalies, such as when you updated a person's date of birth in the employee record but forgot to update it on the "send a card" listing.

I wholeheartedly agree with this statement in principle. But the problem is that sometimes this isn't the same fact at all, even if it *looks* the same. Let's consider the notion of a `Customer`, an `Order` and what the `Address` property means in this case. The `Customer` entity has an `Address` property that holds the address of the customer, and the `Order` has a `ShipTo` property for the shipping address.

Why are we duplicating this information? Well, this isn't actually the same information, even if the content is the same. On the one hand, we have the `Customer.Address`, which represents the customer's *current* address. On the other hand, we have the `Order.ShipTo` which represents a *point-in-time* copy of the customer address, captured at the time we created the order. Those are important distinctions.

One of the more common objections to the kind of modeling advice found in this chapter is that the data is denormalized. And that's true. But for the most part, even if the same data appears in multiple locations, it doesn't have the same semantic meaning. And the notion of point-in-time data is important in many fields.

RavenDB has quite a few features that help in working with normalized data (`LoadDocument` and projections are the two main ones, and they're covered later in Chapter 12), but you need to consider whether it make sense to traverse document references in your model or whether you're breaking document coherency.

The most useful question you can ask yourself in this situation is whether you're looking at the *current* value (in which case you need normalization) or the *point-in-time* value (in which case you'd use a copy).

And with all of this information now in our heads, we can turn to concrete modeling scenarios and see how to deal with them.

4.4. Common modeling scenarios

Giving you advice on how to model your application is beyond the scope of this book. While I highly recommend the DDD book in general, it isn't always the best choice. Proper DDD modeling takes a lot of discipline and hard work, and it's most appropriate only in specific parts of your system (typically the most highly valued ones). RavenDB doesn't have any requirement regarding how you should model your data in your application. You can use DDD, you can use business objects or data transfer objects, it doesn't matter to RavenDB.

What matters is how you *structure* the data inside RavenDB, and that's what we'll be talking about for the rest of this chapter. We'll focus on concrete scenarios rather than generic modeling advice. Instead of trying to advise you on how to model your entire system, I'm going to focus on giving you the tools to build the model as you need it to be, so it will play best with RavenDB.

To make things interesting, we're going to use a kindergarten as our data model. As you can expect, we have the concepts of children, parents, registrations, etc. On the face of it, this is a pretty simple model. We can model it using the code in Listing 4.1.

Listing 4.1. Simple kindergarten model

```
public class Parent
{
    public string Name { get; set; }
}

public class Registration
{
    public DateTime EnrolledAt { get; set; }
    public EnrollmentType Type { get; set; }
}

public class Child
{
    public string Name { get; set; }
    public DateTime Birthday { get; set; }
    public Parent Father { get; set; }
    public Parent Mother { get; set; }
    public Registration Registration { get; set; }
}
```

The code in Listing 4.1 is obviously a simplified model, but it's a good place to start our discussion. A core tenet of modeling in RavenDB is that we need to identify what pieces of information belong together and what pieces are independent of one another. Recall our discussion on the basics of document modeling design. A good document model has documents that are independent, isolated and coherent.

With the basic model in Listing 4.1 and our understanding of a kindergarten, we can now proceed to explore aspects of document modeling. We'll start with the most obvious one. In this model, we don't have documents, we have *a* document: just a single one per child. That's because we're embedding all information about the child in the document itself. Why would we do that?

4.4.1. Embedded documents

A document model is very different from a relational model. It would be typical in a relational model to have a separate table for each class shown in Listing 4.1. A concrete example might work better, and Listing 4.2 shows how this model works for registering Alice to our kindergarten.

Listing 4.2. Kindergarten record of Alice in Wonderland

```
// children/alice-liddell
{
  "Name": "Alice Liddell",
  "Birthday": "2012-05-04T00:00:00.000000Z",
  "Mother": {
    "Name": "Lorina Hanna Liddell",
  },
  "Father": {
    "Name": "Henry Liddell"
  },
  "Registration": {
    "EnrolledAt": "2014-11-24T00:00:00.000000Z",
    "Type": "FullDay"
  }
}
```

All of the information is in one place. For the document ID you can see that we used a semantic ID, `children/alice-liddell`, which includes the child name. This works well for data that's well known and predictable. The data itself is centrally located and easy to access. Looking at the data, it's easy to see all relevant information about our Alice.^[4]

For the most part, embedding information is our default approach because it leads us to more coherent documents, which contain all the information relevant to processing them. We aren't limited by format of schema; we can represent arbitrarily complex data without any issue, and we want to take full advantage of that.

So if the default approach is to embed data, when *wouldn't* we want to do that? There are a few cases. One is when the data doesn't belong to the same document because it's owned by another entity. A good document model gives each document a single reason to change, and that's the primary force for splitting the document apart.

In the case of the kindergarten record, the obvious example here are the parents. Henry and Lorina are independent entities and are not fully owned by the Alice record. We need to split them into independent documents. On the other side of things, Henry and Lorina have more children than just Alice: there's also Harry and Edith.^[5] So we need to consider how to model such information.

4.4.2. Many-to-one relationship

How do we model a kindergarten where Alice, Harry and Edith are all the children of Henry and Lorina? The technical term for this relationship is *many to one*. Unlike the previous example, where we embedded the parents inside the child document, now we want to model the data so there's a distinct difference between the different entities. You can see the document model as JSON in Listing 4.3.

Listing 4.3. Many to one modeling with children and parents

```
// children/alice-liddell
{
  "Name": "Alice Liddell",
  "MotherId": "parents/1923-A",
  "FatherId": "parents/1921-A",
}

// children/henry-liddell
{
  "Name": "Henry Liddell",
  "MotherId": "parents/1923-A",
  "FatherId": "parents/1921-A",
}
```

Listing 4.3 shows ^[6] both Alice and Henry (you can figure out what Edith's document looks like on your own) with references to their parents. I've intentionally not included semantic IDs for the parents to avoid confusion about what information is stored on the side holding the reference. Alice and Henry (and Edith) only hold the *identifier* for their parents' documents, nothing else.

How is this model reflected in our code? Let's look at Listing 4.4 to see that (again, with some information redacted to make it easier to focus on the relevant parts).

Listing 4.4. Child class representing the model

```
public class Child
{
  public string Name { get; set; }
  public string FatherId { get; set; }
  public string MotherId { get; set; }
}
```

Instead of storing the parent as an embedded document, we just hold the ID to that parent. And when we need to traverse from the child document to the parent document, we do that by following the ID. To make things faster, we'll commonly use the *Include* feature to make sure that we load all those documents in one remote call, but that doesn't impact the data model we use.

What about the other side, when we need to find all of Lorina's children? We use a query, as shown in Listing 4.5.

Listing 4.5. Loading Lorina and her children

```
using (var session = store.OpenSession())
{
    var lorina = session.Load<Parent>("parents/1923-A");
    var lorinaChildren = (
        from c in session.Query<Child>()
        where c.MotherId == lorina.Id
        select c
    ).ToList();
}
```

You can also run this query directly in the Studio by going to Indexes and then Query and using the following query: `from Children where MotherId = 'parents/1923-A'`.

As you can see in Listing 4.4 and Listing 4.5, we're being very explicit when we move between documents. RavenDB doesn't allow you to transparently move between different documents. Each document is a standalone entity. This helps ensure you don't create silent dependencies in your model since each document is clearly delineated.

Using lazy operations

In Listing 4.5, you can see we have to make two separate calls to the server to get all the information we want. When going from a child to a parent, we can use the `Include` feature to reduce the number of calls. Going the other way, `Include` wouldn't work, but we don't have to give up. We have the option of making a few lazy operations and only going to the server once. We'll see exactly how this is possible in the next chapter.

The many-to-one relation is probably the simplest one, and it's incredibly common. However, when using it, you need to carefully consider whether the association should cross a document boundary or remain inside the document. In the case of parents and children, it's obvious that each is a separate entity, but orders and order lines are the reverse. In the case of orders and order lines, it's just as obvious that order lines do *not* belong in a separate document but should be part and parcel of the order document.

There are many cases where that distinction isn't quite so obvious, and you need to give it some thought. The decision can be situational and, frequently, it's highly dependent on the way you *use* the data. An equally valid decision in the kindergarten case would be to embed the parents' information in the child document and duplicate that information if we have two or more siblings in kindergarten at the same time.

It depends what kind of work is done with the parents. If all or nearly all the work is done with the children, there's no point in creating a parent document. (It isn't meaningful inside the domain of the kindergarten outside the scope of the children.) However, if we wanted to keep track of parents as well (for example, maybe we want to note that Mrs. Liddell takes two sugars in her tea), then we'll likely use a separate document.

Children and parents are all well and good, but what about when we ramp the example up a bit

and explore the world of grandchildren and grandparents? Now we need to talk about many-to-many relationships.

4.4.3. Many-to-many relationship

A many-to-many relationship is a lot more complex than a many-to-one relationship because it's usually used differently. In our kindergarten example, let's consider how we can model the grandparents. Each grandchild has multiple grandparents, and each grandparent can have multiple grandchildren.

When we were working with parents and children, it was obvious that we needed to place the association on the children. But how should we model grandparents? One way to do it would be to simply model the hierarchy. A grandparent is the parent of a parent. That seems like an elegant reflection of the relationship, but it will lead to a poor model. Grandparents and grandchildren have an association between them that's completely separate from the one going through the parent, and that deserves to be modeled on its own, not as a side effect.

The next question is where to put the relationship. We can add a `Grandparents` array to the child document, which will hold all the document IDs of the grandparents. We can add a `Grandchildren` array to the grandparent document and store the children IDs there. Or we can do both. What should we choose?

In the context of many-to-many associations, we always place the record of the association on the smaller side. In other words, since a child is likely to have fewer grandparents than a grandparent has children, the association should be kept on the child document.

The users and groups model

A more technical example that frequently comes up within the context of many-to-many associations is the users and groups model. A user can belong to many groups, and a group can have many users. How do we model this?

A user typically belongs to a few groups, but a group can have a *lot* of users. So we record the relationship on the smaller side by having a `Groups` property on the user document.

Traversing a many-to-many association from the grandchild (smaller) side can be done by simply including and loading the grandparents, as shown in Listing 4.6.

Listing 4.6. Alice and her grandparents

```
using (var session = store.OpenSession())
{
    Child alice = session
        .Include<Child>(c => c.Grandparents)
        .Load("children/alice-liddell");
    Dictionary<string, Parent> grandparents =
        session.Load<Parent>(alice.Grandparents);
}
```

Following the association from the other side requires us to query, as shown in Listing 4.7.

Listing 4.7. Grandchildren for a grandparent

```
using (var session = store.OpenSession())
{
    Parent grandparent = session.Load<Parent>("parent/1923-A");
    List<Child> grandchildren = (
        from c in session.Query<Child>()
        where c.Grandparents.Contain(grandparent.Id)
        select c
    ).ToList();
}
```

You can also run this query directly in the Studio by going to Indexes and then Query and using the following query: `from Children where Grandparents[] in ('parents/1923-A')`.

The code in Listing 4.7 will load Alice’s mother and all of her grandchildren. In this case, we see a slightly more complex query, but the essence of it remains the same as the one in Listing 4.5. We have separate documents and clear separation between them. In other words, we can query for related data, but we don’t just traverse the object graph from one document to another.

4.4.4. One-to-one relationship

A one-to-one relationship is a pretty strange sight. If there’s a one-to-one relationship, shouldn’t it be an embedded document instead of having a separate document? Indeed, in nearly all cases, that would be a better idea.

There are a few reasons why I’d want to store a part of a document in a separate document. Usually this would be the case if we have a document that’s conceptually the same but has very different access patterns. In the order example, we might have the order header, which is frequently accessed and looked at. And then there’s the full order, which might be very big (lots of line items) and which we don’t need to access often.

In this case, it might make sense to create a document just for the order header (call it `orders/2834/header`). But using a projection will be almost as good, and we’ll discuss those in the next chapter. This saves us the need to split our data. The typical way you’ll build one-to-one relationships in RavenDB is to utilize document ID postfixes to create a set of related documents (`orders/2834` and `orders/2834/header`, for example).

This tends to result in clearer intent since it’s obvious what each part is doing and what it’s meant to be. But even so, it’s often not advisable. Just putting it all in a single document is easier in most cases.

Another scenario that might require you to split a document is if you have a real need to have concurrent activities on a document. This is generally pretty rare. Since a document is the unit of concurrency in RavenDB, you’ll model your documents so they’ll have a single reason to change. But sometimes there are reasons to allow concurrent updates on a document. For example, if you have properties on the document that are useful for the application, not the business model.

Case in point, imagine in an order system that we want to add a feature for tracking an order. A customer representative may mark an order for tracking so they can see what kind of steps a particular order goes through and take action at various points in the workflow. Such an action needs to be recorded, but it doesn't actually impact the behavior of the system in any way. It is fine to add or remove tracking on an order at any point in time, including during concurrent updates to the order.

One way to do that would be to always use patching (also discussed in the next chapter) to update the document, but a better way to handle this kind of requirement is to create a dedicated document `orders/2834/tracking` that would hold all the details about the users tracking this order. This makes it an explicit action in your domain instead of just tacking it onto the existing objects.

4.5. Advanced modeling scenarios

The RavenDB modeling techniques that we've explored so far are good for modeling standard business data. You know how to build your entities, how to deal with relationships between them and how to identify whether they should be separate documents or nested in the same document. But there's also a good deal of sophistication we can apply to non-standard scenarios, which is the topic of this section.

4.5.1. Reference data

Reference data is common, and it can be anything from a list of states to tax rates to localized text. The common thread for reference data is that it's typically small and not really interesting in isolation. Such data only gets interesting when you start working with a lot of it.

It's typical to ignore the modeling concerns for such items. Just throw them in a pile somewhere and not give it any thought. With the list of states example, the most natural way to do it is to define a document whose ID is `states/ny` and has a single `Name` property with the value `New York`. That would *work*, but it's hardly the best way to go about it.

Going back to basic principles, such a document will hardly be coherent or independent. Indeed, such a document makes little sense on its own. Instead of storing each state and each configuration value as its own document, we'll raise the bar a bit and introduce configuration documents. You can see the example in Listing 4.8.

Listing 4.8. Storing the states list as a configuration document

```
// config/states
{
  "AL": "Alabama",
  "AK": "Alaska",
  "AS": "American Samoa",
  "..redacted..": "...",
  "WV": "West Virginia",
  "WI": "Wisconsin",
  "WY": "Wyoming"
}
```

Modeling reference data in the manner shown in Listing 4.8 has several advantages. It's much easier to work with the data. Instead of issuing a query, we can just load the document in one shot, and it's ready for our consumption. It means the database has less work to do, and it plays nicely into the way RavenDB caches data. It also means we can reduce deserialization costs and make it easier to edit and work with the reference data.

In fact, because this is a single document, we can also get RavenDB to do some tasks for us, such as the build-in revisions capability. We'll discuss this in more detail in the next chapter. This can be very useful when you're talking about reference or configuration data since it's easy to see what the changes were (or revert them).

4.5.2. Hierarchical information

Working with hierarchical data is complex because there are cases where you need to traverse the hierarchy, and that traditionally has been expensive. In many cases, hierarchies are recursive and have no limits to their number of levels (although, in practice, the number is usually known to be small).

We need to make a distinction between several types of hierarchies. The first hierarchy is quite simple and can comfortably fit into a single document. An example of such a hierarchy is comments in a discussion thread. The entire thread is a single document, and all the comments in the thread always reside in that document. In such a case, storing and working with the hierarchical nature of the data is quite trivial since you'll often just traverse the data directly after loading the document from the server.

A commonly-seen example of a hierarchy that doesn't fit the "everything in a single document" model is the company hierarchy. Each employee is an independent entity, and we can't store them all as a single document. Instead, we'll strive to model the hierarchy explicitly as an independent concept from the employees. In this case, we'll have the notion of a department, which will record just the chains of who reports to whom. The idea is that we separate the hierarchy out since the position of an employee in the hierarchy is orthogonal to most aspects of the employee.

The notion of a separated hierarchy document gives us a lot of simplicity. Hierarchical operations and queries (all direct and indirect reports, for example) are easy and natural, and the separated hierarchy document works well with caching and the use of Include.

The final example for modeling hierarchies in RavenDB is when we need to model the hierarchy directly in the model. Departments in an organization might be a good example; their location in the hierarchy is important to what they are. In this case, we'll typically model such a relationship as many to one or many to many, at each individual level. That's the simplest method for handling such a requirement.

This works as long as we only need to handle a single level, but it doesn't handle hierarchical queries. In the sample dataset (the Northwind we created in Chapter 2), we have the Employees collection, which has a hierarchical relationship. Each employee has a ReportsTo property, which holds the ID of the manager for that employee. You can see in Listing 4.9 how we use declare function to walk through the hierarchy and build a list of all the bosses for each employee.


```
declare function reportingHierarchy(bossId) {
  const bosses = [];
  while(bossId){
    const e = load(bossId);
    bosses.push(e.FirstName + " " + e.LastName);
    bossId = e.ReportsTo;
  }
  return bosses;
}
from "Employees" as e
select {
  Name: e.FirstName + " " + e.LastName,
  Bosses: reportingHierarchy(e.ReportsTo)
}
```

Listing 4.9 uses a number of features that we haven't talked about until now. We use `declare function` to create a JavaScript function that we can call from the query. This function uses `load` to load documents by their ID. In the `select`, we are using JavaScript notation and can call perform computation (concatenation of the employee's name) or call the previous defined function.

Notice that we are querying from the bottom *up*, so we can walk the hierarchy. If we wanted to go from the top down, we'd need to do something else. Asking for all the employees that report to Andrew Fuller (directly or indirectly) is a bit more complex. We'll need to define an index that is able to walk through the hierarchy. We haven't talked about complex operations in indexes, so I'll just mention that RavenDB's indexes can use `Recurse` to work with hierarchical data and leave that topic to Chapter 13, where we'll cover it in depth.

4.5.3. Temporal data model

Temporal data is often a challenge because it can really mess with the way you think about information. Despite the fancy name, temporal data is just a way to store data that has a relation to time. The best example I've seen for temporal data is payroll. Consider the notion of a paycheck. An employer and employee have a contract stipulating that, for a given amount of work, a given amount of money will be exchanged.

The problem is that this contract can *change* over time. An employee can get a raise, earn additional vacation days, negotiate better overtime terms or all of the above. For extra fun, you may get some of those changes retroactively. This sort of thing makes it hard to figure out what exactly you're supposed to do. (What if these terms change mid-month? How would you handle overtime calculation on a shift between 8 PM and 4 AM that falls in that switch?)

Temporal data vs. time-series

In the previous chapter, we talked about time-series data. Temporal data is different from time-series data. Time-series records the changes in a value over time. Temporal data is data that has a relation to time, but it isn't necessarily a sequence of values. The classic examples

for each is tracking the changes in a stock's price over time (time-series) vs. the terms of a contract (temporal).

The way to model temporal data in RavenDB is to embrace its document nature fully, especially because in most temporal domains the notion of documents and views over time is so important. Consider this situation: a paystub was issued on May 1st, and then a retroactive pay raise was given. How is that money counted? It's easy to see that when we model the data as physical documents, we don't try to model a paystub as a mutable entity but rather a point-in-time view. Any changes that were made during the time frame it covered will be reflected in the *next* paystub.

This approach makes it much easier on you. You don't have to keep track of valid time, effective time and bitemporal time all at once. You just store facts and the time at which they were stored, just as if you were keeping all your printed paystubs in a drawer somewhere.

The same applies to contracts and other things that mutate over time. Consider the documents seen in Figure 4.2. They represent the same contract, with modifications over time as things change.

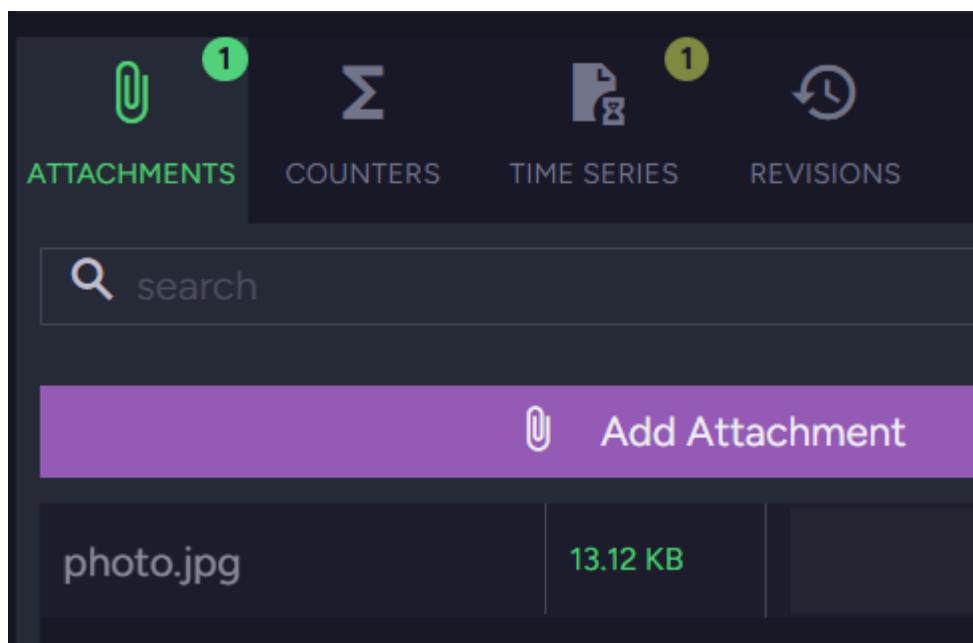


Figure 4.2. Contract documents and changes over time

The key is that, when we consider references to this contract, we can select what *kind* of reference we want. When looking at the employee's record, we'll have a reference to `contracts/hourly/1234-A`, which is the current version of the contract. But when issuing a paystub, we'll always reference a fixed revision of this contract, such as `contracts/hourly/1234-A/2013-05-21`. This way, we set ourselves up to choose whether we want the point-in-time information or the current (continuously updated) version.

If this sounds similar to the way we decide if we'll copy data from another document to get a point-in-time snapshot of it or reference it by ID to always get the latest data, that's probably because it is similar. And it makes dealing with temporal data significantly simpler.

4.6. Additional considerations for document modeling

A major challenge in modeling data with RavenDB is that different features and behaviors all have a very different impact on the cost of various operations, and that in turn impacts how you'll design and work with your model. In this section, we'll explore a few of the features that we haven't gotten to know yet and their impact on modeling data in RavenDB.

I recommend going over this section briefly now and returning to it when you've finished this book. By then, you'll have the complete picture and will be able to make more informed decisions.

4.6.1. Handling unbounded document growth

What size should a document be? It's an interesting question that frequently comes up with regard to modeling. A good range for a document is somewhere on the order of kilobytes. In other words, if your document is very small (a few dozen bytes), is it really a document? Or is it just a single value that would be better off as a reference data document, as we discussed earlier in this chapter? On the other hand, if your document is multiple megabytes, how easy it is to work with it?

Maximum document size

RavenDB has a hard limit on document sizes, and it's around the 2 GB mark. (Documents are internally compressed by RavenDB, so a 2 GB JSON document will typically be smaller inside RavenDB.) But the biggest document I recall seeing in the field was about 800 MB. We had a discussion with the customer regarding modeling and about the fact that, while RavenDB is a document database, it's perfectly capable of handling *multiple* documents.

RavenDB itself, by the way, is just fine with large documents. The problem is everything else. Consider a page that needs to display a 12 MB document. What are the costs involved in doing this?

- Reading 12 MB over the network.
- Parsing 12 MB of JSON into domain entities (this often means using a lot more memory than just the 12 MB of serialized JSON).
- Generating a web page with some or all of the information in the 12 MB document.
- Discarding all of this information and letting it be garbage-collected.

In short, there's a *lot* of work going on, and it can be very expensive. On the other hand, if you have anemic documents, we'll typically need to read many of them to get anything meaningful done, which means a lot of work for the database engine.

Here's the rule of thumb we use: as long as it makes sense to measure the document size in kilobytes, you're good. RavenDB will generate a warning in the Studio when it encounters documents that are too large,^[7] but it has no impact whatsoever on the behavior or performance of the system.

There are two common reasons why a document can be very large. Either it holds a single (or a few) very large properties, such as a lot of text, binary data, etc., or it contains a collection whose size is not bounded.

In the first case, we need to consider whether the large amount of data is actually a core part of the document or if it should be stored externally. RavenDB supports the notion of attachments, which allows you to, well, *attach* binary data to a document. If the data can be stored outside the document, regardless of whether it's binary or textual, that's preferable. Attachments in RavenDB don't have a size limit and aren't accessed when loading the document, so that's a net win. We already talked about attachments in Chapter 3 and we'll discuss the role of attachment in document modeling toward the end of this chapter.

The more complex case is if we have a collection or collections inside the document that have no upper bound. Consider the case of an order document. In retail, an order can contain up to a few hundred items, but in business-to-business scenarios, it's not uncommon to have orders that contain tens and hundreds of thousands of items. Putting all of that in a single document is going to lead to problems, as we've already seen.

Who modifies the big documents?

While big documents are typically frowned upon because of the awkwardness of working with large amounts of data (network, parsing and memory costs) there's another, more theoretical aspect to the problems they bring: who *owns* a very big document?

Remember that good document modeling implies there's only a single reason to change a document. But if an order document contains so many items, that likely means there are multiple sources that add items to the order, such as different departments working with the same supplier. At that point, it isn't a single document we're working with. It's multiple independent orders merged into a single, final shipping/billing order.

Again, falling back to the real-world example, we'd find it strange if a van stopped by our store and started unpacking crates full of pages detailing all the things a customer wanted to purchase. But when we're working on a single document, this is pretty much what we're doing. Instead, we'll typically accept individual requests from each contact person/department and associate them with the appropriate billing/shipping cycle.

In other words, the *reason* we split the document isn't so much to artificially reduce its size but because that's how the business will typically work, and doing it any other way is *really* hard. Splitting it so the document structure follows the structure of the business is usually the right thing to do.

So how do we handle large documents? We typically split them into smaller pieces along some sort of business boundary. Let's consider the case of a bank account and the transactions we need to track. Keeping all the transactions inside the account document isn't feasible on a long-term basis. But how are we going to split them? Here, we look at the business and see that the bank itself doesn't look at the account and all its history as a single unit.

Instead, the bank talks about transactions that happened on a particular business day or in a particular month. And we can model our documents accordingly. In the bank example, we'll end up with the following documents:

- accounts/1234: account details, status, ownership, conditions, etc.

- `accounts/1234/txs/2017-05`: the transactions in this account on May 2017
- `accounts/1234/txs/2017-04`: the transactions in this account on April 2017

Within a given time frame, we can be reasonably certain that there's going to be an upper bound to the number of transactions in an account. I quite like this approach because it aligns closely with how the business is thinking about the domain and it results in clear separation of documents. But you can't always split things apart on such a convenient boundary. What happens if there isn't a time element that you can use?

At that point, you still split the document, but you do so arbitrarily. You can decide that you'll generate a new document for the items in the collection every *N* items, for example. Often, that *N* value is some multiple of the typical page size you have in your system. If we go back to the example of order and items in the order, assuming we don't go the route of splitting the order itself into separate requests, we'll just split the order every 100 items. So, we'll end up with the following documents:

- `orders/1234`: the order header, shipping details, billing, etc.
- `orders/1234/items/1`: the first 100 items for this order
- `orders/1234/items/2`: the second 100 items for this order

The order document will contain the IDs of all the items documents (so we can easily include them), as well as the ID of the last items document, so we'll know where to add an item to this order.

This approach is a bit clumsier, in my eyes. But if we don't have another alternative natural boundary, this will work fine.

4.6.2. Cached queries properties

A common modeling technique is to keep track of some cached global state inside our entities. The most trivial example I can think of is the `customer.NumberOfOrders` property. I want to call out this behavior specifically because it's common, it makes sense and it's usually a bad idea.

But why? Well, let's consider why we need to have this property in the first place. In most domains, a property such as `NumberOfOrders` doesn't comprise an intrinsic part of the customer. In other words, the information is interesting, but it doesn't *belong* inside the customer document. Moreover, in order to keep this property up to date, we need to update it whenever we add an order, decrement it when an order is deleted, etc. For that matter, what do we do about an order that was returned? Does it count? What about the replacement order?

In other words, this is a pretty hefty business decision. But most of the time, this property is added because we want to show that value in the user interface, and issuing an aggregation query can be expensive. Or, rather, an aggregation query is expensive if you aren't using RavenDB.

With RavenDB, aggregation is handled via MapReduce operations, and you don't pay any cost at all for querying on the results of the aggregation. We dedicated the whole of Chapter 14 to discussing this topic, so I'll go into the details there, but the impact of such a feature is profound. Cheap aggregation means that you can do a lot more of it, and you don't have to manage it yourself.

That frees you from a lot of worries, and it means you can focus on building your model as it's used.

You can let RavenDB handle the side channel operations, like calculating exactly how many orders a customer has and in what state.

Another problem with `customer.NumberOfOrders` is how you update it. Two orders generated at once for the same customer may result in lost updates and invalid data. How does RavenDB handle concurrency, and how does it impact your model?

4.6.3. Concurrency control

RavenDB is inherently concurrent, capable of handling hundreds of thousands(!)^[8] of requests per second.

That leads to a set of interesting problems regarding concurrency. What happens if two requests are trying to modify the same document at the same time? That depends on what, exactly, you asked RavenDB to do. If you didn't do anything, RavenDB will execute those two modifications one at a time, and the last one will win. There's no way to control which would be last. Note that *both* operations will execute.

That means that in certain cases, it will leave marks. Consider two concurrent update to the `children/alice` For example, consider the `Child` document from our kindergarten model. Suppose two users concurrently updates to the Alice record from Listing 4.2. One transaction is updating the `Name` property to "Alice Kingsley" as well as the `Registration.Type` to `HalfDay` while the other transaction is updating the `Birthday` property to "2022-06-01" and the `Name` to Alice K. Liddell.

You won't usually see this behavior in practice, though.

Even when you run into concurrent updates, the idiomatic manner to update documents in RavenDB updates the *entire* document, so you'll not see partial modification of the document. If you'll use patching to update the document (discussed in [ravnedb-client-api]), you might get this behavior. However, typically the only visible mark of the last write wins strategy is when you have revisions.

If the first transaction happens to be the last one to save, the resulting document will look like Listing 4.10.

Listing 4.10. Alice record after concurrent updates

```
{
  "Name": "Alice Kingsley",
  "Birthday": "2022-06-01",
  "Registration": {
    "Type": "HalfDay",
    "StartDate": "2017-09-01"
  }
}
```

In other words, the `Name` property was modified by both transactions, and the last one to save won. The `Birthday` and the `Registration.Type` properties were modified by only one transaction, so they

were preserved.

Change Vectors in RavenDB

RavenDB makes extensive use of the notion of change vectors. A change vector is composed of a list of node IDs and etags. A node ID uniquely identifies a node, while an etag is a 64 bit number that is incremented on every operation in a database. Each time a document is modified, it's associated with the current etag value and a new change vector is generated.

This change vector uniquely marks the specific version of the document globally and is used for optimistic concurrency control, for various internal operations and caching. We'll discuss change vectors in detail in the Chapter 9, since they are crucial to understanding how RavenDB handles data replication.

The last write wins model is the default option in RavenDB except when you're creating a new document. When the RavenDB client API is generating the ID, it *knows* it intends to create a new document, and it sets the expected change vector accordingly. So if the document already exists, an error will be generated.

What about when you want to take advantage of this feature? You can ask RavenDB to enable optimistic concurrency at several levels. The following code enables optimistic concurrency for all sessions created by this document store.

```
store.Conventions.UseOptimisticConcurrency = true;
```

And you can also enable this on a more selective basis, on a particular session, using the following snippet:

```
session.Advanced.UseOptimisticConcurrency = true;
```

Or you can enable it on a single particular operation:

```
session.Store(entity, changeVector);
```

In all cases, the RavenDB client API will send the expected change vector to the server, which will compare it to the current change vector. If the current change vector does not match the expected change vector, a `ConcurrencyException` will be thrown, aborting the entire transaction.

The first two options use the change vector that the server supplied when the document was loaded. In other words, they would error if the document was modified between the time we fetched it from the server and the time we wrote it back. The third option is a lot more interesting, though.

The ability to specify the change vector on a specific entity is quite important because it allows you to perform *offline* optimistic concurrency checks. You can render a web page to the user and, along

with the document data there, include the change vector that you got when you loaded that document. The user can look at the document, modify it, and then send it back to your application. At that point, you'll store the document with the change vector as it was when we rendered the page to the user and call `SaveChanges`. If the document has been modified by anyone in the meantime, the operation will fail.

In this manner, you don't need to keep the session around. You just need to pass the change vector back and forth. As you can see, the change vector is quite important for concurrency, but it also plays another important role with caching.

4.6.4. Caching

RavenDB uses REST over HTTP for communication between client and server. That means whenever you load a document or perform a query, you're actually sending an HTTP call over the wire. This lets RavenDB take advantage of the nature of HTTP to play some nice tricks.

In particular, the RavenDB client API is capable of using the nature of HTTP to enable transparent caching of all requests from the server, which provides the client API with some assistance. Here's how it works. Each response from the server contains an ETag header. In the case of loading a single document, that ETag header is also the document's change vector, but if we're querying or loading multiple documents, that ETag header will contain a hash of the current server's state and the relevant documents.

On the client side, we'll cache the request from the server alongside the URL and the ETag value. When a new request comes in for the same URL, we'll send it to server, but we'll also let the server know we have the result of the request with a specific ETag value. On the server side, there are dedicated code paths that can cheaply check if the ETag we have on the client is the same as the current one. If it is, we'll just let the client know that it can use the cached version, and that's it.

We don't have to execute any further code or send anything other than 304 Not Modified to the client. And on the client side, there's no need to download any data from the server. We can access our local copy, already parsed and primed for us. That can represent significant speed savings in some cases because we have to do a lot less work.

If the data has changed, we'll have a different ETag for the request, and the server will process it normally. The client will replace the cached value and the process will repeat. You can take advantage of that when designing your software because you know certain operations are likely to be cached and thus cheap.

RavenDB also has a feature called Aggressive Caching, which allows the RavenDB client API to register for changes from the server. At that point, we don't even need to make a call to the server if we already have a cached value. We can wait for the server to let us know that something has changed and that we need to call back to the database to see what that change was. For certain types of data (config/reference documents in particular, like the config/states we saw in Listing 4.8) that can provide major performance savings.

4.6.5. Reference handling, Include and LoadDocument

Earlier in this chapter, we looked at various types of relationships between objects and documents,

like embedding a value inside a larger document, many-to-one and many-to-many relationships and more. While we were focused on building our model, I skipped a bit on explaining how we work with relationships beyond the diagram stage. I'd like to return to that now.

There are two separate operations that we need to take into account. Fetching related information during a query or a document load can be done using an `Include` call, which we already covered in Chapter 2. I'm pointing this out again because it's a very important feature that deserves more attention (and use). It's an incredibly powerful tool to reduce the number of server requests when working with complex data.

The second operation is querying, or more to the point, when I want to query for a particular document based on the properties of a related document. Going back to the kindergarten example, searching for all the children whose mothers' names are Lorina would be a great example. The child document no longer contains the name of the parent, so how are we going to search for that? RavenDB doesn't allow you to perform joins, but it does allow you to index related data during the indexing phase and then query on that. So querying the children by their mothers' names is quite easy to do, using the `LoadDocument` feature. `LoadDocument` is discussed in full in Chapter 13.

I'm mentioning it here because knowing that it exists has an impact on the way we model data. Be sure to read all about what `LoadDocument` can do and how it works. It can be of great help when deciding how to model related data. In the end, though, the decision almost always comes down to whether we want a point-in-time view of the data or the current value. For the first option, we'll typically copy the value from the source to its own document, and for the second, we can use `LoadDocument` during indexing and querying and `Include` when fetching the data from the server.

4.6.6. Attachments and binary data

RavenDB is a JSON document database, but not all data can be (or should be) represented in JSON. This is especially true when you consider that a document might reasonably include binary data. For example, when working with an order document, the invoice PDF might be something we need to keep.^[9] If we have a user document, the profile picture is another piece of binary data that is both part of the document and separate from it.

We discussed attachments previous in Chapter 3, I'm touching the topic here as well because it also has important implications for modeling. Attachments are binary data that can be attached to documents. An attachment is always associated with a document, and beyond the binary data, there's an attachment name (which must be unique within the scope of the parent document). Attachments are kept on the same storage as the documents (but can be set up to retire to S3 in a transparent manner), and can be worked on together with documents.

That means attachments can participate in the same transaction as documents with the same ACID semantics (all in or none at all). There's no size limit for attachments, and a single document can have multiple attachments.

The ability to store binary data easily is important for our modeling, especially because we can treat attachment changes as part of our transaction. That means an operation such as "the lease is signed" can be done in a single transaction that includes both the update of the Lease document and the storage of the signed lease scan in the same operation. That can save you a bit of a headache with error handling.

When modeling, consider which external data is strongly related to the document and should be stored as an attachment. The easiest mental model I have for doing this is to consider attachments in email. Imagine that the document is the email content, and the attachments are just like attachments in email. Typically, such attachments provide additional information about the topic in question, and that's a good use case for attachments in RavenDB.

4.6.7. Revisions and Auditing

Tracking changes in data over time is a challenging task. Depending on the domain in question,^[10] we need to be able to show all changes that happened to a document. RavenDB supports that with its revisions feature and we discussed that in Chapter 3.

The database administrator can configure RavenDB to keep track of document revisions. Whenever a document is changed, an immutable revision will be created, which can be used later on to follow all changes that happened to a document. RavenDB can be set to track only specific collections and to only keep track of the last N revisions, but often you'll choose "track everything" since disk space is relatively cheap and those domains need this accountability. It's better to keep too much than not enough.

Revisions also can apply to deletes, so you can restore a deleted document if you need to. One way of looking at revisions is as a way to have a copy of all changes on a per-document level.^[11]

Auditing is a *bit* different. While revisions tell you *what* changed, auditing tells you by *whom* and typically what *for*. RavenDB supports this kind of auditing using client-side listeners, which can provide additional context for the document whenever it's changed. We'll discuss listeners and their use in more depth in [ravnedb-client-api].

4.6.8. Expiration

When storing documents inside RavenDB, you can specify that they will expire at a given point in time. RavenDB will periodically remove all the expired documents automatically. This is a relatively small feature, but it's nice to be able to implement functionality like "the link in this email will expire in 12 hours."

We'll see how to use the expiration feature in [ravnedb-client-api]. There isn't much to say about it besides that it works, but it should be noted that the expiration is a soft one. In other words, if you specified an expiration time, the document might still be there after that time has passed because RavenDB didn't get around to cleaning it yet. By default, RavenDB will clear expired documents every minute. So the document might live just a bit longer than expected, but not terribly so.

4.7. ACID vs. BASE

The final topic in the modeling chapter is one of the more important ones. ACID stands for atomic, consistent, isolated and durable, while BASE stands for basically available, soft state, eventually consistent. Those are two very different approaches for dealing with data.

RavenDB uses both, in different parts of its operations. In general, ACID is what we always strive for. A fully consistent model makes it easy to build upon and reason about, but it also can be quite expensive to build and maintain. In a distributed system, ensuring atomicity can be quite

expensive, since you need to talk to multiple machines for each operation, for example. On the other hand, BASE give us a lot more freedom, and that translates into a lot more optimization opportunities.

In RavenDB, all operations on a document or attachment using its ID (put, modify, delete) are always consistent and run in an ACID transaction. Bulk operations over sets of documents (update all that match a particular query, bulk insert, etc.) are composed of multiple separate transactions instead of one very big one.

By default, when you save a document into RavenDB, we'll acknowledge the write when the data has been saved on one node in a durable fashion.^[12] You can also ask the write to be acknowledged only when the write has been made durable on multiple nodes (using cluster-wide transactions or `WaitForReplicationAfterSaveChanges` method) for additional safety. We discuss all such details in Chapter 9.

One of the typical balancing acts that database administrators have to do is to choose how many indexes they will have. Too many indexes and the write process grinds to a halt. Not enough indexes and your queries are going to require table scans, which are *horrendously* expensive even on moderately sized databases.

The reason for the tradeoff is that a transaction must update all the relevant indexes on every change. That means that index updates are right there in the main pathway for updating data, which explains why they can so severely degrade performance. When designing our indexing implementation, we took a different approach.

Indexing in RavenDB is handled as an asynchronous task running in the background whenever there are updates to the database. This means that we don't have to wait for the index to finish updating before completing the write. And that means that we have far better opportunities for optimizations. For example, we can roll up multiple changes that happened in different transactions into the same index update batch.

This also allows us to be able to prioritize certain operations on the fly. If we're under a load of pressure right now, we can reduce the amount of time we spend indexing in order to serve more requests. This follows the idea that we always want to be able to return a result from RavenDB as soon as possible and that, in many cases, a triple-checked answer that came too late is worse than useless.

Indexes in RavenDB are BASE in the sense that they can lag behind the documents they reflect. In practice, indexes are kept up to date and the lag time between a document update and an index update is typically measured in microseconds. The BASE nature of indexes allows us to achieve a number of desirable properties. Adding an index to the system doesn't block anything, and updating an index definition can be done in a side-by-side manner. Various optimizations are possible because of this (mostly related to batching and avoidance of locks).

What are the costs in waiting

Waiting for indexing, cluster-wide transaction or waiting for replication is something that you have to explicitly ask for because it costs. But what *are* those costs? In the case of cluster-write transactions or replication, it means waiting for another server (or servers) to get the

information and persist it safely to disk. That can add latency in the order of tens of milliseconds when running on a local data center and hundreds of milliseconds if your cluster is going over the public internet.

For indexing, this is usually a short amount of time since the indexes are updated quickly. If there's a new index, it may take longer because the write will have to wait for the index to catch up not just with this write but also with all the writes that the new index is processing.

We typically measure RavenDB performance in tens or hundreds of thousands of requests per second. Even going with just 10,000 requests per second, that gives us a budget of 1/10 of a millisecond to process a request. Inside RavenDB, requests are handled in an async manner, so it isn't as if we're holding hundreds of threads open. But additional wait time for the request is something we want to avoid.

For the most part, those costs are pretty small, but there are times where they might be higher. If there's a new index running, and we asked to wait for all indexes, we may be waiting for a while. And we want the default behavior to be as predictable as possible. That isn't meant to discourage you from using these features, but we suggest that you give it some thought.

In particular, spraying the waits at any call is possible and it will work, but it's usually not the best solution. You typically want to apply them in specific scenarios: either high-value writes (and you want to make sure that it's saved on multiple nodes) or if you intend to issue a query immediately after the write and want to ensure that it will be included in the results. Otherwise, you should accept the defaults and allow RavenDB the freedom to optimize its operations.

The BASE on indexing behavior is **optional**. You can ask RavenDB to wait for the indexes to update (using the `WaitForIndexesAfterSaveChanges` method) so you have the option of choosing. If this is a write you intend to immediately query, you can force a wait. Usually, that isn't required, so you don't need to pay the cost here. You can also ask RavenDB to wait for the indexes during *queries* (using `WaitForNonStaleResults` method).

The overall idea is that you have the choice to decide for specific operations how you want to handle the tradeoff between waiting and consistency. You need to be aware of the distinctions here between queries (which can be BASE and lag a bit ^[13]) and bulk operations (many small transactions) and operations on specific documents (either one at a time or in groups), which happen as ACID transactions. Taking advantage of the BASE nature of indexing allows you to reduce the cost of querying and writing, as well as selectively apply the decision on a case-by-case basis.

4.8. Summary

Modeling data in RavenDB isn't a trivial topic, and it isn't over just because you finished this chapter. I tried to highlight certain features and their impact on modeling, but I suggest finishing the book and then reading this chapter again, with fresh perspective and a better understanding of how things are playing together.

We started by looking at RavenDB as a simple key/value store. Just throw data in (such as session

information, shopping cart, etc.) and access it by a well-known key. That has the advantage of giving us fast operations without hiding the data behind an opaque barrier. We can also use the expiration feature to store time dependent data.

From the trivial model of key/value store, we moved to considering real document models. By looking at how documents in the real world (physical ones, printed on actual paper) are modeled, we made it much easier to explain document models to the business. Reducing the impedance mismatch between the business and the application is always a good thing, and proper modeling can help there. Of course, don't go too far. Remember that documents are virtual concepts and aren't limited to the physical world. We then looked at the issue of denormalization and noted the difference between the same *value* of the data and the same *meaning* of the data.

The next topic was actual modeling. We looked at how we should structure embedded values to store data that's intrinsically part of the document instead of holding that data outside of the document. We looked at how to model relations and collections, many-to-one and many-to-many associations. Then we covered more advanced modeling techniques, such as how to handle references and configuration data and how to deal with temporal information and hierarchical structures.

Next, we went over some of the constraints that we have to take into account when modeling, like how to deal with the growth of a document and what constitutes a good size for a document in RavenDB. We looked at concurrency control and how change vectors are useful for optimistic concurrency and for caching. And we examined why we should avoid caching aggregated data in our model (NumberOfOrders).

We looked at handling binary data with attachments, as well as auditing and change tracking using the revisions feature, and we learned that we can expire documents natively in RavenDB. Reference handling at indexing and query time was briefly covered (we'll cover it in depth in Chapter 13), as it's important to how you model your documents.

A repeating theme was the use of semantic IDs to give you better control over documents and their meaning. Not so much for RavenDB's sake footnote:[The database doesn't care what your document IDs look like.] but because it increases understanding and visibility in your domain. Using document IDs to "nest" documents such as accounts/1234/tx/2017-05 or having meaningful document IDs such as config/states helps a lot in setting out the structure for your model.

The final topic we covered was ACID vs. BASE in RavenDB. Documents in RavenDB are stored and accessed in an ACID manner, while we default to BASE queries to get higher performance and have more chances for optimizations. This behavior is controlled by the user on a case-by-case basis, so you can select the appropriate mode for each scenario.

Our next chapter is going to cover the client API in depth, going over all the tools that you have to create some really awesome behaviors. After that, we'll get to running clusters of RavenDB and understanding how the distributed portion of RavenDB is handled.

[1] This ended up being sorted out eventually by uttering the magic words: "computer error." But it was very exciting for a while there.

[2] The Domain Driven Design book [<https://www.amazon.com/Domain-Driven-Design-Tackling-Complexity-Software/dp/0321125215>] is a bit dry, but I remember being very impressed when I read it the first time.

[3] Circle the appropriate choice.

[4] Except maybe which rabbit hole she wandered down...

[5] The real Harry and Lorina had a total of 10 children [https://en.wikipedia.org/wiki/Alice_Liddell], by the way.

[6] I removed all extra information from the documents to make it clearer.

[7] By default, that's set to 5 MB, and it's configurable.

[8] Not a typo! In our benchmarks, we're always saturating the network long before we saturate any other resource, so the current limit is how fast your network cards are in packet processing.

[9] A signed PDF invoice is pretty common, and you're required to keep it for tax purposes and can't just generate it on the fly.

[10] This is common in insurance and healthcare, for example.

[11] Obviously, this does not alleviate the need to have proper backups.

[12] That means that the data has been written to disk and `fsync()` or the equivalent was called, so the data is safe from power loss.

[13] The typical lag time for indexing is under 1 ms.

Chapter 5. Deep Dive Into the RavenDB Client API

In this chapter, we're going to take a deep dive into how the client API works. We're going to show mostly C# code examples, but the same concepts apply to any of the RavenDB client APIs, regardless of platform, with minor changes needed to make things applicable.

There are still some concepts that we haven't gotten around to (clustering or indexing, for example), which will be covered in their own chapters. But the client API is rich and has a lot of useful functionality on its own, quite aside from the server-side behavior.

We already looked into `DocumentStore` and the session, the basic building blocks of CRUD in RavenDB. But in this chapter, we're going to look beyond the obvious and into the more advanced features.

One thing we'll *not* talk about in this chapter is querying. We'll cover that extensively in Chapter 12, so let's keep it there. You already know the basics of querying in RavenDB, but there's a *lot* more power for you to discover.

This chapter is going to contain a large number of code examples, and it will discuss the nitty-gritty details of using the client. It's divided into brief sections each dealing with a specific feature or behavior. I suggest reading this over to note the capabilities of RavenDB and coming back to it as needed in your application.

For the rest of this chapter, we'll use the classes shown in Listing 5.1 as our model, using a simplified help desk as our example.

Listing 5.1. Simplified Help Desk sample model

```
public class Customer
{
    public string Id { get; set; }
    public string Name { get; set; }
    public bool IsActive { get; set; }
}

public class SupportCall
{
    public string Id { get; set; }
    public string CustomerId { get; set; }
    public DateTime Started { get; set; }
    public DateTime? Ended { get; set; }
    public string Issue { get; set; }
    public int Votes { get; set; }
    public List<string> Comments { get; set; }
}
```

5.1. Writing documents

Writing documents in RavenDB is easy, as we saw in Chapter 2. If we want to create a new support call, we can use the code in Listing 5.2 to do so.

Listing 5.2. Creating a new support call using the session

```
using (var session = store.OpenSession())
{
    var call = new SupportCall
    {
        Started = DateTime.UtcNow,
        Issue = customerIssue,
        CustomerId = customerId
    };
    session.Store(call);
    session.SaveChanges();
}
```

This is the basic behavior of RavenDB and how you would typically work with saving data. But there are a lot of additional things that we can do when writing data. For example, let's say the user might have sent us some screenshots that we want to include in the support call.

The async API mirrors the synchronous one

Most of the code samples use the synchronous API because it is simpler. With the async API, you'll need to add `await`, sometimes use `ConfigureAwait(false)`, and consider cancellation tokens. The goal here is to explain how RavenDB *works*, rather than to cover every detail of the API.

You can easily switch from `OpenSession` to `OpenAsyncSession` to use the async API. The main difference is that you call `await session.SaveChangesAsync()` instead of `session.SaveChanges()`, `await session.StoreAsync()` instead of `session.Store()`, etc.

Hint: If you see an API call in the async session that does *not* use the `Async` suffix and returns a task, you can be sure it operates purely in memory. For example, both synchronous and async sessions have a `void Delete(string documentId)` method. Because the async version does not have a `Task DeleteAsync(string documentId)` method, you *know* that it simply updates the session's internal bookkeeping. The actual deletion on the server will occur as part of the `SaveChangesAsync()` call.

5.2. Working with attachments

We already talked in depth about attachments in Chapter 3. I cover them here as well to look at them from the client API perspective, not just how they work on the server. You can add attachments to a RavenDB document to store binary data related to that document. Let's assume the user sent us a screenshot of the problem along with the call. Listing 5.3 shows how we can store

and retrieve the attachments.

Listing 5.3. Saving attachments to RavenDB as part of opening the support call

```
using (var session = store.OpenSession())
{
    var call = new SupportCall
    {
        Started = DateTime.UtcNow,
        Issue = customerIssue,
        CustomerId = customerId
    };
    session.Store(call);

    foreach (var file in attachedFiles)
    {
        session.Advanced.StoreAttachment(call, file.Name,
            file.OpenStream());
    }

    session.SaveChanges();
}
```

Note that we're using the session to store both the support call document and any attachments the user might have sent. An attachment is basically a file name and a stream that will be sent to the server (with an optional content type). When the call to `SaveChanges` is made, the RavenDB client API will send both the new document and all of its attachments to the server in a single call, which will be treated as a transaction. Both the document and the attachments will be saved, or both will fail.

That was easy enough, but now how do we retrieve the attachments? The list of attachments for a particular document is accessible via the session, as shown in Listing 5.4.

Listing 5.4. Getting the list of attachments for a support call

```
using (var session = store.OpenSession())
{
    var call = session.Load<SupportCall>("SupportCalls/238-B");
    var attachments = session.Advanced.GetAttachmentNames(call);

    // render the call and the attachment names
}
```

Calling `GetAttachmentNames` is cheap; the attachments on a document are already present in the document metadata, which we loaded as part of getting the document. There is no server-side call involved. Note that the result of `GetAttachmentNames` doesn't include the *content* of the attachments. To get the attachment itself and not just its name, you need to make an additional call, as shown in Listing 5.5.

Listing 5.5. Getting an attachment content

```
using (var session = store.OpenSession())
{
    var call = session.Load<SupportCall>("SupportCalls/238-B");
    var attachments = session.Advanced.GetAttachmentNames(call);

    using (var stream = session.Advanced.GetAttachment(call,
        attachments[0].Name))
    {
        // process the content of the attachment
    }
}
```

Each call to `GetAttachment` will make a separate call to the server to fetch the attachment. If you have a lot of attachments, be aware that fetching all their information can be expensive due to the number of remote calls that are involved.

5.3. Working with the document metadata

In the previous section, we noted that attachment information is stored in the document metadata. RavenDB uses the metadata for a lot of things. Most of them you don't generally care about (etag, change vector, etc.). But the document metadata is also available to you for your own needs and use.

An actual use case for direct use of the document metadata is pretty rare. If you want to store information, you'll typically want to store it in the document itself, not throw it to the metadata sidelines. Typical use cases for storing data in the metadata are cross-cutting concerns. The preeminent one is auditing. You may want to see who edited a document, for example.

In order to demonstrate working with the metadata, we'll consider creating a support call. Handling a support call can be a complex process that has to go through several steps. In this case, we will save the new support call document to RavenDB with a draft status in the metadata. Typical modeling advice would be to model this explicitly in the domain (so you'll have an `IsDraft` or `Status` property on your model), but for this example, we'll use the metadata. You can see the code for setting a draft status in the metadata in Listing 5.6.

Listing 5.6. Setting a metadata flag as part of creating a new support call

```
using (var session = store.OpenSession())
{
    var call = new SupportCall
    {
        Started = DateTime.UtcNow,
        Issue = customerIssue,
        CustomerId = customerId
    };
    session.Store(call);
}
```

```

var metadata = session.Advanced.GetMetadataFor(call);
metadata["Status"] = "Draft";

session.SaveChanges();
}

```

We can call `GetMetadataFor` on any document that has been associated with the session. A document is associated with the session either by loading it from the server or by calling `Store`. After the document has been associated with the session, we can get its metadata and manipulate it.

Changes to the metadata count as changes to the document and will cause the document to be saved to the server when `SaveChanges` is called. For reading the metadata, you call to `GetMetadataFor` on the entity instance and get an `IMetadataDictionary` which contains methods such as: `GetBoolean`, `GetObject`, `GetString`, etc.

5.4. Cross cutting concerns with session events

The client API exposes a number of events that you can register to in order to deal with cross cutting concerns. Those events allow you to effect the behavior of the RavenDB client API in a global fashion.

The most commonly used events are list below (check the online documentation for the full list):

- `OnBeforeStore` - called for each new or updated entity that we store to the server in `SaveChanges`, *before* the server call.
- `OnAfterSaveChanges` - called for each entity that we send to the server in `SaveChanges`, *after* the server replied back *successfully*.
- `OnBeforeDelete` - called from `SaveChanges` on any entity that was marked to be deleted.
- `OnSessionCreated` / `OnSessionDisposing` - allow to run your own logic on session creationg & disposal.
- `OnBeforeQuery` - called before sending a query to the server. Useful for logging or adding conditions to the query in a global fashion.
- `OnBeforeConversionToDocument` / `OnAfterConversionToDocument` and `OnBeforeConversionToEntity` / `OnAfterConversionToEntity` - allow to customize serialization and deserialization from JSON documents to your domain entities.

You can register to those events globally, at the `DocumentStore` level or at the individual session level. If you want to register to the `DocumentStore`, you *must* do so before you call `documentStore.Initialize()`. All sessions inherit their `DocumentStore` events (you can register more event handlers on the session as well, if needed).

Listing 5.7 shows how we can use the `OnBeforeStore` to add audit metadata to our documents.

Listing 5.7. Using `OnBeforeStore` to register the current user in modified documents metadata

```

using var store = new DocumentStore
{

```

```

        Urls = ["http://localhost:8080"],
        Database = "Northwind"
    };
    store.OnBeforeStore += (sender, e) =>
    {
        e.DocumentMetadata["Modified-By"] =
            System.Environment.UserName;
    };
    store.Initialize();
    using (var session = store.OpenSession())
    {
        session.Store(new Employee
        {
            FirstName = "John",
            LastName = "Doe",
        });
        session.SaveChanges();
    }

```

The result of running the code in Listing 5.7 is shown in Listing 5.8, where we show the resulting metadata on the server side.

Listing 5.8. The metadata of the employee document saved in Listing 5.7, including the Modified-By audit field.

```

{
    "@metadata": {
        "@collection": "Employees",
        "Raven-Clr-Type": "Tryouts.Program+Employee, Tryouts",
        "Modified-By": "Oren Eini"
    }
}

```

① The audit field added by the OnBeforeStore event handler.

Note that the OnBeforeStore and all the other events are *not* async in nature. You can access the session using the provided event args (e.Session), but you don't have access to the Load<T>() / Store<T>() or LoadAsync<T>() / StoreAsync<T>() easily. The session is hardened to allow accessing the session (including loading other documents, storing new documents or having a last chance to modify them) while inside the event, but it makes for a pretty complex system, from an architecture standpoint.

Another example of using those events is using OnBeforeDelete to validate the behavior of the system, as you can see in Listing 5.9.

Listing 5.9. Using OnBeforeDelete event to validate we cannot delete an open support call

```

store.OnBeforeDelete += (sender, e) =>
{
    switch (e.Entity)

```

```

    {
        case SupportCall { Ended: null }:
            throw new InvalidOperationException(
                "Cannot delete a support call that is still open"
            );
    }
};

```

Listing 5.9 checks if we are trying to delete a support call that is still open, throwing an exception (and canceling) the `SaveChanges()` if that is the case.

Another example of using events is to ensure that we always include a particular clause in our queries, as you can see in Listing 5.10.

Listing 5.10. Never read inactive customer

```

store.OnBeforeQueryExecuted += (sender, args) =>
{
    if (args.QueryCustomization is IDocumentQuery<Customer> query)
    {
        query.WhereEquals("IsActive", true);
    }
}

```

The code in Listing 5.10 will apply to all queries operating on `Customer` and will ensure that all the results returned have the `IsActive` property set to `true`. This can also be used in multi-tenancy situations, where you want to add the current tenant ID to all queries.

The full details of what you can do with conventions and listeners are in RavenDB's online documentation. I encourage you to browse the documentation and consider when it might make sense to use listeners in your application. Using them can be quite a time saver because listeners can be applied globally with ease. You can do a lot with those events, but you need to be aware that:

- These events run in the *client side* - as such, a different client (or manual operation) will not execute those.
- It is easy to use these events for cross-cutting concerns, but it is also easy to miss their actions. If you make use of them be sure to properly document their effect on the system.

I like making sure that if I have a before store event, I add *something* to the document metadata that makes it clear that it was modified by an event handler. The `Modified-By` metadata field is a good example of that.

5.5. Change tracking and SaveChanges

The document session implements change tracking on your documents, as you can see in Listing 5.11.

Listing 5.11. Saving the changes to a document without manually tracking what changed

```
using (var session = store.OpenSession())
{
    var call = session.Load<SupportCall>("SupportCalls/238-B");

    call.Ended = DateTime.UtcNow;

    session.SaveChanges();
}
```

The session's change tracking (and identity map) means that you don't have to keep track of what changed and manually call `Store`. Instead, when you call `SaveChanges`, all your changes will be sent to the server in a single request.

You have a few knobs available to tweak the process. `session.Advanced.HasChanges` will let you know if calling `SaveChanges` will result in a call to the server. And `session.Advanced.HasChanged(entity)` will tell you when a particular entity has changed. You can also take it up a notch and ask RavenDB to tell you *what* changed, using `session.Advanced.WhatChanged()`. This will give you all the changes that happened in the session. The `WhatChanged` feature can be nice if you want to highlight changes for user approval, for example, or if you just want to see what modifications were made to your model after a certain operation.

You can also tell RavenDB not to update a particular instance by calling `session.Advanced.IgnoreChangesFor(entity)`. The document will remain attached to the session and will be part of any identity map operations, but it won't be saved to the server when `SaveChanges` is called. Alternatively, you can call `session.Advanced.Evict(entity)` to make the session completely forget about a document.

These operations tend to be useful only in specific cases, but they are very powerful when utilized properly.

5.6. Optimistic & pessimistic concurrency

We covered optimistic concurrency in Chapter 4, but only in the most general terms. Now, let's take a look and see how we can use optimistic concurrency in practice. Listing 5.12 shows two simultaneous sessions modifying the same support call.

Listing 5.12. Concurrent modifications of a support call

```
using (var sessionOne = store.OpenSession()) ①
using (var sessionTwo = store.OpenSession())
{
    var callOne = sessionOne.Load<SupportCall>("SupportCalls/238-B"); ②
    var callTwo = sessionTwo.Load<SupportCall>("SupportCalls/238-B");

    callOne.Ended = DateTime.Today; ③
    callTwo.Ended = DateTime.Today.AddDays(1);
}
```

```

    sessionOne.SaveChanges();
    sessionTwo.SaveChanges();
}

```

④

- ① Opening two *distinct* sessions.
- ② Loading the *same* document on both sessions at once.
- ③ Modify the two entities that represent the same in different sessions.
- ④ Save the changes in both sessions, simulating concurrent work by two threads.

In the case of the code in Listing 5.12, we're always going to end up with the support call end date set to tomorrow. This is because, by default, RavenDB uses the Last Write Wins model. You can change that by setting `store.Conventions.UseOptimisticConcurrency` to true, which will affect all sessions, or you can change it on a case-by-case basis by setting `session.Advanced.UseOptimisticConcurrency` to true on the session directly.

In either case, when this flag is set and `SaveChanges` is called, we'll send the modified documents to the server alongside their change vectors that were received from the server when loading the documents. This allows the server to reject any stale writes. If the flag were set to true, the code in Listing 5.12 would result in a `ConcurrencyException` on the `sessionTwo.SaveChanges()` call.

This ensures that you can't overwrite changes you didn't see, and if you set `UseOptimisticConcurrency`, you need to handle this error in some manner.

5.6.1. Pessimistic locking

When changes happen behind our backs to the document we modified, optimistic locking handles it. Pessimistic locking, on the other hand, prevents those changes entirely. RavenDB does *not* support pessimistic locking. And while you really need support from the database engine to properly implement pessimistic locking, we fake it in an interesting way. The following is a recipe for using approximating pessimistic locking in RavenDB. We mention it not so much because it's a good idea, but because it allows us to explore several different features and see how they work together.

Using pessimistic locking, we can lock a document for modification until we release the lock or until a certain amount of time has gone by. We can build a pessimistic lock in RavenDB by utilizing the document metadata and optimistic concurrency. It's easier to explain with code, and you can find a (pretty problematic) Lock implementation in Listing 5.13.^[1]

Listing 5.13. Extension method to add (pretty bad) pessimistic locking to the session

```

public static IDisposable Lock(
    this IDocumentSession session,
    string docToLock)
{
    var doc = session.Load<object>(docToLock);
    if (doc == null)
        throw new DocumentDoesNotExistException("The document " +
            docToLock + " does not exists and cannot be locked");
}

```

①

```

var metadata = session.Advanced.GetMetadataFor(doc);           ②
if (metadata.GetBoolean("Pessimistic-Locked"))                 ③
{
    // the document is locked and the lock is still value
    var ticks = metadata.GetNumber("Pessimistic-Lock-Timeout"); ④
    var lockedUntil = new DateTime(ticks);
    if (DateTime.UtcNow <= lockedUntil)                         ⑤
        throw new ConcurrencyException("Document " +
            docToLock + " is locked using pessimistic locking");
}

metadata["Pessimistic-Locked"] = true;                         ⑥
metadata["Pessimistic-Lock-Timeout"] =
    DateTime.UtcNow.AddSeconds(15).Ticks;

// will throw if someone else took the lock in the meantime
session.Advanced.UseOptimisticConcurrency = true;
session.SaveChanges();                                       ⑦

return new DisposableAction(() =>
{
    metadata.Remove("Pessimistic-Locked");
    metadata.Remove("Pessimistic-Lock-Timeout");
    Debug.Assert(session.Advanced.UseOptimisticConcurrency); ⑧
    session.SaveChanges();
});
}

```

- ① Load the document to lock from the session
- ② Get the document data, which contains the pessimistic lock details
- ③ Check if this document was pessimistically locked.
- ④ Get the expiration value of the lock.
- ⑤ Check if the lock is still valid and throw if that is still the case.
- ⑥ Update the document metadata to reflect its locked status.
- ⑦ Save the metadata changes after making sure that the session enabled optimistic concurrency.
- ⑧ Remove the pessimistic locking metadata and save the session again, with optimistic concurrency.

There's quite a bit of code in Listing 5.13, but there isn't actually a lot that gets done. We load a document and check if its metadata says it is locked using `Pessimistic-Locked` and `Pessimistic-Lock-Timeout`. The key is that we rely on optimistic concurrency to ensure that our locking mechanism is safe in the face of concurrency. If anyone else is trying to lock the document at the same time, we are ensured that only a single session will be successful, and all other contenders will fail.

The `Lock` method returns an `IDisposable` instance that handles releasing the lock. This is done by removing the metadata values and then calling `SaveChanges` again. If the lock has timed out and someone took the lock, we'll fail here with a concurrency exception as well.

Avoid writing your own distributed pessimistic locks

There's a reason why RavenDB does not include a pessimistic lock feature, and I strongly recommend you avoid using the recipe above. It's here to show how you'd use several different features at once to achieve a goal.

Actually handling a distributed lock is a non-trivial issue. Consider a RavenDB cluster with multiple nodes. If two lock requests go to two distinct nodes at the same time, *both* of them will succeed. The two nodes will quickly discover the conflicting updates and generate a conflict. But it isn't guaranteed they'll discover it inside the lock/unlock period.

Another issue is the subject of timing. If two clients have enough of a clock skew, a client might consider a lock to have expired even though it's still valid. Proper distributed locking requires a consensus protocol of some kind, and those aren't trivial to build or use. See Listing 5.14 for an example that uses RavenDB's consensus capabilities to build a proper cooperative distributed pessimistic lock.

A typical use for pessimistic locks is to lock a document while a user is editing it. That might sound like a good idea, but experience has shown that, in most cases, it leads to trouble. Consider, for example, version control systems (such as Git or Subversion). If you're reading this book, you've likely used a source control of some kind. If you haven't, I suggest you pick up a book about source control and prioritize it over this one: learning about that is far more foundational.

Early source control systems (SourceSafe is a good example) used locks as their concurrency model, and that led to a lot of problems. Say Joe locks a file and then leaves on vacation. All of his coworkers then have to wait until he gets back before committing code to that file. This is a typical problem that arises in such cases. The same happens when you have pessimistic locks. Implementing pessimistic locks requires you to also implement forced lock release, a feature that tells you who locked the document and a whole slew of management functions around it. Typically, implementing optimistic concurrency or merging is easier and matches most users' expectations.

Having said that, a *proper* way to build a pessimistic lock in RavenDB relies on using the Compare Exchange feature we covered briefly in Chapter 2. As a reminder, RavenDB's Compare Exchange allow you to essentially implement `Interlocked.CompareExchange()` in a distributed environment. You can see the code for this in Listing 5.14.

Listing 5.14. Implementing a distributed pessimistic lock in RavenDB using the compare exchange feature

```
async Task<IAsyncDisposable> LockAsync(
    IDocumentStore store,
    string key,
    string owner,
    TimeSpan timeout)
{
    using var session = store.OpenAsyncSession(new SessionOptions ①
    {
        TransactionMode = TransactionMode.ClusterWide
    });
    CompareExchangeValue<string> value = session.Advanced
```

```

        .ClusterTransaction.CreateCompareExchangeValue(key, owner); ②
value.Metadata["@expires"] =
    DateTime.UtcNow.Add(timeout).ToString("o"); ③

await session.SaveChangesAsync(); ④

return new DisposableAsyncAction(async () => ⑤
{
    await UnlockAsync(store, key, value.Index); ⑥
});
}

async Task UnlockAsync(IDocumentStore store, string key, long index)
{
    using var session = store.OpenAsyncSession(new SessionOptions ⑦
    {
        TransactionMode = TransactionMode.ClusterWide
    });
    session.Advanced.ClusterTransaction
        .DeleteCompareExchangeValue<string>(key, index); ⑧
    await session.SaveChangesAsync(); ⑨
}

```

- ① Open a cluster-wide session (to ensure consistency in a distributed cluster).
- ② Creates the compare exchange value using the specified key (ensuring that it is new). We don't care about the value in this case, so we register the owner of the key here, for debugging purposes.
- ③ Telling RavenDB that it should expire this value (RavenDB handles this part automatically).
- ④ Save the changes (and get a `ConcurrencyException` if the key already exists).
- ⑤ The unlocking part of the process, to allow using `await using(Lock(...)) { ... }` to handle cleanup.
- ⑥ Calling the `UnlockAsync` method with the original `value.Index` to ensure consistency of the operation.
- ⑦ Creating a second cluster-wide session, to handle the deletion of the compare exchange value.
- ⑧ Delete the compare exchange value, using the previously defined index to ensure consistency of the operation.
- ⑨ Saving the changes (or getting `ConcurrencyException` if the key was already removed or changed).

Listing 5.14 is relying on the nature of the compare exchange feature in RavenDB, to allow us to have an atomic operation in a distributed cluster. We also use the `@expires` option to let RavenDB handle the cleanup and expiry of the lock on its own. We'll discuss this feature in more detail later. We'll discuss more about `@expires` and documents expiration in Chapter 6.

The sort of pessimistic lock that is shown in Listing 5.14 is a cooperative one. All users must call to the `Lock` method to ensure proper synchronization. It doesn't rely on an existing connection to be kept alive and will be cleaned up solely based on expiration time. That means that you can also use

that in an *offline* manner. In Listing 5.14 you can switch to offline pessimistic concurrency by *not* putting the `LockAsync` in a `using` block, but rather pass the `value.Index` to the caller and have them call `UnlockAsync` when they are done. A proper example on how to do this with optimistic concurrency mode is shown in the next section.

5.6.2. Offline optimistic concurrency

We looked at online optimistic concurrency in Listing 5.12, where we loaded a document into the session, modified it and then saved. In that time frame, if there was a change, we'd get a concurrency exception. But most software doesn't work like that. In a web application, you aren't going to keep the document session open for as long as the user is on the site. Instead, most likely you'll use a session-per-request model. The user will load a page with the document's content in one request and modify it in another request. There isn't a shared session in sight, so how can we implement optimistic concurrency?

All you need to do is send the change vector of the document to the user and accept it back when the user wants to save the document. Listing 5.15 shows an example using two separate sessions with concurrency handling between them.

Listing 5.15. Concurrent modifications of a support call (offline optimistic concurrency)

```
string changeVector;
Supportcall callOne;

using (var sessionOne = store.OpenSession())           ①
{
    callOne = sessionOne.Load<SupportCall>("SupportCalls/238-B");
    changeVector = sessionOne.Advanced.GetChangeVectorFor(callOne);
}

using (var sessionTwo = store.OpenSession())           ②
{
    var callTwo = sessionTwo.Load<SupportCall>("SupportCalls/238-B");
    callTwo.Ended = DateTime.Today.AddDays(1);
    sessionTwo.SaveChanges();
}

callOne.Ended = DateTime.Today;                        ③

using (var sessionThree = store.OpenSession())
{
    sessionThree.Advanced.UseOptimisticConcurrency = true;
    sessionThree.Store(callOne, changeVector, callOne.Id); ④
    sessionThree.SaveChanges(); // will raise ConcurrencyException ⑤
}
```

- ① Load the document and record its current change vector in one session.
- ② In another session, load and modify the document.
- ③ Modify the first version we loaded, simulating changing the entity by the user.

- ④ Associating the *new* session (with optimistic concurrency) with the *original* version of the document, with the original change vector.
- ⑤ Saving the changes in the session (and getting a `ConcurrencyException` because it was already changed)

The code in Listing 5.15 simulates how we open a session to load a document and send it to the client. Then update it in a separate session when we get the reply from the client. Interleaved with that, we have another session that modified the same document. We provide the *original* change vector to the final session, ensuring that we can detect any changes behind our back.

Calling `Store()` with the original change vector gives the RavenDB client API enough information for us to do an optimistic concurrency check from the time we loaded `callOne` in the first session. In a web scenario, you'll typically send the change vector alongside the actual data and get it back from the client to do the check. You might also want to check out the `Changes` API, which is covered a little later in this chapter. This might help you get early change notifications when you need to implement offline optimistic concurrency.

5.7. Patching documents and concurrent modifications

The typical workflow with RavenDB is to open a session, load a document, run some business logic and call `SaveChanges`. When you follow those steps, the session will figure out what documents have changed and send them to the server. This model is simple and easy to follow, and it's the recommended way to work.

However, there are a few scenarios in which we don't want to send the entire document back to the server. For example, if our document is very large and we want to make a small change, we can avoid that cost. Another reason to avoid the full document save is a scenario that calls for concurrent work on a document.

Let's consider the `SupportCall.Votes` property. Two users may very well want to vote on the same support call at the same time. One way to handle that is to load the support call, increment the `Votes` property and call `SaveChanges`. In order to handle concurrent modifications, we can utilize optimistic concurrency and retries. But that's quite a lot of work to write, and if the document is large, there's also a lot of data going back and forth over the network for little reason. Listing 5.16 shows how we can do much better.

Listing 5.16. Incrementing a property using the Patch API

```
using (var session = store.OpenSession())
{
    session.Advanced.Increment<SupportCall, int>(
        "SupportCalls/238-B",
        c => c.Votes,
        1);

    session.SaveChanges();
}
```

What the code in Listing 5.16 does is generate a patch request, rather than load and save the full document. That request is stored in the session, and when `SaveChanges` is called, it will be sent to the server (alongside any other changes/operations made on the session, as usual). On the server side, we'll apply this operation to the document. This patch request is safe to call concurrently, since there's no loss of data when executing patches.

Which is faster, patching or load/save?

The hasty answer is that patching is faster. We send a lot less data, and we need one less round trip to do so. Winning all around, right?

But the real answer is that things are bit more complex. The patch request is actually a JavaScript function that we send. That means we need to parse and run it on the server side, potentially marshal values into the script environment and then marshal it back. Conversely, the code path for loading and saving documents in RavenDB is well trodden and has been optimized plenty. That means in many cases it might be easier to just load and modify the document directly, rather than use a patch.

Patching *isn't* expensive; I want to emphasize that. But at the same time, I've seen codebases where *all* writes had to be made using patching because of perceived performance benefits. That resulted in an extremely hard-to-understand system that was resistant to change. The general recommendation is to utilize patching only when you need to support concurrent modifications.

Note that in most cases concurrent modifications of the same document is *not* the default. A properly modeled document should have a single reason to change, but it's common that documents have additional data on them (like the `Votes` property or `Comments`) that are important to save but don't have any real business logic attached to them.

That kind of change is fine to do using patching. If you find yourself trying to run serious business logic in patch scripts (we'll see exactly how to do this in a bit), you should move that into your own business logic. For a numerical value like `Votes`, the *proper* answer in RavenDB is to use counters, not a document property and patching.

An important consideration to take into account is that there is no guarantee as to the order in which the patches will run. But, you don't need to worry about concurrency between patches on the same document as there is no concurrent/interleaved execution of the scripts on the same document.

A slightly more complex example of the use of patching is adding a comment to a `SupportCall`. Just like before, we want to support adding a comment concurrently. But to make things a bit more interesting, we'll add a business rule: a call that has been ended cannot have additional comments added to it. Listing 5.17 shows the obvious way to accomplish this.

Listing 5.17. Adding a comment to a support call using patch

```
using (var session = store.OpenSession())  
{
```

```

var call = session.Load<SupportCall>("SupportCalls/238-B");
if (call.Ended != null)
    throw new InvalidOperationException("Cannot comment on closed call");

session.Advanced.Patch(call, c => c.Comments,
    comments => comments.Add("This is important stuff!"));

session.SaveChanges();
}

```

In Listing 5.17, you can see how we moved from using the simple `Increment` call to the `Patch`, which allows us to either replace a property value completely or add an item to a collection. If you look closely at the code in Listing 5.17, you'll find that there's a hidden race condition there. The business rule is that we can't add comments to a closed call; we're able to load the call, check that its `Ended` property is null and then send the patch request to the server. However, in the meantime, another client could have closed the call, and yet we'd still add the comment.

The seriousness of this issue depends entirely on the domain and the model. It's possible that you *do* want to add comments during that period, or it's possible that allowing it could break important business invariants.

Patch requests are sent as part of `SaveChanges`

It's probably obvious, but I wanted to spell this out explicitly. Calls to `Patch`, `Increment` or `Defer` don't go to the server immediately. Instead, they're added to the list of operations the session needs to execute and will be sent to the server in a single batch (along with any modified documents) when `SaveChanges` is called. If you have multiple patch operations in the same session on the same document, they'll be merged into a single patch. And if there are multiple patches on different documents, they'll all be executed within the same transaction, as a single unit.

There are two ways to avoid the race condition. We can send a change vector to the server asking it to fail with a concurrency exception if the document has been modified since we last saw it. That *works*, but it defeats the whole point of using patches for concurrent modification of the document. The second alternative is to move the invariant check into the script itself. Calls to `Increment` and `Patch` are actually just wrappers around the `Defer` call, which allows you to add work to the session that's to be sent to the server when `SaveChanges` is called.

In Listing 5.18, we're dropping down to using `Defer` directly to manipulate the patch request ourselves, with no wrappers. As you can see, this is a bit involved, but overall it's pretty straightforward.

Listing 5.18. Using a patch script to maintain call invariants

```

using (var session = store.OpenSession())
{
    session.Advanced.Defer(new PatchCommandData(
        id: "SupportCalls/238-B",

```

```

        changeVector: null,
        patch: new PatchRequest
        {
            Script = @"
                if (this.Ended != null)
                    throw 'Cannot add a comment to a closed call';
                this.Comments.push($comment);
            ",
            Values =
            {
                ["comment"] = "This is important stuff!!"
            }
        },
        patchIfMissing: null));

    session.SaveChanges();
}

```

The code in Listing 5.18 passes a `PatchCommandData`, containing the relevant document ID, to `Defer`. The key part is in the `PatchRequest.Script`. We do the check on the document and fail if the call has already been closed. If it hasn't, we add the comment to the call. You can also see that we don't have to deal with string concatenation here since we can pass arguments to the scripts directly. There's also the option to run a script if the document does not exist (using `patchIfMissing`). This gives you the option to do a "modify or create" style of operation.

Using this method, we can be sure that we'll never violate the rule about adding comments to a closed call. A cautionary word, though: this is more complex than anything we've dealt with before, and I would only recommend doing it if you really must.

5.8. Deferring commands

In the previous section, we used the `Defer` method to register a `PatchCommandData` on the session, to be executed when `SaveChanges` is called. But `Defer` is more generic than this. It's a general mechanism for the user to register arbitrary commands that will be part of the same transaction as the `SaveChanges`.

The RavenDB API is like an ogre. It has layers

Like the popular character Shrek, RavenDB API is composed of layers. At the top, you have the document store and the document session. Those are built using the operations concept, which you typically won't use directly. The operations are handled via the request executor, which allows you to generate requests directly to the server (and take advantage of RavenDB's authentication and automatic failover).

The case of `Defer` is a good example. Instead of forcing you to drop down all the way, we expose an extension point in the session so you can plug in a command of your own and piggyback on the session handling of transactions.

The available commands range from putting and deleting documents and attachments to applying patches that delete documents having some prefix. Aside from the patch operation, the rest are only useful in rare cases. The most common use of `Defer` beyond patch is when you need fine-grained control over the operations that will be executed in a single transaction, to the point where you want to control the ordering of the operations.

RavenDB doesn't allow changing the document's collection by updating the `@collection` metadata. If you need to move a document to a different collection you must delete the old document and then create a new one with the appropriate collection. The session doesn't allow you to both delete and modify a document in the same high-level operation. For the purpose of discussion, let's assume we *have* to do this in a single transaction so no other client sees a transient deleted state.

To be clear, this is a strange situation, dreamt up specifically to showcase a feature that should be used only in special circumstances. This escape hatch in the API exists to prevent you from being blocked if you need something we didn't foresee, but it is probably a bad idea to make it the front door.

Another reason to avoid using `Defer` is that it's lower in the RavenDB client API layers. Instead of dealing with high level concepts like entities, you'll be dealing with the direct representation RavenDB uses for JSON (the blittable format, discussed later). That format is optimized for performance; developer convenience was secondary in its design.

5.9. Bulk inserting documents to RavenDB

RavenDB is fast, *really* fast, but it still needs to face operational realities. The fallacies of distributed computing [https://en.wikipedia.org/wiki/Fallacies_of_distributed_computing] still apply, and I/O takes a non-trivial amount of time. This means that when you want to get the best speed out of RavenDB, you need to help it achieve that.

Stacking the deck

I'm going to be talking performance numbers in this section, and I wanted to make it clear that I've intentionally chosen the worst possible situation for RavenDB and then compounded the issue by using the wrong approaches. This is so I can show the real costs in a manner that's highly visible.

I'll refer you again to the fallacies of distributed computing [https://en.wikipedia.org/wiki/Fallacies_of_distributed_computing]. I'm trying to select a scenario that would break as many of those fallacies as possible and show how RavenDB is able to handle them.

Listing 5.19 shows the absolute slowest way to write 10,000 documents into RavenDB.

Listing 5.19. Writing 10,000 documents, one at a time

```
var sp = Stopwatch.StartNew();
for (int i = 0; i < 10_000; i++)
{
    using (var session = store.OpenSession())
```



```

    {
        session.Store(new Customer
        {
            Name = "Customer #" + i
        });
        session.SaveChanges();
    }
}
Console.WriteLine(sp.Elapsed);

```

For fun, I decided to run the code in Listing 5.19 against the live test [<http://live-test.ravendb.net/>] instance we have. That instance was in San Francisco, and I was testing this from Israel. The test instance was also running as a container inside an AWS t2.medium machine (two cores and 4 GB of memory, with burst-only mode). In other words, this performance test was heavily biased against RavenDB, and the results were not great. In fact, they were *bad*.

This is because we're running each write as an independent operation, and we have to wait for the previous operation to complete before we can start the new one. What's more, the database server handles just a single request concurrently, which means we have no way to amortize I/O costs across multiple requests. This is the absolute worst way you can write a large amount of documents into RavenDB because most of the time is spent just going back and forth between the client and the application. For each request, we have to make another REST call, send a packet to the server, etc. On the other side, the server accepts a new request, processes it and commits it to disk. During the entire process, it's effectively idle, since most of the time is spent waiting for I/O. That's a big waste all around.

Each request took about 220–260 milliseconds to run. Writing the first 1,000 documents took four minutes and six seconds, and 2,000 requests took eight minutes on the dot. The full 10,000 documents would take 40 minutes or so. Granted, we're intentionally going to a remote server, but still...

What happens when we're running the writes in parallel? The code in Listing 5.20 shows how to do this.

Listing 5.20. Writing 10,000 documents, with a bit of parallelism thrown in

```

var sp = Stopwatch.StartNew();
Parallel.For(0, 10_000, i =>                                     ①
{
    using (var session = store.OpenSession())
    {
        session.Store(new Customer
        {
            Name = "Customer #" + i
        });
        session.SaveChanges();
    }
});
Console.WriteLine(sp.Elapsed);

```

① Using `Parallel.For()` to execute the inner lambda 10,000 times in parallel in the thread pool.

Using the method in Listing 5.20, I was able to write 1,000 documents in 56 seconds. We got to 2,000 in a minute and a half, 3,000 in a minute and 50 seconds, etc. The reason for the speed up is actually related to how thread pooling is handled on the client side. Since we make a lot of blocking requests, the thread pool figures out that we have plenty of blocking work and creates more threads. That means we have the chance to do more concurrent work. So as time goes by, more threads are created and we make additional concurrent requests to RavenDB.

The total time of writing 10,000 documents in this setup was two minutes and 52 seconds. So we've gotten done 20 times faster than we did using sequential writes. The code in Listing 5.20 is still using synchronous calls, which means the client side is spinning threads to handle the load and we're limited by the rate of new thread creation on the client.

RavenDB also supports an async API, which is much more suitable for scale-out scenarios because we aren't holding a thread for the duration of the connection. Listing 5.21 shows how we can write all those documents in parallel, using the async API. The code is a tad complex because we want to control the number of concurrent requests we make. Spinning 10,000 concurrent requests will likely load the network and require careful attention to how they are managed, which is out of scope for this book. Instead, I limited the number of concurrent connections to 128.

Listing 5.21. Writing 10,000 documents, using async API

```
var sp = Stopwatch.StartNew();
var semaphore = new SemaphoreSlim(128);           ①

async Task WriteDocument(int i)                  ②
{
    using (var session = store.OpenAsyncSession())
    {
        await session.StoreAsync(new Customer
        {
            Name = "Customer #" + i
        });
        await session.SaveChangesAsync();
    }
    semaphore.Release();                           ③
}

var tasks = new List<Task>();
for (int i = 0; i < 10_000; i++)                 ④
{
    semaphore.Wait();
    tasks.Add(WriteDocument(i));
}

Task.WaitAll(tasks.ToArray());

Console.WriteLine(sp.Elapsed);
```

- ① Semaphore set to 128 to limit maximum concurrency.
- ② The actual individual async store operation to the database.
- ③ Releasing the semaphore when we are done saving the document.
- ④ Loop 10,000 times to execute the WriteDocument, but limiting to the 128 concurrent tasks.

The code in Listing 5.21 writes 1,000 documents in just under 10 seconds, and it completes the full 10,000 writes in under 30 seconds (29.6, on my machine). The speed difference is, again, related to the client learning our pattern of behavior and adjusting itself accordingly (creating enough buffers, threads and other resources needed; warming up the TCP connections.^[2]).

However, we really had to make an effort. We wrote explicit async code and managed it, rate-limited our behavior and jumped through several hoops to get a more reasonable level of performance. Note that we went from over 40 minutes to less than 30 seconds in the span of a few pages. Also note that we haven't actually modified *what* we're doing. We only changed how we're talking to the server, but it had a huge impact on performance.

You can take it as a given that RavenDB is able to process as much data as you can feed it. The typical concern in handling writes is how fast we can get the data to the server, not how fast the server can handle it.

RavenDB contains a dedicated API and behavior that makes it easier to deal with bulk loading scenarios. The bulk insert API uses a single connection to talk to the server and is able to make much better use of the network. The entire process is carefully orchestrated by both the client and the server to optimize performance. Let's look at the code in Listing 5.22 first and then discuss the details.

Listing 5.22. using bulk insert to write 100,000 documents, quickly

```
var sp = Stopwatch.StartNew();

using (var bulkInsert = store.BulkInsert())
{
    for (int i = 0; i < 100_000; i++)
    {
        bulkInsert.Store(new Customer
        {
            Name = "Customer #" + i
        });
    }
}

Console.WriteLine(sp.Elapsed);
```

The code in Listing 5.22 took two minutes and 10 seconds to run on my machine. Which is interesting, because it seems slower than the async API usage sample, right? Except there's one problem. I made a typo when writing the code and wrote a *hundred* thousand documents instead of *ten* thousand. If I was writing merely 10,000 documents, it would complete in about 18 seconds. The code is fairly trivial to write, similar to our first sample in Listing 4.14, but the performance is many

times faster.

To compare the costs, I ran the same code against a local machine, giving me a total time of 11 seconds to insert 100,000 documents (instead of two minutes remotely). If we wanted to compare apples with apples, then the cost for writing 10,000 documents is shown in Table 5.1.

Table 5.1. Bulk insert costs locally and remotely

	Remote	Local
Session	41 minutes	20 seconds
Bulk Insert	18 seconds	6.5 seconds

You can see that while bulk insert is significantly faster in all cases, being over three times faster than the session option (Listing 5.19) locally seems insignificant considering it's over 130 times faster in the remote case. The major difference, as you can imagine, is the cost of going over the network, but even on the local machine (and we're not even talking about the local network), there's a significant performance benefit for bulk insert.

Amusingly enough, using bulk insert still doesn't saturate the server. For large datasets, it's advisable to have parallel bulk insert operations going at the same time. This gives the server more work to do, and it allows us to do optimizations that increase the ingest rate of the server.

The way bulk insert works is by opening a single long running request to the server and writing the raw data directly into the database. That means we don't need to go back and forth between the client and the server and can rely on a single network roundtrip to do all the work. The server, for its part, will read the data from the network and write it to disk when it's best to do so. In other words, bulk inserts are *not* transactional. A bulk insert is actually composed of many smaller transactions, whose size and scope are determined by the server based on its own calculations, in order to maximize performance.

When the bulk insert is completed, you can rest assured that all the data has been safely committed to disk properly. But during the process, the data is committed incrementally instead of going with a single big-bang approach.

For the most part, RavenDB performance is ruled by how many requests you can send it. The more requests, the higher the degree of parallelism and the more efficiently RavenDB can work. In our internal tests, we routinely bumped into hardware limits (the network card cannot process packets any faster, the disk I/O is saturated, etc.), not software ones.

5.10. Reading documents

We just spent a lot of time learning how we can write documents to RavenDB in all sorts of interesting ways. But for reading, how much is there really to know? We already learned how to load and query a document; we covered that in Chapter 2. We also covered Include and how to use it to effortlessly get referenced documents from the server. What else is there to talk about? As it turns out, quite a bit.

In most applications, reads are far more numerous than writes, often by an order of magnitude. That means RavenDB needs to be prepared to handle a *lot* of reads, and those applications typically

have a number of ways in which they access, shape and consume the data. RavenDB needs to be able to provide an answer to all those needs.

The first feature I want to present allows you to dramatically increase your overall performance by being a little lazy.

5.10.1. Lazy requests

In the previous section about bulk insert, we saw how important the role of the network is. Running the same code on the local network vs. the public internet results in large differences due to latency. Moving from many requests to a single bulk insert request is the primary reason we cut costs dramatically in both local and remote cases.

The latency of going to the server and making a remote call is often much higher than the cost of actually processing the request on the server. In production, your database will typically run on a dedicated machine, so you'll usually pay the network cost to get to it. RavenDB helps in several ways. A session has a budget on the number of remote calls it can make (`session.Advanced.MaxNumberOfRequestsPerSession`). If it goes over that limit, an exception is thrown. That motivated designing techniques to reduce the number of remote calls.

Include is one such technique: tell the server to include additional documents in the response so you don't have to issue another request. But Include can't solve every pattern. Consider the pair of queries in Listing 5.23 that cannot be optimized with Include.

Listing 5.23. Loading a customer and the count of support calls for that customer

```
using (var session = store.OpenSession())
{
    var customer = session.Load<Customer>("customers/8243-C");
    var countOfCalls = session.Query<SupportCall>()
        .Where(c => c.CustomerId == "customers/8243-C")
        .Count();

    // show the customer and the number of calls to the user
}
```

A scenario like the one outlined in Listing 5.23 is incredibly common. We have many cases where we need to show the user information from multiple sources, and that's a concern. Each of those calls turns out to be a *remote* call, requiring us to go over the network. There are ways to optimize this specific scenario. We can define a MapReduce index and run a query and Include on it. We haven't yet gone over exactly what this means (we'll cover that in Chapter 13), but this is a pretty complex solution, and it isn't relevant when you have different types of queries. If we wanted to also load the logged-in user, for example, that wouldn't work.

RavenDB's solution for this issue is the notion of lazy requests. A lazy request isn't actually being executed when you make it. Instead, it's stored in the session, and you get a `Lazy<T>` instance back. You can make multiple lazy requests, one after another, and no network activity will occur. However, as soon as you access the *value* of one of those lazy instances, all the lazy requests held up by the session will be sent to the server as a single unit.

All those requests will be processed by the server, and all the replies will be sent as a single unit. So no matter how many lazy requests you have, there will only ever be a single network round trip to the server. You can see the code sample in Listing 5.24.

Listing 5.24. Lazily loading a customer and their count of support calls

```
using (var session = store.OpenSession())
{
    Lazy<Customer> lazyCustomer = session.Advanced.Lazily
        .Load<Customer>("customers/8243-C");           ①

    Lazy<int> lazyCountOfCalls = session.Query<SupportCall>()
        .Where(c => c.CustomerId == "customers/8243-C")
        .CountLazily();                                ②

    // force execution of pending lazy operations explicitly
    session.Advanced.Eagerly.ExecuteAllPendingLazyOperations(); ③

    // if ExecuteAllPendingLazyOperations wasn't called, it
    // will be implicitly called when accessing Value.
    int countOfCalls = lazyCountOfCalls.Value;           ④
    Customer customer = lazyCustomer.Value;

    // show the customer and the number of calls to the user
}
```

- ① *Lazily* load the Customer - no network access is required here.
- ② *Lazily* execute a count query - no network access is required here.
- ③ Execute both previous pending operations in a *single* remote call to the server.
- ④ Access the *value* of the Lazy<T> instance. Implicitly call to ExecuteAllPendingLazyOperations() if it wasn't called previously.

As the code in Listing 5.24 shows, we can define multiple lazy operations. At that stage, they're pending. They're stored in the session but haven't been sent to the server yet. We can either call ExecuteAllPendingLazyOperations to force all pending operations to execute, or we can have that happen implicitly by accessing the Value property on any of the lazy instances we received.

Why do we need ExecuteAllPendingLazyOperations?

The existence of ExecuteAllPendingLazyOperations is strange. It's explicitly doing something that will happen implicitly anyway. So why is it needed? This method exists to allow users to have fine-grained control over the execution of requests. In particular, it allows you to set up a stage in your pipeline that will request all the data it's going to need. Then it will call ExecuteAllPendingLazyOperations to fetch this explicitly.

The next stage is supposed to operate on the pure in-memory data inside the session and not require any calls to the server. This is important in advanced scenarios, when you need this level of control and want to prevent the code from making unexpected remote calls in

performance-critical sections of your code.

The performance gain from Lazy is directly correlated to the number of lazy requests it's able to batch and how far away the actual server is. The more requests that can be batched and the further away the database server, the faster this method becomes. On the local machine, it's rarely worth going to the trouble, but once we go to production, this can get you some real benefits.

Note that, as useful as Lazy is, it's limited to requests that you can make with the information you have on hand. If you need to make queries based on the results of another query, you won't be able to use Lazy for that. For most of those scenarios, you can use Include. And of course, Lazy and Include can work together, so that will usually suffice.

5.10.2. Streaming data

When dealing with large amounts of data, the typical API we use to talk to RavenDB is not really suitable for the task. Let's consider the case of the code in Listing 5.25.

Listing 5.25. Query all support calls for a customer

```
using (var session = store.OpenSession())
{
    List<SupportCall> calls = session.Query<SupportCall>()
        .Where(c => c.CustomerId == "customers/8243-C")
        .ToList();
}
```

What will happen if this is a particularly troublesome customer that opened a *lot* of calls? If this customer had just 30 calls, it's easy to see that we'll get them all in the list. But what happens if this customer has 30,000 calls? Figure 5.1 shows how a query is processed on the server in this case.

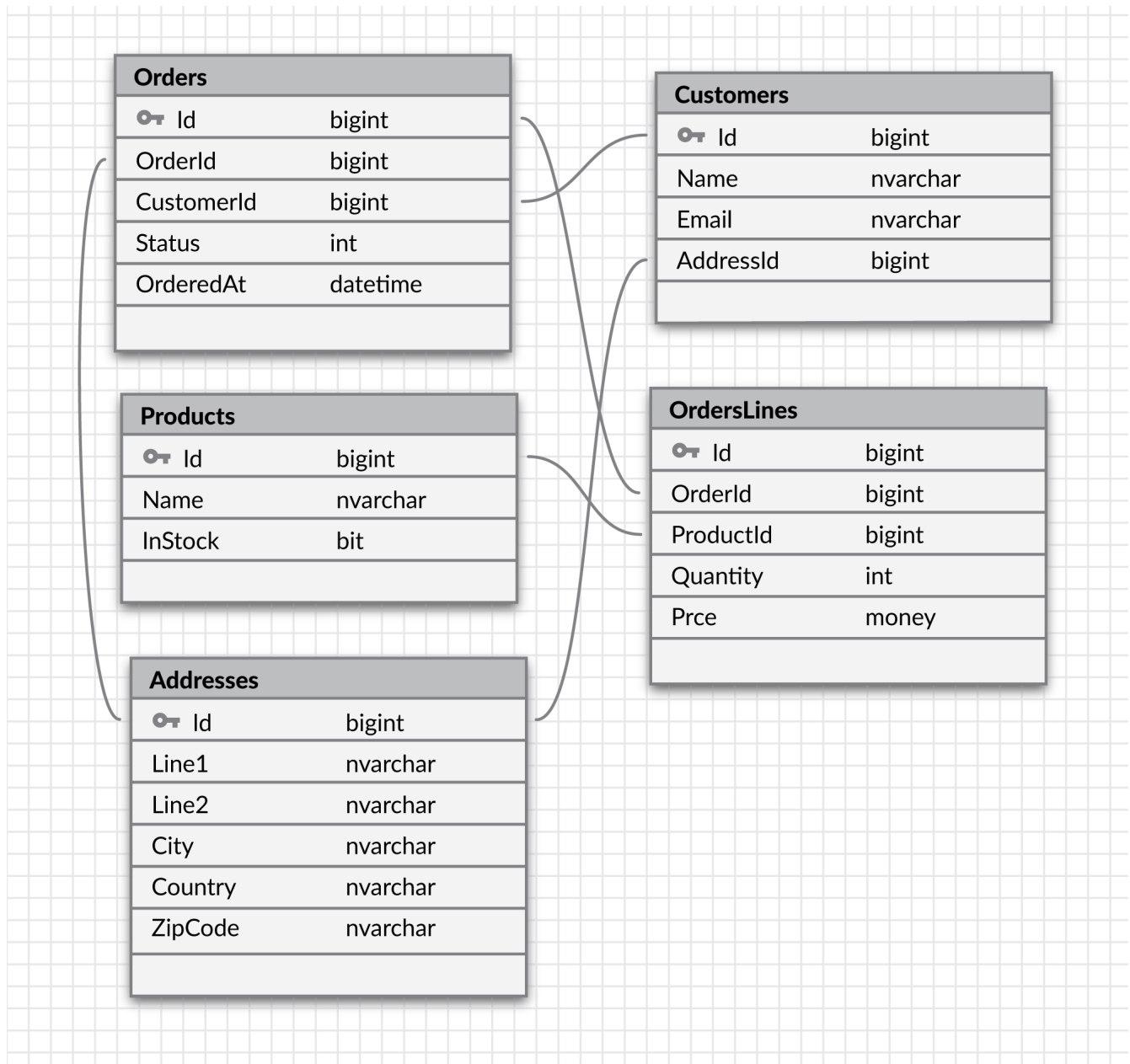


Figure 5.1. A query using request buffering

The server will accept the query, find all matches, prepare the results to send and then send them all over the network. On the client side, we'll read the results from the network and batch them all into the list that we'll return to the application.

If there are 30 results in all, that's great, but if we have 30,000, we'll likely suffer from issues. Sending 30,000 results over the network, reading 30,000 results *from* the network and then populating a list of 30,000 (potentially complex) objects is going to take some time. In terms of memory usage, we'll need to hold all those results in memory, possibly for an extended period of time.

Due to the way memory management works in .NET,^[3] allocating a list with a lot of objects over a period of time (because we are reading them from the network) will likely push the list instance, and all of its contents, into a higher generation. This means that, when you're done using it, the memory will not be collected without a more expensive Gen1 or even Gen2 round.

In short, for a large number of results, the code in Listing 5.25 will take more time, consume more

memory and force more expensive GC in the future. In previous versions of RavenDB, we had guards in place to prevent this scenario entirely. It's easy to start writing code like that in Listing 5.25 and over time have more and more results come in. Our logic was that, at some point, there needed to be a cutoff point where an exception is thrown before this kind of behavior poisoned your application.

As it turned out, our users *really* didn't like this behavior. In many cases, they would rather the application do more work (typically unnecessarily) than to have it throw an error. This allowed them to fix a performance problem rather than a "system is down" issue. As a result of this feedback, this limitation was removed, but we still recommend always using a Take clause in your queries to prevent just this kind of issue.

All queries should have a Take clause.

A query that doesn't have a take clause can be a poison pill for your application. As data size grows, the cost of making this query also grows until the entire thing goes down.

The RavenDB client API contains a convention setting called `ThrowIfQueryPageSizeIsNotSet`, which will force all queries to specify a Take clause and will error otherwise. We recommend that, during development, you set this value to true to ensure your code will always be generating queries that have a limit to the number of results they get.

Very large queries are bad, it seems, but that isn't actually the topic of this section. Instead, it's just the preface explaining why *buffered* large queries are a bad idea. RavenDB also supports the notion of *streaming* queries. You can see what that would look like in Figure 5.2.

☐	Id
☐	contracts/hourly/1234-A/2013-05-21
☐	contracts/hourly/1234-A/2012-02-13
☐	contracts/hourly/1234-A

Figure 5.2. A query using request streaming

Unlike the previous example, with streaming, neither client nor server need to hold the full response in memory. Instead, as soon as the server has a single result, it sends that result to the client. The client will read the result from the network, materialize the instance and hand it off to the application immediately. In this manner, the application can start processing the results of the

query before the server is done sending it, and it doesn't have to wait. You can see the code for that in Listing 5.26.

Listing 5.26. Stream all support calls for a customer

```
using (var session = store.OpenSession())
{
    var callsQuery = session.Query<SupportCall>()
        .Where(c => c.CustomerId == "customers/8243-C");

    using (var stream = session.Advanced.Stream(callsQuery))
    {
        while (stream.MoveNext())
        {
            SupportCall current = stream.Current;
            // do something with this instance
        }
    }
}
```

Instead of getting all the results in one go, the code in Listing 5.26 will pull them from the stream one at a time. This way, the server, the client API and the application can all work in parallel with one another to process the results of the query. This technique is suitable for processing a large number of results (in the millions).

The use of streaming queries requires you to keep a few things in mind:

- The results of a streaming query are *not* tracked by the session. Changes made to them will not be sent to the server when `SaveChanges` is called. This is because we expect streaming queries to have a high number of results, and we don't want to hold all the references for them in the session. If we did, we would prevent the GC from collecting them.
- Since streaming is happening as a single large request, there's a limit to how long you can delay before you call `MoveNext` again. If you wait too long, it's possible for the server to give up sending the rest of the request to you (since you didn't respond in time) and abort the connection. Typically, you'll be writing the results of the stream somewhere (to a file, to the network, etc.).
- If you want to modify all the results of the query, don't call `session.Store` on each as they're pulled from the stream. You'll just generate an excess of work for the session and eventually end up with a truly humongous batch to send to the server. Typically, if you want to read a lot of results and modify them, you'll use a stream and a `Bulk Insert` at the same time. You'll read from the stream and call `Store` on the bulk insert for each. This way, you'll have both streaming for the query as you read and streaming via the bulk insert on the write.
- The stream results are a snapshot of the data at the time when the query is computed by the server. Results that match the query after it was already processed are *not* streamed to the client.

When you need to select whether to use a regular (buffered) query or a streaming query, consider the number of items that you expect to get from the query and what you intend to do with them. If the number is small, you'll likely want to use a query for its simple API. If you need to process many

results, you should use the streaming API.

Note that the streaming API is *intentionally* a bit harder to use. We made sure the API exposes the streaming nature of the operation. You should strive to avoid wrapping that. Streaming should be used on the edge of your system, where you're doing something with the results and passing them directly to the outside in some manner.

5.10.3. Caching

An important consideration when speeding up an application is caching. In particular, we can avoid expensive operations if we cache the results from the last time we accessed them. Unfortunately, caching is *hard*. Phil Karlton said [<https://martinfowler.com/bliki/TwoHardThings.html>]:

There are only two hard things in Computer Science:
cache invalidation and naming things.

Caching itself is pretty trivial to get right. The hard part is how you're going to handle cache invalidation. If you're serving stale information from the cache, the results can range from nothing much to critical, depending on what exactly you're doing.

With RavenDB, we decided early on that caching was a complex topic, so we'd better handle it properly. It's done in two parts. The server side generates an etag for all read operations. This etag is computed by the server and can be used by the client later on. The client, on the other hand, is able to cache the request from the server internally. The next time a similar read request is made, the client will send the cached etag to the server alongside the request.

When the server gets such a request with an etag, it follows a dedicated code path, optimized specifically for that, to check whether the results of the operation have changed. If they didn't, the server can return to the client immediately, letting it know that it's safe to use the cached copy. In this manner, we save computation costs on the server and network transfer costs between the server and the client.

Externally, from the API consumer point of view, there's no way to tell that caching happened. Consider the code in Listing 5.27.

Listing 5.27. Query caching in action

```
using (var session = store.OpenSession())
{
    var calls = session.Query<SupportCall>()
        .Where(c => c.CustomerId == "customers/8243-C")
        .ToList();
}

using (var session = store.OpenSession())
{
    // this query will result in the server
    // returning a "this is cached" notification
    var calls = session.Query<SupportCall>()
```

```
.Where(c => c.CustomerId == "customers/8243-C"))  
.ToList();  
}
```

The client code doesn't need to change in any way to take advantage of this feature. This is on by default and is always there to try to speed up your requests. Caching is prevalent in RavenDB, so although the example in Listing 5.27 uses queries, loading a document will also use the cache, as will most other read operations.

The cache that's kept on the client side is the already-parsed results, so we saved not only the network round-trip time but also the parsing costs. We keep the data in *unmanaged* memory because it's easier to keep track of the size of the memory and avoid promoting objects into Gen2 just because they've been in the cache for a while. The cache is scoped to the document store, so all sessions from the same document store will share the cache and its benefits.

Time to skip the cache

Caching by default can be a problem with a particular set of queries: those that use the notion of the current time. Consider the case of the following query:

```
session.Query<SupportCall>()  
    .Where(c =>  
        c.StartedAt >= DateTime.Now.AddDays(-7)  
        && c.Ended == null);
```

This query is asking for all opened support calls that are over a week old.

This kind of query is quite innocent looking, but together with the cache, it can have surprising results. Because the query uses `DateTime.Now`, on every call, it will generate a different query. That query will never match any previously cached results, so it will always have to be processed on the server side. What's worse, every instance of this query will sit in the cache, waiting to be evicted, never to be used.

A much better alternative would be to use the following:

```
c.StartedAt >= DateTime.Today.AddDays(-7)
```

By using `'Today'`, we ensure that we can reuse the cached entry for multiple calls. Even if you need more granularity than that, just truncating the current time to a minute/hour interval can be very beneficial.

The cache works by utilizing HTTP caching behavior. Whenever we make a GET request, we check the cache to see if we previously made a request with the same URL and query string parameters. If we did, we fetch the cached etag for that request and send it to the server. The server will then use

that etag to check if the results have changed. If they didn't, the server returns a 304 Not Modified response. The client will then just use the cached response that it already has.

While most read requests are cached, there are a few that aren't. Anything that will always be different, such as stats calls, will never be cached. This is because stats and debug endpoints must return fresh information any time they're called. Attachments are also not cached because they can be very large and are typically handled differently by the application.

5.11. Aggressive caching

Caching is great, it would seem. But in order to utilize caching by RavenDB, we still need to go back to the server. That ensures we'll get a server confirmation that the data we're going to return for that request is indeed fresh and valid. However, there are many cases where we don't care much about the freshness of the data.

On a heavily used page, showing data that might be stale for a few minutes is absolutely fine, and the performance benefits gained from not having to go to the server can be quite nice. In order to address this scenario, RavenDB supports the notion of aggressive caching. You can see an example of that in Listing 5.28.

Listing 5.28. Aggressive caching in action

```
using (var session = store.OpenSession())
using (session.Advanced.DocumentStore.AggressivelyCache())
{
    var customer = session.Load<SupportCall>(
        "customers/8243-C");
}
```

The code in Listing 5.28 uses aggressive caching. In this mode, if the request is in the cache, the RavenDB client API will *never* even ask the server if it's up to date. Instead, it will immediately serve the request from the cache, skipping all network traffic. This can significantly speed up operations for which you can live with a stale view of the world for a certain period.

However, operating in this mode indefinitely would be pretty bad since you'd never see new results. This leads us back to the problem of cache invalidation. Aggressive caching isn't just going to blindly cache all data at all times. Instead, the first time it encounters an instruction to aggressively cache, the client is going to open a connection to the server and ask the server to let it know whenever something changes in the database. This is done using the Changes API, which is covered in the next section of this chapter.

Whenever the server lets the client know that something has changed, the client will ensure that the next cached request will actually hit the server for the possibly updated value. The client will not be asking specifically for changes it has in its cache. Instead, it will ask for all changes on the server. (Take note of that, as you might be caching a lot of different queries, documents, etc.) The client knows to check with the server for anything in the cache that is older than that notification. When this is done, the cached etag is still being sent, so if that particular response hasn't changed, the server will still respond with a 304 Not Modified and we'll use the cached value (and update its

timestamp).

The idea is that, with this behavior, you get the best of both worlds. If nothing has changed, immediate caching is available without having to go to the server. But if something might have changed, we'll check the server for the most up-to-date response. Given typical behavioral patterns for the application, we'll often be able to use the aggressive cache for quite some time before a write will come in and make us check with the server.

Why isn't aggressive caching on by default?

Aggressive caching isn't on by default because it may violate a core constraint of RavenDB: that a request will always give you the latest information. With the requirement that aggressive caching must be turned on explicitly, you're aware that there's a period of time where the response you receive might have diverged from the result on the server.

Caching behavior in a cluster

Typically, RavenDB is deployed in a cluster, and a database will reside on multiple machines. How does caching work in this context? The caching is built on the full URL of the request, and that takes into account the particular server that we'll be making the request to. That means that the cache will store a separate result for each server, even if the request is identical otherwise.

For the most part, the etags generated for HTTP requests among servers should be identical for identical data since it uses the change vectors of the documents to compute it. However, different servers may receive documents in a different order, which may result in a difference in the actual results. That shouldn't impact the behavior of the system, but be aware that RavenDB doesn't do cross node caching.

You'll learn more about RavenDB's behavior in a cluster in Chapter 9, but it is worth to talk about how multi-node clusters work with caching and aggressive-caching. When the data is replicated across multiple nodes, RavenDB will try to select a primary node that does most of the reads & writes for that database. There are options for spreading the load across all the nodes, either using round-robin or using more sophisticated techniques. Those are fairly advanced and we'll discuss them in Chapter 9 in detail.

5.12. Changes API

We mentioned the Changes API in the previous section since the aggressive caching is using it. The Changes API is a way for us to connect to the server and ask it to let us know when a particular event has happened, as shown in Listing 5.29.

Listing 5.29. Getting notified when a document changes

```
var subscription = store.Changes()  
    .ForDocument("customers/8243-C")
```

```
.Subscribe(change => {  
    // let user know the document changed  
});  
  
// dispose to stop getting future notifications  
subscription.Dispose();
```

Typically, we use the code in Listing 5.29 when implementing an edit page for a document. When the user starts editing the document, we register for notifications on changes to this document. If it does change, we let the user know immediately. That allows us to avoid having to wait for the save action to discover we need to redo all our work.

The Changes API works by opening a WebSocket to the server and letting the server know exactly what kind of changes it's interested in. We can register for a particular document, a collection, documents that start with a particular key, or even global events, such as operations or indexes.

Changes API in a cluster

In a cluster, the Changes API will always connect to a *single* node, and changes must first flow to that node before the changes will be sent to the client. The failure of the node will cause us to reconnect (possibly to a different server) and resume waiting for the changes we're interested in.

The Changes API is meant to get *non-critical* notifications. It's cheap to run and is pretty simple, but it's possible that a failure scenario will cause you to miss updates. If the connection has been reset, you might lose notifications that happened while you were reconnecting. It's recommended that you use the Changes API for enrichment purposes and not rely on it. For example, you might use it to tell if a document has changed so you can give the user early notification. At the same time, you *should* also utilize optimistic concurrency turned on so it will catch the change on save anyway.

Another example of a way you might use the Changes API is with aggressive caching. If we missed a single notification, that isn't too bad. The next notification will put us in the same state. And we'll be fine because the user explicitly chose performance over getting the latest version, in this case. Yet another example of Changes API use might be for monitoring. You want to know what's going on in the server, but it's fine to lose something if there's an error because you're interested in what's happening now, not the full and complete history of actions on the server.

For critical operations — ones where you can't afford to miss even a single change — you can use Subscriptions, which are covered in Chapter 7. They're suited for such a scenario, since they guarantee that all notifications will be properly sent and acknowledged.

5.13. Projecting data in queries

In the previous chapter, we talked a lot about modeling and how we should structure our documents to be independent, isolated and coherent. That makes for an excellent system for transaction processing (OLTP) scenarios. But there are quite a few cases where, even in a business application, we have to look at the data differently. Let's take a look at our support case example. If

I'm a help desk engineer and I'm looking at the list of open support calls, I want to see all the recent support calls and the customer that opened them.

Based on what we know so far, it would be trivial to write the code in Listing 5.30.

Listing 5.30. Recent orders and their customers

```
using (var session = store.OpenSession())
{
    List<SupportCall> recentOrders = session.Query<SupportCall>()
        .Include(c => c.CustomerId) // include customers
        .Where(c => c.Ended == null) // open order
        .OrderByDescending(c => c.Started) // get recent
        .Take(25) // limit the results
        .ToList();
}
```

The code in Listing 5.30 is doing a *lot*. It gets us the 25 most recently opened support calls, and it also includes the corresponding customers. We can now show this to the user quite easily, and we were able to do that in a single request to the server. Under most circumstances, this is exactly what you'll want to do.

However, in order to pull the few fields we need to show to the user in the application, we had to pull the full support calls and customers documents. If those documents are large, that can be expensive. Another way to handle this is by letting the server know exactly what we need returned and then letting it do the work on the server side.

This can be done by specifying a projection during the query. Projections allow us to control exactly what is being returned from the query. On the most basic level it allows us to decide what fields we want to get from the server, but we can actually do a lot more than that.

Let's see how we can use a query to project just the data that we are interested in. You can see the details in Listing 5.31.

Listing 5.31. Recent orders and their customers, using projection

```
using (var session = store.OpenSession())
{
    var recentOrders = (
        from call in session.Query<SupportCall>()
        where call.Ended == null
        orderby call.Started descending
        let customer = RavenQuery.Load<Customer>(call.CustomerId) ①
        select new ②
        {
            CustomerName = customer.Name, ③
            call.CustomerId,
            call.Started,
            call.Issue,
            call.Votes
        }
    )
}
```



```

    }
)
.Take(25)
.ToList();
}

```

- ① Load (at the server side) the related customer.
- ② Return a projection of the results, not the full documents.
- ③ Return a property from the related Customer document, not just the SupportCall document.

There are a few things to note in Listing 5.31. First, we no longer include the Customer in the query. We don't need that because the result of this query isn't a list of SupportCall documents but a list of projections that already include the CustomerName that we want. The key part, as far as we're concerned, is the call to the `RavenQuery.Load<Customer>()` method. This is translated into a server side load operation that fetches the related Customer document and extracts just the Name field from it.

The output of a query like the one in Listing 5.31 is not a document, it is a projection and as a result of that, it isn't tracked by the session. This means that changes to a projection won't be saved back to the server when `SaveChanges` is called. You *can* call `Store` on the result of a projection, but be aware that this will create a new document, which is probably isn't what you intended to happen. You can see the actual query that the code in Listing 5.31 generates in Listing 5.32.

Listing 5.32. The Raven Query Language query generated from Listing 5.31.

```

from 'SupportCalls' as call
load call.Customer as customer
select {
    CustomerName = customer.Name,
    CustomerId: call.CustomerId,
    Started: call.Started,
    Issue: call.Issue,
    Votes: call.Votes
}

```

5.14. Cross-cutting concerns on the client

The RavenDB client API is quite big. We already discussed the notion of layers in the design of the client API, allowing you to select at what level you want to work at any given point in time. There is quite a lot that you can configure and customize in the client behavior. The most common customization is changing the conventions of the client by providing your own logic.

Most of the decisions that the RavenDB client API makes are actually controlled by the `DocumentConventions` class. This class allows you to modify all sorts of behaviors, from how RavenDB should treat complex values in queries to selecting what property to use as the document ID in entities.

If you need fine-grained control over the serialization and deserialization of your documents, this is

the place to look. The `DocumentConventions` class holds important configurations, such as the maximum number of requests to allow per server or whether we should allow queries without a `Take` clause. Listing 5.33 shows an example of controlling what collection an object will find itself in.

Listing 5.33. Customize the collection to allow polymorphism

```
store.Conventions.FindCollectionName = type =>
    typeof(Customer).IsAssignableFrom(type)
        ? "Customers"
        : DocumentConventions.DefaultGetCollectionName(type);
```

In Listing 5.33, we're letting RavenDB know that `Customer` or any derived type should be in the `Customers` collection. That means that we can create a class called `VIPCustomer` that will have additional properties, but it will still be treated as a `Customer` by anything else (indexing, queries, etc.). Such options allow you to have absolute control over how RavenDB will work within your environment.

5.15. How RavenDB uses JSON

The RavenDB server and the RavenDB C# client API use a dedicated binary format to represent JSON in memory. The details on this format are too low level for this book and generally shouldn't be of much interest to outside parties, but it's worth understanding a bit about how RavenDB handles JSON even at this stage. Typically, you'll work with JSON documents in their stringified form: a set of UTF8 characters with the JSON format. That is human-readable, cheap to parse and simple to work with.

But JSON parsing requires you to work in a streaming manner, which means that to pull up just a few values from a big document, you still need to parse the full document. As it turns out, once a document is inside RavenDB, there are *lot* of cases where we want to just get a couple of values from it. Indexing a few fields is common, and parsing the JSON each and every time can be incredibly costly. Instead, RavenDB accepts the JSON string on write and turns it into an internal format called blittable.^[4]

A blittable JSON document is a format that allows RavenDB random access to any piece of information in the document without having to parse the document, with a traversal cost of (amortised) $O(1)$. Over the wire, RavenDB is sending JSON strings, but internally, it's all blittable. The C# client is also using the blittable format internally since that greatly helps with memory consumption and control. You generally won't see that in the public API, but certain low-level operations may expose you to it.

Blittable documents are immutable once created and *must* be disposed of once you're done with them. Since the document session will typically hold such blittable objects, the session must also be disposed of to make sure all the memory it's holding is released. An important consideration for the overall performance of RavenDB is that blittable documents always reside in native memory. This allows RavenDB fine-grained control over where and how the memory is used and reused, as well as its life cycle.

On the client side, using the blittable format means we have reduced memory consumption and

reduced fragmentation. It also reduces the cost of caching significantly.

5.16. Summary

In this chapter, we've gone over a lot of the advanced features in the client API. We looked at working with attachments and understanding how we can use them to store binary data. Then we moved on to working with the document metadata in general. The document metadata is a convenient place to stash information about our documents that doesn't actually belong to the document itself. Auditing is one such example, and we saw how we can use listeners on the events that the client API exposes to us.

We looked at how change tracking is managed and how we can get detailed information from the session about what exactly changed in our documents. Then we examined how we should handle concurrency in our application. We looked at optimistic concurrency in RavenDB and even implemented *pessimistic* locking to support global distributed locks using RavenDB's compare exchange feature. Online optimistic concurrency can be handled for us automatically by the session, or we can send the change vector value to the client and get it back on the next request, thus implementing offline optimistic concurrency.

Another way to handle concurrency or save the trouble of shuffling lots of data between client and server is to use patching. The client API offers patching at several levels. Setting a value or incrementing a number is supported by a strongly typed API, but more complex tasks can be handled using *Defer*, which also offers you the ability to write JavaScript code that will be executed on the server to mutate your documents.

We also looked at various ways to get a lot of data into RavenDB, from a sequential *SaveChanges* per document, to running them in parallel, to using the bulk insert API to efficiently push data into RavenDB. We saw that the major limiting factor was typically the cost of going to the database and that different approaches could produce significantly different performance profiles.

After looking at the various ways we could write data to RavenDB, it was time to look at the other side: seeing how we can optimize reads from the server. We had already gone over *Include* in Chapter 2 and in this chapter we looked at lazy requests, allowing us to combine several different requests to the server into a single round trip.

The mirror image to bulk insert is the streaming feature, suitable for handling an immense amount of data. Streaming allows us to start processing the request from the server immediately, without having to wait for the complete results. This allows us to parallelize the work between client and server and gives us the option to immediately start sending results to the user.

Following the reading and writing of documents, we looked into caching them. The client API has sophisticated caching behaviors, and we delved into exactly how that works, as well as how it reduces the time we need to provide answers to the user. Caching allows us to tell the server we already have the result of a previous call to the same URL. And it allows the server to let us know if that hasn't been modified. If that's the case, the server doesn't need to send any results on the wire, and the client can use the cached (parsed and processed) data immediately. RavenDB also supports the notion of aggressive caching, which allows us to skip going to the server entirely. This is done by asking the server to notify the client when things change, and only then go to the server to fetch those changes.

That option is also exposed in the client API, using the Changes API. The Changes API gives you the ability to ask the server to tell you when a particular document, collection or set of documents with a given prefix has changed. This lets users know that someone has changed the document they're working on, and allows them to implement features such as "this data has changed," etc.

Next, we looked at how we can project the results of queries and document loads on the server side using projections. A projection allows you to modify the shape of the data that RavenDB returns to the client. This can be done by simply returning a subset of the data from the documents. More advanced scenarios allow you to load additional documents and merging the data from associated documents into a single result.

We looked at cross-cutting concerns on the client and how we can apply behavior throughout the client once. We can modify the client behavior by controlling how RavenDB decides what class belongs in what collection, as well as serialization and deserialization. Listeners allow you to attach behavior to certain actions in the client API, giving you the option to customize certain behaviors. We looked at adding auditing to our application in about three lines of code and even saw how we can limit all the queries on the client to only include active users as a cross-cutting behavior.

Our final endeavor was to cover at a high level the native serialization format that RavenDB uses, the blittable format. This format is meant to be extremely efficient in representing JSON. It's typically stored in native memory to reduce managed memory consumption and garbage collection costs down the road. You don't typically need to concern yourself with this, except to remember that it's important to dispose of sessions when you're done with them.

This chapter is a long one, but it still doesn't cover the full functionality. There are plenty of useful features, but they tend to be useful in specific, narrow circumstances or only make sense to talk about in a larger scope. We've barely discussed queries so far, but we'll do so extensively when we get to Chapter 11 and when we discuss indexing.

The next chapter is a good example of this. We'll dedicate that chapter to handling data subscriptions and all the myriad ways they make batch data processing tasks easier for you.

[1] This is *not* a recommended way to do this, there are a lot of problems with using this approach, see later in this section for a much better alternative.

[2] TCP slow start can be a killer on benchmarks.

[3] The behavior on the JVM is the same. Other clients environment have different policies.

[4] I don't like the name, but we couldn't come up with anything better.

Chapter 6. Advanced Documents Capabilities

In Chapter 5 we talked about advanced client API usage in RavenDB and in Chapter 3 we covered other types of data that you can store in RavenDB, such as attachments, counters, time series and revisions. In this chapter, we are going to cover more advanced capabilities that RavenDB offers, at the database side. This covers features in which RavenDB itself is operating on your documents, without any nudges from the client side.

This isn't going to be a long chapter, as most of these features we discuss are straightforward and meant to address specific scenarios. They are very useful in many scenarios, so it is important to know about them.

6.1. Document Expiration

Sometimes, a document comes with an expiration date. For example, you may want to store a `ResetPasswordLink` document that is only valid for 30 minutes. After that, it should be removed. You can handle the validity period by checking the date of the document when the user click on the link, but what if they never do? How do you clean up the data after the time expired? You can write a task that would check for `ResetPasswordLink` documents that are too old and delete them. Then you'll need to schedule that task to run periodically, monitor it, etc.

RavenDB comes with a builtin expiration mechanism that can take care of this for you. You just need to mark the documents that you want to expire with a special metadata field (`@expires`), and RavenDB will take care of the rest for you. You can see how to mark a document as expirable in Listing 6.1 using the `@expires` metadata field.

Listing 6.1. Set the expiration data on a document.

```
using (var session = store.OpenSession())
{
    var token = Guid.NewGuid().ToString();
    var link = new ResetPasswordLink("users/1-A", token);
    session.Store(link);
    var metadata = session.Advanced.GetMetadataFor(link); ①
    var expires = DateTime.UtcNow.AddMinutes(30)
        .ToString("O"); ②
    metadata["@expires"] = expires; ③
    session.SaveChanges();
}
```

① Get the metadata for the newly stored reset link document.

② Expiration uses UTC date in ISO 8601 format (yyyy-MM-ddTHH:mm:ss.fffffffZ).

③ Set the `@expires` metadata field to the desired expiration date.

The code in Listing 6.1 will create a `ResetPasswordLink` document that will expire in 30 minutes. Note that for expiration, we *must* always use UTC time, not local time. RavenDB will automatically delete the document after that time passes, so you don't have to do anything else. Well, almost... If you'll run the code in Listing 6.1 without enabling expiration on the database, you'll see the

warning in the Studio shown in Figure 6.1.

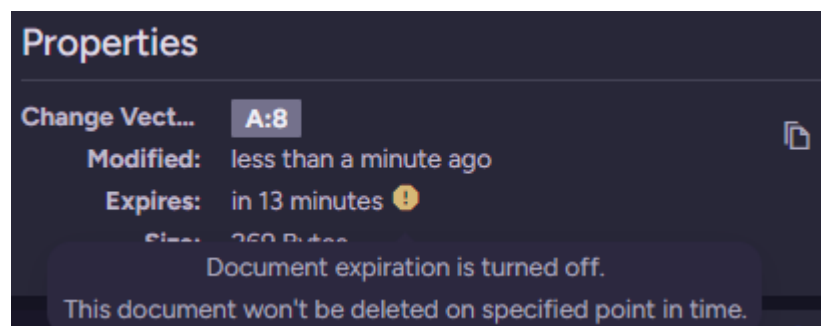


Figure 6.1. The Studio warns about setting expiration when it is not enabled

By default, expiration needs to be *explicitly enabled* on the database side. You can do that using the Studio, or by using the `ConfigureExpirationOperation` operation using the API. In the RavenDB Studio, go to the Settings' tab of your database, and then to the Document Expiration section, as shown in Figure 6.2, enable `Enable Document Expiration` and click on `Save`.

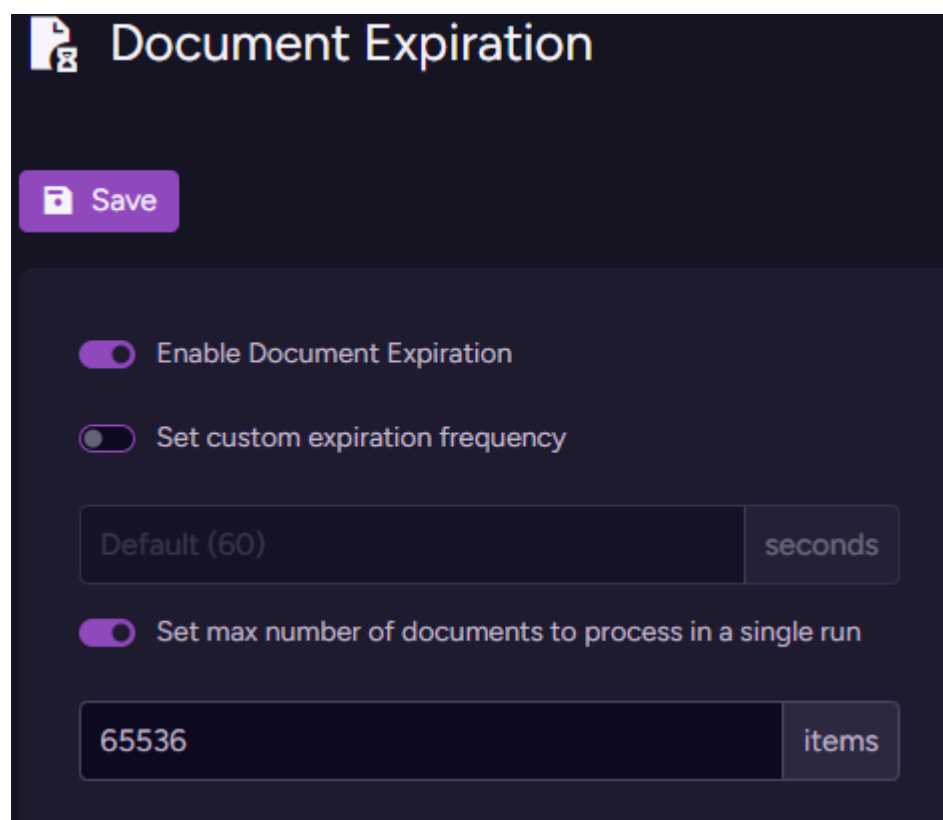


Figure 6.2. Enabling expiration in the Studio

After enabling expiration on the database, the documents that you marked for expiration will be automatically deleted after their expiration date passes. Note the options you can see in Figure 6.2: you can set the frequency in which RavenDB will check for expired documents (the default is every 60 seconds). In other words, if a document expires at 12:00:00, it will be deleted at some time around that time (based on the frequency you set and how many other documents you have that also needs to be expired).

I mention that because in some instances you may have a *lot* of documents set to expire at a given point in time. For example, you may have a promotional offer to all your customers that all expire at midnight on a given day. In that case, you may have millions of documents to expire at once. This

is also why you have the option to set the batch size in Figure 6.2. Instead of having to immediately delete all expired documents, RavenDB will delete them in batches, allowing it to control the amount of load that it has to deal with all at once.

Documents expiration in a cluster

When running in a cluster, with multiple nodes hosting the same database, how does RavenDB handles expiration?. Running expiration on multiple nodes at the same time would be inefficient and could lead to conflicts. To avoid that, expiration is handled by the database leader node. The expiration task will run only on the leader, and the deletions will be replicated to the other nodes. You'll learn more about the distributed nature of RavenDB in Chapter 9.

The practical impact of this behavior is that you should not rely on expiration to happen at an *exact* time. RavenDB does the expiration check periodically and in batches, so there may be a delay between the expiration time and the actual deletion of the document. In practice, however, that doesn't usually matter. If an expiration link was set with expiration and lasted for 30 minutes or 30 minutes and 40 seconds, it is fine. Same for a promotional offer that lasted for a few extra minutes after midnight before being deleted. If you do care about the exact time of expiration, you should also check that that in your application logic as well.

6.2. Refreshing Documents

In addition to expiring documents using `@expires`, RavenDB also supports refreshing documents using the `@refresh` metadata field. The refresh mechanism is similar to expiration, but instead of deleting the document, it will update the document to remove the `@refresh` metadata field when the time comes. That is all it does. We'll first take a look at how this works and then we'll discuss *why* this is useful. Let's see how to set the `@refresh` metadata field on a document in Listing 6.2.

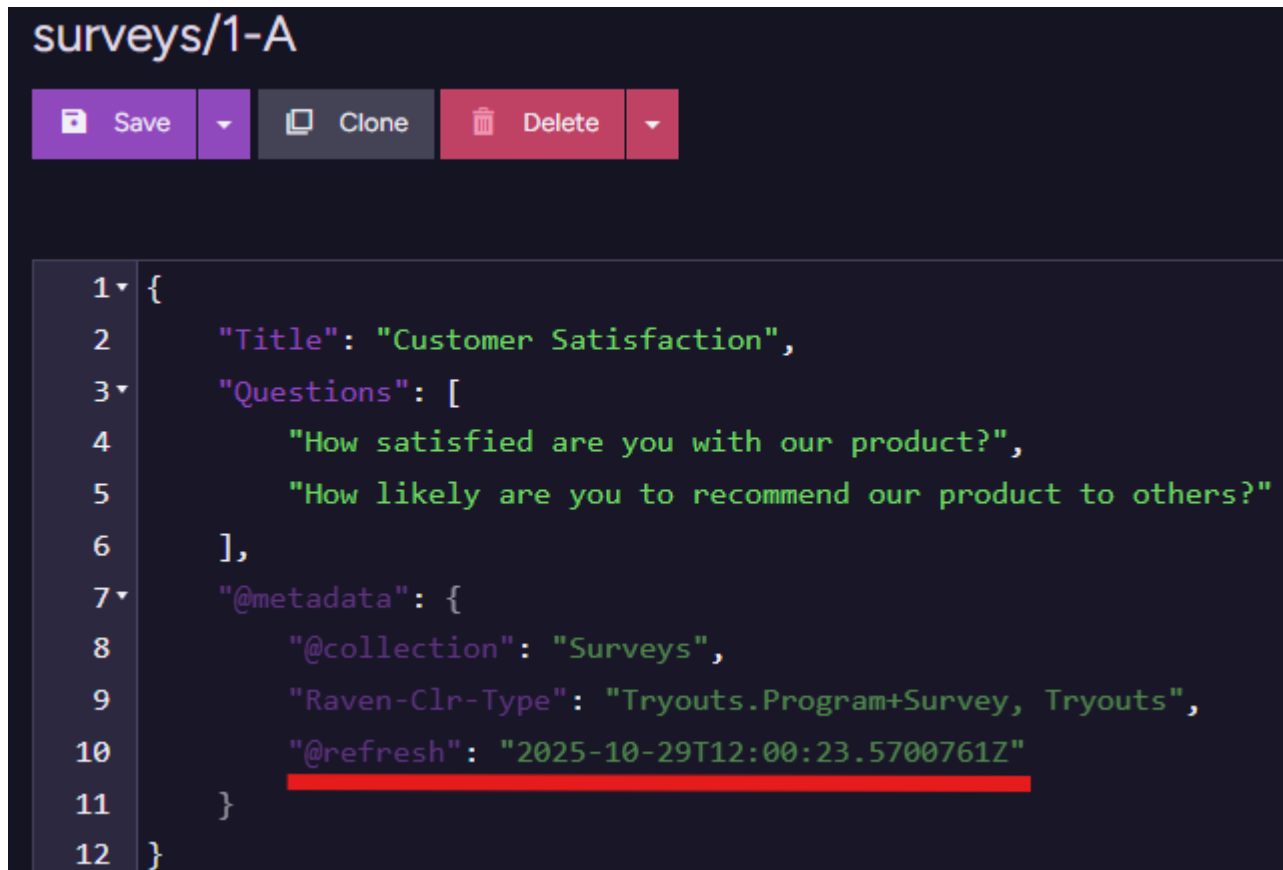
Listing 6.2. Set the refresh data on a document.

```
using (var session = store.OpenSession())
{
    Survey survey = new Survey
    {
        Title = "Customer Satisfaction",
        Questions =
        [
            "How satisfied are you with our product?",
            "How likely are you to recommend our product to others?"
        ]
    };
    session.Store(survey);
    var metadata = session.Advanced.GetMetadataFor(survey); ①
    var refresh = DateTime.UtcNow.AddDays(3).ToString("O"); ②
    metadata["@refresh"] = refresh;                             ③
    session.SaveChanges();
}
```

```
}
```

- ① Get the metadata for the newly stored survey document.
- ② Refresh uses UTC date in ISO 8601 format (yyyy-MM-ddTHH:mm:ss.ffffffZ).
- ③ Set the @refresh metadata field to the desired refresh date.

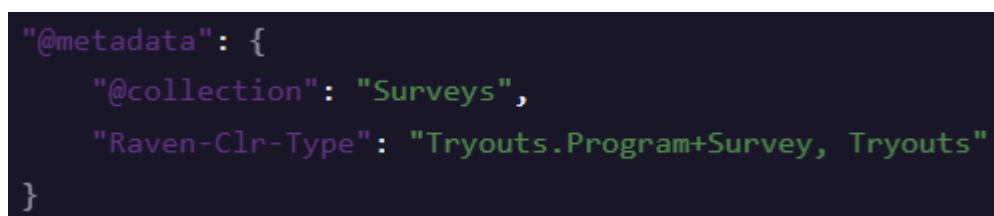
Running the code in Listing 6.2 will create a Survey document that will be refreshed in 3 days. You can see how that document will look in the Studio in Figure 6.3.



```
1 {  
2   "Title": "Customer Satisfaction",  
3   "Questions": [  
4     "How satisfied are you with our product?",  
5     "How likely are you to recommend our product to others?"  
6   ],  
7   "@metadata": {  
8     "@collection": "Surveys",  
9     "Raven-Clr-Type": "Tryouts.Program+Survey, Tryouts",  
10    "@refresh": "2025-10-29T12:00:23.5700761Z"  
11  }  
12 }
```

Figure 6.3. A survey document with the @refresh metadata field set

After 3 days pass, RavenDB will automatically update the document to remove the @refresh metadata field, as shown in Figure 6.4.



```
"@metadata": {  
  "@collection": "Surveys",  
  "Raven-Clr-Type": "Tryouts.Program+Survey, Tryouts"  
}
```

Figure 6.4. The survey document after being refreshed

Note that in Figure 6.4, the document is otherwise unchanged. Only the @refresh metadata field was removed.

You might be wondering about why this feature even exists. The refresh feature is meant to be used in conjunction with other RavenDB features, not on its own. The actual point of using @refresh is to trigger *other parts* of RavenDB, which act on document modification. You'll see an example of how

we can use `@refresh` in Chapter 7, we use it to trigger processing of documents using subscriptions at a given time.

RavenDB has quite a number of features that are triggered by document modifications, such as indexing, subscriptions, ETL, AI integration, and more. By using `@refresh`, we can schedule things so that they will happen at a given time in the future, without having to have an external process to do that for us.

Refresh is one-shot only

The `@refresh` metadata field is removed at the scheduled time, so it is a one-shot operation. If you want to have a recurring refresh, you'll need to set the `@refresh` field again. There is no way to set a recurring refresh in RavenDB.

In almost all respects, the `@refresh` mechanism is similar to the `@expires` mechanism. The `@refresh` feature also needs to be explicitly enabled on the database side, and both features run on the primary database node and replicate the changes to the other nodes in a cluster. You can enable the `@refresh` feature in the Studio, by going to the Settings tab of your database and then to the Document Refresh section or using `ConfigureRefreshOperation` API.

On its own, the `@refresh` feature is useless, but when combined with other RavenDB features, it becomes very powerful mechanism to schedule work to be done at a later time.

6.3. Archiving Documents

Once your database hits a certain size, you need to start thinking about what exactly you are *doing* with all this data. In many cases, you may have documents that are no longer actively used, and while RavenDB has no problems handling large datasets, you may want to move old documents out of the way, to keep your active working set smaller.

An easy way to handle this is to mark document with `@expires` metadata field, so they will be automatically deleted after a certain time. But deleting data is a permanent operation, and not something to do just to make the task of the database easier. After all, regardless of the size of the data in question, it is the role of the *database* to handle that. RavenDB is perfectly happy with large datasets, including multi-terabyte databases. If you want to go beyond the amount of data that a single server can handle, you can use sharding to spread the load across multiple servers (see Chapter 11 for more details).

However, in many cases, you want to both keep the old data around, but it doesn't make *sense* to keep orders data from three years ago in the same manner as your latest order. RavenDB allows you to define an archiving time for your documents, which lets RavenDB aggressively optimize how it deals with those documents, assuming that they are rare accessed.

Just like `@expires` and `@refresh`, you can set the archiving time using the `@archive-at` metadata field on your documents. And just like the other two features, you need to enable archiving explicitly. You can do that in the RavenDB Studio by going to the Settings tab of your database, and then to the Data Archival section. Given that you just read about `@expires` and `@refresh`, this should be very familiar to you by now and I'm going to skip the details here. Check the previous sections to get the

full picture, since @expires, @refresh and @archive-at work in a very similar manner.

When you store a document with the @archive-at metadata field set, RavenDB will automatically move that document to the archived mode when the time comes. What exactly does that mean? An archived document is still fully accessible, you can read it and modify it normally. However, there are a number of very important differences in how RavenDB handles archived documents:

- Indexes exclude archived documents by default, when a document is marked as archived, RavenDB will remove it from the index (as if it was deleted). This lets RavenDB keep its indexes smaller and more efficient. If an archived document is modified later on, it will be ignored by the index. You can also configure an index to exclude archive documents (the default), include archive documents, or *only* index archive documents.
- Subscriptions (discussed in Chapter 7) also exclude archived documents by default. Just like with indexes, you can configure this behavior on a per subscription basis.
- Archived documents are stored on the disk in a compressed format, which reduces their disk usage significantly, at the cost of some CPU usage when reading them. We'll talk about the compression mechanism in the next section.

The end result is that archived documents are stored in a compressed form, aren't touched by indexes or subscriptions. That lets RavenDB discard it from the working set of the database, so it will not take any memory from the server. They are stored in "cold storage" on the disk, and only touched when requested directly by id. This allows you to keep large amounts of old data around, without impacting the performance of your day to day operations.

Note that those three behaviors (ignoring during indexes, skipping in subscriptions and compressing on the disk) are the only differences in how RavenDB handles archived documents. In particular, ETL processes or replication are *not* impacted by the archived state of documents, they will be replicated or take part of ETL processes normally.

A typical use case for document archiving in an e-commerce system would be to archive orders documents that are over two years old. Those documents are rarely accessed, but you may need to get to them once in a while for auditing or other purposes. When you create an order document, you can set the @archive-at metadata field to the last day of the year, two years in the future. For example, "@archive-at": "2028-01-01T00:00:00.000000Z" for an order created today (November 2025). That way, at the start of 2028, RavenDB will automatically archive all the documents, removing it from indexes.

Selecting the archive date has an impact on your system

You can simply set the archive date to be two years from the creation date of the document. That would mean that the archiving process will be spread evenly over time. In practice, that isn't usually how businesses operate. Most businesses have a yearly cycle. It is very common to need to want to search over data for the last year or the previous one. For that reason, it is often better to set the archive date to be the same for all documents in a given year. For example, for all orders created in 2025, set the archive date to be 2028-01-01T00:00:00.000000Z.

That is something that you want from a business perspective, but it also has technical

advantages. When the new year rolls by, RavenDB now needs to archive all the documents from two years ago. That can be a significant amount of data, and RavenDB will handle that in the background. Note that archiving, just like `@expires` and `@refresh`, happens on the primary node in a cluster, and the changes are replicated to the other nodes. And like expiration and refresh, RavenDB is going to *start* archiving the documents at the specified time, but it may take some time to complete the process.

Following the archiving in Jan 2028, queries on orders for a customer, will only consider the documents from 2026 and 2027. If you need to read an archived order, you can do that directly by id. Alternatively, you can create a dedicated index that includes archived documents, to be able to query them as well. The "Orders/Archived" index is configured to only index archived documents, and it is queried only on very specific scenarios, such as auditing or generating reports over old data.

Using the example above, with the archive date set to the start of the year, that index for archiving documents will only need to index documents once a year, when the archiving process runs. I'm jumping ahead a bit, since we'll cover indexing in depth in Chapter 13, but I wanted to explain how archiving and indexing interacts. This "Orders/Archived" index will run once a year, and if (as expected) there are no queries on this index, RavenDB is able to idle the index completely. In other words, the archive index will consume no resources from the system while there are no queries on it.

6.4. Documents Compression

Documents in RavenDB are stored on the disk in a binary format. This can be a surprise to some, since RavenDB uses JSON as its document format. However, for internal storage, RavenDB uses a binary format that is much more efficient to store and process. The primary reason why RavenDB uses a binary format is performance. Parsing JSON is a costly operation, and RavenDB needs to be able to read and write documents very quickly. The binary format used internally is designed to allow to process documents in a zero copy manner, without the need to parse or allocate memory for the documents.

The details of the binary format are not relevant for this book, primarily because it is an internal implementation detail. It is worth to understand the implications of this format. In particular, how this format deals with strings, and especially large strings. If you have a document with a large string property (HTML content, large text, etc), RavenDB natively applies compression to the value.

There are actually *several* layers of compression that RavenDB applies to documents. If we are dealing with a string that is less than 128 characters, RavenDB uses an internal compression mechanism for small string compression. If you are interested in the details, the following blog post [<https://ayende.com/blog/172865/reverse-engineering-the-smaz-compression-library>] goes into the details of how that works.

When the string in question is larger than 128 characters, RavenDB uses the LZ4 compression algorithm to compress the string value. LZ4 is a very fast compression algorithm that provides good compression ratios at high speed. This compression is done transparently by RavenDB, so you don't have to do anything special to take advantage of it. A good example of the impact that this can have on large string properties is when you have HTML content. For example, consider the posts/203141-

A document in Figure 6.5. You can find the actual blog post [<https://ayende.com/blog/203141-A/a-deep-dive-into-ravendb-ai-agents>] directly, by the way.

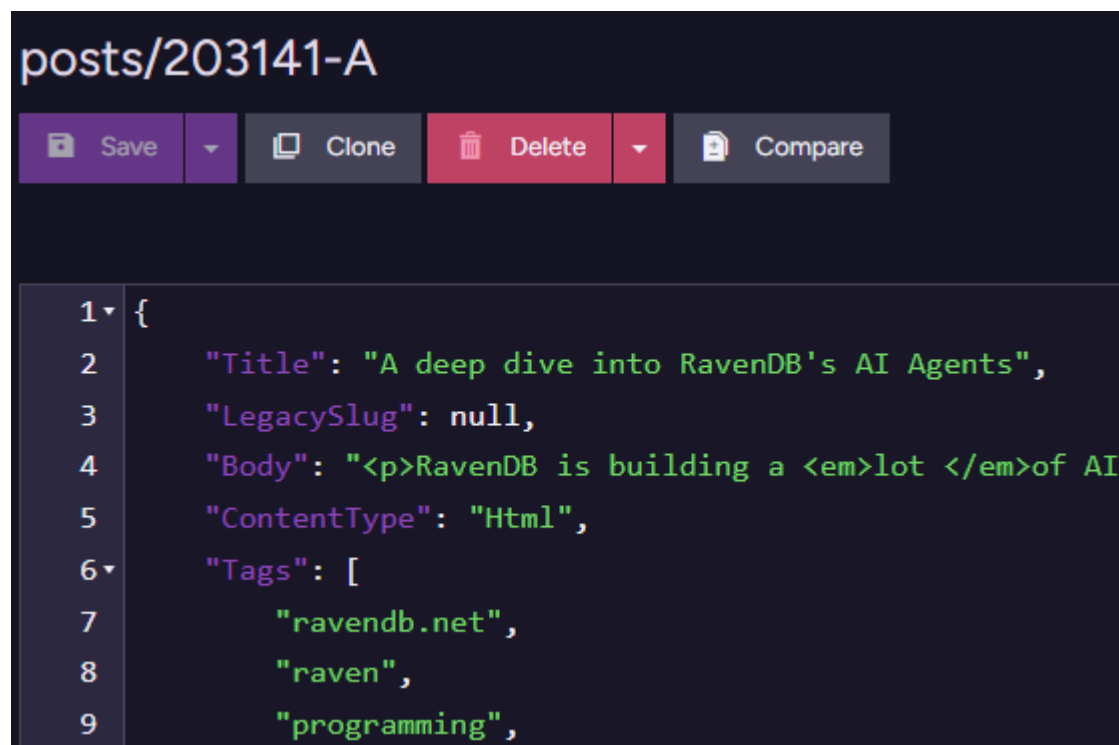


Figure 6.5. A BlogPost document with large HTML content

You can see in line 4 in Figure 6.5 that the Body property contains a large HTML content. In this case, it is 39,939 characters long. And the total document size, as JSON, it is 40.78 KB. RavenDB is able to compress that down to 22.82 KB. You can see those details in the RavenDB Studio, in the document properties, as you can see in Figure 6.6.

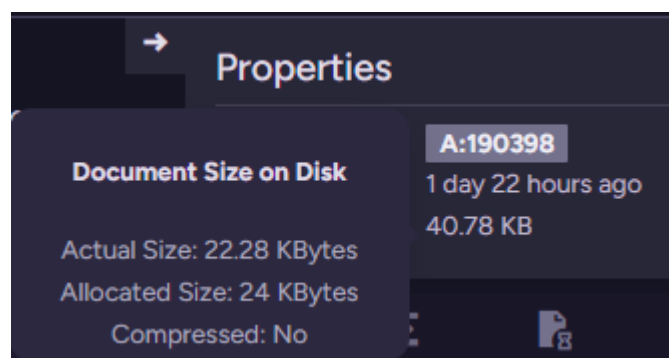


Figure 6.6. Showing the size of a large document in the Studio, as JSON and on disk

This behavior means that there are actually three separate sizes for a document inside of RavenDB. Using the posts/203141-A shown in Figure 6.5 and Figure 6.6, we have:

- The size of the document using JSON, which is 41.35 KB. This is relevant when transferring the document over the network, since RavenDB uses JSON as its wire format.
- The compressed size of the document on the disk, which is 22.82 KB.
- The allocated size of the document on disk, which is 24.00 KB. Internally, RavenDB allocates disk space in 8KB increments (called pages), so the actual size on the disk is rounded up to the next page size. Documents that are smaller than 4KB will be stored in a more compact manner, to avoid wasting an entire 8KB page for a single small document.

Technically speaking, the maximum size of a document inside of RavenDB is 2GB. In practice, automatic large string compression means that you can store larger documents, since we care about the size of the document on disk, not its size as JSON. That is not an endorsement for large documents, just an explanation of how things looks like from the inside.

Compressing a single string property is often suprising effective, but it has limits. If the document contains an array of complex objects, for example, a large part of the actual document size is the overhead of JSON itself. For example, the repeated property names for each item in the array. RavenDB deals with that as well, and will store only a single copy of the property name for each unique property name in a document. Again, this is part of the internal binary format, not something that you should be aware of. I mention it here to explain how RavenDB is able to store documents efficiently on the disk.

RavenDB does not apply compression for attachments

It is worth laying it out expclicitly: RavenDB does *not* apply any compression to attachments. Attachments are stored as-is on the disk, without any modification. This is because attachments can be of any type, and RavenDB has no way to know if the attachment is already compressed. Most modern file types are already natively compressed (JPEG, PNG, PDF, DOCX, XLSX, ZIP, etc), so applying additional compression would be a waste of CPU cycles with little to no gain.

If you store attachments in RavenDB, make sure that it is either in an already compressed format or compress it yourself before storing it in RavenDB.

Having explained all the internal details about how RavenDB stores JSON documents, I want to talk about the next step. Compressing documents, beyond just avoiding repetitive property names and compressing large strings. The simplest way to go about compressing documents is to simply use a general purpose compression algorithm, such as LZ4 or ZSTD, to compress the entire document. That would be simplest option, but it isn't the most effective one. To understand exactly why, we need to understand exactly what compression does.

Compression algorithms work by finding repetitive patterns in the data and replacing them with shorter representations. The problem is that you need to have enough data to be able to find those patterns. If most of your documents are small, there is not enough repetitions inside the document itself to be able to compress it effectively. The best way to deal with that is to compress multiple documents together, so the compression algorithm can find patterns across documents. However, if we compress multiple documents together, we need to decompress all of them when we want to read a single document. That makes reading a single document a *lot* more expensive, since we need to decompress multiple documents just to get to the one we want.

RavenDB deals with this problem in quite an interesting manner. RavenDB allows you to enable document compression for specific collections and for revisions, as you can see in Figure 6.7, where we specified that the `Orders` collection should have be compressed, as well as revisions for all collections.

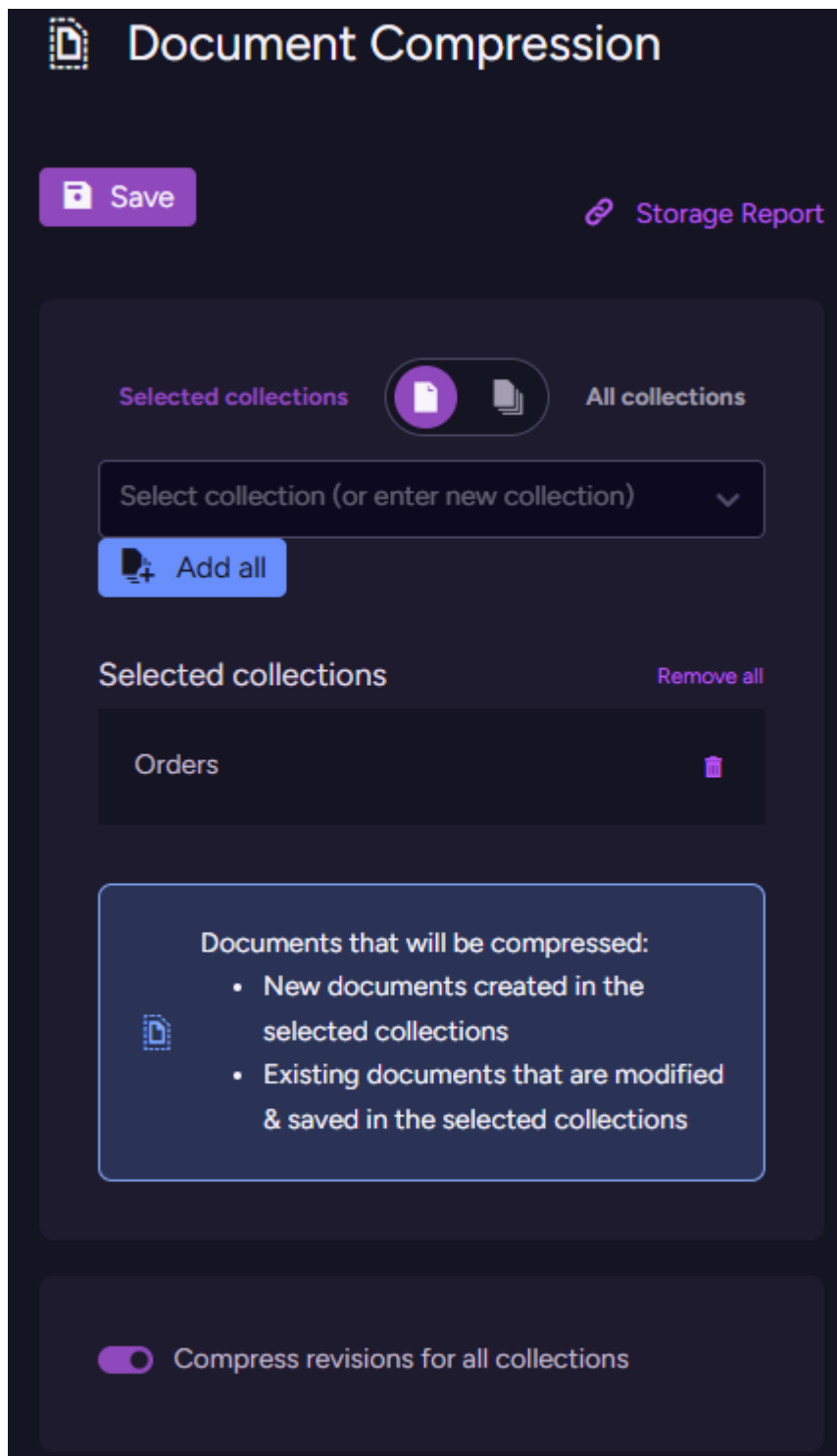


Figure 6.7. Enabling document compression for a collection in the Studio

At the bottom of Figure 6.7, you can see that enabling compression will only impact new documents. Existing documents will remain uncompressed until they are modified. Now, let's talk about how RavenDB actually compresses those documents.

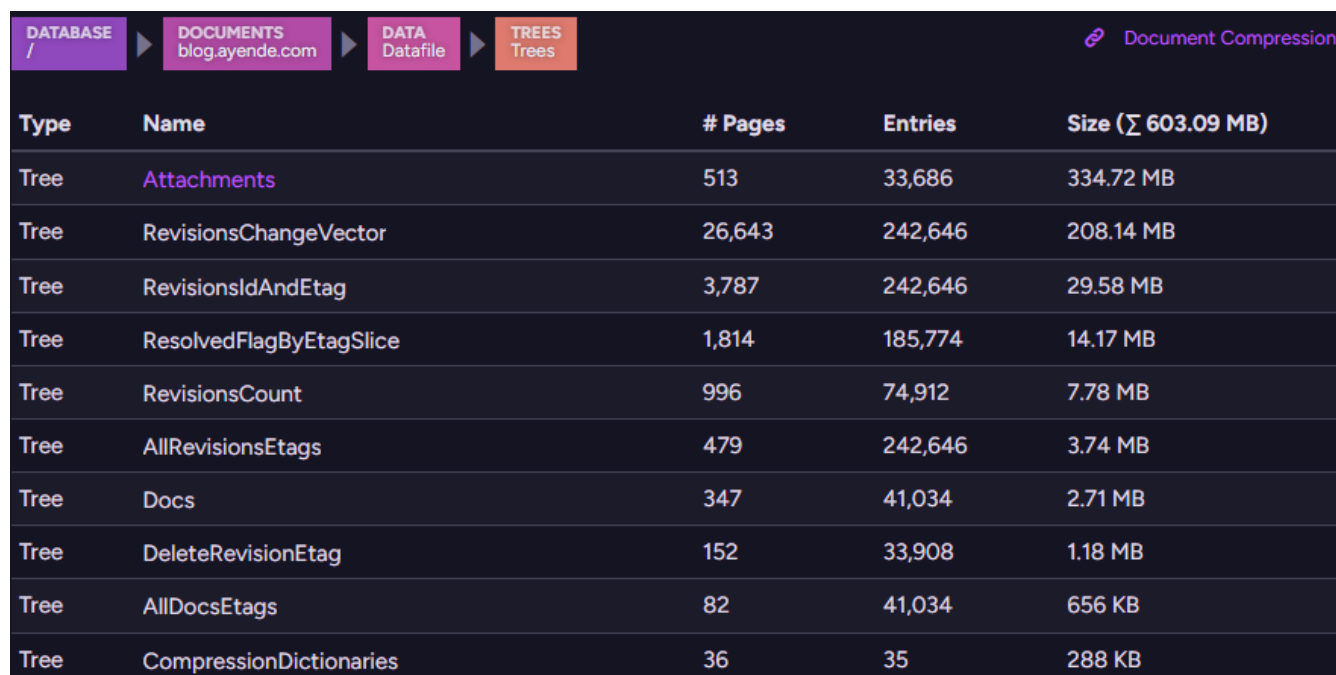
To get efficient compression for documents, including being able to take advantage of patterns across documents, RavenDB uses a technique called dictionary compression. The idea is that it scans a bunch of documents together, and builds a dictionary of common patterns across those documents. Then, when storing each document, RavenDB uses that dictionary to compress the document individually. In other words, using the example of `posts/203141-A`, RavenDB will detect that the string `Body` or `ContentType` appears in many documents, and will store a single copy of that string in the dictionary. When compressing the document using the dictionary, RavenDB can detect

that the string `Body` appears in the document, and replace that with a reference to the dictionary entry. In other words, instead of storing 11 bytes for `ContentType`, RavenDB can store a lot less.

RavenDB isn't limited to just compressing property names. It can also detect common values across documents. For example, if you have a date value and many of your documents share the same `2026-01-` because they were all created in January 2026, RavenDB can store that common prefix in the dictionary as well. The same applies to other common values, such as status codes, common strings, etc.

I'm giving the example about dates intentionally, because it is obvious that as soon as the month or year roll by, we'll lose that common prefix. RavenDB assumes that the repetitive patterns that it takes advantage of will change over time. For that reason, RavenDB monitors the compression rate of the current compression dictionary for each collection. If the compression rate goes below a certain threshold, RavenDB will rebuild the compression dictionary for that collection and test to see if we can find better patterns to take advantage of. If the new dictionary provides better compression, it will start using that one instead.

At any given point, RavenDB will have (for each collection) a set of compression dictionaries that is used, as well as a single active dictionary that it will use for new or modified documents. That is exposed in the RavenDB Studio under `Stats > Storage Report`, and then under `Database > Documents > Datafile > Trees > CompressionDictionaries`. You can see how this looks like in Figure 6.8, where the last entry shows the number of compression dictionaries that were generated.



Type	Name	# Pages	Entries	Size (Σ 603.09 MB)
Tree	Attachments	513	33,686	334.72 MB
Tree	RevisionsChangeVector	26,643	242,646	208.14 MB
Tree	RevisionsIdAndEtag	3,787	242,646	29.58 MB
Tree	ResolvedFlagByEtagSlice	1,814	185,774	14.17 MB
Tree	RevisionsCount	996	74,912	7.78 MB
Tree	AllRevisionsEtags	479	242,646	3.74 MB
Tree	Docs	347	41,034	2.71 MB
Tree	DeleteRevisionEtag	152	33,908	1.18 MB
Tree	AllDocsEtags	82	41,034	656 KB
Tree	CompressionDictionaries	36	35	288 KB

Figure 6.8. Compression dictionaries in the Studio

Each dictionary uses about 8KB in size, and once created, can never be deleted. This is because existing documents may still be using that dictionary for compression, so RavenDB needs to keep it around. In practice, however, the number of dictionaries per collection is usually small, so this isn't a significant overhead. RavenDB will only create a new dictionary when the current one is no longer effective, so in many cases, you will use the same dictionary until your data patterns change significantly.

The costs of document compression

Compression is a trade off. On the one hand, you reduce the amount of disk space that documents take. On the other hand, you need to spend CPU cycles to compress and decompress documents. When we built the documents compression feature, we assumed that you'll want to use it primarily for collections that are not frequently modified or read, especially if you have a lot of them. A good example of that are log entries, where you may have millions of documents that are rarely accessed. In that case, the savings in disk space are worth the extra CPU cycles needed to read. Archived documents are also another example of when we want to spend CPU cycles to reduce your costs. Indeed, archived documents are compressed using this exact technique.

That was the plan, at least. In practice, we found that even for frequently accessed collections, overall performance improved when using document compression. The reason for that is that by compressing the document data on the disk, we also reduce the amount of I/O operations that we required for both reads & writes. The I/O costs are often the bottleneck in database operations, so the reduction in I/O more than makes up for the extra CPU cycles needed for compression.

This is especially true on the cloud, where you are rarely burning through all your CPU cycles, but are often starved for I/O. Document compression can significantly improve the overall performance as well as reduce your cloud costs.

By default, documents compression is enabled for revisions, but not for collections. You need to explicitly enable it for each collection that you want to compress. You can also enable that globally for all collections. It isn't a panacea, but it is something that you should at least look at in terms of reducing disk usage and improving performance as part of your initial testing. Note that you can also *disable* document compression after you enabled that. However, that will not decompress existing documents. Instead, RavenDB will simply stop compressing modified documents going forward.

An easy way to see the impact of document compression is to enable or disable compression and then ask RavenDB to compact the database, as shown in Figure 6.9.

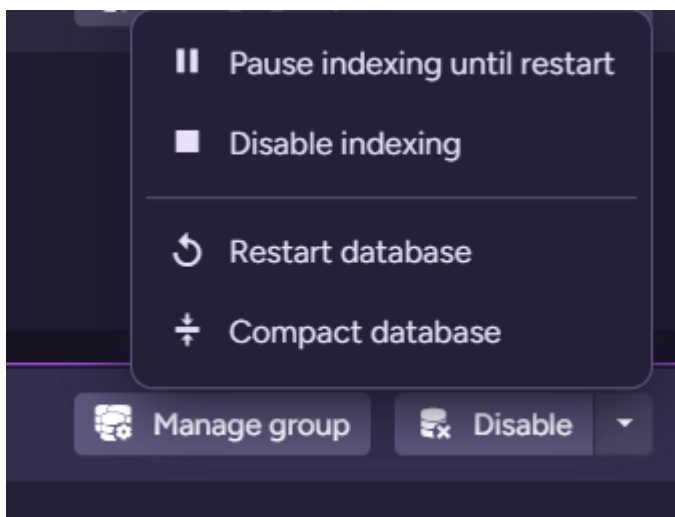


Figure 6.9. Starting database compaction for the database in the Studio

Clicking on the Compact database shown in Figure 6.9 will start the compaction process for the database. This forces RavenDB to rebuild the database file from scratch, along the way it will compress or decompress documents as needed. After the compaction is done, you can check the size of the database file on the disk to see the impact of document compression. Note that compaction is an offline operation, so the database wouldn't be available while it is running, and that it may take a significant amount of time, depending on the size of your database. We'll discuss database compaction in more detail in Chapter 20.

6.5. Schema validation

!!!TODO!!!

6.6. Remote attachments

!!!TODO!!!

Chapter 7. Batch Processing With Subscriptions

RavenDB is typically used for transaction-oriented operations. In other words, we usually have a request that needs to read or write a small number of documents, we load the relevant documents, modify them based on our business logic and save them back to the database. In those cases, there is a user waiting for the operation to complete. In other words, this is RavenDB deployed as the backend data store for an application or an API.

In most systems, beyond processing client requests, there is also non trivial amount of background work that needs to be done. A new employee in the company needs to go through orientation, and an ordered marked Ready needs to be picked up and processed, etc. For example, we have a new requirement. All London based employees must go through online safety course. As you can see in Listing 7.1, that is very simple to do.

Listing 7.1. Send all the London employees to an online safety course

```
using (var session = store.OpenSession())
{
    var londonEmployees = session.Query<Employee>()
        .Where(x => x.Address.City == "London")
        .ToList();
    foreach (var employee in londonEmployees)
    {
        // ... redacted ...
    }
}
```

The code in Listing 7.1 is very simple. But there is a problem. It works for the *current* employees only. What happens if we hire a new employee? Or if an employee from Seattle move to London? The code in Listing 7.1 won't handle that. We'll need to run the code Listing 7.1 again, find out which employees were already sent to the course, handle the case if we have a *lot* of employees in London, etc.

You *can* implement this sort of functionality, but it is pretty complex to do that properly. Furthermore, you'll need to implement something similar whenever you need to do any such background processing for any other scenario. Luckily, RavenDB provides a more elegant solution. RavenDB supports a dedicated batch processing mode, using the notion of subscriptions.

7.1. Processing a batch of documents in a subscription

A subscription is simply a way to register a query with the database and have the database send us all the documents that match the query. So far, this sounds very much like the streaming feature, which we covered in the previous chapter. However, subscriptions are *meant* for batch processing. We'll first look at an example of using subscriptions, and then dive more deeply into exactly what is going on here. You can see our first example of using subscription in Listing 7.2.

Listing 7.2. Using batch processing to send all London employees to online safety course

```
store.Subscriptions.Create<Employee>(                                ①
    x => x.Address.City == "London",
    new PredicateSubscriptionCreationOptions
    {
        Name = "london-employees"
    });
var worker = store.Subscriptions
    .GetSubscriptionWorker<Employee>("london-employees");           ②
await worker.Run(async batch =>                                     ③
{
    foreach (var employee in batch.Items)                           ④
    {
        // ... redacted ...
    }
});
```

- ① Create a new subscription for all employees in London, named london-employees
- ② Get the subscription worker for the london-employees subscription
- ③ Run the subscription, with a lambda that accepts a batch of matching employees.
- ④ Process each employee in the batch.

Listing 7.2 creates a new subscription, get the worker for that subscription and run it. The worker will then get a batch of matching employees and execute the provided lambda on the batch of employees. The key here is that this code will *handle* all the details of keeping track of which employees were already processed. New employees or employees that moved from Seattle to London will be included in the next batch, etc. You just need to provide the code that will process each batch of employees, and RavenDB takes care of everything else.

Subscription gets really interesting when it's done going through all the matching documents in the database. This is where things At this point, the subscription *isn't* done. Instead, it's kept alive and waiting for new or updated documents that match the query. When that happens, the server will immediately send that document to the subscription. In other words, not only did we get traditional batch processing, but in many cases, we also have *live* batch processes.

Instead of waiting for a nightly process to run, you can keep the subscription open and handle changes as they come in. This keeps you from having to do things like polling and remembering the last items that you read. RavenDB's subscription also includes error handling, high availability and on-the-fly updates.

With subscriptions, work is divided into two distinct operations. First we need to create the subscription. Then we need to open it. Subscriptions aren't like queries; they aren't ephemeral. A subscription is a persistent *state* of a specific business process. It indicates what documents that particular subscription has processed, the query it is using, etc.

You can also create the subscription through the RavenDB Studio. In the same database that we used in the previous chapter, go to Settings and Manage Ongoing Tasks, click on Add a Database Task and in the Filter by Category select `Subscriptions. You'll see the task selection dialog shown in

Figure 7.1.

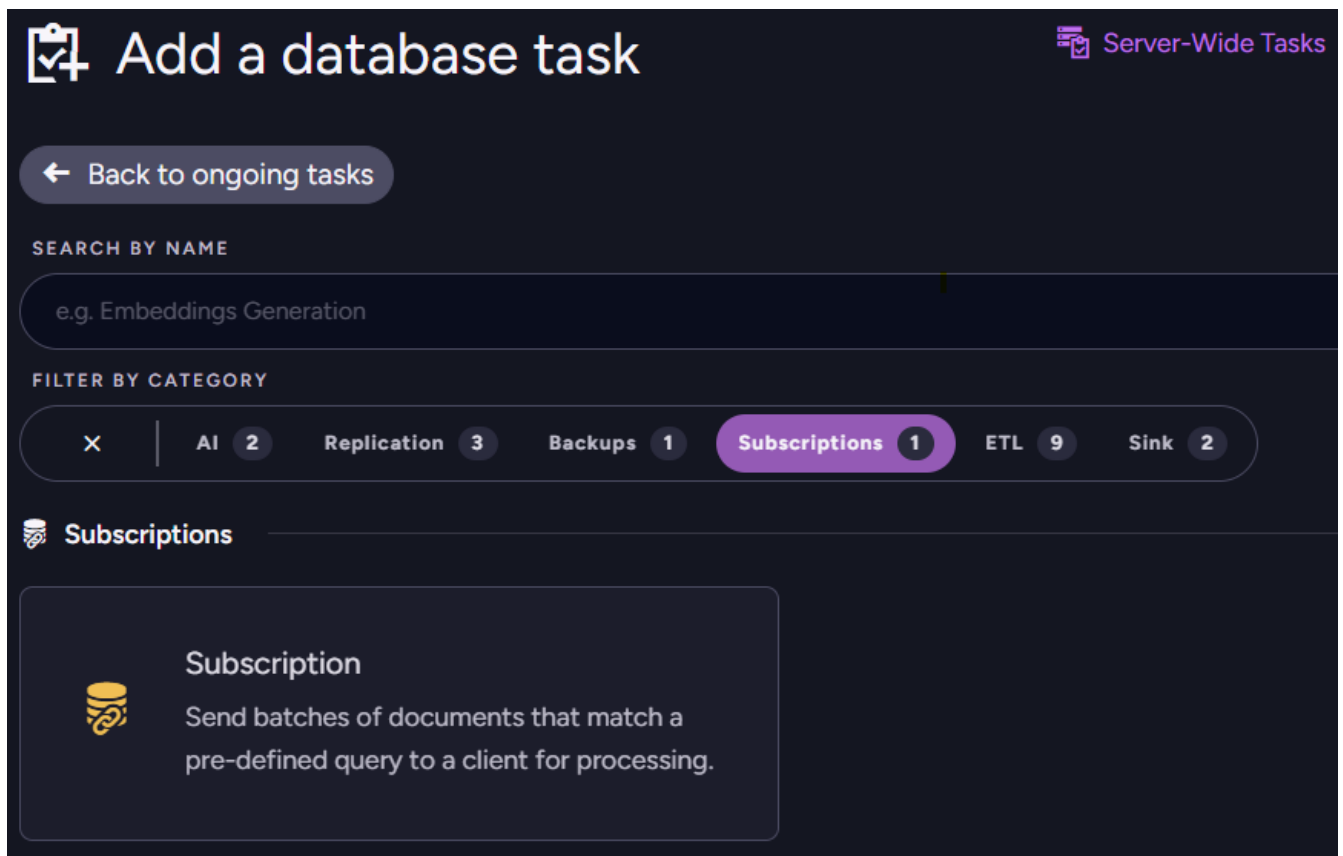


Figure 7.1. Add a new database task

You'll see the subscription edit form. Now, name the task "CustomersSubscription" and enter from Customers in the query field. That's it. You can save the subscription. The result of this is shown in Figure 7.2.

New Subscription

Info Hub

Save Cancel Test Subscription

Task Name london-employees

Task State Enabled

Subscription RQL: Syntax ?

```
1 from "Employees"
2 where Address.City = 'London'
```

Press Shift+F11 to enter full screen mode

Figure 7.2. The new subscription dialog, matching employees in London

The subscription in Figure 7.2 match the one we defined in Listing 7.2 and regardless of whether you created the subscription manually or via code, the result is a named subscription. At the top right of Figure 7.2, you can see the button to test a subscription. If you click on Test Subscription, you'll see a result similar to Figure 7.3.

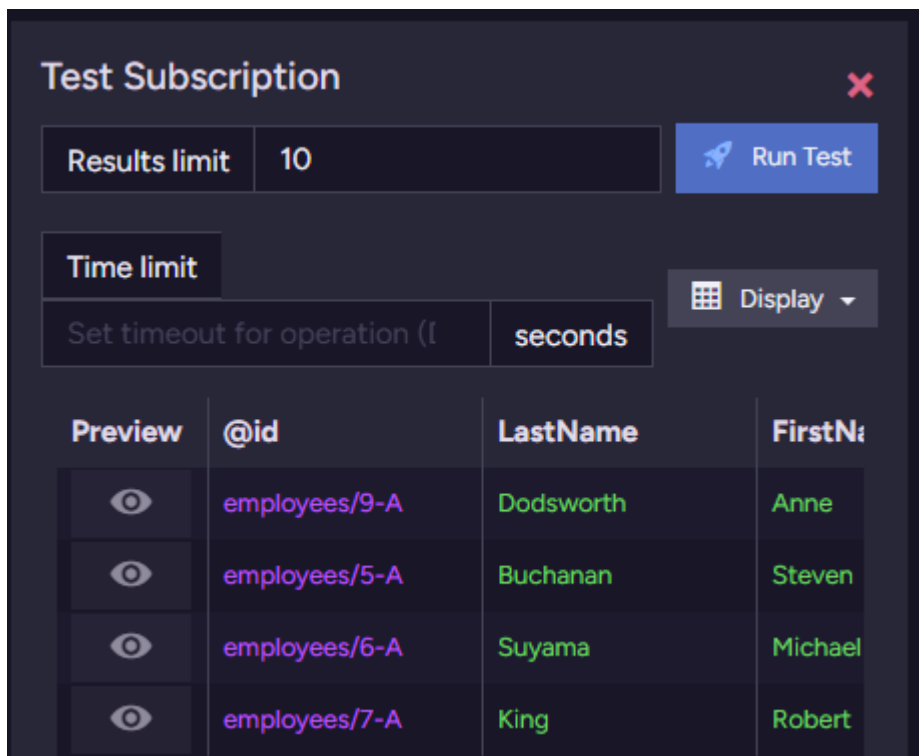


Figure 7.3. The results of testing the employees subscription

Those are the documents in the Employees collection that match the where clause we defined. If we were to run the same query in the Query window, we'll see the exact same result. On its own, a separate way to query documents isn't really that interesting. The point of subscriptions isn't for you to manually go to the Studio and inspect things. The point is that RavenDB will send those details to your code. You can see this at the bottom of Listing 7.2, where we open the subscription and start handling batches of documents on the client.

Listing 7.2 shows the typical way of using a subscription. The subscription is opened with the subscription name that we created previously and then we Run the subscription. The actual batch processing of the documents is done by the lambda that's passed to the Run call on a background thread.

After calling Run, we wait on the returned task. Why do we do that? In general, a subscription will live for a very long time. Usually the lifetime of the process that's running it. In fact, you'll often have a process dedicated just to running subscriptions (or even a process per subscription). What we're doing at the end of Listing 7.2 is basically waiting until the subscription exits.

A subscription will not exit on its own. It will keep running, processing documents, until we explicitly tell it to stop. Problems with the network will simply cause it to retry or failover to another node. If it processed all the data that the database had to offer, it will sit there and wait until it has something more to give, rather than exiting.

There are a few cases where the subscription will actually exit. The first, of course, is if you intentionally close it by disposing the subscription. It's safe to dispose the subscription while it's running, and that's how you typically do an orderly shutdown of a subscription. We might have been kicked off the subscription for some reason. The admin may have deleted the subscription or the database. Or perhaps the credentials we used were invalid. Maybe the subscription connection was taken over by another client. We'll cover more on this in the subscription deployment section later on in this chapter.

The subscription ID should be persistent, usually stored in configuration or loaded from the database. It's used to get the state of the subscription and what documents have already been processed by that particular subscription.

You can set your own subscription ID during the Create call, which gives you a well known subscription name to use, or you can ask RavenDB to choose one for you. Note that even if you used your own subscription name, it still needs to be created before you can use it. But why do we have all those moving parts? We create the subscription, open it, run it and wait on the resulting task, and we haven't even gotten to the part where we actually do something using it.

The reason for this is that subscriptions are long-lived processes, which are resilient to failure. Once a subscription is created, a client will open it and keep a connection open to the server, getting fed with all the documents that match the subscription criteria.

Once we've gone over all the documents currently in the database, the subscription will become idle but remain connected to the server. Whenever a new or updated document matches the subscription query, it will be sent to the subscription. Errors during this process, either in the network, the server or the client, are tolerated and recoverable. The subscription will ensure that a client will receive each matching document at least once.^[1]

Subscription in a cluster

The subscription will connect to a server in the cluster, which may redirect the subscription to a more suitable server for that particular subscription. Once the subscription has found the appropriate server, it will open on that server and start getting documents from it. A failure of the client will result in a retry (either from the same client or another one that was waiting to take over). A failure of the server will cause the client to transparently switch over to another server.

The entire process is highly available on both client and server. The idea is that once you set up your subscriptions, you just need to make sure that the processes that open and process your subscriptions are running, and the entire system will hum along, automatically recovering from any failures along the way.

7.1.1. Understanding the subscription lifecycle

A subscription that has already processed all existing documents in the database has a lag between a new document coming in and the subscription receiving it of a few milliseconds. Under load, when there are many such documents, we'll batch documents and send them to the client for processing as fast as it can handle them. The entire model of subscription is based on the notion of batch processing. While it's true that subscriptions can remain up constantly and get fed with all the changes in the database as they come, that doesn't have to be the case. If a subscription isn't opened, it isn't going to miss anything; once it's re-opened, it will get all the documents that have changed while it was gone.

This allows you to build business processes that can either run continuously or at off-peak times in your system. Your code doesn't change, nor does it matter to RavenDB. A subscription that isn't opened consumes no resources. In fact, a good administrator will know that he can reduce the

system load by shutting down subscriptions that are handling non-time-critical information with the knowledge that once the load has passed, starting the subscriptions up again will allow them to catch up from their last processed batch.

This is made possible due to the following flow: on each batch that the client receives, it acknowledges the server once it has successfully completed processing that batch. The server will not move forward and send the next batch until it receives this acknowledgment. It is this acknowledgment that makes this process reliable. Usually you'll also have a *single* subscription worker running at any one time (although you can run multiple workers on the same subscription, if you want to scale out processing).

Now that we know what subscriptions are, and have some idea about how to create, test and run them. Let's talk about the impact of the subscription feature on your overall system design. Because of the way subscriptions work, they process a full batch of documents from the server, and it is a good way to run heavy processing on them. We aren't limited by the data streaming over the network, and unlike streaming, we don't have to worry about timing out if we have a long computation to run. As long as the client remains connected, the server will be happy to keep waiting for the batch to complete. The server will ping the client every now and then to see what its state is and to detect client disconnection at the network level. If that's detected, the connection on the server will be aborted and all resources will be released.

Subscriptions are background tasks

It may be obvious, but I wanted to state this explicitly: subscriptions are background tasks for the server. There's no requirement that a subscription will be opened at any given point in time, and a subscription that wasn't opened will simply get all the documents it needs since the last acknowledged batch. A subscription that don't have a client connected to it also consume no resources and require no work from the server.

That means that if a document was modified multiple times, it's possible that the subscription will only be called upon once. See the section about versioned subscriptions if you care about this scenario.

A simple way to think about subscription is that they are merely a cursor on the collection the subscription covers based on it's last update time. Note that this isn't how it works, but it is a useful mental model. When a client is connected to a subscription, RavenDB will start reading from that collection from the cursor position forward. For each document, it will evaluate the subscription where clause (if any) and if the document matches the filter, it will be added to the batch that will be sent to the client.

When the server accumulate enough documents for a batch, it send that to the client for processing. On the client side, the subscription worker will accept the batch from the server and invoke the lambda to process the batch. If the batch completed successfully, the client will acknowledge the server that the batch was processed. At that point, the cursor is moved forward past the last document in the batch.

If the client disconnects or the invoked lambda throws an error, the cursor remains where it was, and when the client reconnects, it will resume from the initial cursor position. In other words, the

client will get the same batch again. This is how subscriptions ensure that you won't miss any documents.

Subscriptions deal with distributed state

The model of subscriptions as a cursor on the last update time on the collection is an oversimplification, but it is a useful one. The actual implementation is more complex, because RavenDB is a distributed database, there are multiple nodes in the system which can accept writes and the server node that is processing the subscription doesn't have to be the node that accepted the write on a document.

In general, you can assume that the subscription will accept the documents in *rough* modification order. You can also move the "cursor" that the subscription points to if needed by setting the Use a defined starting point in the edit subscription form or using the ChangeVector through the API. We'll learn more about change vectors in Chapter 9.

One of the things to be aware of is that right from the subscription lambda itself we can open a new session, modify the document we got from the subscription, Store the document and call SaveChanges on it. That is perfectly valid, but also a bit problematic. Remember that the subscription is essentially a cursor on the last modified time of the documents in the collection. If we modify the document, it will get a new last modified time, which means that it will be after the cursor position.

At this point, the subscription will process it again. And if it is a match to the subscription query, it will include that in the batch again. You need to be aware of that and protect against such infinite loops. You can use a field to the document that indicates that it was already processed by this subscription and filter based on that field in the query. There are also a few other subtle issues that we need to handle with regards to running in a cluster and failover. We'll discuss those issues later in this chapter.

7.2. The first real subscription

Subscriptions so far are useful but not really something to get excited about. But the fun part starts now. Let's say that we want to send a survey to all customers with whom we had a complex support call. The first step for that is to create a subscription using the code in Listing 7.3.

Listing 7.3. Creating a subscription for complex calls

```
var subId = store.Subscriptions.Create<SupportCall>(
    call =>
        call.Comments.Count > 25 &&
        call.Votes > 10 &&
        call.Survey == false
);
```

Figure 7.4 shows the newly created subscription on the server side. You can see that the filter we provided on the client side was turned into a server side script that decided whatever a particular SupportCall is a match for this subscription.

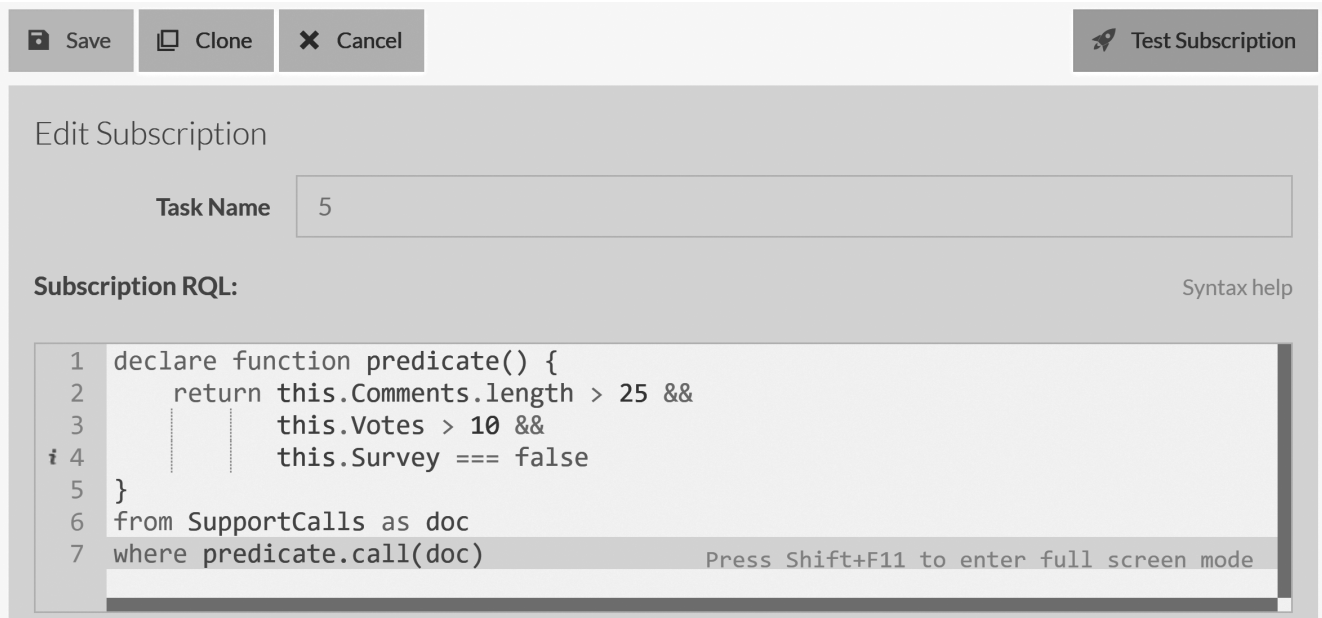


Figure 7.4. Subscription query defined from code, filtering complex support calls

We’ve registered for support calls that have more than 10 votes and over 25 comments, and we add a flag to record whether or not we’ve sent the survey. It’s important to note that this filtering is happening on the *server side*, not on the client. Internally we’ll transform the conditional into a full query (see Figure 7.4) and send it to the server to be evaluated on each document in turn. Any matching document will be sent to the client for processing. Of course, this is just part of the work. We still need to handle the subscription itself. This is done in Listing 7.4.

Listing 7.4. Taking surveys of complex calls

```
await subscription.Run(batch =>
{
    foreach (var item in batch.Items)           ①
    {
        SupportCall call = item.Result;

        var age = DateTime.Today - call.Started;
        if (age > DateTime.FromDays(14))         ②
            continue; // no need to send survey for old stuff

        using (var session = store.OpenSession()) ③
        {
            var customer = session.Load<Customer>(call.CustomerId);

            call.Survey = true;                   ④

            session.Store(call, item.ChangeVector, item.Id); ⑤

            try
            {
                session.SaveChanges();
            }
            catch (ConcurrenyException)           ⑥
            {
            }
        }
    }
}
```

```

        // will be retried by the subscription
        continue;
    }

    SendSurveyEmailTo(customer, call); ⑦
}
}
});

```

- ① Iterate over all the items in the batch.
- ② Filter support calls that are too old to be worth sending a survey.
- ③ Open a new session from the store (*not* tied to the current batch, see more on this later).
- ④ Modify the current call so we won't get it again (will be filtered by the where clause in Listing 7.3)
- ⑤ Store the modified call in the session, with the change vector to ensure optimistic concurrency check.
- ⑥ Handle a concurrency exception by not doing anything (and letting the subscription send us the modified document again in the next batch).
- ⑦ Send the survey email to the customer (assuming this can never fail for the moment).

There's a lot going on in Listing 7.4. For each item in the batch, we'll create a new session, load the customer for this support call and mark the call as having sent the survey. Then we'll call `Store` and pass it not only the instance that we got from the subscription from but also the change vector and ID for this document. This ensures that when we call `SaveChanges`, if the document has changed in the meantime on the server side, we'll get an error.

In Listing 7.4 we assume that we don't have to deal with failures for `SendSurveyEmailTo` for the sake of simplicity. A more complete implementation would first send the email and only then set the Survey flag on the support call. That way, if we need to retry the batch, we'll resume from the current batch, instead of marking it as done. Of course, that leads to the opposite problem of getting a failure in saving the update to Survey and then sending the email twice. Typical mail systems have options to ignore duplicate emails in a certain time period, which is probably how you would solve this in production.

Subscribing to documents on the database we're writing to

There are a few things to remember if you're using the subscription to write back to the same database you're subscribing to:

- Avoid the subscription/modify loop. When you modify a document that you're subscribing to, the server will send it to the subscription again. If you modify it every time it's processed, you'll effectively create an infinite loop, with all the costs this entails. You can see in Listing 7.4 that we're careful to avoid this by setting the Survey flag when we process a document and filter on the flag in Listing 7.3.
- The document you received may have already been changed on the server. Typically, the lag time between a document being modified and the subscription processing the document is very short. That can lead you to think this happens instantaneously or, even

worse, as part of the same operation as modifying the document. Nothing could be further from the truth. Assuming we process two support calls in a batch, and the first `SendSurveyEmailTo` takes 30 seconds, that gives a *huge* gap of time for the second document to be modified while we're processing the first one. In Listing 7.4, we handled that explicitly using optimistic concurrency to get an error if it was changed in the meantime. In other cases, you may want to use patching to avoid having to deal with the issue.

- If you're using subscriptions to integrate with other pieces of your infrastructure (such as sending emails), you have to be ready for failure on that end and have a meaningful strategy for handling it. Your options are to either propagate the error up the chain, which will force the subscription to close (and retry from the last successful batch), or you can catch the exception and handle it in some manner.

7.2.1. Distributed transactions and RavenDB

Subscription are commonly used to bridge between RavenDB and other systems. For example, in Listing 7.4, we send an email to the customer after marking the support call as having sent the survey. Failure handling in such a scenario is complex, because we're forced to integrate between two systems that don't share a transaction boundary. In other words, theoretically speaking, if we could create a transaction covering sending the email and writing to RavenDB, the problem would be solved.

In practice, RavenDB supported distributed transactions with multiple resources up until version 3.x, where we deprecated this feature. Version 4.0 removed it completely. Distributed transactions (also known as two-phase commit or 2PC) *sound* wonderful. Here you have a complex interaction between several different components in your system, and you can use a transaction to orchestrate it all in a nice and simple manner.

But it doesn't actually work like this. Any distributed transaction system that I've worked with had issues related to failure handling and partial success. A distributed transaction coordinator basically requires all participants in the transaction to promise that, if it tells them to commit the transaction, it will be successful. In fact, the way a coordinator usually works is by having one round of promises, and if all participants have been able to make that promise, there will be a second round with confirmations. The "two-phase commit" name is literally the description of the algorithm.

The problem starts when you've gotten a promise from all the participants, you've already confirmed with a few of them that the transaction has been committed and one of the participants fails to commit for whatever reason (hardware failure, for example). In that case, the transaction is in a funny, half-committed state.

The coordinator will tell you this is a bug in the participant. A participant shouldn't have made a promise it couldn't keep. And commonly, coordinators will retry such transactions (manually or automatically) and recover from transient errors. But the problem with the line of thinking that deems this an issue with this particular participant and not the coordinator is that those kind of errors are happening in production.

For one project, we had to restart the coordinator and manually resolve hanging transactions on a biweekly basis, and it wasn't a very large or busy website. Joe Armstrong [<https://en.wikipedia.org/wiki/>

Joe_Armstrong_(programming)], inventor of Erlang, described the problem far better than I could:^[2]

The "Two Generals' Problem" is reality, but the computer industry says it doesn't believe in mathematics: two-phase commit always works!^[3]

7.3. Optimizing subscription processing

There's another issue with the code in Listing 7.4. It is incredibly wasteful in the number of remote calls that it is making. One of the key benefits of using batch processing is the fact that we can handle things, well, in a batch. However, Listing 7.4 will create a session per document in the batch. The default batch size (assuming we have enough documents to send to fill a batch) is 4,096 items. That means that if we have a full batch, the code in either one of these listings will generate 8,192 remote calls. That's a *lot* of work to send to the server, all of which is handled in a serial fashion.

We can take advantage of the batch nature of subscriptions to do much better. Turn your attention to Listing 7.5, if you would.

Listing 7.5. Efficiently process the batch

```
await subscription.Run(batch =>
{
    using (var session = batch.OpenSession())           ①
    {
        var customerIds = batch.Items                   ②
            .Select(item => item.Result.CustomerId)
            .Distinct()
            .ToList();

        // force load of all the customers in the batch
        // in a single request to the server
        session.Load<Customer>(customerIds);             ③

        foreach (var item in batch.Items)
        {
            SupportCall call = item.Result;
            var age = DateTime.Today - call.Started;
            if (age > DateTime.FromDays(14))
                return; // no need to send survey for old stuff

            // customer was already loaded into the session
            // no remote call will be made
            var customer = session.Load<Customer>(
                call.CustomerId);

            // register the change on the session
            // no remote call will be made
            session.Advanced.Patch<SupportCall, bool>(    ④
                call.Id,
                c => c.Survey,
```

```

        true);

        SendSurveyEmailTo(customer, call);           ⑤
    }

    // send a single request with all the
    // changes registered on the session
    session.SaveChanges();                           ⑥
}
});

```

- ① Open a single session for the entire batch (using `batch.OpenSession` rather than `store.OpenSession`).
- ② Extract all the customer IDs from the batch.
- ③ Load all the customers in a single request to the server.
- ④ Use patching to register the change to the support call (no remote call is made at this point), and no need for optimistic concurrency check.
- ⑤ Send the survey email to the customer.
- ⑥ Call `SaveChanges` once for the entire batch, sending all the changes to the server in a single request.

Listing 7.5 takes a different approach than Listing 7.4. Instead of opening a session per document in the batch, we open a single session for the entire batch. We also use `Patch` instead of optimistic concurrency to update the Survey flag, avoiding the need to deal with concurrent updates entirely. Note that we also changed the behavior. Previously, we would update each document Survey flag and then send the email. Now we send the email first, then update the Survey for all the documents in the batch in one shot. A failure on `SendSurveyEmailTo` will force us to retry the entire batch.

The batch object has the `Items` property, which is the list of documents in the batch. Those documents are already loaded into the batch (and into the `batch.OpenSession`). You can iterate them multiple time at no cost, as we do in Listing 7.5, once to load all the customers, and the second time to process all the calls.

Don't use the `store.OpenSession` in batch processing

The code in Listing 7.4 uses `batch.OpenSession` instead of `store.OpenSession`. Why is that? Well, this is where I need to skip head a bit and explain some concepts we haven't seen yet. When running in a cluster, RavenDB divides the work between the various nodes in a database. That means that a subscription may run on node B while the cluster as a whole will consider node A as the preferred node to write data to.

RavenDB handles that transparently, by replicating the information across the nodes in the cluster, including optimistic concurrency using the document's change vector, as we discussed in the previous chapter. However, reading from one node and writing to another can lead to subtle concurrency issues. RavenDB can handle these issues, but they can be surprising to users. It's generally better to write back to the same node you're reading from.

In order to avoid that, you should use `batch.OpenSession` (or `batch.OpenAsyncSession`) to create the session. This will ensure that the session you've created will operate against the same node you're reading from and thus will allow us to reason about the state of the system with fewer variables.

Listing 7.4 relies on the Unit of Work nature of the session. Once we've loaded a document into it, trying to load it again will give us the already loaded version without going to the server. Without this feature, the amount of calls to Load would have probably forced us over the budget of remote calls allowed for the session.^[4]

Another option to deal with the number of Load calls is to utilize `include`. We can modify the subscription query to the one show in `[subscription-with-include]`.

Listing 7.6. Subscription using string query with include

```
store.Subscriptions.Create<SupportCall>(
    new SubscriptionCreationOptions {
        Query = @"
        from SupportCalls as call
        where call.Comments.Count > 25 and call.Votes > 10
            and call.Survey == false
        include call.Customer"
    });
```

If we'll use the subscription defined in `[subscription-with-include]`, the customer will be automatically included in the batch that we get from the server. That is one of the reasons why `batch.OpenSession` is preferred over `store.OpenSession`. The session that we get from the batch is already preloaded with all the documents that were included in the subscription query.

With the subscription shown in `[subscription-with-include]`, we can drop the need to call `session.Load<Customer>(customerIds);` in Listing 7.5. The customers will already be loaded into the batch's session from the `include`, and the rest of the code behaves the same (and exhibit the same low number of remote calls).

7.3.1. Complex conditionals

We already saw how we can create a subscription that filters documents on the server side: it was shown in Listing 7.3. The code there used a Linq expression, and the client API was able to turn that into a query that was sent to the server. Listing 7.3 was a pretty simple expression, and the code that handles the translation between Linq expressions and queries is quite smart. It's able to handle much more complex conditions. In `[subscription-with-include]`, we also saw that we can use a string query to define the subscription.

In Listing 7.7 we are taking that to the next level and show how we can use a more complex filtering mechanism to subscribe to all the support calls that require special attention.

Listing 7.7. Creating a subscription using JavaScript filtering

```
var options = new SubscriptionCreationOptions
```



```

{
  Name = subscriptionName,
  Query = @"
    declare function isAnnoyedCustomer(call) {
      const watchwords = ['annoy', 'hard', 'silly'];

      const lastIndex = call['@metadata']['Last-Monitored-Index'] || 0;

      for (let i = lastIndex; i < call.Comments.length; i++)
      {
        let comment = call.Comments[i].toLowerCase();
        for (let j = 0; j < watchwords.length; j++)
        {
          if (comment.indexOf(watchwords[j]) !== -1)
            return true;
        }
      }
      return false;

      from SupportCalls as call
      where isAnnoyedCustomer(call)"
};

store.Subscriptions.Create(options);

```

The interesting code in Listing 7.7 is in `isAnnoyedCustomer` inside the query. We're defining a few words that we'll watch for, and if we see them in the comments of a support call, we want to give that call some special attention via this subscription. We do that by simply scanning through the array of `Comments` and checking if any comment contains any of the words we're looking for.

Use the subscription test feature

Any time that you need to run non-trivial code, you should consider how you'll debug it. Subscriptions give you a *lot* of power and they can carry the load of significant portions of your business logic. Because of that, it's very important that you will be able to test and play with them while developing.

For this reason, RavenDB has the subscription test feature. This feature will apply the subscription logic on top of the database and allow you to see the results of your filter or transformation scripts. We've already gotten a taste of that in Figure 7.4, but the more complex your queries are, the more useful this feature will become.

There's one item to consider here, though. A subscription test works just like a subscription. It matches all the documents in the particular collection you're operating on and applies the script to each and every one of them, giving you those documents that matched successfully. However, if your script filters out many of the matches and you have a large amount of data, it may take the subscription test a while to find enough documents that match your subscription filter.

You can reduce this time by selecting a document from which the subscription test will start the scanning process. This way it will not have to scan all the documents in the collection. Selecting such a document is done in the Send Documents From dropdown. While you'll typically use "Beginning of Time" as the default, you can save some time if you position the subscription directly on the first of the documents you want to process.

The one interesting tidbit is the use of `call['@metadata']['Last-Monitored-Index']`. What's that for? Remember that a subscription will be sent all the documents that match its criteria. And whenever a document is changed, it triggers a check for a match with the subscription. That means that if we didn't have some sort of check to stop it, our subscription would process any support call that had a comment with one of the words we watch for *every single time that call is processed*.

In order to avoid that scenario, we set a metadata value named Last-Monitored-Index when we process the subscription. You can see how that works in Listing 7.8.

Listing 7.8. Escalating problematic calls

```
const string script = @" ①
var metadata = this['@metadata'];
var existing = metadata['Last-Monitored-Index'] || 0;
metadata['Last-Monitored-Index'] = Math.max($idx, existing);";

await subscription.Run(batch =>
{
    using (var session = batch.OpenSession())
    {
        foreach (var item in batch.Items)
        {
            // mark the last index that we
            // already observed using Patch
            session.Advanced.Defer( ②
                new PatchCommandData(
                    id: item.Id,
                    patch: new PatchRequest
                    {
                        Script = script,
                        Values =
                        {
                            ["idx"] = item.Result.Comments.Count
                        }
                    },
                    patchIfMissing: null,
                ));
        }
        // actually escalate the call
        session.SaveChanges(); ③
    }
});
```

- ① The JavaScript code that will set the Last-Monitored-Index metadata value to the size of the Comments array.
- ② Use `Defer` to register a patch command to set the Last-Monitored-Index metadata value.
- ③ Call `SaveChanges` once for the entire batch, sending all the changes to the server in one remote call.

We are simply setting the Last-Monitored-Index to the size of the Comments on the call and saving it back to the server. This will ensure that we'll only get the support call again if there are *new* comments with the words we're watching for. The code in Listing 7.8 is going out of its way to be a good citizen and not go to the server any more times than it needs to. This is also a good chance to demonstrate a real use case for `Defer` in production code. The use of `Defer` means two things: one, we don't need to worry about the number of calls and, two, we've handled concurrency.

Maintaining per-document subscription state

Subscriptions often need to maintain some sort of state on a per-document basis. In the case of Listing 7.8, we needed to keep track of the last monitored index, but other times you'll have much more complex state to track. For example, imagine that we need to kick off a workflow that will escalate a call once it passes a certain threshold. We might need to keep track of the state of the workflow and have that be accounted for in the subscription itself.

Using the metadata to do this works quite nicely if we're talking about small and simple state. However, as the complexity grows, it isn't viable to keep all that in the document metadata, and we'll typically introduce a separate document to maintain the state of the subscription. In it, we'll track support calls, and `SupportCalls/238-B` will have a `SupportCalls/238-B/EscalationState` document that contains the relevant information for the subscription.

Listing 7.7 and Listing 7.8 together show us how a subscription can perform rather complex operations and open up some really interesting options for processing documents. But even so, we aren't done. We can do even more with subscriptions.

7.3.2. Complex subscription queries and projections

We've used conditional subscriptions to filter the documents that we want to process, and since this filtering is happening on the server side, it allows us to reduce the number of documents that we have to send to the subscription. This is awesome in itself, but a really interesting feature of subscriptions is that we don't actually *need* to send the full documents to the client. We can select just the relevant details to send.

Say we want to do some processing on highly voted support calls. We don't need to get the full document; we just need the actual issue and the number of votes for that call. Instead of sending the full document over the wire, we can use the code in Listing 7.9 to achieve our goals more efficiently.

Listing 7.9. Getting just the relevant details in the subscription

```
var options = new SubscriptionCreationOptions
```

```
{
    Name = subscriptionName,
    Query = @"
        from SupportCalls as call
        where call.Votes >= 10
        select {
            Issue: call.Issue,
            Votes: call.Votes
        }"
};
store.Subscriptions.Create(options);
```

What we're doing in Listing 7.9 is filtering the support calls. If the call has less than 10 votes, we'll just filter it. However, we aren't limited to just filtering the full document. We can also return an object of our own. That object can be built by us and contain just what we want to send directly to the client, and RavenDB will send it.

But there is an issue here. In Listing 7.9, we're creating a subscription on `SupportCall`. However, the value that will be sent to the client for the subscription to process is *not* a `SupportCall` document. It's a projection created during the subscription run. That means that, on the client side, we need to know how to handle such a thing. This requires a bit of a change in how we open the subscription, as you can see in Listing 7.10.

Listing 7.10. Opening subscription with a different target type

```
public class SupportCallSubscriptionOutput
{
    public string Issue;
    public int Votes;
}

var options = new SubscriptionWorkerOptions(subscriptionName);
var subscription = store.Subscriptions
    .GetSubscriptionWorker<SupportCallSubscriptionOutput>(options);

await subscription.Run(batch =>
{
    foreach (var item in batch.Items)
    {
        SupportCallSubscriptionOutput result = item.Result;
        // do something with the
        // result.Issue, result.Votes
    }
});
```

In order to consume the subscription in a type-safe way, we create a class that matches the output that we'll get from the subscription script, and we'll use that when we open the subscription. Scenarios like the one outlined in Listing 7.9 and Listing 7.10 make this flexibility very useful.

If this was all we could do with the subscription script, it would have been a good way to reduce the amount of data that's sent over the wire. But there are actually more options available for us that we haven't gotten around to yet. Consider Listing 7.5 where we get the support call and immediately have to load the associated customer. That can lead to a remote call per item in the batch. We've already gone over why this can be a bad idea in terms of overall performance. Even with the optimization we implemented in [subscription-with-include], there's still a remote call to do. We can do better.

We can ask RavenDB to handle this as part of the subscription processing directly. Take a look at Listing 7.11, which does just that.

Listing 7.11. Getting just the relevant details in the subscription

```
var options = new SubscriptionCreationOptions<SupportCall>
{
    Name = subscriptionName,
    Query = @"
        from SupportCalls as call
        where call.Votes >= 10
        load call.CustomerId as customer
        select {
            Issue: call.Issue,
            Votes: call.Votes,
            Customer: {
                Name: customer.Name,
                Email: customer.Email
            }
        }"
};
store.Subscriptions.Create(options);
```

In Listing 7.11, we're using `load` as part of the processing of the subscription on the server side. This allows us to get the customer instance and send pieces of it back to the client. In order to consume it, we'll need to change the `SupportCallSubscriptionOutput` class that we introduced in Listing 7.9 to add the new fields.

When processing the output of this subscription, we can directly process the results without making any other remote calls, not even to load the associated document. In Listing 7.12, you can see how we'd process such a subscription.

Listing 7.12. Opening subscription with a different target

```
public class SupportCallSubscriptionOutput           ①
{
    public class Customer                             ②
    {
        public string Name;
        public string Email;
    }
    public string Issue;
```

```

    public int Votes;
    public Customer Customer;
}

var options = new SubscriptionWorkerOptions(subscriptionName);
var subscription = store.Subscriptions
    .GetSubscriptionWorker<SupportCallSubscriptionOutput>(options);

await subscription.Run(batch =>
{
    foreach (var item in batch.Items)           ③
    {
        SupportCallSubscriptionOutput result = item.Result;
        SendEscalationEmail(result.Customer, item.Id);
        // other stuff related to processing the call
    }
});

```

- ① The output class for the subscription.
- ② An inner class to represent the customer details that we get from the subscription.
- ③ Processing the subscription batch.

You can see that we use an inner class to scope the meaning of the `Customer` here. This isn't required. It's merely a convention we use to bring some order to the client-side types. Note that unlike before, we are no longer working with documents. Using `batch.OpenSession` won't have them in the session, nor do we benefit from the Unit of Work pattern. If we wanted to modify the documents in this subscription, we'll need to either use patching or to load them. Of course, we can combine both load and include to get the benefit of both worlds.

7.4. Error handling and recovery with subscriptions

What happens when there's an error in the processing of a document? Imagine that we had code inside the lambda in the `Run` method and that code threw an exception. Unless you set `SubscriptionWorkerOptions.IgnoreSubscriberErrors`,^[5] we'll abort processing of the subscription and the `Run` will raise an error. Typical handling in that scenario is to dispose the subscription and immediately open it again.

Assuming the error is transient, we'll start processing from the last batch we received and continue forward from there. If the error isn't transient, however, the error keep occurring over and over. Consider a `NullReferenceException` because of a null where we didn't expect it. You might want to set an upper limit to the number of errors you'll try to recover from in a given time period, choosing to just fail completely afterward. This depends heavily on the kind of error reporting and recovery you're using in your applications.

Note that this applies only to errors that come from the code processing the document. All other errors (connection to server, failover between servers, etc.) are already handled by RavenDB. The reason that we abort the subscription in the case of subscriber error is that there really isn't any good way to recover from it in a safe manner. We don't want to skip processing the document. And

just logging the error is possible (in fact, that's exactly what we do if `IgnoreSubscriberErrors` is set), but no one ever reads the log until the problem is already discovered, which is typically very late in the game.

RavenDB errors & subscription recovery

The subscription infrastructure is going to handle all the errors related to RavenDB. It will failover between servers to recover, retry if there is a network hiccup, etc. An important aspect of that is that when we refer to this we refer to all the errors that come from the *infrastructure*. If the lambda that the subscription invokes raises an exception, there is no special handling there.

For example, if the code in Listing 7.4 raise a `ConcurrencyException`, even though it is a RavenDB exception, it wasn't raised by the subscription infrastructure but by the lambda itself. That is a *very* important distinction to make between the types of errors that is handled for you and the kind that bubbles up.

In other words, RavenDB will take care of all the issues related to talking to the database, but the error handling related to your code is on you (including when your code talks to RavenDB directly, of course). In practice, you generally don't have to worry about it. An error thrown during document processing will kill your subscription. We saw in Listing 7.2 that after we call `Run`, we need to wait on the resulting task. If an error is raised during document processing, the subscription will close and that error will be raised to the caller of the `Run` method.

The typical manner in which you'll handle errors with subscriptions is to just retry the whole subscription, as shown in Listing 7.13. There's a lot of things going on in this listing, so take the time to carefully read through the error handling.

Listing 7.13. Retrying subscription on error

```
while (true)                                     ①
{
    var options = new SubscriptionWorkerOptions(subscriptionName) ②
    {
        MaxErroneousPeriod = TimeSpan.FromHours(2),
        TimeToWaitBeforeConnectionRetry = TimeSpan.FromMinutes(2)
    };

    var subscriptionWorker = store.Subscriptions
        .GetSubscriptionWorker<Order>(options);

    try
    {
        subscriptionWorker.OnSubscriptionConnectionRetry += exception => ③
        {
            Logger.Error("Error during subscription processing: "
                + subscriptionName, exception);
        };
    }
}
```

```

    await subscriptionWorker.Run(async batch =>                                ④
    {
        foreach (var item in batch.Items)
        {
            if (item.Result.Company == "companies/832-A")                    ⑤
                throw new UnsupportedCompanyException(
                    "Company Id can't be 'companies/832-A', FIXME!");

            await ProcessOrder(item.Result);
        }
    }, cancellationToken);

    // Run will complete normally if you have disposed the subscription
    break;
}
catch (Exception e)
{
    if (e is SubscriberErrorException se)                                    ⑥
    {
        if (se.InnerException is UnsupportedCompanyException)
        {
            throw; // an actual error that we raise to the caller
        }

        continue;                                                            ⑦
    }

    if (e is SubscriptionClosedException)
        return; // closed explicitly by admin, probably, not an error

    throw; // not recoverable                                                ⑧
}
finally
{
    subscriptionWorker.Dispose();
}
}

```

- ① Infinite loop, so if we failed processing the subscription, we'll just open it again and keep trying.
- ② Configure that we allow a down time of up to 2 hours and before raising an error to the caller if the subscription infrastructure is getting errors from the cluster nodes. It will also wait 2 minutes between retries.
- ③ Register to get the errors raised by the subscription infrastructure (which are usually handled internally).
- ④ The actual processing of the subscription, as we normally do.
- ⑤ Add an exceptional state that will cause the subscription to throw an error.
- ⑥ Handling exception thrown from the subscription's lambda invocation.

- ⑦ Assuming that this is a transient error and going back to the beginning of the loop, to try opening the subscription again.
- ⑧ Raise the error to the calling code.

Listing 7.13 shows a typical way to handle errors in subscriptions. There are a few non-recoverable scenarios, such as the database not existing, the subscription doesn't exist or is misconfigured, or the credentials we have are invalid. There's no way to recover from those kinds of errors without administrator involvement, so we should just raise this up the stack until we catch the attention of someone who can actually fix this.

The next part handles errors when the subscription was closed explicitly by an administrator. RavenDB will automatically recover from failures and accidental disconnects, but the administrator can also choose to explicitly kill a subscription connection. In this case, we'll report this to the caller, who can decide what to do about it. Simply retrying is probably not a good idea in this case. A subscription can also fail because another client is already holding onto it, or a client came in and kicked our subscription from the subscription. Both of these cases are strongly tied to the deployment mode you have and will be discussed in the next section.

What about subscription script errors?

We've talked plenty about what happens when there are errors on the server side (automatically recover), on the client side (thrown to caller and handled by your code) and the like. However, there's one class of errors that we didn't consider: what would happen if there's an error in the query that we use to evaluate each document?

Consider the following snippet: `select { Rate: (10 / call.Votes) }`. It isn't meant to be meaningful, just to generate an error when the `Votes` property is set to zero. In this case, when evaluating the script on a support call with no votes, we'll get an error attempting to divide by zero. That error will be sent to the client and be part of the batch.

When the client accesses the `item.Result` property on *that particular document*, the exception will be raised on the client side. In this way, you can select how to handle such errors. You can handle this on the client side in some manner and continue forward normally, or you can let it bubble up as an error in the subscription, which should trigger your error-handling policy, as we saw in Listing 7.13.

All other exceptions would typically be raised from within the batch processing code. A very simple error handling strategy can be to just try an error backoff for retries. So on the first error, you'll retry immediately, if there is still an error, wait for 5 seconds and try again. Keep adding 5 seconds to the sleep time until a certain maximum (such as 1 minute). In many cases, this is enough to handle transient errors without having to get an admin involved in the process.

Exactly how you'll handle error recovery in your system is up to you and the operations teams that will maintain the application in production, and it's tied very closely to how you're using subscriptions and deploying them.

7.5. Subscription deployment patterns

Talking about a single subscription in isolation helps us understand how it works, but we also need to consider how subscriptions are deployed and managed in your systems. Subscriptions are used to handle most batch processing needs in RavenDB, which puts them in an interesting place regarding their use. On the one hand, they're often performing critical business functions. But on the other hand, they aren't visible. (A failing subscription will not cause an error page to be shown to the user.)

Subscriptions are usually deployed in either a batch-and-stop mode or in a continuous run. The continuous run is a process (or a set of processes) that are running your subscriptions constantly. The database will feed any new data to them as soon as it's able, and all processing happens live. That isn't required, mind you. Under load, the administrator is perfectly able to shut down a subscription (or even all of them) to free up processing capacity. When they're started up again, they'll pick up where they left off.

In batch mode, the subscriptions are run on a schedule. When they run out of work to do, they'll shut themselves down until being run again. Listing 7.14 shows how you can write a self-stopping subscription.

Listing 7.14. Stop the subscription after 15 minutes of inactivity

```
var options = new SubscriptionWorkerOptions(subscriptionName)
{
    CloseWhenNoDocsLeft = true
}
using (var subscription = store
    .Subscriptions.GetSubscriptionWorker<SupportCall>(options))
{
    await subscription.Run(batch =>
    {
        // process batches until we run out of docs
        // that match the subscription
    });
}
```

The `CloseWhenNoDocsLeft` option will cause RavenDB to abort the subscription if there are no documents to send. This is useful for processes that start, run through everything that happened since their last run and then shut down again.

Another aspect of subscription deployment is the notion of high availability. On the server side, the cluster will ensure that if there was a failure, we will transparently failover to another node. But who takes care of high availability on the client? For high availability you'll want to have at least a couple of machines running subscriptions, so if one of them goes down, the other one will carry on. However, if the same subscription is running concurrently on two machines, that can lead to duplicate processing. Now, your code needs to handle that anyway, since subscriptions guarantee processing at least once and various error conditions can cause a batch to be re-sent. However, there's a major difference between handling that once in a blue moon and having to deal with it constantly, leaving aside the issue that we'll also have higher resource usage because we need to

process the subscription on multiple nodes.

Luckily, RavenDB subscriptions don't work like that. Instead, a subscription is usually opened by one and only one client (we'll talk about concurrent subscriptions and the exception to this rule later in this chapter). By default, if two clients attempt to open the same subscription, one of them will succeed and the other one will raise an error because it couldn't take hold of the subscription. This is controlled by the `SubscriptionOpeningStrategy` option set on the `Strategy` property. The various options of this property are:

- `OpenIfFree` is the default. The client will attempt to open the subscription but will fail if another client is already holding it. This is suitable in cases where you're running subscriptions in batch/schedule mode. If another client is already processing it, there's no need to process anything yourself.
- `TakeOver` will attempt to open the subscription even if a client is already connected to it. If there's a connected client, its connection will be closed. This is used to force the subscription open operation to succeed for a particular client. If an incoming subscription tries to take over an existing one, which also has a "take over" strategy, the incoming subscription will "lose" the contest.
- `WaitForFree` will attempt to open the subscription, but if there's already a connected client, it will keep trying to acquire the connection. This is suitable for high availability mode, when you have multiple clients attempting to acquire the same subscription and you want one of them to succeed while the rest stand ready in case that one fails.
- `Concurrent` allows multiple subscription instances at the same time, each processing their own set of documents. Will be discuss at length later in this chapter.

For high availability processing on the client, you'll set up multiple machines that will run your subscriptions and open them using the `WaitForFree` option. In this way, those clients will compete for the subscriptions. If one of them fails, the other will take over and continue processing them. The use of multiple machines for handling subscriptions also allows you to split your processing between the machines.

Failure conditions in a distributed environment

Failure handling in a distributed environment is *hard*. When both clients and servers are distributed, this makes for some really interesting failure modes. In particular, while we promise that a subscription will only be opened by a single client at a time, it's possible for the network to split in such a way that two clients have successfully connected to two different servers and are trying to process the same subscription from them.

That scenario should be rare, and it will only last for the duration of a single batch, until the next synchronization point for the subscription (which ensures global consistency for the subscription). One of those servers will fail to process the batch acknowledgement and return that error to the client, eventually aborting the connection.

You may decide that you want certain operations to be handled on one worker node ^[6] and configure those subscriptions to `TakeOver` on that worker node and `WaitForFree` on any other worker nodes. This way, a particular subscription has a preferred worker node that it will run on, but if

that one fails, it will run on another. In other words, each worker node will run only its own preferred subscriptions unless there's a failure.

When your deployment contains multiple worker nodes that are all processing subscriptions, you need to be even more aware of the fact that you're running in a distributed system and that concurrency in a distributed environment should always be a consideration. We already talked about optimistic concurrency in Chapter 5, but with distributed databases, we also need to take into account eventual consistency and conflicts between machines. We'll start talking about the distributed nature of RavenDB in Chapter 9 in depth.

For subscriptions, we typically don't care about this. The batch processing mode means that we already see the world differently and it doesn't matter if we get a document in a particular batch or in the next. One thing to be aware of is that different subscriptions may be running on different nodes and thus getting the documents in a different order. (All of them will end up getting all the documents. There's just no guaranteed absolute order across the distributed system.)

7.6. Using subscription for queuing tasks

In this section, I want to utilize subscriptions as a processing queue for a few reasons. For one thing, it's a very common scenario for using subscriptions. For another, it lets us explore a lot of the RavenDB functionality and how different pieces are held together. What we want to do is to be able to write an operation to the database and, when we've successfully processed it, automatically delete it.

Going back to the email-sending example, our code will write `EmailToSend` documents that will be picked up by a subscription and handled. The code for actually doing this is shown in Listing 7.15.

Listing 7.15. Delete the emails to send after successful send

```
using (var subscription = store.Subscriptions
    .GetSubscriptionWorker<EmailToSend>(options))
{
    await subscription.Run(async batch =>
    {
        using (var session = batch.OpenAsyncSession())
        {
            foreach (var item in batch.Items)
            {
                try
                {
                    SendEmail(item.Result);
                }
                catch
                {
                    // logging / warning / etc
                    continue;
                }
                session.Delete(item.Id);
            }
        }
        await session.SaveChangesAsync();
    });
}
```

```
}  
});  
}
```

- ① If we failed to send the email, we *ignore* the failure. We'll not process the `EmailToSend` document again (we "successfully" processed it by having it in a batch that didn't throw an error). But we'll also not delete it.
- ② Successfully sending the email means that we'll delete this document entirely.

There's actually a lot going on in Listing 7.15 that may not be obvious. First, we're processing a batch of documents, trying to send each one in turn. If there's an error in sending one of those emails, we'll skip further processing. However, if we were able to successfully send the email, we'll register the document to be deleted. At the end of the batch, we'll delete all the documents that we've successfully sent.

We don't touch the documents that we fail to send. The idea is to dedicate the `EmailToSend` collection for this task only. That means that in the `EmailToSend` collection, we'll only ever have one of two types of documents:

- `EmailToSend` documents that haven't been processed yet by the subscription and will be sent (and deleted) soon.
- Documents that we tried to send but failed to.

Those documents that we fail to send are interesting. Since the subscription has already processed them, we won't be seeing them again. However, we didn't lose any data, and an administrator can compare the current state of the subscription to the state of the `EmailToSend` collection and get all the failed documents.

At that point, the admin can either fix whatever is wrong with those documents (which will cause them to be sent to the subscription again) or they can reset the position of the subscription and have it reprocess all those documents again (for example, if there was some environmental problem). This mode of operation is really nice for processing queues. For simple cases, you can just rely on RavenDB to do it all for you. In fact, given that RavenDB subscriptions can work with failover of both clients and servers, this gives you a robust solution to handle task queues that are local to your application.

RavenDB is *not* a queuing solution, and this code doesn't pretend that it is. The code in Listing 7.15 is a great way to handle task queues, but proper queuing systems will offer additional features (like monitoring and built-in error handling) that you might want to consider. For most simple tasks, the fire-and-forget operations, you can use RavenDB in this mode. But for more complex situations, you should at least explore whether a proper queue can offer a better solution.

7.7. The Outbox design pattern

Earlier in this chapter I talked about distributed transactions and how hard they are. The Outbox pattern is a nice way to handle this scenario without a lot of complexity. Consider the case of Listing 7.4, where we needed to send survey emails as well as update the database. Compare that to the code in Listing 7.15 which *only* handles sending emails. The error handling aspect in Listing 7.15 is

much simpler than the one in Listing 7.4.

Instead of trying to have a distributed transaction across two different systems, we'll use the Outbox pattern. Instead of performing the actual operation (`SendSurveyEmailTo`), we'll write a document to RavenDB that declare the *intent* to do so (`EmailToSend`). That intent (the actual `EmailToSend` document) is then processed behind the scenes like we saw in Listing 7.15.

Take a look at Listing 7.16 to see how this is done. We're back to processing support calls, and we update the `Survey` field on the support call as well as store a new `EmailToSend`. The final step of the subscription's lambda is to call `SaveChangesAsync` and save both the updates to the support calls as well as the new `EmailToSend` documents in a *single* transaction.

Listing 7.16. Using the outbox pattern to send emails

```
await subscription.Run(async batch =>
{
    using var session = batch.OpenSession();

    foreach (var item in batch.Items)
    {
        SupportCall call = item.Result;
        var age = DateTime.Today - call.Started;
        if (age > DateTime.FromDays(14))
            return; // no need to send survey for old stuff

        var customer = await session.LoadAsync<Customer>(call.CustomerId);

        call.Survey = true;                                ①

        EmailToSend email = EmailToSendFactory.SupportSurvey(call, customer);
        await session.StoreAsync(email);                    ②
    }
    await session.SaveChangesAsync();                        ③
});
```

- ① Update the `Survey` flag on the current call.
- ② Store the new `EmailToSend`.
- ③ Save both changes to the database in a single transaction.

The code in Listing 7.16 does only half the job. It perform all the business logic that is required, and handles the transactional aspects very cleanly. But it *doesn't send an email*! That task was cleanly separated out to a different task. As you can imagine, a separate subscription running the code in Listing 7.15 would perfectly complement this behavior. The Outbox pattern then allows us to split the problem into distinct steps, and then handle each one of them in a far simpler manner.

7.8. Prallel work inside a subscription batch

By default, there is a single running subscription worker at any given time for a subscription. The reasoning behind it is that it makes it a *lot* simpler to understand what is going on. In general, here

is the flow of documents in a subscription can be seen in Figure 7.5. The subscription get a batch of documents, process them and then acknowledge their successful processing. And then cycle starts anew.

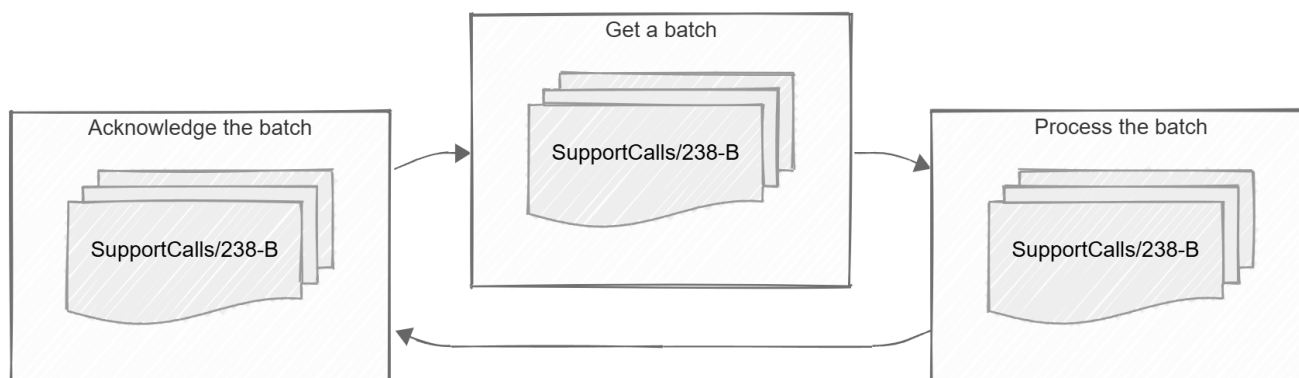


Figure 7.5. Flow of documents batches in a subscription

The process outline in Figure 7.5 is easy to understand and deal with. It is also inherently *serial* process. At any given point in time, there is just a single batch being processed. What happens if we have an expensive process to run through? Let's say that it takes us a second or two to send an email. When we want to process a batch that may have dozens of documents, the code in Listing 7.15 will be *very* slow.

We can deal with this using parallel processing. Subscription batches are handled in a serial fashion, but inside the lambda, you are free to use concurrency as you see fit. In Listing 7.17 you can see the parallel version of Listing 7.15.

Listing 7.17. Parallel processing all the emails in the batch at once

```

using (var subscription = store.Subscriptions
    .GetSubscriptionWorker<EmailToSend>(options))
{
    await subscription.Run(async batch =>
    {
        using (var session = batch.OpenAsyncSession())
        {
            List<Task> emailTasks = [];
            foreach (var item in batch.Items)           ①
            {
                emailTasks.Add(SendEmailAsync(item.Result));
            }

            try
            {
                await Task.WhenAll(emailTasks);           ②
            }
            catch { /* ignored / expected */ }

            for (int i = 0; i < emailTasks.Count; i++)     ③
            {
                if (emailTasks[i].IsCompletedSuccessfully)
            }
        }
    }
}
  
```

```

        session.Delete(batch.Items[i].Id);           ④
    }

    await session.SaveChangesAsync();
}
});
}

```

- ① Run over the items in the batch and spawn a separate `SendEmailAsync` for each of them.
- ② Wait for all the tasks to complete (we don't care if they completed successfully or not).
- ③ Scan through the completed tasks and check their successful status.
- ④ If the email was sent successfully, delete the matching `EmailToSend` document.

The session is single threaded

In Listing 7.17, we send multiple emails in parallel by calling `SendEmailAsync` without calling `await` after each one. In other words, there are concurrent calls to `SendEmailAsync` at the same time. However, note that when we call to the session using `session.Delete()`, we do that *after* all the parallel tasks has been completed and in a completely singled threaded fashion.

Session in RavenDB are *not* safe for multi-threading. The `DocuemntStore` is, of course, but the session can only be used from a single thread at a time. The code in Listing 7.17 is very careful to ensure that we never try to access the session from multiple threads at the same time.

Being able to parallelize work inside the batch is great, but it isn't really helpful when we want to parallelize work *between* batches. We'll discuss that in the next section.

7.9. Concurrent subscription processing

In the previous version, we dealt with performing the work within a batch in parallel. In this section, we'll discuss how we can run concurrent *batches*. Before we get to the code and how to make it work, let's talk about what this means and how this works. One of the problems we have with concurrent batches is what happens if we want to process the *same* docuemnt in concurrent batches. Look at Figure 7.5 again, and note that we are processing the document `SupportCalls/238-B` in the batch.

If we have two workers working concurrently on the same subscription, and we start processing a batch with `SupportCalls/238-B` in it, what happens when that document is modified (while the first batch is still running)? In the normal course of things, the second worker will get the *updated* version of the *same* docuemnt to process at the *same time*. As you can probably tell from the amount of emphasis I added to the previous statement, that is a Bad Thing. It can lead to really complex and hard to follow behavior in your application.

For this reason, when you have concurrent subscription workers running at the same time, RavenDB takes care to ensure that each document can only be handed out to a batch *once*. In other

words, if a batch is currently processing SupportCalls/238-B, even if there were later modifications to this document and there subscription workers that want work, RavenDB will not hand this docuemnt to any worker until the ongoing batch either acknowledge a successful batch or errors.

Concurrent batch processing assume that the subscription does some work

When the subscription acknowledges that it successfully completed a batch, it needs to register that in the cluster. We'll talk more about this in Chapter 9, but the gist of it is that it needs to run this acknowledgement through a consensus protocol. In other words, acknowledging a batch (concurrent one or not) is a global cluster operation which represents a synchronization point.

This is important because it means that the value you'll get from concurrent subscription will be apparent if you going to benefit from mutliple concurrent batches being processed at the same time. The overhead of fetching a new batch or acknoledging a completed one should be vastly less than the cost of actually processing a batch. Otherwise, there is not much benefit in going concurrent.

Concurrent subscriptions requires the RavenDB cluster to do additional book-keeping to ensure that we don't hand the same docuemnt to workers at the same time. Conceptually, you can imagine that RavenDB "checks out" the documents while the subscription is processing a batch. When a new batch is created for another concurrent instance, we check each document we load against the documents that are currently "checked out". If there is a match, RavenDB skips adding the document to the batch, but record the fact that it should be sent (because it was updated after it was already sent out for processing).

When the batch with that document is acknowledged (or errored), we will then "release" the documents there. And any document that was skipped previously because on this reapon will be included in the next batch for processing. Importantly, that means that while *usually* subscriptions will process documents in rough modification order, for concurrent subscriptions, that doesn't always hold. You may get documents in a different order, since they were previously skiped and now needs to be processed out of order.

After all of this explanation, actually *making use* of concurrent subscription might be a let down. There isn't much to it, as you can see in Listing 7.18.

Listing 7.18. Using concurrent subscriptions

```
var subscriptionWorker1 = store.Subscriptions
    .GetSubscriptionWorker<SupportCall>(
        new SubscriptionWorkerOptions(subscriptionName)
        {
            Strategy = SubscriptionOpeningStrategy.Concurrent,
        });
①

var subscriptionWorker2 = store.Subscriptions
    .GetSubscriptionWorker<Order>(
②
```

```
new SubscriptionWorkerOptions(subscriptionName)
{
    Strategy = SubscriptionOpeningStrategy.Concurrent,
});
```

```
var sub1 = subscriptionWorker1.Run(batch => { /* process batch */ });    ③
var sub1 = subscriptionWorker2.Run(batch => { /* process batch */ });    ④
```

- ① Create the first subscription worker with Concurrent strategy.
- ② Create the second subscription worker with Concurrent strategy.
- ③ Start processing batches from the first worker.
- ④ Start processing batches from the second worker concurrently with the first.

Both workers in Listing 7.18 will run in concurrently and process *different* documents at the same time. We previously talked about the Strategy option for the subscription worker. It controls how RavenDB deals with concurrent requests for the same subscription. When using the Concurrent strategy, *all* the subscription workers need to agree to use the Concurrent strategy. If you'll try to create a subscription worker with Concurrent while there is an existing one that doesn't have Concurrent setup, it will error. Conversely, trying to create a subscription worker *without* Concurrent if there is a Concurrent worker running will also fail.

While a Concurrent subscription worker is running, only workers with Concurrent can be created. But that limitation holds only as long as there is a running subscription worker. In other words, close all the Concurrent subscription workers, and you can open a new one with `OpenIfFree` without issue.

7.10. Versioned subscriptions

The subscriptions we've used so far are always operating on the *current* state of the document. Consider the following case:

- You create a subscription on the Customer collection.
- You create a customer document and modify it three times.
- You open and run the subscription and observe the incoming items in the batch.

What will the subscription get? It will get the *last* version of the customer document. The same can happen when the subscription is already running. If you've made multiple modifications to a document, when the next batch starts, we'll just grab the current version of the document and send it to the subscription. This can be a problem if your subscription code assumes that you'll always get the document every time it changes.

The good news is that we can utilize another RavenDB feature, revisions (discussed in Chapter 3), to allow us to see all the changes that were made to a document.

The first step to exploring this feature is to enable revisions on the collection you want. In this case, it's the Customers collection. Set the minimum retention time for two weeks, and again make a few modifications to a Customer document. When you run your subscription once more, note that you're

again getting just the latest version of the Customer document.

The reason you only get the latest version of the document is that the subscription, too, needs to let the server know that it's versioned. This is because the data model for versioned subscriptions is different. Let's take a look at Listing 7.19 for an example of a simple versioned subscription that can detect when a customer changed their name.

Listing 7.19. Subscribing to versioned customers

```
store.Subscriptions.Create<Revision<Customer>>(  
    new SubscriptionCreationOptions  
    {  
        Name = subscriptionName,  
    }  
);  
  
using (var subscription = store.Subscriptions  
    .GetSubscriptionWorker<Revision<Customer>>(  
        subscriptionName  
    ))  
{  
    await subscription.Run(batch =>  
    {  
        foreach (var item in batch.Items)  
        {  
            Revision<Customer> customer = item.Result;  
  
            if (customer.Previous.Name !=  
                customer.Current.Name)  
            {  
                // report customer name change  
            }  
        }  
    });  
}
```

The key parts to the code in Listing 7.19 is the use of `Revision<Customer>` as the type that we're subscribing on. RavenDB recognizes that as a versioned subscription and will feed us each and every revision for the Customer collection.

Note that the data model as well has changed. Instead of getting the document itself, we get an object that has `Previous` and `Current` properties, representing the changes that happened to the document. In other words, you're able to inspect the current and previous versions and make decisions based on the changes that happened to the entity.

This feature opens up a *lot* of options regarding analytics because you aren't seeing just a snapshot of the state but all the intermediate steps along the way. This has uses in business analytics, fraud, outliers detection and forensics, in addition to the general benefit of being able to reconstruct the flow of data through your system. The way versioned subscriptions work, we're going to get all the changes that match our criteria, in the same order they happened in the database.^[7]

If I wrote to customers/8243-C, created a new customers/13252-B and then wrote to customers/8243-C again, I'll get the changes (either in a single batch or across batches) in the following order:

- customers/8243-C: (Previous → Current)
- customers/13252-B: (null → Current)
- customers/8243-C: (Previous → Current)

This can be useful when running forensics or just when you want to inspect what's going on in your system.

Handling revisioned document creation and deletion

When a new document is created, we'll get it in the versioned subscription with the `Previous` property set to `null` and the `Current` set to the newly created version. Conversely, if the document is deleted, we'll get it with the `Current` set to `null` and the `Previous` set to the last version of the document.

If a document is deleted and recreated, we'll have one entry with (`Previous`, `null`) and then another with (`null`, `Current`). This feature is also useful if you just want to detect the deletion of documents in a subscription.

Just like regular subscriptions, we aren't limited to just getting all the data from the server. We can filter it, as you can see in Listing 7.20.

Listing 7.20. Setting filters on a versioned subscription

```
store.Subscriptions.Create(  
    new SubscriptionCreationOptions  
    {  
        Name = subscriptionName,  
        Query = @"  
            from Customers (Revisions = true)  
            where Prervious.Name != Current.Name"  
        }  
    );
```

The code in Listing 7.20 will only send you the Customer documents whose names have been changed. The rest of the code in Listing 7.19 can remain the same, but we now don't need to check if the names have changed on the client side. We've done that already on the server.

Note that the RQL syntax for using versioned subscription directly from Customers (`Revisions = true`), which tells RavenDB that you want to use the revisions as your source of data, rather than the docuemnts thmeselves.

A natural extension of working on revisions is to skip sending the full data to the client and get just the data we need. An example of this can be shown in Listing 7.21.

```
store.Subscriptions.Create(  
    new SubscriptionCreationOptions<Versioned<Customer>>  
    {  
        Name = subscriptionName,  
        Query = @"  
            from Customers (Revisions = true)  
            where Prervious.Name != Current.Name  
            select {  
                OldName: this.Previous.Name,  
                NewName: this.Current.Name  
            }"  
    }  
);
```

The code in Listing 7.21 isn't that interesting except for what it's actually doing. It's filtering the data on the server side and only sending us the old and new names. The subscription handling code on the client side just needs to take the old name and new name and notify whoever needs to know about the name change. Remember that on the client side you need to match the projection structure, as we saw in Listing 7.12.

Versioned subscriptions give you a lot of power to work with your data (and the changes in your data) in an easy way. The ability is of particular interest in reversed event sourcing because you can go over your data, detect a pattern that matches a particular event you want to raise and publish it.

Versioned subscriptions use the document revisions

This might be an obvious statement, but it needs to be explicitly said. The backing store for the versioned subscription is all the revisions that are stored for the documents as they're changed. That means that it's important to know the minimum retention time that's been configured. If you have a weekly subscription run and the versioning is configured to keep data for a day, you're going to miss out on revisions that have already been deleted.

So when using a versioned subscription, the administrator must verify that the revisions retention time matches the subscription run time.

7.11. Subscriptions, background jobs and the role of the admin

Systems typically accrue additional behavior over time. As business processes mature and evolve, so does the demand on our systems. The way we've talked about subscriptions so far is mainly as a core part of the application itself. A subscription that the developer team would write, build and manage. But that doesn't have to be the case. In fact, it's common for subscriptions to be used as "aftermarket" entry points for modifying, extending and supporting the system.

Consider the case of wanting to send all new users a welcome email after they've registered. We

could do that as part of the application itself, but given that this is a self-contained feature, we don't actually *have* to involve the developers. We can write a subscription that would monitor new customers and send them the email.

For flexibility, we don't want to write it in C#. We don't need an application. We just need to be able to run a script to process those documents. I'm using the Python client API in Listing 7.22 to perform this task. (I could have used node.js, Ruby, etc. I'm using Python as the example here because it's a great scripting language, and one I'm very fond of.)

Listing 7.22. Sending welcome emails to new customers

```
#!/usr/bin/env python3

from dataclasses import dataclass
from ravendb.documents.store.definition import DocumentStore

store = DocumentStore (
    urls = ["http://localhost:8080"],
    database = "HelpDesk"
)
store.initialize()

with store.subscriptions.get_subscription_worker_by_name(
    "new-customers-welcome-email"
) as subscription:

    def send_welcome_email(customer):
        pass # redacted

    def handle_batch(batch):
        with batch.open_session() as session:
            for item in batch.items:
                customer = item.result
                send_welcome_email(customer)
                customer.welcome_email_sent = True
            session.save_changes()

subscription.run(handle_batch).join()
```

The code in Listing 7.22 is quite similar to the way we would do things in C#. We open a document store, open the subscription and pass a method to process the document batches. The use of Python typically means shorter development time and more freedom for the operations team to modify things. That can be a great thing or a horrible thing.

Subscription in a dynamic language

A major advantage of writing such scripts in a dynamic language is the low overhead and friction. We don't need to define classes. We can just use the shape of the data as it came down the wire. That makes everything simpler.

On the other hand, you need to be aware that doing so requires discipline. You must be sure to version your subscription's code in step with the rest of the code. This is especially true if you are writing the sort of "aftermarket" scripts like the one shown in Listing 7.22, the development team may not always give a heads up when the shape of the data changes.

Subscriptions, even if they're written in another platform, are still considered to be a part of your system. RavenDB is meant to be an application database, not a shared database between a multitude of applications using the same data. Even if there are additional processes going on, they should be (eventually, at least) all owned by the application itself.

The beauty in using a scripting language for those kind of tasks is that it allows you to simply set up a subscription without any sort of application, compilation, etc. A single script per business process is usually enough. Since those are background batch processes, it doesn't usually matter if the language isn't very fast. You can trade off convenience for speed in this case with a clear conscience.

One thing that *isn't* in Listing 7.22 is error handling. In previous examples in this chapter, I spent some time discussing error handling, and any subscription scripts you have most certainly do need to have something there. But a conscientious admin will have already set up a watchdog for this, reporting errors and raising alerts accordingly. Any production system is made of a lot of moving parts. Most likely that the infrastructure to manage such scripts, restart them if there is an error and alert you if they haven't recovered is already in place.

For example Listing 7.23 uses the `recur` tool [<https://github.com/dbohdan/recur>] to run the Python subscription in a way that would recover from errors, manage exponential backups, etc.

Listing 7.23. Executing the subscription script under a watchdog

```
recur --forever --max-delay 5m --reset -15s \  
exponential ./new-customers-welcome-email.py
```

Listing 7.23 is shown not as a specific recommendation for `recur` that is shown there. I mention the need for actual orchestration of how your subscription code runs because it's important that it *is* in place. The nasty side of this kind of approach is that it's easy for what would turn out to be a critical business process to be developed in a completely "Shadow IT" approach; that is, this subscription is running on some server somewhere and will continue to do so until a reboot is done, where stuff breaks and no one really knows what.

7.12. Subscription and the RavenDB Cluster

We'll deal more with RavenDB clusters in Chapter 9, but since subscriptions are operating in a clustered environment, I wanted to touch on the topic briefly. We'll assume that we are running on a three node RavenDB cluster, and processing documents in a database that is replicated to all those nodes. The RavenDB client will direct read & writes from the session to one of the nodes automatically (we'll discuss that in a lot more depth in Chapter 9). How do subscriptions work when your database is running on multiple nodes?

The RavenDB cluster will assign one of the nodes hosting the database to be the owner of this

subscription. All the work for this subscription will be coordinated through that node. Of course, if that node goes down, RavenDB will automatically failover the ownership of the subscription to another node in the cluster. This is important, because the owning node of a subscription doesn't have to be the same node that the client considers the primary node.

RavenDB spreads the subscription load across all nodes

It makes sense to make the subscription owner and the client's primary node the same node, no? Why would we want to have them be different? The primary reason is that you are likely to have *multiple* subscriptions, and RavenDB can spread their load across multiple nodes. A more complete answer is that there isn't really a primary node in RavenDB. The default model uses a primary node, but that is just *a* default. You can use round-robin, you can use a designated node per client, etc.

Note that in practice, this doesn't matter. A write to any node will be replicated to all the other nodes of the database, and then the subscription will process that. Which node the subscription is connected to is not really something that you'll likely care about.

You can see in Figure 7.6 that the owning node of the subscription is A and that there is a single connected subscription worker that has subscribed to this subscription.

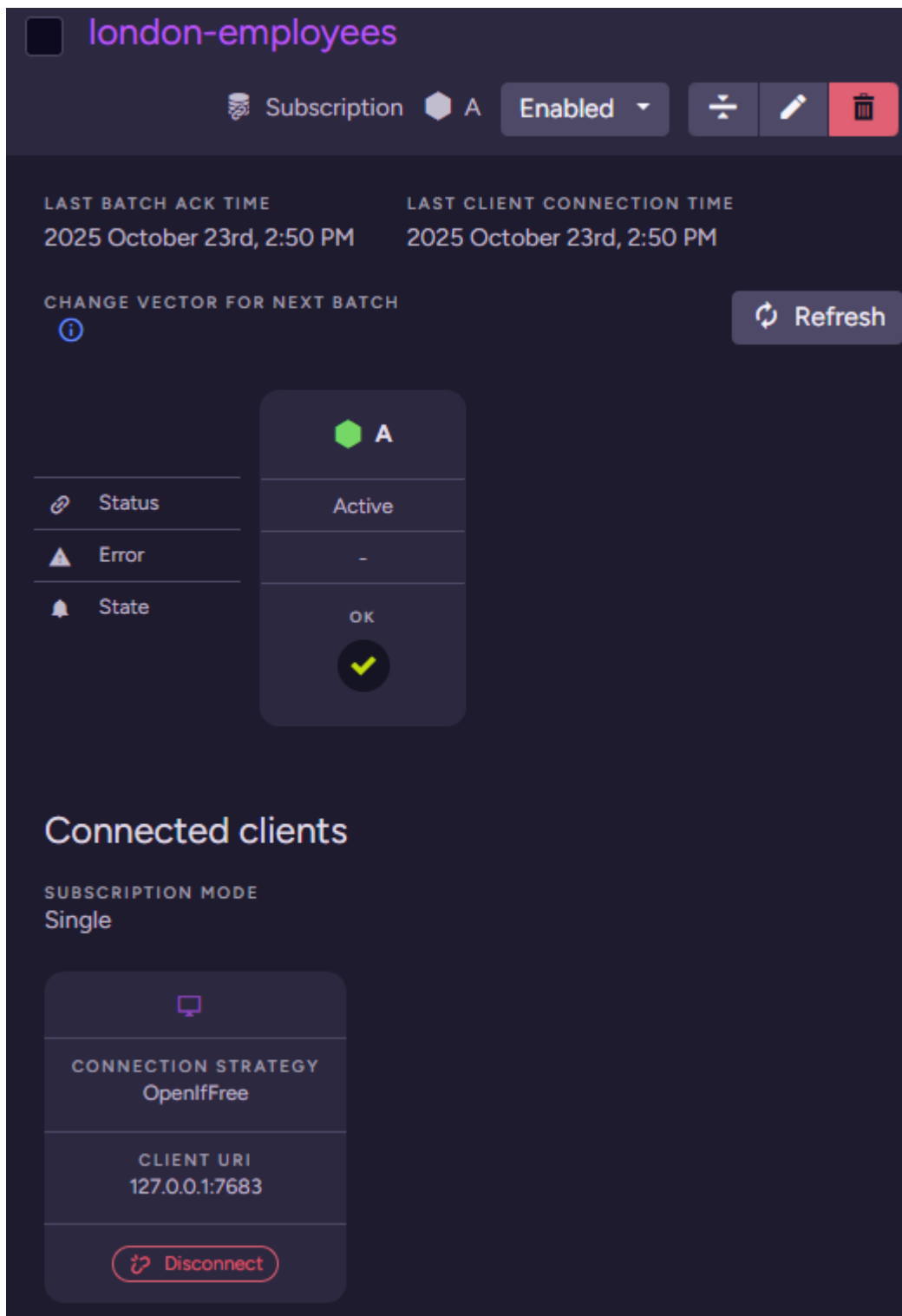


Figure 7.6. RavenDB Studio showing a client connected to the london-employees subscription

If you have multiple subscriptions, each will have a separate owning node, so we split the workload across all the nodes in the cluster. Of course, you can also override that decision by RavenDB and explicitly designate a node as the owning node of a subscription.

For concurrent subscriptions, the same rules apply. There is a single owner node, regardless of how many concurrent subscription workers you have. That node is responsible for sending out batches of documents to all the concurrent workers.

7.13. Subscriptions behind the scenes

When you create a subscription, you specify a query that defines the documents you want to subscribe to. Internally, a subscription is composed of two pieces:

- The query to run to find the documents to send to the worker.
- The last processed position for the subscription.

There is a bunch of other book-keeping involved, of course, but those are the two main pieces. When a subscription connects to RavenDB, we *scan* through the documents in the database in modification order. The query goes through each document since the last processed position and check if it matches the subscription query. If it does, we can add that to the batch and send it to the worker.

The internal etag queue

When RavenDB stores documents on disk, it assigns each document an etag, which is a strictly increasing number that represents the order in which the documents were modified. That etag is a *single node* representation of when a document was modified. Note that in a cluster, this doesn't *quite* work. Consider nodes A and B in a cluster. I modify users/1-A on node A and users/2-B on node B. Each node will then replicate the changes to the other node. However, the etags on each node may not reflect the same order of modifications. On node A, users/1-A may have etag 100 and users/2-B may have etag 101, while on node B, users/2-B may have etag 150 and users/1-A may have etag 153. At the cluster level, RavenDB uses change vectors, discussed in depth in Chapter 9.

The key is that whenever a document is modified, we'll update its etag. So updating users/1-A will give it a new etag, say 103 and the order of documents in node A will be users/2-B (101), users/1-A (103).

Subscriptions operate on the internal etag queue, remembering the last etag position that was successfully acknowledged by the worker. That means that whenever a document is modified, the subscription will process it *again*. That is either desirable, or something that we want to avoid, so we'll add that as a filter to the subscription query, as we did in Listing 7.6 when we checked the `call.Survey = false` at the query and marked the document with `Survey = true` when we processed them.

The reason RavenDB operates in this manner is that it means that it needs to keep track of a single value to keep the last processed position. That makes it very easy to handle the subscription state and recover from failures.

When the worker acknowledges that the batch has been processed successfully, we update the last processed position to the last document in the batch and start scanning through the remaining documents to gather enough items to send in the *next* batch. For concurrent subscription, we need to keep track of multiple concurrent batches being "checked-out" simultaneously. That is required so we won't send an update document to a worker while another worker instance is already processing an older version of the document.

Batch processing can fail if the subscription worker explicitly indicates a failure back to the RavenDB server, or if the subscription connection has dropped. In that case, we don't update the last processed position, and the next time a worker connects, we resend the same batch of documents again. For concurrent subscriptions, we may need to scan through the "checked-out" documents that failed to process and send them to the next available worker.

The biggest cost subscription has on the server side is the need to scan through all the documents in the collection. RavenDB translates the subscription query into a JavaScript function that it applies to each document. A subscription that is connected but isn't currently processing documents require the server to just send a ping every few seconds (to verify that the client on the other side is indeed connected). A subscription that is *not* connected has no cost at all on the server side.

In other words, subscriptions are very cheap to maintain on the server side. The main cost is when you have a subscription worker needs a new batch of documents to process. Assuming that the client does *any* processing at all, that means that most of the time, the server isn't actually doing much. Having a lot of subscriptions operate at once is fine, since they aren't going to be putting a lot of load on the servers. At the same time, you can close subscriptions that are running to reduce overall system load and resume them (from where they left off) if you need to provide extra breathing room for your system.

While the subscription itself is cheap on the server side, there is the worker process that handle batches of documents. That work can be expensive, depending on what you're doing. And being able to delay background work to less busy times is a great way to ensure that your system remains responsive to user requests.

7.14. Subscribing to documents in the future

Subscriptions operate on documents as they are *now*. The subscription query is applied to the current state of the document, and whenever a document is modified, we re-apply the subscription query to see if it matches. When happens if I want to subscribe a document that was modified now, but I want to be called in 3 days? A good example is dealing with timeouts. We send a survey to the user, and we want to follow up if they haven't responded in 3 days. Essentially, we want to subscribe to documents in the future.

The problem is that the subscriptions don't work like that. In the previous section, we discussed how subscription operate purely on the modification order of documents. We cannot "schedule" a document to be processed in the future. However, we can use a small trick to achieve the same effect. RavenDB has the notion of `@refresh` metadata, you can see how we use that in Listing 7.24.

Listing 7.24. Creating a survey document with @refresh metadata three days for now

```
using (var session = store.OpenSession())
{
    Survey survey = new Survey
    {
        Title = "Customer Satisfaction",
        Questions =
        [
            "How satisfied are you with our product?",
```

```

        "How likely are you to recommend our product to others?"
    ]
};
session.Store(survey);
var metadata = session.Advanced.GetMetadataFor(survey);
metadata["@refresh"] = DateTime.Today.AddDays(3);
session.SaveChanges();
}

```

The code in Listing 7.24 creates a Survey document and sets the @refresh metadata to three days in the future. You can see what this looks in in the RavenDB Studio in Figure 7.7. The idea of @refresh is really simple, at the specified time, RavenDB will simply remove the @refresh metadata from the document, that is all. That doesn't look like something particularly useful, isn't it?

When RavenDB removes the @refresh metadata from the document, it is *modifying* the document. That in turn means that anything that processes documents as they are updated will reprocess them. That includes indexes, ETL processes and most relevant for our current discussion, subscriptions.



Figure 7.7. Showing the @refresh metadata set on a document

We can take advantage of this behavior to implement "subscriptions in the future". We create a subscription that looks for Survey documents that don't have the @refresh metadata set, as shown in Listing 7.25. The key here is that the query will filter based on where '@metadata'. '@refresh' == null.

Listing 7.25. Subscription query for Survey documents without @refresh metadata

```

from "Surveys"
where '@metadata'. '@refresh' == null

```

In other words, we expect each Survey document to be processed *twice* by the subscription. First, we'll check if when it is created (as shown in Listing 7.24), and we'll filter it out since it *has* a @refresh value in its metadata. Then, three days later, RavenDB will remove the @refresh metadata, modifying the document. At that point, the subscription will see the document again, and this time it will match the query, and the subscription worker will be able to process it. In effect, we've created a "subscription in the future" by using the @refresh metadata as a delay mechanism.

This @refresh technique to delay the processing of documents with subscriptions is a common pattern for implementing timeouts, delayed processing, etc. We'll talk more about @refresh in Chapter 6, where we'll introduce a few more advanced capabilities of RavenDB documents.

7.15. Summary

When RavenDB implemented subscriptions for the first time, it was just a small section: a minor feature that was meant to handle an esoteric scenario that only a single customer had run into. From that modest beginning, this feature has blown up to completely replace most batch processing handling in RavenDB. It's not the first thing you'll use in every situation, but for anything that *isn't* about responding to a user request, it's usually a match.

In this chapter, we looked into what subscriptions are, how to do batch processes with RavenDB and subscriptions, how to filter and modify the data on the server and how to accept and process the data on the client side and write it back to RavenDB in an optimal fashion. We looked at integrating with external systems via the email-sending example, including how to handle failures and partial failures both in RavenDB and in the external systems.

Using subscription for ETL work

It's tempting to use subscriptions to handle ETL tasks,^[8] such as writing to a reporting database. While it is possible, RavenDB has better options to handle ETL. See the discussion on this topic in Chapter 15.

We spent a lot of time discussing how to handle errors in subscriptions, how to deploy subscriptions into production and how we can ensure failover and load balancing of the work among both client and server using subscriptions. The subscription open strategies allow us to have a hot standby client for a subscription, ready to take over from a failed node and continue processing or multiple concurrent workers that subscribe to same subscription. On the server side, RavenDB will automatically failover subscriptions from a failed database node to a healthy one in a completely transparent manner.

Another great use case for subscriptions is implementing task queues. And we looked at that, including error handling and recovery. We also discussed the ability to fix an issue with a document that failed and have it automatically reprocessed by the subscription.

We then looked at versioned subscriptions, where we ask RavenDB to give us a before-and-after snapshot of each and every change for the documents we care about. This is a great help when producing timelines, tracking changes and handling forensics. This feature relies on the revisions configuration on the database and exposes it directly for batch processing needs. We can even write a subscription script that would filter and send just the relevant details from both old and new revisions of the document to the client side. Finally, we looked at how we can subscribe to documents in the future using `@refresh`, allowing us to implement timeouts and delayed processing in a simple manner.

Coming up next, in Chapter 8 we are going to explore how to leverage these advanced capabilities for AI integration and utilizing large language models in your systems...

[1] Although errors may cause you to receive the same document multiple times, you're guaranteed to never miss a document.

[2] This particular lecture was over a decade ago, but I still vividly remember it. It was *that* good.

[3] There's also the *three*-phase commit, which just adds to the fun and doesn't actually solve the issue.

[4] Remember, that budget is configurable, but it's mostly there to help you realize that generating so many requests is probably

not healthy for you.

[5] And you probably shouldn't do that.

[6] I'm using the term worker node here to refer to machines that are running business processes, subscriptions, etc. This is to distinguish them from RavenDB nodes.

[7] Again, different nodes may have observed the events in a different order, but it should roughly match across the cluster.

[8] Extract, transform, load: the process of moving data around between different data storage systems.

Chapter 8. Large Language Models & AI Integration

AI has conquered the world by storm. There is a high likelihood that these words are being read by an AI model so it can summarize the contents for you. As a developer, using AI is a core part of how you build software. Whether it is asking the model questions or using it to generate code, for developers, AI is now a first-class tool in their toolbox. At the same time, when it comes the time to actually build AI features for *your* systems, developers are faced with a daunting task. You first need to understand what models can do, select the appropriate model, figure out how to reduce costs, secure your data and integrate that into your application in a way that make sense for your users.

At the lowest rung of the ladder, you can throw your documentation into a model and get a basic chat bot that has some understanding of your system. But this is limited in scope and usually only works for checking the "we must have an AI feature" box. The real power of AI comes when you can deeply integrate it into your system, but that is *hard*. One of the reasons RavenDB *exists* is to make hard things easy. The core philosophy of RavenDB is to take complex, difficult, and error-prone tasks and make them simple, reliable, and easy to use. That is why we started working on that in the first place.

We aim to bring this same philosophy to AI integration. RavenDB has a number of features that make it easy to build AI-powered applications. From semantic search to generative AI to AI agents, RavenDB provides a comprehensive suite of tools that make it easy to build intelligent applications. We'll spend this chapter understanding how you can use these features to build smart applications and integrate AI deeply into your systems.

A bit of foreshadowing

AI integration is a Big Deal, and it actually builds on a bunch of RavenDB features that we haven't touched yet. I'm going to be focused specially on how you can *use* AI in this chapter, and leave the details about how RavenDB make everything ticks to the next chapters. In particular, we'll talk about vector search and the HNSW algorithm - which are covered in more depth in Chapter 13 and most of the actual AI integration capabilities we'll discuss (Embedding Generation and Gen AI) are built on top of RavenDB's ETL capabilities, which we'll cover in Chapter 15. There is no need to read those chapters first, but if you want to understand the internals of how everything works, you should definitely check them out.

AI and Large Language Models (LLMs) is a *big* topic, but in the context of RavenDB, it comes down to following key features:

- Vector Search - allowing to search for the meaning of data, not just the words.
- Embedding Generation - generating high-quality vector representations of your data using powerful models.
- Generative AI Tasks - applying AI models to your documents in a structured, repeatable way.
- AI Agents Creator - building conversational agents that can interact with your data and perform

actions on your behalf.

Most of these features build on top of each other and work together with other RavenDB features to provide a comprehensive AI solution. We'll see some examples of that later in the chapter. Vector search is the foundation, allowing you to search for the meaning of your data. Embedding generation allows you to utilize external models to generate high-quality vectors for your data, which are then used by vector search. Generative AI tasks allow you to apply AI models to your documents in a structured, repeatable way. That includes making use of embedding generation and vector search to provide context to the model. Finally, the AI Agents Creator builds on top of generative AI tasks, allowing you to build conversational agents that can interact with your data and perform actions on your behalf.

8.1. Vector search

I think that the best way to explain this feature is to show, rather than tell. Take a look at the query in Listing 8.1. Running this query on the Northwind sample database, will give you the shown in Figure 8.1.

Listing 8.1. Searching for sweets in the Northwind sample database using vector search

```
$query = 'sweets'  
from Products  
where vector.search(embedding.text(Name), $query)  
select id(), Name  
limit 2
```

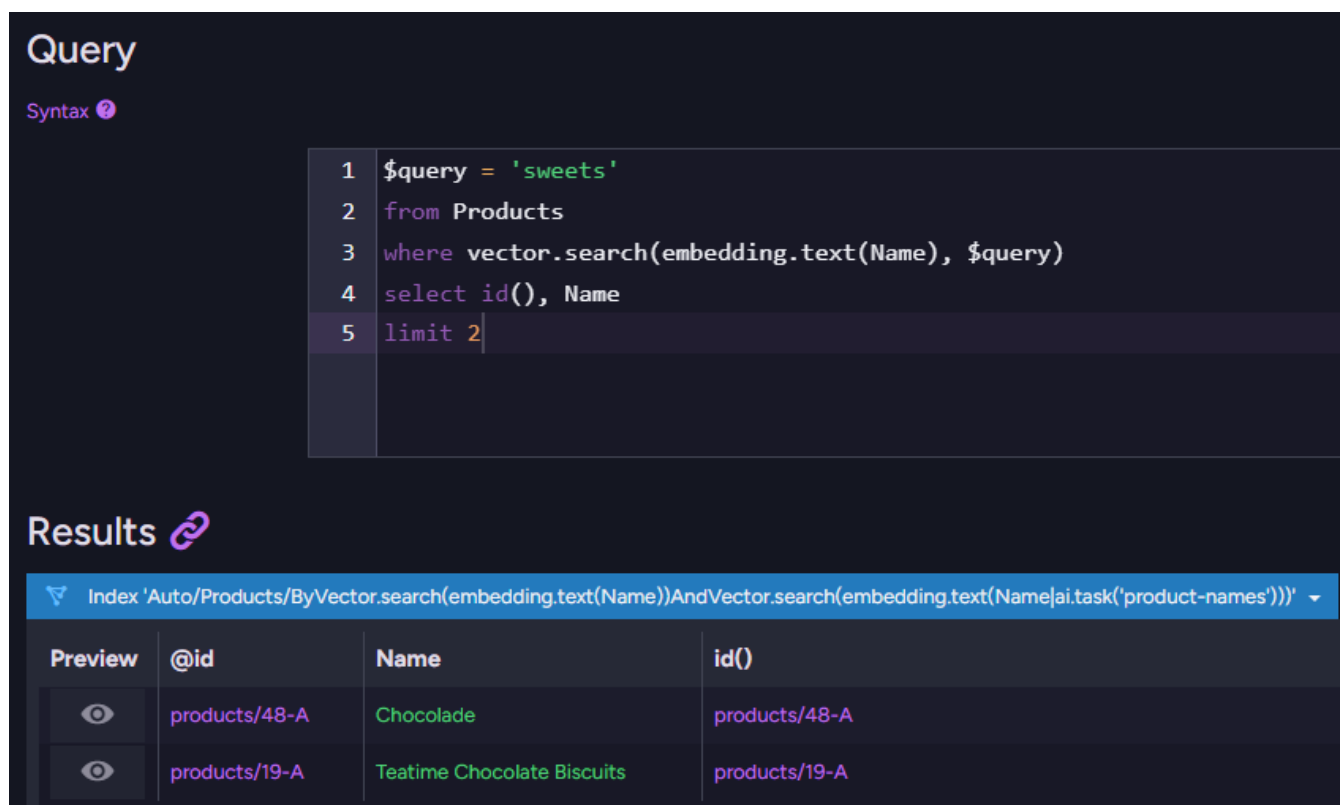


Figure 8.1. Result of searching for sweets in using vector search query in the Northwind sample database.

The query in Listing 8.1 is really interesting. Even just a few years ago, this would have been

impossible to do. In the results, we get products named "Chocolade" and "Teatime Chocolate Biscuits". How does that work? There is no word "sweets" in either of those product names, so a traditional keyword search would fail. This is where vector search shines, it is not just searching for keywords, but is able to dig deeper into the *meaning* of the what you are looking for.

Seaching by intent, instead of by keywords

I want to stop for a moment and talk a bit more about Figure 8.1 and what you see there. We are querying on "sweets" and finding chocolate and biscuits. We elevated our search capabilities by several orders of magnitude by being able to search by *intent*, rather than by keywords. This is a *huge* deal. It means that you can now search for things without needing to hit on the exact magic incantation. In terms of user experience, that is a huge leap.

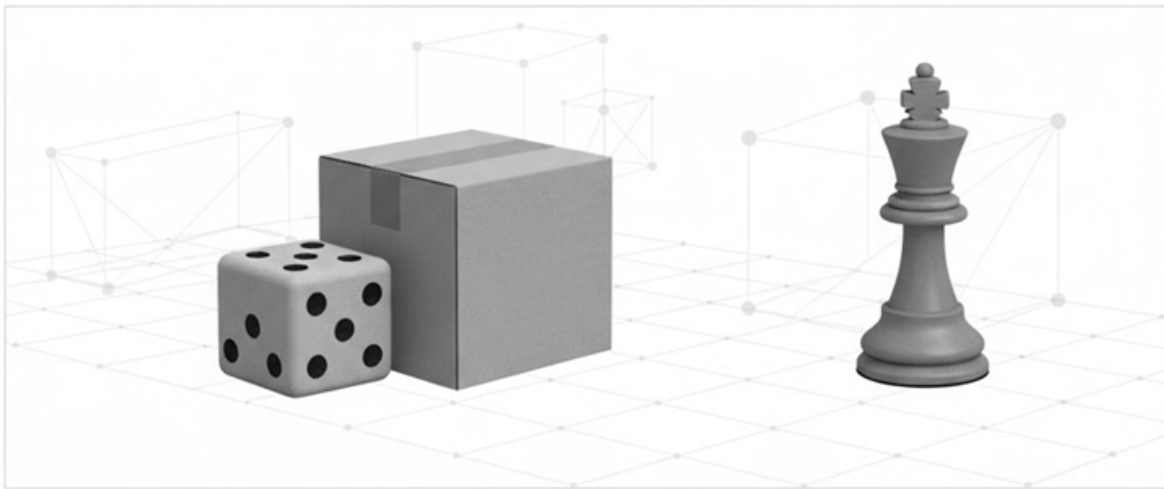
This is also a *big* deal for developers, you now have access to this level of search without going through a lot of hassle. To give some context, to do the same thing without vector seach, you would need to deal with synonyms, misspellings, stemming, thesauri and a lot of other things. You would have to build a complex search index, and even then, the results would be limited.

With vector search, you lean *hard* on the model itself. You want to enable multilingual search? Use a model that supports that, like cohere-embed-multilingual, and let your users search in their native language. All you need to do is just specify which model to use, RavenDB will take care of the rest and you can wow your users.

Let's talk first about what vector search *is*, and then we'll see how RavenDB implements it. A vector (or embedding) is a high-dimensional representation of data. It is a series of numbers that represent the meaning of the data as understood by a large language model. The actual numerical meaning isn't really *that* interesting, to be honest. What matters is that the model we can check the distance between two vectors. The closer the vectors are, the more similar the meaning of the data is as far as the model is concerned.

Figure 8.2 shows an example of how the same set of items can be grouped in different ways depending on the model used to generate the vectors. In Figure 8.2 we grouped the items at the top by shape (so dice & cardboard box are close together). At the bottom, we grouped them by functionality (so the dice and the chess piece are close together). It is important to understand that *both* groupings are valid, they just represent different aspects of the data. You can train the model to focus on different aspects of the data, and that will change how the vectors are generated.

TOP: Grouping A: Model 1 (Similarity by Geometric Shape)



BOTTOM: Grouping B: Model 2 (Similarity by Usage)

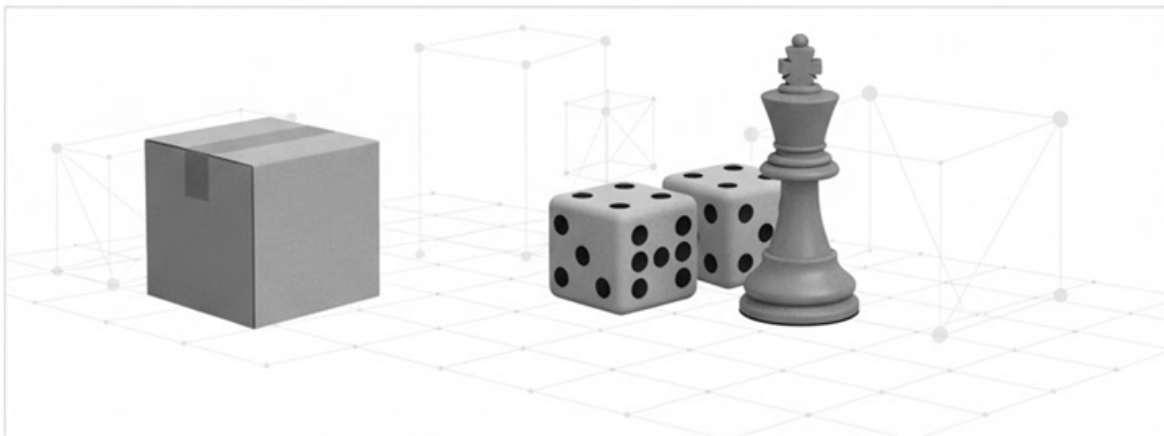


Figure 8.2. Showing two grouping of the same items using different models

The reason that I am going into this much detail is that it is important to understand that vector search is not magic. It is simply finding the closest matches between your search query and the data in your database. The key is that it does that using similarity search based on high dimensional vectors that capture the model's understanding on the query and the data.

With a better understanding of what vector search is, let's go back to Listing 8.1 and see what exactly is going on there. The key part of the query is the `vector.search(embedding.text(Name), $query)` expression. Let's dissect that for a bit. The `embedding.text(Name)` part is telling RavenDB that we are using a getting text that we need to convert to a vector. We can then compare that to the vectors generated from the `Name` field.

RavenDB embeds an embedding model (`bge-micro-v2`) internally. This model is a small, efficient model that can run on most systems. Since we didn't specify anything else, RavenDB is going to use this model for embedding generation. Then it will perform a vector search to find the nearest

matches using the model's understanding of the data and the query.

The bge-micro-v2 model and its use cases

The bge-micro-v2 is a *small* embeddings model. RavenDB use this model because it allows you to get started with vector search with no hassles. There is no need to setup API Keys or pay for external models. You can just start using vector search right away, zero setup required. The bge-micro-v2 model can run on the CPU, so you don't need to provision a database server with a GPU.

The bge-micro-v2 model is *great* for demos and getting you off the ground when you start. It provides good results for many use cases. However, it is still a fairly small model and you can get better results from other models.

I mention this because it is easy to think that vector search isn't that useful. If you use bge-micro-v2, which has inherently limited capabilities, it may give underwhelming results. When evaluating the usefulness of vector search, be sure to also evaluate other models via the Embedding Generation feature, we'll cover it in depth later in this chapter.

The bge-micro-v2 model outputs an embedding vector with 384 dimensions (an array with 384 float values). With OpenAI, we have text-embedding-3-small with 1,536 dimensions and text-embedding-3-large with 3,072 dimensions. The larger the model, the more nuanced the understanding of the data is likely to be. On the other hand, larger models are more expensive to use, so there is a trade-off between cost and quality.

Another tradeoff to consider is the *size* of the vectors. A 384 dimension vector is about 1.5KB in size (384 floats at 4 bytes each). A 1,536 dimension vector is about 6KB in size and a 3,072 dimension vector is about 12KB in size. When you have millions of documents, the size of the vectors can add up quickly. That is why RavenDB supports quantization, which can reduce the size of the vectors significantly, at the cost of some accuracy.

RavenDB supports quantizing the data using Int8 and binary quantizations. Int8 quantization can reduce the size of a 1,536 dimension vector from 6KB to about 1.5KB, while binary quantization can reduce it to about 192 bytes. You'll need to experiment to see what balance of accuracy and performance works best for your use case. Note that you need to evaluate both different models and different quantizations.

If you aren't working with a *lot* of vectors, there is usually no need to really bother. Just use the model that gives you the best results and don't worry about the size of the vectors. I did want to mention that RavenDB has the inherent capability to handle quantization for you, should you need it.

8.1.1. Exact and Approximate Nearest Neighbors Search

So we now understand what vectors are. What are we actually doing with them? Listing 8.2 shows the output of the bge-micro-v2 model for the words "sweets" and "chocolate", and how we can compute the distance between the two vectors using cosine similarity. To be clear, that isn't what is actually happening inside of RavenDB, but that is a good way to understand what is going on at a

conceptual level. The higher the similarity value, the closer the vectors are, and the more similar the meaning of the data is as far as the model is concerned.

Listing 8.2. Showing the output of the bge-micro-v2 model for the word "sweets" and "chocolate"

```
float[] sweetsEmbedding = new float[384];
bge_micro_v2("sweets", sweetsEmbedding);
// -0.18093847,0.04937424,0.19464265,0.011313453,-0.1382206,0.518904,      ①
// 0.56170607,0.11258009,-0.22343214,0.11457193,0.051699474,-0.7476225,
// 372 more numbers elided for brevity

float[] chocolateEmbedding = new float[384];
bge_micro_v2("chocolate", chocolateEmbedding);
// -0.08718172,-0.42426482,0.20022313,0.12990747,0.053507004,0.118514456, ②
// 0.4829657,0.4304149,-0.30294275,0.0854362,0.23490147,-0.55278444,
// 372 more numbers elided for brevity

var similarity = TensorPrimitives.CosineSimilarity(                      ③
    sweetsEmbedding, chocolateEmbedding);
// similarity = 0.7771348
```

- ① The embedding vector for the word "sweets".
- ② The embedding vector for the word "chocolate".
- ③ Calculating the similarity between the two vectors using cosine similarity.

Listing 8.2 shows how we compare the similarity between the two vectors using cosine similarity. But when we use the `vector.search()` function in RavenDB, what is actually happening? We need to compare the current query that we are searching for to *all* the vectors in the database. Listing 8.3 shows how that could work.

Listing 8.3. Exact nearest neighbors search algorithm

```
float[] sweetsEmbedding = new float[384];
bge_micro_v2("sweets", sweetsEmbedding);
var topMatchesQueue = new PriorityQueue<EmbeddingResult, float>();
foreach(var embedding in AllEmbeddingsFor("Name"))
{
    var similarity = TensorPrimitives.CosineSimilarity(
        sweetsEmbedding, embedding);
    var result = new EmbeddingResult(embedding, similarity)
    if (topMatchesQueue.Count < N)
    {
        topMatchesQueue.Enqueue(result, -similarity);
    }
    else if (similarity > topMatchesQueue.Peek().Similarity)
    {
        topMatchesQueue.EnqueueDequeue(result, -similarity);
    }
}
```

```
// topMatchesQueue now contains the N most similar embeddings
```

The code in Listing 8.3 is doing an exact nearest neighbors search. It is comparing the embedding for "sweets" to *all* the embeddings in the database, and keeping track of the top N matches. Note that there is some fancy footwork to keep the priority queue sorted by max similarity while the underlying priority queue is a min-heap. The details aren't that important, the key point is that we are doing a linear scan of all the embeddings in the database.

RavenDB *can* do something like that, but you need to tell it to do so very explicitly. If you want to use exact nearest neighbors search, you can use: `exact(vector.search(...))`. In the case of Listing 8.1, you would write the query like this: `where exact(vector.search(embedding.text(Name), $query))`. This will force RavenDB to do an exact nearest neighbors search, in a very similar manner to the code in Listing 8.3. You have to force RavenDB to go this route because it is *slow* when you have a lot of vectors.

RavenDB typically uses a much more efficient algorithm called HNSW (Hierarchical Navigable Small World) for vector search. HNSW is an approximate nearest neighbors search algorithm that is much faster than exact nearest neighbors search. The details on the HNSW algorithm are beyond the scope of this book, but you can read more about it in the online documentation.

HNSW is an approximate algorithm. You can see that the code in Listing 8.3 will scan all the embeddings in the database and will find us the best matches. HNSW, on the other hand, will use a graph-based approach to find the nearest neighbors. This makes it *much* faster, but also means that it is not guaranteed to find the *exact* nearest neighbors. In practice, it usually does and when it doesn't, the results are usually still very good, but it is something to be aware of.

The key problem is that you usually simply *cannot* do a full scan of the data, there is too much of it. If you have a million documents, and each embedding is 1,536 dimensions (6KB), that is 6GB of data to scan through for a single query when using exact nearest neighbors search. At the same time, HNSW can find good matches in milliseconds, even with tens of millions of embeddings.

The HNSW algorithm

The HNSW algorithm is based on building a layer graph, where each layer in the graph is a subset of the layer below it. The bottom layer contains all the embeddings, while the top layer contains only a few. When searching for the nearest neighbors, HNSW starts at the top layer and works its way down to the bottom layer. At each layer, it uses a greedy algorithm to find the nearest neighbors. This allows HNSW to quickly narrow down the search space and find the nearest neighbors in a fraction of the time it would take to do an exact nearest neighbors search.

If you are interested in the details of the HNSW algorithm, you can find a lot of paper, talks and sample code implementing it. I like reading through code to understand how things work, so I recommend checking out the `deepfates/hnsw` project on GitHub [<https://github.com/deepfates/hnsw>]. It is a simple, clean implementation of the HNSW algorithm in TypeScript, which is a great way to explore how this algorithm works.

The HNSW algorithm that is implemented in RavenDB is a lot more complex. Primarily

because we are using a persistent storage engine, rather than an in-memory data structure and that we optimize to run on all sorts of hardware, including pretty modest machines. If you are interested, I wrote a blog post on the details on optimizing HNSW in RavenDB for low end hardware [<https://ravendb.net/articles/scaling-hnsw-in-ravendb-optimizing-for-inadequate-hardware>].

With proper understanding of vector search, we now need to understand how to get the vectors in the first place. That is where Embedding Generation comes into play.

8.2. Connecting RavenDB to AI Models

Before we get to actually working with models, we need to setup RavenDB to be able to connect to them. Aside from bge-micro-v2, which is meant to get you started quickly, all other models are external. That means that you need to tell RavenDB about the model you want to use and how to connect to it. There are two types of models that RavenDB can connect to: embedding models and generative models. Embedding models are used to generate vectors for vector search, while generative models are used to process and generate data.

For embedding, RavenDB natively supports (at the time of this writing) OpenAI, Open Router, Ollama, Hugging Face, Google AI, Vergex, Mistral and Azure OpenAI. Note that supporting OpenAI for embedding means that any model that offers the v1/embeddings endpoint will work just fine. That includes a lot of models that are not explicitly listed here.

For generative models, RavenDB natively supports OpenAI, Ollama and Azure OpenAI. Again, any model that offers the v1/chat/completions endpoint will work just fine. Here RavenDB requires a bit more, since generative models are more complex than embedding models. In particular, RavenDB need the model in question to be able to support tool usage and structured output. Most models support that out of the box, but it is something to be aware of.

For simplicity, we'll focus on OpenAI and Ollama as the models we'll be using in the examples. The process is similar for other models, check the online documentation for the full details. I'm assuming that you have an OpenAI account and has the OpenAI API key ready to go. For Ollama, you can run the code shown in Listing 8.4 to get started.

I'm using Ollama as a way to run the models completely on your own hardware, and OpenAI as a way to use a hosted model. That way, you can see how to use both hosted and self-hosted models with RavenDB.

Listing 8.4. Running Ollama locally using Docker

```
docker run -d -v ollama:/root/.ollama -p 11434:11434 -name ollama ollama/ollama
curl --retry-max-time 30 -s http://localhost:11434/
docker exec ollama ollama pull llama3.2:3b
docker exec ollama ollama pull nomic-embed-text
```

The script in Listing 8.4 will run Ollama in a Docker container, and will pull the llama3.2:3b model for generative tasks and the nomic-embed-text model for embedding tasks. You can use any model that Ollama supports, of course. Those are just here to get you started. Once you have Ollama running, you can connect to it using RavenDB.

8.2.1. Connecting RavenDB to Ollama

Connecting RavenDB to an AI model is simple. Go to the RavenDB Studio and go to the Northwind database, then the AI Hub section and the AI Connection Strings option. Click on Add New, give the connection string a name (we'll use Ollama Embedding for now). You can then select an identifier for the connection string (we'll use ollama-embedding).

Identifiers in RavenDB AI Integration

You'll find a lot of identifiers in RavenDB's AI integration. Those are simply unique identifiers (without spaces) that RavenDB uses to refer to connection strings, tasks, agents, etc. The idea is that you can have a human friendly name (like Ollama Embedding) and a machine friendly identifier (like ollama-embedding). For references, you'll always use the identifier, while for display purposes, you'll use the name.

Next, you need to select the model type. We'll use Text Generation for now. Finally, you need to specify Ollama as the connector and `http://localhost:11434/` as the URL. Clicking on the Model drop down should list the models that Ollama has available (we pulled `llama3.2:3b` and `nomic-embed-text` earlier, so they should be listed there). Select `nomic-embed-text` as the model and click on Save connection string. You can see the full process in Figure 8.3.

Create a new connection string

Name

Ollama Embedding

Identifier (optional) ⓘ

ollama-embedding

Regenerate

Model type ⓘ

Chat

Conversational AI and content generation model

Text embeddings

Embedding generation model for vector search and similarity comparison

Connector

Ollama

URI ⓘ

http://localhost:11434/

Model ⓘ

nomie-embed-text:latest

Max Concurrent Query Batches (optional) ⓘ

Test connection

Cancel

Save connection string

Figure 8.3. Creating an Ollama connection string in RavenDB Studio

You can repeat the same process as in Figure 8.3 to create a connection string for generative tasks. We'll call it Ollama Generative, with the identifier `ollama-generative`, use Chat as the model type, Ollama as the connector type, `http://localhost:11434/` as the URL and select the `llama3.2:3b` model.

8.2.2. Connecting RavenDB to OpenAI and OpenAI compatible models

Connecting RavenDB to OpenAI is similar to connecting to Ollama. Go to the AI Hub section in

RavenDB Studio, then the AI Connection Strings option. Click on Add New, give the connection string a name (we'll use OpenAI Embedding for now). You can then select an identifier for the connection string (we'll use openai-embedding). Next, you need to select Text Generation for the model type then specify OpenAI as the connector.

You now need to provide your OpenAI API key. You can do that by setting the API Key field to your OpenAI API key. Finally, you need to select the URL to use. For OpenAI, you can use the default URL (<https://api.openai.com/v1/>). If you are using Grok, on the other hand, you'll use <https://api.x.ai/v1> and using OpenRouter you'll use <https://openrouter.ai/api/v1>.

You can now select the model to use. For embedding, you can select either text-embedding-3-small. Click on Save connection string to finish. You can see the full process in Figure 8.4.

Create a new connection string

Name

OpenAI Embedding

Identifier (optional) ⓘ

openai-embedding ↻ Regenerate

Model type ⓘ

Chat

🗯️ Conversational AI and content generation model

Text embeddings

📄 Embedding generation model for vector search and similarity comparison

Connector

🌀 OpenAI ▼

API Key ⓘ

.....👁️

Endpoint ⓘ

https://api.openai.com/v1/ ▼

Model ⓘ

text-embedding-3-small ▼

Figure 8.4. Creating an OpenAI connection string in RavenDB Studio

You can repeat the same process as in Figure 8.4 to create a connection string for generative tasks. We'll call it OpenAI Generative, with the identifier openai-generative, use Chat as the model type, OpenAI as the connector type, provide your OpenAI API key and the model URL then select the gpt-4.1-mini model.

We'll discuss more about model selection later in the chapter, but for now, this is enough to get you started. Withh those logistics out of the way, let's see how we can use those features in practice.

8.3. Embedding Generation

Until now, we didn't need to consider how the vectors were generated. RavenDB used the embedded model (bge-micro-v2) to handle that for us automatically. That is great to get started, but you'll usually want to graduate to using more powerful models. That is where Embedding Generation comes into play. Before we get to the details of how this works, I want to explain a bit about the constraints in which embedding generation operates.

Models usually operate best using a GPU, and *most* database servers don't have a powerful GPU (or cost way too much if they do). Embedding is computationally expensive, so it is important to reduce both the amount of data you want to embed and the number of time you need to do so.

You'll usually run the embedding model externally. Whether as a cloud service (like OpenAI, Cohere, etc) or your own hosted model (using something like Ollama or LlamaIndex). In other words, embedding generation is *not* done directly inside of RavenDB, but externally using remote calls. Furthermore, in most external models, you have to consider rate limits. You cannot just blast the model with embedding requests, you need to throttle your requests. When running on your own hardware, you need to avoid overloading the GPU, so you have effective rate limits there as well.

All of that means that embedding generation is not something that we *can* do in a synchronous manner, not at scale. Using the Product's Name as an example, we cannot just expect RavenDB to compute the embedding for the Name field when we update a document. We *can*, but that would have an unacceptable performance penalty. Instead, we need to do that asynchronously, in the background, and in a batched manner (since that is more efficient for the model).

RavenDB's Embedding Generation feature is built to handle exactly that for you. Taking complete ownership of the entire process of embedding generation, from tracking which documents need to be embedded, batching requests to the model, and storing the resulting vectors. RavenDB also handles all the logistics around that. If you updated the Product document and did *not* changed the Name field, RavenDB will not re-embed the document. If the embedding model is temporarily unavailable (because of an error or exceeding the rate limits), RavenDB will back off and retry later.



Embedding Generation in RavenDB is built on top of RavenDB's ETL (Extract, Transform, Load) capabilities, which we'll cover in depth in Chapter 15.

To create an embedding generation task, go to the AI Hub section in RavenDB Studio, then to AI Tasks and then on Add AI Task. Select Embedding Generation as the task type. You can then give the task a name (we'll use Product Name Embedding (Ollama)), and an identifier (we'll use product-name-embedding-ollama). In the connection string, we'll select Ollama Embedding (the one we created earlier). You can see what the initial configuration looks like in Figure 8.5.

New Embeddings Generation

 Save  Cancel

Task Name

Identifier 
  Regenerate

Task State

☐ Set responsible node

Connection String
  Create a new AI connection string

 Test Connection

Figure 8.5. Creating an Embedding Generation task in RavenDB Studio

Next, we need to specify the collection to operate on. You can select Products from the drop down. Finally, we need to specify the field Name to embed and then click on Add Path Configuration. You can see the full configuration in Figure 8.6. You can click on Save to finish creating the task and getting RavenDB started on embedding your products' names.

Collection

Products

Embeddings Source

Paths ▾

Path Configurations

Source Text Path

Enter Source Text Path

Chunking Method

Plain Text: Split Lines ▾

Max Tokens per Chunk

Default: 2048

Overlap Tokens

Default: 0

+ Add path configuration

Path	Chunking Method	Max Tokens per Chunk	Overlap Tokens
Name	PlainTextSplit	2048	0 

Quantization ⓘ

Single (no quantization) ▾

Embeddings Cache Expiration ⓘ

90

0

0

0

DAYS

HOURS

MINUTES

SECONDS

Figure 8.6. Full Embedding Generation task configuration in RavenDB Studio

In Figure 8.6, you can see that there are a *lot* of options that we didn't touch. We'll explain what they mean shortly, but for now, you should be able to see that RavenDB is already working in the background on generating the embeddings for your data. You should be able to see two new document collections in the database: `@embeddings/Products` and `@embeddings-cache`. Those are used by RavenDB to manage the state of the embedding generation and store the resulting vectors.

You should also generate another embedding generation task for OpenAI. You can repeat the same process as in Figure 8.5, but this time, we'll call it `Product Name Embedding (OpenAI)`, with the identifier `product-name-embedding-openai` and use the OpenAI Embedding connection string.

8.3.1. Using the generated embeddings

Once RavenDB has generated the embeddings for your data, you can use them in your queries. You can use the same vector search query as in Listing 8.1, but this time, we'll specify the model to use. We'll use the `text-embedding-3-small` model from OpenAI that we set up earlier. You can see how we can use these generated embedding for queries in Listing 8.5.

Listing 8.5. Searching for sweets in the Northwind sample database using vector search with OpenAI embeddings

```
$query = 'sweets'  
from Products  
where vector.search(embedding.text(Name,  
    ai.task("product-name-embedding-openai")), $query)  
select id(), Name  
limit 5
```

The query in Listing 8.5 is similar to the one in Listing 8.1, but this time, we are specifying the embedding generation task to use. We do that by using the `ai.task("product-name-embedding-openai")` argument to the `embedding.text()` function. This tells RavenDB to use the embeddings generated by the `product-name-embedding-openai` task.

Embedding generation also handles queries

When we use `embedding.text()`, it also runs the `$query` text through the same embedding model, so both the query and the data are represented in the same vector space. This works when you are using the default `bge-micro-v2` model or `ai.task()` to explicitly specify the embedding model. This saves you the need to manually generate the embedding for the query text. It is RavenDB's responsibility to handle this chore.

This little tidbit of ergonomics is really important. It means that you can just write your queries as you normally would, and RavenDB will take care of the embedding generation for you. That includes dealing with rate limits, caching, batching, retries, etc. You can just focus on your application logic and let RavenDB handle the rest.

The results of the query in Listing 8.5 are decidedly *not* the same as the results from Listing 8.1. Take a look at Table 8.2 and Table 8.3. Note that I increased the `limit` to 5, to show more results. I also added a `Meaning` column to better explain what each product is, given that some of the names are not in English.

Table 8.2. Result of searching for sweets in using vector search query in the Northwind sample database with OpenAI's text-embedding-3-small embeddings.

ID	Name	Meaning
products/62-A	Tarte au sucre	Sweet sugar pie
products/61-A	Sirop d'érable	Sweet syrup from maple tree
products/48-A	Chocolade	Sweet treat made from cocoa
products/3-A	Aniseed Syrup	Sweet liquid made from anise seeds
products/44-A	Gula Malacca	Sweet brown sugar from Malacca

Table 8.3. Result of searching for sweets in using vector search query in the Northwind sample database with

bge-micro-v2 *embeddings*.

ID	Name	Meaning
products/48-A	Chocolade	Sweet treat made from cocoa
products/19-A	Teatime Chocolate Biscuits	Chocolate cookies to eat with tea
products/3-A	Aniseed Syrup	Sweet liquid made from anise seeds
products/5-A	Chef Anton's Gumbo Mix	Spicy soup mix made by Chef Anton
products/26-A	Gumbär Gummibärchen	Yummy little bear-shaped chewy candies

If you go over the list, you can see that the results are quite different. The OpenAI model is able to find more relevant results, like the sugar pie and the maple syrup. The model seems to have a broader understanding of what "sweets" are, especially given multilingual context. The bge-micro-v2 model, on the other hand, is more focused on chocolate and sweets that are explicitly named as such. We also have the spicy soup mix in the results for bge-micro-v2, which is clearly not a sweet.

8.3.2. Understanding closest neighbors search

Why did we get "Gumbo Mix" (a spicy soup) when searching for "sweets" with the bge-micro-v2 model? To understand that, we need to look at how vector search works. Vector search is all about finding the closest neighbors in the vector space. Each vector represents a point in a high-dimensional space, and the distance between two vectors represents how similar their meanings are.

In Table 8.4 you can see an example of how colors can be represented as vectors in a 3D space (using RGB values). The distance between the colors represents how similar they are. For example, red and blue are far apart, while red and yellow are closer together.

Table 8.4. RGB colors as "vectors" and their similarity to pink

Color	RGB Vector [R,G,B]	Similarity to Pink [255, 0, 128]
Red	[255, 0, 0]	0.894
Blue	[0, 0, 255]	0.449
Green	[0, 255, 0]	0.000
Yellow	[255, 255, 0]	0.632
Purple	[128, 0, 128]	0.949

The table also shows the similarity to the color pink. Using the table in Table 8.4, finding the top three most similar colors to pink would yield:

1. Purple (0.949)
2. Red (0.894)

3. Yellow (0.632)

If I were to rank those colors, I would probably say that red is closer to pink, and I don't think that I would consider that yellow at all. But those are the results that math gives us, based on the values we put in. In other words, the results are only as good as the vectors that we have. That is where the specifics of the model you use for embedding generation come into play.

Limiting the results of vector search

You can specify a limit for `vector.search()` that would limit how far from the source vector RavenDB would search. That is done using the minimum similarity argument. For example, you can write: `vector.search(embedding.text(Name), $query, 0.7)`. This would limit the search to only those vectors that have a similarity of 0.7 or higher to the query vector.

The problem with this approach is that you may filter out results that you actually want. In the case of "sweets" and "Gubmo Mix", the distance is 0.583. But the distance between "sweets" and "Chocolate" is 0.641. Setting the minimum similarity to 0.7 would filter out both results. And it is hard to know what the right value is to use that would actually match your expectations.

By default, RavenDB does not set a minimum similarity, so it will return the closest matches it can find. This is usually the best approach, since it allows you to get the most relevant results and you can always set the minimum similarity based on your specific use case, dataset and the model used.

8.3.3. Turning text into embeddings

Conceptually, embedding generation is fairly simple. You take some text, send it to the model, and get back a vector. It is as easy as running: `embed(text, model: "nomic-embed-text")` and getting back the resulting vector. However, there are a quite a few details to consider. To start with, you cannot just throw text at the model and expect it to be fine with it. Most models have a limit on the amount of text they can handle in a single request. That means that you need to consider how much text you are sending to the model.

The next issue is that the text you want to embed may not be in a form that is suitable for embedding. For example, the text may contain HTML tags, and sending those to the model would add no additional value, but add significant overhead. You also have to deal with the size of the text, if you have large text, you need to break it to smaller chunks so the model can process it.

Finally, there are the realities of working with external models. You need to consider rate limits, transient errors, proper batching techniques, and more. RavenDB's Embedding Generation feature is built to handle all of that for you transparently, while giving you the right knobs to tune the behavior to your needs.

You can go over the full details in the documentation, but the most interesting options for you to consider is the Chunking Method shown in Figure 8.6. Alongs with the Max Tokens and Overlap Tokens options, these form the basis for how you control how RavenDB transforms the data to be embedded for the model.

There are three chunking methods available in RavenDB:

- Plain Text - This method simply takes the text as is, and breaks it into chunks based on the Max Tokens and Overlap Tokens settings. This is useful when you have plain text data that doesn't require any special processing.
- Markdown - This method is designed to handle Markdown formatted text. It will parse the Markdown and extract the relevant text, while ignoring the formatting. This is useful when you have Markdown documents that you want to embed.
- HTML - This method is designed to handle HTML formatted text. It will parse the HTML and extract the relevant text, while ignoring the tags. This is useful when you have HTML documents that you want to embed.

In all cases, RavenDB will apply the Max Tokens and Overlap Tokens settings to control how the text is chunked. Let's talk about what that means. Let's assume that you have a large text to embed (say, a long product description). You can set the Max Tokens to 500 and the Overlap Tokens to 50. That means that RavenDB will break the text into chunks of 500 tokens each, with an overlap of 50 tokens between each chunk. In this way, the model has better context about what the text is about.

Consider the following product description: "This amazing chocolate cake that is perfect for any occasion". If we chunk this text with Max Tokens of 10 and Overlap Tokens of 2, we would get chunks like the following:

1. "This amazing chocolate cake that is"
2. "chocolate cake that is perfect for"
3. "that is perfect for any occasion."

The overlap between the chunks helps the model to understand the context of the text better. In this example, the overlap of 2 tokens ensures that the model sees "chocolate cake" in both chunks 1 and 2, which helps it to understand that the text is about a chocolate cake. Note that we are dealing with *tokens* here, not words, and that tokens are usually smaller than words. For example, the word "chocolate" can be two tokens: "choco" and "late". The actual tokenization is done by the model itself, so different models may have different tokenization strategies.

There are whole books written about tokenization and chunking strategies, so I won't go into more detail here. As you can probably see from the example above, using very short chunks isn't really meaningful. You should check the documentation of the model you use, but the usual recommendation is to use chunks of about 256 tokens for the purpose of fact finding, with high precision. For general knowledge retrieval, you can use chunks of about 512 tokens. If your intent is to mostly get summary / analytics information, you can use chunks of about 1,024 tokens. Using more than that typically doesn't yield better results, unless your model is specifically designed to handle larger contexts. For overlap, a common recommendation is to use about 10% of the chunk size. So for a chunk size of 256 tokens, you can use an overlap of about 25 tokens.

Running Embedding Generation in a cluster

When running RavenDB in a cluster, the database will choose a node to run the Embedding Generation task on. That means that only that node will be responsible for generating the

embeddings. If that node goes down, another node will take over the task. This is done to avoid multiple nodes trying to generate embeddings for the same documents at the same time. You'll learn more about these sorts of behavior of RavenDB in Chapter 15.

Using those capabilities, RavenDB is able to extract the plain text from your documents, chunk it properly, and send it to the model for embedding generation. Those embeddings are then stored inside RavenDB and are ready to be used for vector search.

8.3.4. Behind the scenes of RavenDB's Embedding Generation

RavenDB may try to take care of all the logistics for you, but it is useful to understand what is going on behind the scenes. After you define an Embedding Generation task (as we did in Figure 8.6), RavenDB will start processing documents in the background. It will scan the Products collection for documents that need to be embedded. For each document, it will extract the Name field, process it according to the chunking method, and send the resulting chunks to the model for embedding generation in an efficient manner. The question is, what will it then *do* with those embeddings?

Those are being persisted to two system collections: @embeddings/Products and @embeddings-cache. We'll touch on the later collection in a bit, but for now, let's focus on the @embeddings/Products collection. For each document in the Products collection, RavenDB will create a corresponding document in the @embeddings/Products collection. For example, let's look at the embeddings/products/48-A document (note that products/48-A is the Chocolate product). You can see the document in Listing 8.6.

Listing 8.6. An example embedding document in the @embeddings/Products collection

```
{
  "product-name-embedding-ollama": {
    "@quantization": "Single",
    "Name": [
      "yd7vjRJp072FtRo8o4JQt1uqc02srHh+G9qjc9rm88Y="
    ]
  },
  "product-name-embedding-openai": {
    "@quantization": "Single",
    "Name": [
      "40duqImomJNPLYvgTpfH9I7pgudikEcmr9mtC2tfSiQ="
    ]
  },
  "@metadata": {
    "@collection": "@embeddings/Products",
    "@flags": "HasAttachments"
  }
}
```

We did quite a lot of work with embeddings in this chapter, so you can see in Listing 8.6 that we have two separate Embedding Generation tasks defined. For each task, we have a separate entry in the document. Each entry contains the quantization method used (we'll touch on that in a bit) and

an array for the Name property. That array holds the attachments for all the chunks for the Name property. In this case, the Chocolate product name is short enough that it fits in a single chunk, so we have a single attachment for each task. You can see the actual attachments in Figure 8.7.

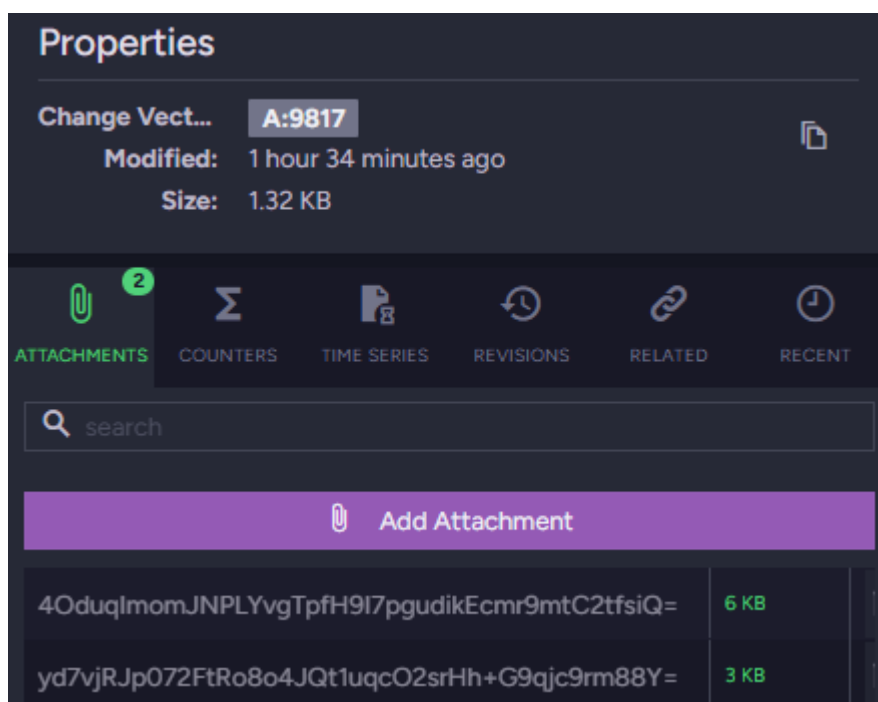


Figure 8.7. The attachment in the embeddings/products/48-A document

The "yd7vjRJp072FtRo8o4JQt1uqcO2srHh+G9qjc9rm88Y=" attachment is the embedding for the Chocolate product name, generated by the nomic-embed-text model in Ollama. And the "40duqImomJNPLYvgTpFH9I7pgudikEcmr9mtC2tfsiQ=" attachment is the embedding generated by OpenAI's text-embedding-3-small model. You can also see in the Figure 8.7 the different sizes of the attachments. In the case of OpenAI's text-embedding-3-small model, the embedding is 1536 dimensions (6KB), while the nomic-embed-text model generates embeddings of 768 dimensions (3KB).

RavenDB uses attachments to store the actual embedding vectors. This is done for efficiency reasons, we can store the embeddings vectors as raw array of float, instead of serializing them through JSON, base64 or the like. RavenDB's attachments also handle de-duplication for us, so an identical vector across multiple documents will use a single attachment under the hood. This saves both disk space and I/O when working with embeddings.

8.3.5. Embedding Quantization

Looking at Listing 8.6, you can see that we have a @quantization property for each embedding generation task. That property tells us how the embedding vector is stored. The bge-micro-v2 generates embeddings of 384 dimensions, nomic-embed-text uses 768 dimensions, OpenAI's text-embedding-3-small has 1,536 dimensions and the text-embedding-3-large model has 3,072 dimensions. A single embedding from the latter model would take 12,288 bytes (12KB) to store as an array of floats. You can see the sizes of the embeddings stored in Figure 8.7, each attachment is quite large.

If you have a large number of embeddings, that can add up quickly. Just a million embeddings from the text-embedding-3-small model would cost about 6GB of disk space. Remember that you may have

chunking in place, so a single document will likely have multiple embeddings. And you may have multiple embedding generation tasks defined, each generating its own set of embeddings. In short, embeddings can *very* quickly consume a lot of disk space. This is made worst by the fact that you also need to go through those embeddings when queries, so it isn't just the size of the disk, but also how many IOPS you need for queries. Disk space this days is pretty cheap, but IOPS are not. That is especially true in cloud environments, where getting good rate of IOPS can be quite expensive.

Compression doesn't really work well for embeddings. The vectors are pretty random in nature, so there isn't much redundancy to exploit. But it turns out that you can trade off accuracy for size. In Table 8.5, you can see the "sweets" embedding using bge-micro-v2 that we saw earlier in Listing 8.2.

Table 8.5. Showing the size and values of an embedding vector using different quantization methods (model: bge-micro-v2, text: "sweets")

Type	Vector	Size (Bytes)
Full (Single)	[-0.18093847, 0.04937424, 0.19464265, ...]	1,536
Quantized (int8)	[-118, 32, 127, ...], scale = 0.001532619	384
Restructured	[-0.180849042, 0.049043808, 0.194642613, ...]	N/A
Quantized (Binary)	[0, 1, 1, ...]	48

Table 8.5 shows how we can take advantage of quantization to reduce the size of the embedding vector. The first row shows the full embedding vector, using single precision floats (4 bytes each). This is the default way that embeddings are stored in RavenDB. The second row shows the embedding vector using int8 quantization. In this case, each float is converted to an 8-bit integer, along with a scale factor to reconstruct the original value. This reduces the size of the embedding vector by a factor of 4, at the cost of some accuracy.

You can actually see the difference between full precision and the reconstructed vector in the third row of Table 8.5. The values are pretty close, but not identical. We saved 75% of the size, but we lost some accuracy in the process. The interesting part is that, in practice, the loss of accuracy is not meaningful. The nearest neighbors search results are usually very similar to using full precision.

When talking about int8 quantization, the loss in accuracy ranges between statistically insignificant to about 2% loss in quality, depending on the model and the dataset. Many models are explicitly trained to work well with int8 quantization, so the loss in accuracy is negligible.

The last row in Table 8.5 shows binary quantization. In this case, each float is converted to a single bit, based on whether it is positive or negative. This reduces the size of the embedding vector by a factor of 64, at the cost of more accuracy loss. However, in practice, binary quantization make sense when your embeddings vector contains a *lot* of dimensions (1,024 or higher). In those cases, the loss in accuracy is still acceptable, given the significant reduction in size.

The reason binary quantization works well with large embeddings is that, in high dimensional spaces, the exact values of the dimensions become less important. With large number of dimensions, you'll likely see a drop of accuracy in the range of 5% to 10%, which is acceptable in

many scenarios. However, with smaller embeddings (under 1,024 dimensions), the loss in accuracy can be significant, so binary quantization is not recommended in those cases.

Quantization applies to both embedding generation and for indexing

In this chapter, we are looking at the quantization options in the context of Embedding Generation. However, the same quantization options are also available when defining vector indexes. That means that you can use embedding generation to store full embeddings, and then use int8 quantization when indexing those embeddings for vector search. This allows you to balance between accuracy and performance based on your specific needs.

We'll discuss those options in more detail in Chapter 13.

RavenDB supports full, int8 and binary quantization for embeddings. You can select the quantization method to use when creating the Embedding Generation task, as shown in Figure 8.6. The rule of thumb is that you want to use int8 quantization for most scenarios, unless your tests shows any significant loss in accuracy. Binary quantization is useful when you have large embeddings (1,024 dimensions or higher) and need to save on disk space and IOPS.

RavenDB gives you plenty of choices, so you can get the right balance for you. Having said that, we found that int8 quantization is a great default choice for most scenarios.

8.3.6. Caching Embeddings

In Figure 8.5, we defined a simple Embedding Generation task that would generate embeddings for the Name field of the Products collection. That raises the question of what happens when the product name changes? What actions do we need to take to ensure that the embeddings are up to date?

With RavenDB's Embedding Generation feature, the answer is: nothing. RavenDB will automatically track changes to the documents in the target collection, and will re-generate the embeddings as needed. If you update a product's name, RavenDB will detect that change and will schedule the document for re-embedding. Of course, that automatically leads to the *next* question, what happens if I make a change to a Product document that does *not* affect the Name field? Will RavenDB re-embed the document anyway (causing unnecessary work and costs)?

The answer is: no. RavenDB is smart enough to detect that the Name field was not changed, and will skip re-embedding the document. If I modified the PricePerUnit field of a product, RavenDB will not re-embed the document, since the Name didn't change since the last time that we run the embedding. Let's explore how this works under the hood.

When RavenDB generates embeddings for a document, it starts from figuring out the transformation and filtering that needs to run. The end result is the text that we want to generate embeddings for. RavenDB then computes a hash of that text, and check if that hash is already in the @embeddings-cache collection. If that is the case, we don't *need* to generate the embedding again, we can just reuse the existing embedding.

RavenDB uses a cryptographic hash function (BLAKE2b) to ensure that the hash is unique and collision resistant. You can see an example of that in Figure 8.8. The document ID in @embeddings-

cache is the name of the embedding generation task and the hash of the text that we embedded.

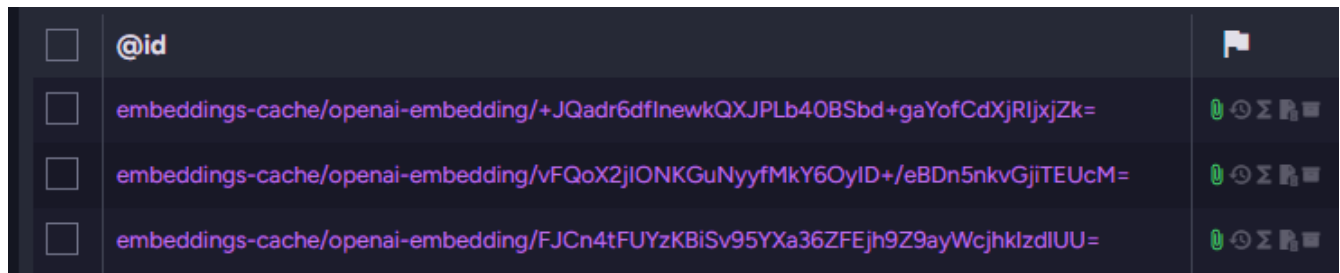


Figure 8.8. Screen shot of the @embeddings-cache collection in RavenDB Studio

The association between the embedding and its source document is maintained in the @embeddings/[Collection Name] collection, with documents named after the source document ID (embedding/[Source Document ID]). In our case, we'll have @embeddings/Products collection and the embedding/products/48-A for the Chocolate product. You saw an example of that in Listing 8.6.

If you'll look inside the the actual documents in the @embeddings-cache collection, such as the one in Figure 8.9 you'll find that they have no actual content, and are simply to hold the attachment with the actual embedding vector. Note that there is no actual relation between the @embeddings-cache document and its source. The same text, coming from different sources and even different collections, will yield the same hash and the same embedding.



Figure 8.9. An embedding cache document example

In Figure 8.9, you can see that the metadata contains an @expires property. RavenDB use the Embedding Cache Expiration setting (shown in Figure 8.6) to determine when to delete old entries from the @embeddings-cache collection. That has the effect of removing the the embedding cache that wasn't used for a long time, potentially saving disk space.

I'm writing potentially, because Embedding Generation is doing some fancy footworks behind the scenes. Consider products/48-A (the Chocolate product). An Embedding Generation task processed the document, and then generated the following output as a result:

- embeddings-cache/openai-embedding/ChhqkEbD5UbSliv7lWARTTIDj9dFK9MFhNxNpjXbBB0= - holding the embedding for the Chocolate name using the Open AI embedding model. This holds the resulting embedding vector as an attachment.
- embeddings/products/48-A - where we have a Name property with the name of the attachment(s) for this property for this embedding task. Note that this document *also* has the attachment with the actual embedding vector.

The Embedding Generation task takes advantage of the fact that RavenDB handles attachment de-

duplication for us. Both the cache document and the embedding document has the *same* attachment, but only a single copy of that attachment exists on disk. At the same time, if we change the Name property of the products/48-A document, we'll drop the old embedding reference from the embeddings/products/48-A document, and add the new one.

The *old* value (the previous embedding) is still referenced by the embeddings-cache/openai-embedding/ChhqkEbD5UbSliv7lWARTTlDj9dFK9MFhNxNpjXbBB0= document, and will be cached until it expires. Once it expires, RavenDB will delete the cache document, and the attachment will be removed from disk. On the other hand, if we never see another update to the products/48-A document, the embedding reference in the embeddings/products/48-A document will remain valid, but after the expiration time, the cache document will be removed (without updating the embeddings/products/48-A, which still points to the attachment).

Whenever RavenDB encounters a value that needs to be embed that is already in the cache, we'll update the expiration time on the @embeddings-cache document, to ensure that it remains available. This way, frequently used embeddings remain in the cache, while infrequently used ones are cleaned up over time. You can also control that behavior by adjusting the Embedding Cache Expiration setting in the Embedding Generation task configuration.

So many hashes

Using Chocolate as our example value, we have the hash ChhqkEbD5UbSliv7lWARTTlDj9dFK9MFhNxNpjXbBB0= used in the @embeddings-cache document ID. That hash is computed using the BLAKE2b hash function on the text CHOCOLADE. We process the text so it is in upper case (and trim it), because for embedding purposes, we found that case sensitive embeddings are not useful. In other words, the embedding for Chocolate and chocolate are identical in RavenDB, since we normalize the text we use as the cache key.

We use utf16 encoding for the hashing (for internal implementation details in RavenDB) I mention that only for completeness' sake. If you'll try to compute the hash using blake2b, you'll likely use utf8 encoding and be confused why it doesn't match. You'll need to ensure you use utf16 encoding to get the same result.

The names of the *attachments*, on the other hand, are *also* hashes. But those are the blake2b hash of the actual embedding vector bytes. There is no meaningful way to reference those values outside of RavenDB, so we just use the hash as the attachment name. (Note that the attachment's name has no bearing on RavenDB's ability to de-duplicate attachments, that is done based on the content of the attachment itself.)

All of the above are strictly implementation details, and may change over time (in a compatible way, of course). I wanted to share them with you so you can understand how RavenDB is actually handling things behind the scenes.

The cache is explicitly held as documents so it will be shared across all Embedding Generation tasks, rather than being state that is specific to a single Embedding Generation task. This caching behavior has several important implications:

- It minimizes the number of calls made to the embedding model, reducing costs and improving

performance.

- It ensures that embeddings are only re-generated when necessary, letting RavenDB easily and predictably detect when we need to re-embed values.
- This caching mechanism works across documents, across collections and will work even if you have multiple Embedding Generation tasks defined for the same model. As long as they use the same AI connection string, they will share the same cache and benefits.
- Deleting or re-creating the Embedding Generation task is safe, since the cache will still be valid and used as needed.

8.3.7. Generating Embeddings for Queries

You can use Embedding Generation tasks in queries as we saw, as you can see in [vector-search-query].

Listing 8.7. An example of using Embedding Generation in a vector search query

```
vector.search(embedding.text(Name,  
    ai.task("product-name-embedding-openai")), $query)
```

In order to run a `vector.search()` query, RavenDB needs to have the embedding for the `$query` value. RavenDB will use the same Embedding Generation task to generate the embedding for the query value, including the caching behavior. The caching behavior is actually much more significant impact for queries than for documents. Generating embeddings for documents is an async background operation, we prefer to optimize for throughput there. Queries, on the other hand, are latency sensitive operations, there is a *user* twiddling their thumbs on the other side.

When you run a query that requires embedding generation, RavenDB will first check the `@embeddings-cache` collection to see if the embedding for the query value is already available. If it is, we can use that embedding directly, and avoid the overhead of calling the external model. Note that this is the same exact cache that is used for documents, so if you query over a value that already exists in the documents, you'll get a guaranteed cache hit.

If you are querying over values that are not in the documents, RavenDB will need to call the external model to generate the embedding. This happens transparently during query time, and also generates an `@embeddings-cache` entry for future use. The next time that you'll run the same query, RavenDB will find the embedding in the cache, and avoid the external call.

Rate limits and queries

Most external models you'll use have some form of rate limit that you have to take into account. That can be a maximum number of tokens per minutes (or even day) or maximum number of requests per minute. RavenDB's Embedding Generation feature is built to handle those limits transparently for you. If you hit a rate limit, RavenDB will back off and retry the request after some time.

That works quite nicely when we generating embeddings for documents, indeed, that where the async & background nature of the Embedding Generation feature shines. However, when

generating embeddings for queries, we run into a problem. We cannot arbitrarily delay a query indefinitely, since that would lead to a poor user experience. To handle that, RavenDB will immediately raise an error for such queries. This is done to ensure that your application can handle such scenarios gracefully, rather than being stuck waiting for a long time.

This is one of those cases where a cache is likely to both be really helpful and quite confusing. If you manage to burn through all your rate limits, you need to consider that *some* queries will continue to work properly (using the already generated embeddings from the cache) but some will fail (since there is no cached embedding for them). This is a partial failure mode that is worth noting, since it is much less obvious than complete failure.

When deploying features that rely on automatic embedding of users' input to work, you need to ensure that your rate limits are set high enough to handle your expected workload as well as monitor their usage.

Embedding Generation task allows you to define different cache durations for values coming from documents and values that were cached because of queries. This is required because you'll likely have a lot more variance in the texts you'll embed for queries, over the usually more stable set of texts that you have in your documents. This matters because embeddings can be pretty large, especially if you have a lot of them.

By default, RavenDB will cache embeddings from documents for 90 days, while embeddings from queries will be cached for 14 days. You can adjust those settings in the Embedding Generation task configuration, as shown in Figure 8.6.

8.3.8. Using Embedding Generation Scripts

In Figure 8.6, we defined a simple Embedding Generation task that would generate embeddings for the Name field of the Products collection. You aren't limited to simply specifying the fields to embed, however. RavenDB allows you to define a custom script that will be used to generate the text to embed. A simple example of that is shown in Listing 8.8.

Listing 8.8. An Embedding Generation script that adds additional context to the product name

```
const category = load(this.Category);
embeddings.generate({
  Query: `
    Product Name: ${this.Name}
    Category: ${category.Name}
  `,
});
```

Listing 8.8 shows how we can add additional context to the product name by including the product category in the text to embed. Consider product such as "Monitor" which can belong to the "Babies" category or the "Computers" category. Or "Pump" which can refer to "Shoes" or "Plumbing".

Note that the script in Listing 8.8 is going beyond merging the name of the product and the name of the category together. It is actually adding some structure to the text, by labeling each part. This

helps the embedding model to understand the context better, leading to more accurate embeddings. Note that we are also *loading* the related Category and using its Name property, not just the Name of the product.

Loading related documents in Embedding Generation scripts

You'll learn more about load and its capabilities in Chapter 13, but for now, it is useful to understand that you can load related documents in Embedding Generation scripts. That allows you to enrich the text that is being embedded with additional context from related documents. However, you need to be aware that RavenDB will *not* re-run the embedding generation if the *related* document changes. In other words, if you change the Name of a Category, RavenDB will not re-embed all the products in that category, since their own document didn't change.

The script also generated a *new* field name: Query. That field name is relevant only for queries, you can run queries over that field, as you can see in Listing 8.9.

Listing 8.9. Using generated field name Query in vector search

```
from "Products"
where vector.search(embedding.text(Query,
    ai.task("product-name-embedding-openai")), $query)
```

You can also do much more complex processing, such as combining multiple fields, applying transformations, and more. The script is a full JavaScript environment, so you can use any JavaScript features you need to generate the text to embed. Note that `embeddings.generate()` can be called multiple times in the script, allowing you to generate embeddings for array values or from multiple fields.

`embeddings.generate()` accepts an object, where each property is a field name, and the value is the text to embed for that field. You can also control the transformation behavior of each individual field as shown in Listing 8.10.

Listing 8.10. An Embedding Generation script with per-field options

```
embeddings.generate({
  Body: html.strip(this.HtmlBody, 2048),
  Description: markdown.splitLines(this.MarkdownDescription, 1024),
});
```

Listing 8.10 shows how we can use different transformation methods for different fields. The Body field is processed as HTML from the `HtmlBody` field, stripping the tags and extracting the plain text, as well as chunking the data on 2,048 tokens. The Description field is processed as Markdown from the `MarkdownDescription` field, splitting the text into lines of up to 1,024 tokens each.

8.4. Generative AI inside your database

8.5. Generative AI & Subscriptions

Subscriptions to handle output from generative AI to evaluate support calls.

Chapter 9. RavenDB Clusters

Chapter 10. RavenDB on Cloud and the Edge

The smallest machine we ever run RavenDB on was a Raspberry PI Zero. It had a single core CPU and 512MB of RAM running a 32-bit OS. RavenDB works on that kind of machine, and users have deployed production machines on that kind of hardware. RavenDB Cloud largest instance size (at this point of time) is a 3 nodes cluster with 48 cores and 384GB of RAM per node. You can go larger, but that is the largest pre-defined instance size. RavenDB is designed to run in a wide variety of environments, from the smallest edge devices to the largest cloud deployments.

In this chapter, we are going to look at how RavenDB works in these different environments, and what are the things that you need to consider when deploying RavenDB in these scenarios.

Chapter 11. Sharding in RavenDB

Chapter 12. Querying in RavenDB

Chapter 13. Indexing in RavenDB

Chapter 14. MapReduce and Aggregations in RavenDB

Chapter 15. Sharing data and making friends with ETL

Chapter 16. Production Deployments

Chapter 17. Securing your Ravens

Chapter 18. Inside the Voron storage engine

Chapter 19. Inside the Corax indexing engine

Chapter 20. Operational Recipes